SLAM十四：从理到践-扉.pdf 1 2017/3/31 16:21:01

视觉SLAM十四讲

从理论到实践

高翔 张涛 刘毅 颜沁睿 著

電子工業出版社
Publishing House of Electronics Industry
北京•BEIJING

内 容 简 介

本书系统介绍了视觉 SLAM（同时定位与地图构建）所需的基本知识与核心算法，既包括数学理论基础，如三维空间的刚体运动、非线性优化，又包括计算机视觉的算法实现，例如多视图几何、回环检测等。此外，我们还提供了大量的实例代码供读者学习研究，从而更深入地掌握这些内容。

本书可以作为对 SLAM 感兴趣的研究人员的入门自学材料，也可以作为 SLAM 相关的高校本科生或研究生课程教材使用。

未经许可，不得以任何方式复制或抄袭本书之部分或全部内容。

版权所有，侵权必究。

图书在版编目（CIP）数据

视觉 SLAM 十四讲：从理论到实践 / 高翔等著.—北京：电子工业出版社，2017.3

ISBN 978-7-121-31104-8

I. ①视…II. ①高…III. ①人工智能—视觉跟踪—研究 IV. ①TP18

中国版本图书馆 CIP 数据核字（2017）第 053910 号

策划编辑：郑柳洁

责任编辑：白　涛

印　　刷：北京季蜂印刷有限公司

装　　订：北京季蜂印刷有限公司

出版发行：电子工业出版社

　　北京市海淀区万寿路 173 信箱　　邮编：100036

开　　本：720×1000　1/16　　印张：25　　字数：560 千字

版　　次：2017 年 3 月第 1 版

印　　次：2018 年 1 月第 7 次印刷

定　　价：75.00 元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：（010）88254888，88258888。

质量投诉请发邮件至 zlts@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

本书咨询联系方式：（010）51260888-819　faq@phei.com.cn。

阅读须知

本书图片

为描述方便，书中介绍图片相关内容时直接使用了对应的色彩名称，对应到纸面上即灰色。此外，部分图片可能需要放大观察，纸面上无法呈现应有效果。为此，书中图片均在博文视点官方网站提供下载。

读者服务

轻松注册成为博文视点社区用户（www.broadview.com.cn），您即可享受以下服务。

- **下载资源：**本书所提供的示例代码及资源文件均可在 下载资源 处下载。
- **提交勘误：**您对书中内容的修改意见可在 提交勘误 处提交，若被采纳，将获赠博文视点社区积分（在您购买电子书时，积分可用来抵扣相应金额）。
- **与作者交流：**在页面下方 读者评论 处留下您的疑问或观点，与作者和其他读者一同学习交流。

页面入口：http://www.broadview.com.cn/31104

第二版序

自《视觉 SLAM 十四讲：从理论到实践》出版以来已经过去了一年多的时间。这一年内，这本书在业界引起了广泛的关注和讨论。大多数读者评价是正面的，不过也有些地方不够令人满意。比如说，这本书作为初学者向，入门则入门矣，有些应该深入的地方还讲得不够深入；再比如说，书的数学符号不够统一，有些地方容易令读者产生了误解。还有的地方，读者认为叙述不够有趣，我没有完全放开，这都是事实，所以我准备在第二版中更加放飞自我一些。

实际上，第一版书是在 2016 年中期开始写的，所有文字、图片和代码都需要从头准备，再加上我也是第一次写这么厚的书，错误部分所在难免。2018 年，我在慕尼黑工大给学生讲 SLAM 课程，期间又积累了一些材料。所以第二版书从内容上也会更丰富，更合理一些。第二版在第一版的基础上做的改动有：

1. 更多的实例。我增加了一些实验代码来介绍算法的原理。在第一版中，多数实践代码调用了各种库中的内置函数，我现在认为更加深入地介绍底层计算会更好一些。所以第二版的许多代码，在调用库函数之外，还提供了底层的实现。
2. 更深入的内容，特别是第七章至第十二章，同时也删除了一些泛泛而谈的边角料。对第一版大部分数学公式进行了审视，重写了那些容易引起误解的地方。
3. 更完善的项目。我将第一版的第九章移至了第十三章。于是，我们可以在介绍了所有必要知识之后，向大家展现一个完整的 SLAM 系统如何工作了。
4. 更通俗、简洁、口语化的表达，我觉得这是一本好书的标准，特别是介绍一些看起来高深莫测的数学知识时。我也重新制作了部分插图，在黑白打印成书之后会看起来更清楚一些。
5. 当然，每章前的简笔画我是不会改的！

总之，我尽量做到深入浅出，也希望第二版书能够给你更加舒服的阅读体验。

目录

第一章 预备知识 1
1.1 本书讲什么 1
1.2 如何使用本书 3
1.2.1 组织方式 3
1.2.2 代码 5
1.2.3 面向的读者 5
1.3 风格约定 6
1.4 致谢和声明 7

第一部分 数学基础 9

第二章 初识 SLAM 11
2.1 引子：小萝卜的例子 13
2.2 经典视觉 SLAM 框架 19
2.2.1 视觉里程计 20
2.2.2 后端优化 21
2.2.3 回环检测 22
2.2.4 建图 23
2.3 SLAM 问题的数学表述 24
2.4 实践：编程基础 27
2.4.1 安装 Linux 操作系统 27
2.4.2 Hello SLAM 29
2.4.3 使用 cmake 30
2.4.4 使用库 32
2.4.5 使用 IDE 34
第三章 三维空间刚体运动 39
3.1 旋转矩阵 41
3.1.1 点和向量，坐标系 41
3.1.2 坐标系间的欧氏变换 42
3.1.3 变换矩阵与齐次坐标 45
3.2 实践：Eigen 46
3.3 旋转向量和欧拉角 50
3.3.1 旋转向量 50
3.3.2 欧拉角 51
3.4 四元数 53
3.4.1 四元数的定义 53
3.4.2 四元数的运算 54
3.4.3 用四元数表示旋转 56
3.4.4 四元数到其他旋转表示的转换 56
3.5 * 相似、仿射、射影变换 58
3.6 实践：Eigen 几何模块 59
3.6.1 Eigen 几何模块的数据演示 59
3.6.2 实际的坐标变换例子 61
3.7 可视化演示 62
3.7.1 显示运动轨迹 62
3.7.2 显示相机的位姿 65
第四章 李群与李代数 67
4.1 李群与李代数基础 69
4.1.1 群 69
4.1.2 李代数的引出 70
4.1.3 李代数的定义 71
4.1.4 李代数 $\mathfrak{so}(3)$ 72
4.1.5 李代数 $\mathfrak{se}(3)$ 73
4.2 指数与对数映射 73
4.2.1 SO(3) 上的指数映射 73
4.2.2 SE(3) 上的指数映射 75
4.3 李代数求导与扰动模型 77
4.3.1 BCH 公式与近似形式 77
4.3.2 SO(3) 李代数上的求导 78
4.3.3 李代数求导 79
4.3.4 扰动模型（左乘） 80

4.3.5 SE(3) 上的李代数求导 80
4.4 实践：Sophus 81
4.4.1 Sophus 的基本使用方法 81
4.4.2 例子：评估轨迹的误差 83
4.5 * 相似变换群与李代数 86
4.6 小结 87
第五章 相机与图像 89
5.1 相机模型 91
5.1.1 针孔相机模型 91
5.1.2 畸变 94
5.1.3 双目相机模型 97
5.1.4 RGB-D 相机模型 98
5.2 图像 100
5.3 实践：计算机中的图像 101
5.3.1 OpenCV 的基础使用方法 101
5.3.2 图像去畸变 105
5.4 实践：3D 视觉 106
5.4.1 双目视觉 106
5.4.2 RGB-D 视觉 107
第六章 非线性优化 113
6.1 状态估计问题 115
6.1.1 批量状态估计与最大后验估计 115
6.1.2 最小二乘的引出 117
6.1.3 例子：批量状态估计 118
6.2 非线性最小二乘 120
6.2.1 一阶和二阶梯度法 120
6.2.2 高斯牛顿法 121
6.2.3 列文伯格—马夸尔特方法 123
6.2.4 小结 125
6.3 实践：曲线拟合问题 125
6.3.1 手写高斯牛顿法 125
6.3.2 使用 Ceres 进行曲线拟合 128
6.3.3 使用 g2o 进行曲线拟合 133
6.3.4 g2o 的编译与安装 134
6.3.5 使用 g2o 拟合曲线 135
6.4 小结 139

第二部分 实践应用 141
第七章 视觉里程计 1 143
7.1 特征点法 145
7.1.1 特征点 145
7.1.2 ORB 特征 147
7.1.3 特征匹配 150
7.2 实践：特征提取和匹配 152
7.2.1 OpenCV 的 ORB 特征 152
7.2.2 手写 ORB 特征 154
7.2.3 计算相机运动 157
7.3 2D−2D: 对极几何 158
7.3.1 对极约束 158
7.3.2 本质矩阵 160
7.3.3 单应矩阵 162
7.4 实践：对极约束求解相机运动 164
7.5 三角测量 169
7.6 实践：三角测量 170
7.6.1 三角测量代码 170
7.6.2 讨论 172
7.7 3D−2D：PnP 172
7.7.1 直接线性变换 174
7.7.2 P3P 175
7.7.3 Bundle Adjustment 177
7.8 实践：求解 PnP 180
7.8.1 使用 EPnP 求解位姿 180
7.8.2 使用 BA 优化 182
7.9 3D−3D：ICP 188
7.9.1 SVD 方法 189

7.9.2 非线性优化方法 191
7.10 实践：求解 ICP 191
7.10.1 SVD 方法 191
7.10.2 非线性优化方法 193
7.11 小结 195
第八章 SLAM：现在与未来 197
8.1 当前的开源方案 198
8.1.1 MonoSLAM 199
8.1.2 PTAM 200
8.1.3 ORB-SLAM 203
8.1.4 LSD-SLAM 205
8.1.5 SVO 206
8.1.6 RTAB-MAP 208
8.1.7 其他 209
8.2 未来的 SLAM 话题 209
8.2.1 视觉 + 惯性导航 SLAM 209
8.2.2 语义 SLAM 211
8.2.3 SLAM 的未来 212
附录 A 高斯分布的性质 213
附录 B 矩阵求导 215
附录 C ROS 入门 217
参考文献 221

第一章　预备知识

1.1　本书讲什么

这是一本介绍视觉 SLAM 的书，也很可能是第一本以视觉 SLAM 为主题的中文书。

那么，SLAM 是什么？

SLAM 是 **S**imultaneous **L**ocalization **a**nd **M**apping 的缩写，中文译作“**同时定位与地图构建**”[1]。它是指搭载特定**传感器**的主体，在**没有环境先验信息**的情况下，于**运动过程中**建立**环境**的模型，同时估计自己的**运动**[2]。如果这里的传感器主要为相机，那就称为“视觉 SLAM”。

本书的主题就是视觉 SLAM。这里我们刻意把许多个定义放到一句话中，希望帮助读者建立一个较明确的概念。首先，SLAM 的目的是解决“定位”与“地图构建”这两个问题。也就是说，一边要估计传感器自身的位置，一边要建立周围环境的模型。那么怎么解决呢？这需要用到传感器的信息。传感器以一定形式观察外部世界，但不同传感器观察的方式是不同的。之所以要花一本书的内容去讨论这个问题，是因为它很难——特别是我们希望**实时地**、在**没有先验知识**的情况下进行 SLAM。当用相机作为传感器时，我们要做的就是根据一张张连续运动的图像（它们形成了一段视频），从中推断相机的运动，以及周围环境的情况。

这似乎是个很直观的问题。我们自己走进陌生的环境时不就是这么做的吗？

在计算机视觉（Computer Vision）创立之初，人们就想象着有朝一日，计算机将和人一样，通过眼睛去观察世界，理解周遭的物体，探索未知的领域——这是一个美妙而又浪漫的梦想，吸引了无数的科研人员日夜为之奋斗[3]。我们曾经以为这件事情并不困难，然而进展却远不如预想的那么顺利。我们眼中的花草树木、虫鱼鸟兽，在计算机中却是那样的不同：它们只是一个个由数字排列而成的矩阵。让计算机理解图像的内容，就像让我们自己理解这些数字一样困难。我们既不了解自己如何理解图像，也不知道计算机该如何理解、探索这个世界。于是我们困惑了很久，直到几十年后的今天，才发现了一点点成功的迹象：通过人工智能（Artificial Intelligence）中的机器学习（Machine Learning）技术，计算机渐渐能够辨别出物体、人脸、声音、文字——尽管它所用的方式（统计建模）与我们是如此不同。另一方面，在 SLAM 发展了将近 30 年之后，我们的相机才渐渐开始能够认识到自身的位置，发觉自己在运动——虽然方式还是和人类有巨大的差异。不过，至少研究者们已经成功地搭建出种种实时 SLAM 系统，有的能够快速跟踪自身位置，有的甚至能够进行实时的三维重

建。

这件事情确实很困难，但我们已经有了很大的进展。更令人兴奋的是，近年来随着科技的发展，涌现出了一大批与 SLAM 相关的应用点。在许多地方，我们都希望知道自身的位置：室内的扫地机和移动机器人需要定位，野外的自动驾驶汽车需要定位，空中的无人机需要定位，虚拟现实和增强现实的设备也需要定位。SLAM 是那样重要。没有它，扫地机就无法在房间自主地移动，只能盲目地游荡；家用机器人就无法按照指令准确到达某个房间；虚拟现实也将永远固定在座椅之上——所有这些新奇的事物都无法出现在现实生活中，那将多么令人遗憾。

今天的研究者和应用开发人员，逐渐意识到了 SLAM 技术的重要性。在国际上，SLAM 已经有近三十年的研究历史，也一直是机器人和计算机视觉的研究热点。21 世纪以来，以视觉传感器为中心的**视觉 SLAM 技术**，在理论和实践上都经历了明显的转变与突破，正逐步从实验室研究迈向市场应用。同时，我们又遗憾地发现，至少在国内，与 SLAM 相关的论文、书籍仍然非常匮乏，让许多对 SLAM 技术感兴趣的初学者无从一窥门径。虽然 SLAM 的理论框架基本趋于稳定，但其编程实现仍然较为复杂，有着较高的技术门槛。刚步入 SLAM 领域的研究者，不得不花很长的时间，学习大量的知识，往往要走过许多弯路才得以接近 SLAM 技术的核心。

本书全面系统地介绍了以视觉传感器为主体的视觉 SLAM 技术，我们希望它能（部分地）填补这方面资料的空白。我们会详细地介绍 SLAM 的理论背景、系统架构，以及各个模块的主流做法。同时，**极其重视实践**：本书介绍的**所有**重要算法，都将给出可以运行的实际代码，以求加深读者的理解。在第二版中，我们对于大多数算法，会讨论内在的原理，而非简单地从函数库中进行调用。之所以这么做，主要是考虑到 SLAM 毕竟是一项和实践紧密相关的技术。再漂亮的数学理论，如果不能转化为可以运行的代码，那就仍是可望而不可即的空中楼阁，没有实际意义。我们相信，实践出真知，实践出真爱。只有实际地演算过各种算法之后，你才能真正地认识 SLAM，真正地喜欢上科研。

自 1986 年提出以来[4]，SLAM 一直是机器人领域的热点问题。关于它的文献数以千计，想要对 SLAM 发展史上的所有算法及变种做一个完整的说明，是十分困难而且没有必要的。本书中会介绍 SLAM 所牵涉的背景知识，例如射影几何、计算机视觉、状态估计理论、李群李代数等，并在这些背景知识之上，给出 SLAM 这棵大树的主干，而略去一部分形状奇特、纹理复杂的枝叶。我们认为这种做法是有效的。如果读者能够掌握主干的精髓，那么自然会有能力去探索那些边缘的、细节的、错综复杂的前沿知识。所以，我们的目的是，让 SLAM 的初学者通过阅读本书快速地成长为能够探索这个领域边缘的研究者。另一方面，即便你已经是 SLAM 领域的研究人员，本书也可能有一些你还觉得陌生的地方，可以让你产生新的见解。

目前，与 SLAM 相关的书籍主要有《概率机器人》（*Probabilistic robotics*）[5]、《计算机视觉中的多视图几何》（*Multiple View Geometry in Computer Vision*）[3]、《机器人学中的状态估计》（*State Estimation for Robotics: A Matrix-Lie-Group Approach*）[6] 等。它们内容丰富、论述全面、推导严谨，是 SLAM 研究者中脍炙人口的经典教材。然而就目前来看，还存在两个重要的问题：其一，这些图书的目的在于介绍基础理论，SLAM 只是其应用之一。因此，它们并不能算是专门讲解 SLAM 的书

籍。其二，它们的内容偏重于数学理论，基本不涉及编程实现，导致读者经常出现“书能看懂却不会编程”的情况。而我们认为，只有读者亲自实现了算法，调试了各个参数，才能谈得上真正理解了问题本身。

我们会提及 SLAM 的历史、理论、算法、现状，并把完整的 SLAM 系统分成几个模块：视觉里程计、后端优化、建图，以及回环检测。我们将陪着读者一点点实现这些模块中的核心部分，探讨它们在什么情况下有效，什么情况下会出问题，并指导大家在自己的机器上运行这些代码。你会接触到一些**必要的**数学理论和许多编程知识，会用到 Eigen、OpenCV、PCL、g2o、Ceres 等库[①]，掌握它们在 Linux 操作系统中的使用方法。

从写作风格上，我们不想把本书写成枯燥的理论书籍。技术类图书应该是严谨可靠的，但严谨不意味着刻板。一本优秀的技术书应该是生动有趣而易于理解的。如果你觉得“这个作者怎么这么不正经”，敬请原谅，因为我并不是一个非常严肃的人[②]。无论如何，有一件事是可以肯定的：只要你对这门新技术感兴趣，在学习本书的过程中肯定会有所收获！您会掌握与 SLAM 相关的理论知识，你的编程能力也将有明显的进步。在很多时候，您会有一种“我在陪你一起做科研”的感觉，这正是我所希望的。但愿您能在此过程中发现研究的乐趣，喜欢这种“通过一番努力，看到事情顺利运行”的成就感。

好了，话不多说，祝你旅行愉快！

1.2　如何使用本书

1.2.1　组织方式

本书名为“视觉 SLAM 十四讲”。顾名思义，我们会像在学校里讲课那样，以“讲”作为本书的基本单元。每一讲都对应一个固定的主题，其中会穿插“理论部分”和“实践部分”两种内容。通常是理论部分在前，实践部分在后。在理论部分中，我们将介绍**理解算法所必需**的数学知识，并且大多数时候以叙述的方式，而不是像数学教科书那样用“定义—定理—推论”的方式，因为我们觉得这样的方式阅读起来更容易一些，尽管有时候显得不那么严谨。实践部分主要是编程实现，讨论程序里各部分的含义及实验结果。看到标题中带有“实践”两个字的章节，你就应该（兴致勃勃地）打开电脑，和我们一起愉快地编写代码了。

值得一提的是，我们只会把与解决问题相关的数学知识放在书里，并尽量保持浅显。因为我是工科生，所以要勇敢地承认，某些做法只要经验上够用，没必要非得在数学上追求完备。只要我们知道这些算法在绝大多数实际场景下能够工作，并且数学家们（通过冗长而且复杂的证明和讨论）说明在什么情况下可能不工作，那么我就表示满意，而不刻意追究那些看似完美的证明（当然它们固有自己不可否认的价值）。由于 SLAM 牵涉到了太多数学背景，为了防止使本书变成数学教科书，

[①] 如果你完全没有听说过它们，那么应该感到兴奋，这说明你会从本书中收获很多知识。

[②] 你会经常在脚注中发现一些神奇的东西。

我们把一些细节上的推导和证明留作习题和补充阅读材料，方便感兴趣的读者进一步阅读参考文献，更深入地掌握相关细节。

每一讲正文之后，我们设计了一些习题。其中，带 * 号的习题是具有一定难度的。我们强烈建议读者把习题都练习一遍，这对你掌握这些知识很有帮助[①]。

全书内容主要分为两个部分。

1. 第一部分为**数学基础**篇，我们会以浅显易懂的方式，铺垫与视觉 SLAM 相关的数学知识，包括：
   - 第 1 讲是前言，介绍这本书的基本信息，习题部分主要包括一些自测题。
   - 第 2 讲为 SLAM 系统概述，介绍一个 SLAM 系统由哪些模块组成，各模块的具体工作是什么。实践部分介绍编程环境的搭建过程以及 IDE 的使用。
   - 第 3 讲介绍三维空间运动，你将接触到旋转矩阵、四元数、欧拉角的相关知识，并且在 Eigen 当中使用它们。
   - 第 4 讲为李群和李代数。即便你现在不懂李代数为何物，也没有关系。你将学到李代数的定义和使用方式，然后通过 Sophus 操作它们。
   - 第 5 讲介绍针孔相机模型以及图像在计算机中的表达。你将用 OpenCV 来调取相机的内外参数。
   - 第 6 讲介绍非线性优化，包括状态估计理论基础、最小二乘问题、梯度下降方法。你会完成一个使用 Ceres 和 g2o 进行曲线拟合的实验。

   这些就是我们要用到的所有数学知识了，当然，其中还隐含了你以前学过的高等数学和线性代数。我保证它们看起来都不会很难，至少没有听上去那么难。当然，若你想进一步深入挖掘，我们会提供一些参考资料供你阅读，那些材料可能会比正文里讲的知识难一些。
2. 第二部分为 **SLAM 技术**篇。我们会使用第一部分所介绍的理论，讲述视觉 SLAM 中各个模块的工作原理。
   - 第 7 讲为特征点法的视觉里程计。该讲内容比较多，包括特征点的提取与匹配、对极几何约束的计算、PnP 和 ICP 等。在实践中，你将用这些方法去估计两个图像之间的运动。
   - 第 8 讲为直接法的视觉里程计。你将学习光流和直接法的原理，然后实现一个简单的直接法运动估计。
   - 第 9 讲为后端优化，主要为对 Bundle Adjustment 的深入讨论，包括基本的 BA，以及如何利用稀疏性加速求解过程。你将用 Ceres 和 g2o 分别书写一个 BA 程序。
   - 第 10 讲主要讲后端优化中的位姿图。位姿图是表达关键帧之间约束的一种更紧凑的形式。我们会介绍 SE(3) 和 Sim(3) 的位姿图，同时你将使用 g2o 对一个位姿球进行优化。
   - 第 11 讲为回环检测，主要介绍以词袋方法为主的回环检测。你将使用 dbow3 书写字典训练程序和回环检测程序。

[①] 它们也可能成为今后相关行业的面试题，或许还能帮你在找工作时留个好印象。

- 第 12 讲为地图构建。我们会讨论如何使用单目进行稠密深度图的估计（以及这是多么不可靠），然后讨论 RGB-D 的稠密地图构建过程。你会书写极线搜索与块匹配的程序，然后在 RGB-D 中遇到点云地图和八叉树地图的构建问题。
- 第 13 讲是工程章，你将搭建一个双目视觉里程计框架，综合运用先前学过的知识，实现它的基本功能。这个过程中，你会碰到一些问题，例如优化的必要性、关键帧的选择等。我们会在 Kitti 数据集上测试它的性能，讨论一些改进的手段。
- 第 14 讲主要介绍当前的开源 SLAM 项目以及未来的发展方向。相信在阅读了前面的知识之后，你会更容易理解它们的原理，实现自己的新想法。

最后，如果你完全看不懂上面在说什么，那么恭喜你！这本书很适合你！加油！

1.2.2　代码

本书所有源代码均托管在 GitHub 上：

`https://github.com/gaoxiang12/slambook2`

注意后面有一个 2，表示这是第二版书的代码。我强烈建议读者下载代码以供随时查看。代码是按章节划分的，比如，第 7 讲的内容就会放在 ch7 文件夹中。此外，对于书中用到的一些小型库，会以压缩包的形式放在 3rdparty 文件夹下。在第二版中，我们用 git submodule 工具来保证读者使用的软件版本与书中的完全一致，所以读者不必操心软件版本问题。对于像 OpenCV 那种大中型库，我们会在它们第一次出现时介绍其安装方法。如果你对代码有任何疑问，请单击 GitHub 上的 Issues 按钮，提交问题。如果确实是代码出现问题，我们会及时进行修改；即使是你的理解有偏差，我也会尽可能回复。如果你不习惯使用 Git，那么单击右侧包含 download 字样的按钮将代码下载至本地即可。

1.2.3　面向的读者

本书面向对 SLAM 感兴趣的学生和研究人员。阅读本书需要一定的基础，我们假设你具备以下知识：

- **高等数学、线性代数、概率论**。这些是大部分读者应该在大学本科阶段接触过的基本数学知识[①]。你应当明白矩阵和向量是什么，或者做微分和积分是什么意思。对于 SLAM 中用到的专业知识，我们会额外加以介绍。
- **C++ 语言基础**。由于我们采用 C++ 作为编码语言，所以建议读者至少熟悉这门语言的语法。比如，你应该知道类是什么，如何使用 C++ 标准库，模板类如何使用，等等。我们会避免过多地使用技巧，但有些地方确实无法避免。此外，我们还使用了一些 C++ 11 标准的内容，不过，我们会在用到的地方加以解释。

[①] 实际当中每个人都至少需要学三遍线性代数：本科一遍，研究生一遍，工作时期一遍。

- **Linux 基础**。我们的开发环境是 Linux 而非 Windows，并且只提供 Linux 下的源程序，**不会**再提供 Windows 下的开发方法介绍。**我们认为，掌握 Linux 是一个 SLAM 研究人员所必需的，请初学者暂时不要问为什么，把本书的知识学好之后相信你会和我们有同样的想法**。各种程序库在 Linux 下的配置都非常便捷，你也会在此过程中体会到 Linux 的便利。如果读者此前从未使用过 Linux，那么最好找一本 Linux 的教材稍加学习（掌握基本知识即可，一般就是相关图书的前面几章内容）。我们不要求读者具备多么高超的 Linux 操作技能，但希望读者至少知道“打开终端，进入代码目录”是如何操作的。本讲的习题里有一些 Linux 知识自测题，如果你清楚自测题的答案，那么阅读本书代码不会有任何问题。

对 SLAM 感兴趣但不具备上述知识的读者，可能在阅读本书时会感到困难。如果你不了解 C++ 的基本知识，可以读一点 *C++ Primer Plus* 之类的图书入门；如果你缺少相关的数学知识，也可以先阅读一些相关数学教材补充知识，不过我们认为，对大多数大学本科水平的朋友，读懂本书所需的数学背景肯定是具备了。代码方面，你最好花点时间亲自输入一遍，再调节里面的参数，看看效果会发生怎样的改变。这会对学习很有帮助。

本书可作为 SLAM 相关课程的教材，亦可作为课外自学材料使用。

1.3 风格约定

本书既有数学理论介绍，也有编程实现，因此，为方便阅读，对不同内容采用了不同排版方式加以区分。

1. 数学公式单独列出，重要的公式还在右侧标了序号，例如：

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}. \tag{1.1}$$

   数学字体采用国标风格。标量使用斜体字（如 a, α），向量和矩阵使用粗斜体（如 $\boldsymbol{a}, \boldsymbol{A}, \boldsymbol{\Sigma}$，希腊字母除外）。空心粗体代表特殊集合，如实数集 $\mathbb{R}$、整数集 $\mathbb{Z}$。李代数部分使用哥特体，如 $\mathfrak{so}(3), \mathfrak{se}(3)$。

2. 程序代码以方框框出，使用不同的字体和小一些的字号，左侧带有行号。如果程序较长，方框会延续到下一页。总之，看起来像这样：

示例代码

```
#include <iostream>
using namespace std;

int main ( int argc, char** argv )
{
  cout<<"Hello"<<endl;
  return 0;
}
```

3. 当代码数量较多或有的部分与之前列出的重复，不适合完全列在书中时，我们会**仅给出重要片段**，并以“片段”二字注明。因此，再说一遍，我们强烈建议读者到 GitHub 上下载所有源代码，完成练习，以更好地掌握本书知识。
4. 由于排版原因，书中展示的代码可能与 GitHub 中的代码有稍许不同，请以 GitHub 上的代码为准。
5. 我们用到的每个库，在第一次出现的时候会有比较详细的安装和使用说明，但在后续的使用中则不再赘述。所以，建议读者按章节顺序阅读本书内容。
6. 每一讲的开头会列出本讲的内容提要，而末尾会有小结和练习题。引用的参考文献在全书末尾列出。
7. 以星号开头的章节是选读部分，读者可以根据兴趣阅读。跳过它们不会对理解后续章节产生影响。
8. 文中重要的内容以**黑体**标出，相信你已经习惯了。
9. 我们设计的实验大多数是演示性质的。看懂了它们不代表你已经熟悉整个库的使用。所以我们建议你在课外花一点时间，对本书经常用的几个库进行深入学习。
10. 本书的习题和选读内容可能需要你自己搜索额外材料，所以你需要学会使用搜索引擎。

1.4　致谢和声明

在本书漫长的写作过程中，我得到了许多人的帮助，包括但不限于：

- 中科院的贺一家博士为第 5 讲的相机模型部分提供了材料。
- 颜沁睿提供了第 7 讲的公式推导材料。
- 华中科大的刘毅博士为本书第 6 讲和第 10 讲提供了材料。
- 众多的老师、同学为本书提供了修改意见：肖锡臻、谢晓佳、张明明、耿欣、李帅杰、刘富强、袁梦、孙志明、陈昊升、王京、朱晏辰、丁文东、范帝楷、衡昱帆、高扬、李少朋、吴博、闫雪娇、张腾、郑帆、卢美奇、杨楠等等。在此向他们表示感谢。

此外，感谢我的导师张涛教授一直以来对我的支持和帮助。感谢电子工业出版社郑柳洁编辑的支持。没有他们的帮助，本书不可能以现在的面貌来到读者面前。本书的成书与出版是所有人共同努力的结晶，尽管我没法把他们都列在作者列表中，但是它的出版离不开他们的工作。[①]

本书写作过程中参考了大量文献和论文。其中大部分数学理论知识是前人研究的成果，并非我的原创。一小部分实验设计亦来自各开源代码的演示程序，不过大部分是我自己编写的。此外，也有一些图片摘自公开发表的期刊或会议论文，文中均已注明。未做说明的图像，或为原创，或来自网络，恕不一一列举。如有问题，请与我们联系，我们会在第一时间加以修正。

本书涉及知识点众多，错漏在所难免。如有疑问，欢迎通过电子邮件与我联系。

[①] 文章中的“我”主要是指本人高翔，前面说“我”不正经不包括上述其他作者。他们都是敬业乐群的好同志。

我的邮箱是：gao.xiang.thu@gmail.com。

感谢我的爱人刘丽莲女士长期的理解和支持。这本书是献给她的。

习题（基本知识自测题）

1. 有线性方程 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$，若已知 $\boldsymbol{A},\boldsymbol{b}$，需要求解 $\boldsymbol{x}$，该如何求解？这对 $\boldsymbol{A}$ 和 $\boldsymbol{b}$ 有哪些要求？提示：从 $\boldsymbol{A}$ 的维度和秩角度来分析。
2. 高斯分布是什么？它的一维形式是什么样子？它的高维形式是什么样子？
3. 你知道 C++ 中的**类**吗？你知道 STL 吗？你使用过它们吗？
4. 你以前怎样书写 C++ 程序？（你完全可以说只在 Visual C++ 6.0 下写过 C++ 工程，只要你有写 C++ 和 C 语言的经验就行。）
5. 你知道 C++11 标准吗？其中哪些新特性你听说过或用过？有没有其他的标准？
6. 你知道 Linux 吗？你有没有至少使用过一种（不算安卓），比如 Ubuntu？
7. Linux 的目录结构是什么样的？你知道哪些基本命令，比如 ls, cat 等？
8. 如何在 Ubuntu 中安装软件（不打开软件中心的情况下）？这些软件被安装在什么地方？如果只知道模糊的软件名称（比如想要装一个名称中含有 eigen 的库），应该如何安装它？
9.* 花一个小时学习一下 Vim，因为你迟早会用它。你可以在终端中输入 vimtutor 阅读一遍所有内容。我们不需要你非常熟练地操作它，只要能够在学习本书的过程中使用它输入代码即可。**不要在它的插件上浪费时间，不要想着把 Vim 用成 IDE，我们只用它做文本编辑的工作。**

第一部分

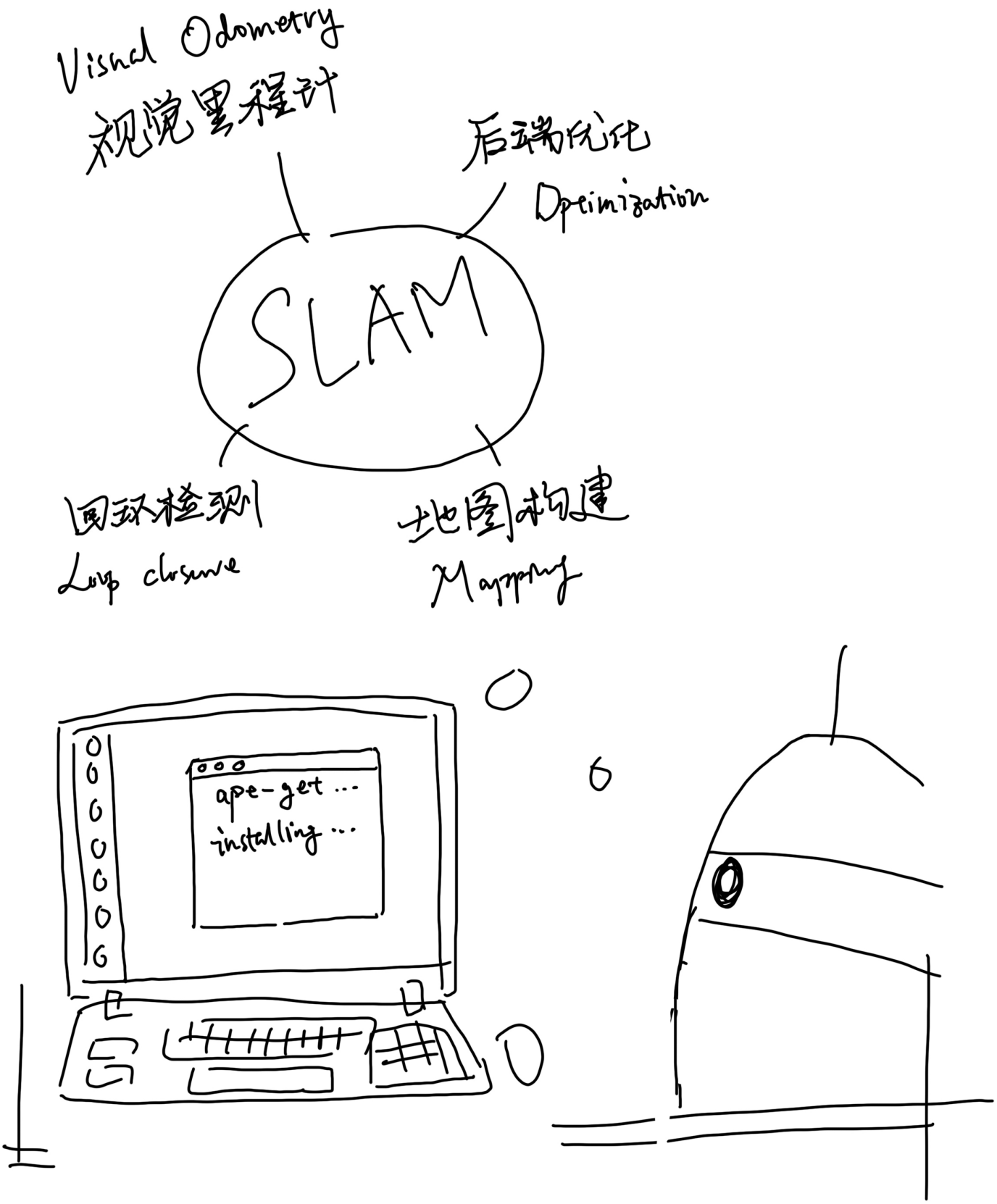
数学基础

第二章　初识 SLAM

主要目标

1. 理解一个视觉 SLAM 框架由哪几个模块组成，各模块的任务是什么。
2. 搭建编程环境，为开发和实验做准备。
3. 理解如何在 Linux 下编译并运行一个程序，如果程序出了问题，又该如何对它进行调试。
4. 掌握 cmake 的基本使用方法。

本讲概括地介绍一个视觉 SLAM 系统的结构，作为后续内容的大纲。实践部分介绍环境搭建、程序基本知识，最后完成一个“Hello SLAM”程序。


Visual Odometry
视觉里程计
后端优化
Optimization
SLAM
回环检测
Loop closure
地图构建
Mapping
ape-get ...
installing ...

2.1　引子：小萝卜的例子

假设我们组装了一台叫作“小萝卜”的机器人，大概的样子如图 2-1所示。

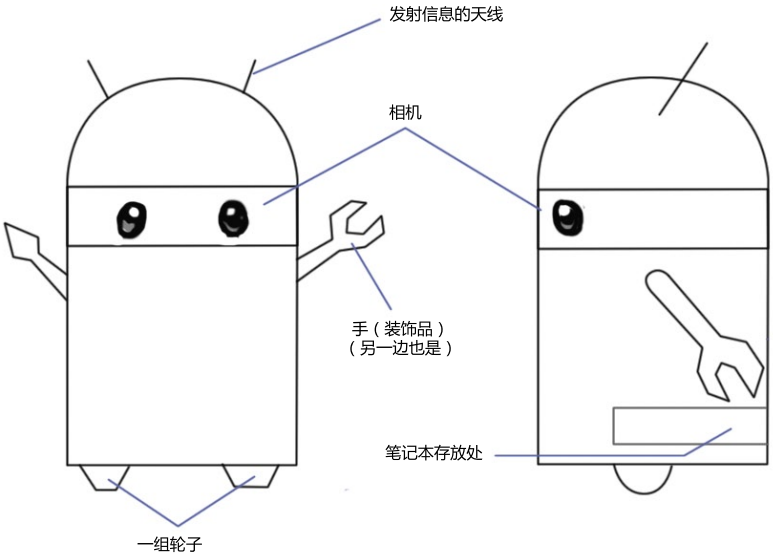


图 2-1　小萝卜设计图。左边：正视图；右边：侧视图。设备有相机、轮子、笔记本，手是装饰品。

先不要管它这个简笔画的风格。另外，虽然有点像“安卓”，但它并不是靠安卓系统来计算的。我们把一台笔记本塞进了它的后备箱内（方便我们随时拿出来调试程序）。它能做点什么呢？

我们希望小萝卜具有**自主运动能力**，这是非常基本的功能。虽然世界上也有放在桌面像摆件一样的机器人，能够和人说话或播放音乐（听说还卖得不错），不过一台平板电脑完全可以胜任这些事情。作为机器人，我们希望小萝卜能够在房间里自由移动。不管我们在哪里招呼一声，它都会滴溜溜地走过来。

你会发现自主运动能力是许多高级功能的前提。不管是扫地也好搬东西也好，首先要让它动起来。要移动就得有轮子和电机，所以我们在小萝卜的下方安装了轮子（足式机器人步态很复杂，留给自动化所的博士们发论文去吧）。有了轮子，机器人就能够四处行动了，但不加规划和控制的话，小萝卜不知道行动的目标，就只能四处乱走，更糟糕的情况下会撞上墙造成损毁。而要规划和控制，首先需要**感知**周边的环境。为此，我们在它的脑袋上安装了一个相机。安装相机的主要动机，是考虑到这样一个机器人**和人类非常相似**——从画面上一眼就能看出。有眼睛、大脑和四肢的人类，能够在任意环境里轻松自在地行走、探索，我们（天真地）觉得机器人也能够完成这件事。为了使小萝卜能够探索一个房间，它至少需要知道两件事：

1. 我在什么地方？——定位。
2. 周围环境是什么样？——建图。

“定位”和“建图”，可以看成感知的“内外之分”。作为一个“内外兼修”的小萝卜，一方面要明白自身的**状态**（即位置），另一方面也要了解外在的**环境**（即地图）。当然，解决这两个问题的方法非常多。比方说，我们可以在房间地板上铺设导引线，在墙壁上贴识别二维码，在桌子上放置无线电定位设备（这其实是现在很多仓储物流机器人的做法）。如果在室外，还可以在小萝卜脑袋上安装 GPS 信号接收器（像手机或汽车一样）。有了这些东西之后，定位问题是否已经解决了呢？我们不妨把这些传感器（见图 2-2）分为两类。

一类传感器是**携带于机器人本体上**的，例如机器人的轮式编码器、相机、激光传感器，等等。另一类是**安装于环境中**的，例如前面讲的导轨、二维码标志，等等。安装于环境中的传感设备，通常能够直接测量到机器人的位置信息，简单有效地解决定位问题。然而，由于它们必须在环境中设置，在一定程度上限制了机器人的使用范围。比方说，有些地方没有 GPS 信号，有些地方无法铺设导轨，或者 GPS 信号精度不够，这时该怎么做定位呢？

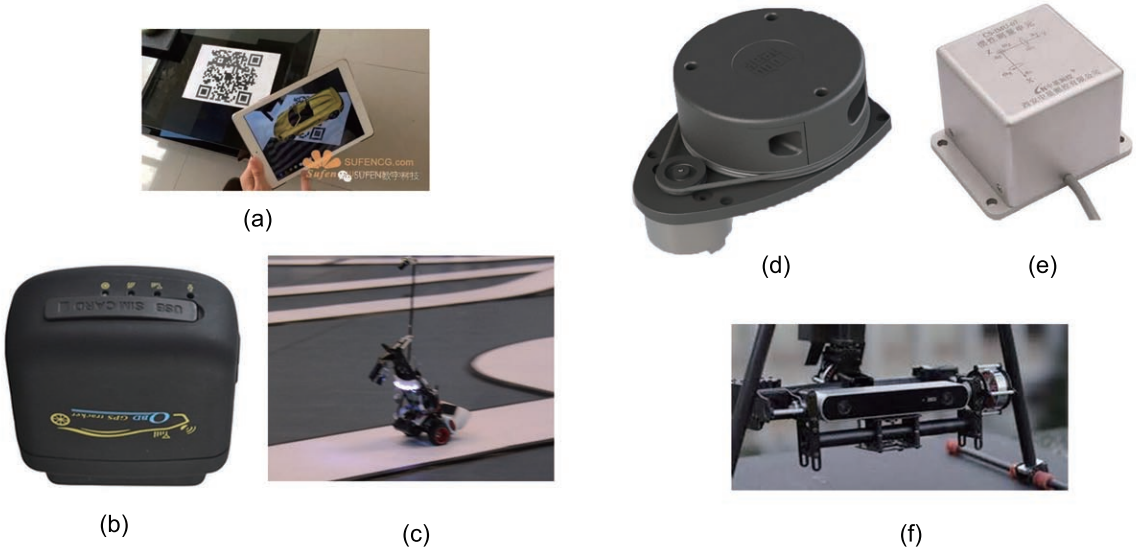


图 2-2 一些传感器。（a）利用二维码进行定位的增强现实软件；（b）GPS 定位装置；（c）铺设导轨的小车；（d）激光雷达；（e）IMU 单元；（f）双目相机。

我们看到，这类传感器**约束**了外部环境。只有在这些约束满足时，基于它们的定位方案才能工作。反之，当约束无法满足时，我们就没法进行定位了。所以说，虽然这类传感器简单可靠，但它们无法提供一个普遍的、通用的解决方案。相对地，那些携带于机器人本体上的传感器，比如激光传感器、相机、轮式编码器、惯性测量单元（Inertial Measurement Unit，IMU）等，它们测到的通常都是一些间接的物理量而不是直接的位置数据。例如，轮式编码器会测到轮子转动的角度，IMU 测量运动的角速度和加速度，相机和激光传感器则读取外部环境的某种观测数据。我们只能通过一些间接的手段，从这些数据推算自己的位置。虽然这听上去是一种迂回战术，但更明显的好处是，它

没有对环境提出任何要求[①]，从而使得这种定位方案可适用于未知环境。

回顾前面讨论过的 SLAM 定义，我们在 SLAM 中非常强调未知环境。所以在理论上，我们不限制小萝卜的使用环境[②]，这意味着我们没法假设像 GPS 或导轨这样的外部传感器都能顺利工作。因此，使用携带式的传感器来完成 SLAM 是我们重点关心的问题。特别地，当谈论视觉 SLAM 时，我们主要是指如何用**相机**解决定位和建图问题。同样，如果传感器主要是激光，那就称为激光 SLAM。激光 SLAM 相对成熟，比如 2005 年的《概率机器人》[5] 就介绍了许多激光 SLAM 的知识。

视觉 SLAM 是本书的主题，所以我们尤其关心小萝卜的眼睛能够做些什么事。SLAM 中使用的相机与我们平时见到的单反摄像头并不是同一个东西。它更加简单，通常不携带昂贵的镜头，而是以一定速率拍摄周围的环境，形成一个连续的视频流。普通的摄像头能以每秒钟 30 张图片的速度采集图像，高速相机则更快一些。按照工作方式的不同，相机可以分为单目相机（Monocular）、双目相机（Stereo）和深度相机（RGB-D）三大类，如图 2-3所示。直观看来，单目相机只有一个摄像头，双目有两个，而 RGB-D 原理较复杂，除了能够采集到彩色图片之外，还能读出每个像素与相机之间的距离。深度相机通常携带多个摄像头，工作原理和普通相机不尽相同，在第 5 讲会详细介绍其工作原理，此处读者只需有一个直观概念即可。此外，SLAM 中还有全景相机[7]、Event 相机[8] 等特殊或新兴的种类。虽然偶尔能看到它们在 SLAM 中的应用，不过到目前为止还没有成为主流。从样子上看，小萝卜使用的似乎是水平的双目相机。

图 2-3　形形色色的相机：单目、双目和深度相机。

我们来分别看一看各种相机用来做 SLAM 时有什么特点。

[①] 话虽如此，实际我们至少要求传感器在该环境下能够工作，所以不要举一些“轮式编码器在宇宙空间中没法工作，相机在黑屋子里看不见东西”的极端例子。

[②] 不过现实中我们都会有一个大概的范围，例如室内和室外的区分。

单目相机

只使用一个摄像头进行 SLAM 的做法称为单目 SLAM（Monocular SLAM）。这种传感器结构特别简单，成本特别低，所以单目 SLAM 非常受研究者关注。你肯定见过单目相机的数据：照片。是的，作为一张照片，它有什么特点呢？

照片本质上是拍摄某个场景（Scene）在相机的成像平面上留下的一个**投影**。**它以二维的形式记录了三维的世界**。显然，这个过程丢掉了场景的一个维度，也就是所谓的深度（或距离）。在单目相机中，我们无法通过单张图片来计算场景中物体与相机之间的**距离**（远近）。之后我们会看到，这个距离将是 SLAM 中非常关键的信息。由于我们人类见过大量的图像，形成了一种天生的直觉，对大部分场景都有一个直观的**距离感（空间感）**，它可以帮助我们判断图像中物体的远近关系。比如说，我们能够辨认出图像中的物体，并且知道其大致的大小；比如，近处的物体会挡住远处的物体，而太阳、月亮等天体一般在很远的地方；再如，物体受光照后会留下影子，等等。这些信息都可以帮助我们判断物体的远近，但也存在一些情况会使这种距离感失效，这时我们就无法判断物体的远近及其真实大小了。图 2-4 就是这样一个例子。在这张图像中，我们无法仅通过它来判断后面那些小人是真实的人，还是小型模型。除非我们转换视角，观察场景的三维结构。换言之，在单张图像里，你无法确定一个物体的真实大小。它可能是一个**很大但很远**的物体，也可能是一个**很近但很小**的物体。由于近大远小的透视关系，它们可能在图像中变成同样大小的样子。

图 2-4　单目视觉中的尴尬：不知道深度时，手掌上的人是真人还是模型？

由于单目相机拍摄的图像只是三维空间的二维投影，所以，如果真想恢复三维结构，必须改变相机的视角。在单目 SLAM 中也是同样的原理。我们必须移动相机，才能估计它的**运动**（Motion），同时估计场景中物体的远近和大小，不妨称之为**结构**（Structure）。那么，怎么估计这些运动和结构呢？想象你坐在一辆运动的列车中。如果列车往右移动，那么我们看到的东西就会往左边移动——

这就给我们推测运动带来了信息。另一方面，我们还知道：**近处的物体移动快，远处的物体则运动缓慢，极远处（无穷远处）的物体（如太阳、月亮）看上去是不动的**。于是，当相机移动时，这些物体在图像上的运动就形成了**视差**（disparity）。通过视差，我们就能定量地判断哪些物体离得远，哪些物体离得近。

然而，即使我们知道了物体远近，它们仍然只是一个相对的值。比如我们在看电影时，虽然能够知道电影场景中哪些物体比另一些大，但无法确定电影里那些物体的“真实尺度”：那些大楼是真实的高楼大厦，还是放在桌上的模型？而摧毁大厦的是真实怪兽，还是穿着特摄服装的演员？直观地说，如果把相机的运动和场景大小同时放大两倍，单目相机所看到的像是一样的。同样地，把这个大小乘以任意倍数，我们都将看到一样的景象。这说明，单目 SLAM 估计的轨迹和地图将与真实的轨迹和地图相差一个因子，也就是所谓的**尺度**（Scale）[①]。由于单目 SLAM 无法仅凭图像确定这个真实尺度，所以又称为**尺度不确定性**（Scale Ambiguity）。

平移之后才能计算深度，以及无法确定真实尺度，这两件事情给单目 SLAM 的应用造成了很大的麻烦。其根本原因是通过单张图像无法确定深度。所以，为了得到这个深度，人们开始使用双目和深度相机。

双目相机和深度相机

使用双目相机和深度相机的目的，在于通过某种手段测量物体与我们之间的距离，克服单目相机无法知道距离的缺点。一旦知道了距离，场景的三维结构就可以通过单个图像恢复出来，同时消除尺度不确定性。尽管都是为了测量距离，但双目相机与深度相机测量深度的原理是不一样的。双目相机由两个单目相机组成，但这两个相机之间的距离〔称为**基线**（Baseline）〕是已知的。我们通过这个基线来估计每个像素的空间位置——这和人眼非常相似。我们人类可以通过左右眼图像的差异判断物体的远近，在计算机上也是同样的道理（见图 2-5）。如果对双目相机进行拓展，也可以搭建多目相机，不过本质上并没有什么不同。

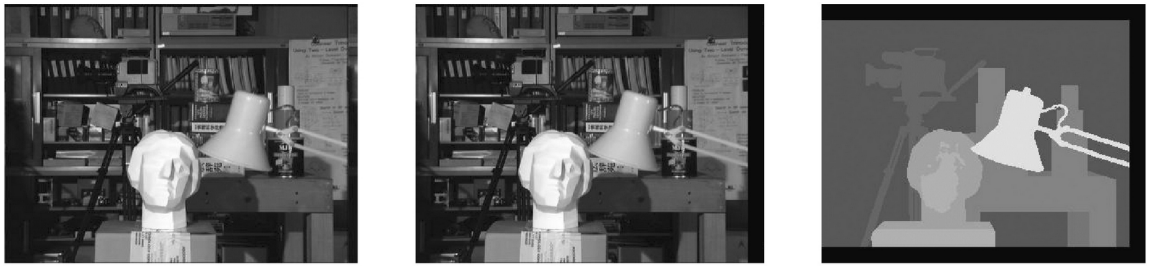


图 2-5　双目相机的数据：左眼图像，右眼图像。通过左右眼的差异，能够判断场景中物体与相机之间的距离。

[①] 数学上的原因将会在视觉里程计一讲中解释。

计算机上的双目相机需要大量的计算才能（不太可靠地）估计每一个像素点的深度，相比于人类真是非常笨拙[①]。双目相机测量到的深度范围与基线相关。基线距离越大，能够测量到的就越远，所以无人车上搭载的双目通常会是个很大的家伙。双目相机的距离估计是比较左右眼的图像获得的，并不依赖其他传感设备，所以它既可以应用在室内，亦可应用于室外。双目或多目相机的缺点是配置与标定均较为复杂，其深度量程和精度受双目的基线与分辨率所限，而且视差的计算非常消耗计算资源，需要使用 GPU 和 FPGA 设备加速后，才能实时输出整张图像的距离信息。因此在现有的条件下，计算量是双目的主要问题之一。

深度相机（又称 RGB-D 相机，在本书中主要使用 RGB-D 这个名称）是 2010 年左右开始兴起的一种相机，它最大的特点是可以通过红外结构光或 Time-of-Flight（ToF）原理，像激光传感器那样，通过主动向物体发射光并接收返回的光，测出物体与相机之间的距离。这部分并不像双目相机那样通过软件计算来解决，而是通过物理的测量手段，所以相比于双目相机可节省大量的计算（见图 2-6）。目前常用的 RGB-D 相机包括 Kinect/Kinect V2、Xtion Pro Live、RealSense 等，在一些手机上人们也用它来识别人脸。不过，现在多数 RGB-D 相机还存在测量范围窄、噪声大、视野小、易受日光干扰、无法测量透射材质等诸多问题，在 SLAM 方面，主要用于室内，室外则较难应用。

图 2-6　RGB-D 数据：深度相机可以直接测量物体的图像和距离，从而恢复三维结构。

我们讨论了几种常见的相机，相信通过以上的说明，你已经对它们有了直观的了解。现在，想象相机在场景中运动的过程，我们将得到一系列连续变化的图像[②]。视觉 SLAM 的目标，是通过这样的一些图像，进行定位和地图构建。这件事情并没有想象的那么简单。它不是某种算法，只要我

[①] 我三个月大的女儿已经能够辨认并抓取放在她面前的玩具了，我觉得她比许多大学实验室里的机器人都要智能。

[②] 你可以用手机录个小视频试试。

们输入数据，就可以往外不断地输出定位和地图信息了。SLAM 需要一个完善的算法框架，而经过研究者们长期的努力工作，现有这个框架已经定型了。

2.2　经典视觉 SLAM 框架

下面来看经典的视觉 SLAM 框架，如图 2-7所示，了解一下视觉 SLAM 究竟由哪几个模块组成。

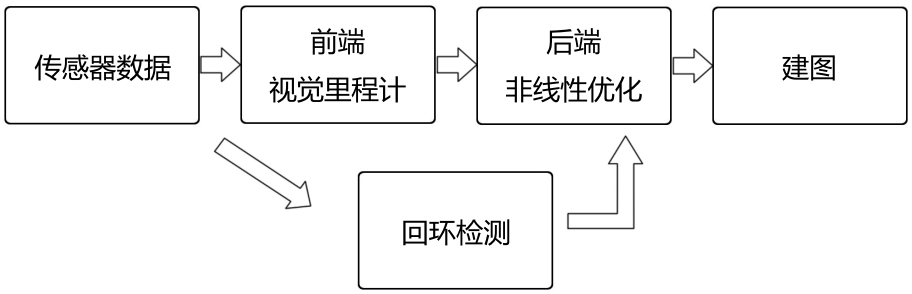


图 2-7　整体视觉 SLAM 流程图。

整个视觉 SLAM 流程包括以下步骤。

1. 传感器信息读取。在视觉 SLAM 中主要为相机图像信息的读取和预处理。如果是在机器人中，还可能有码盘、惯性传感器等信息的读取和同步。
2. **视觉里程计**（Visual Odometry，VO）。视觉里程计的任务是估算相邻图像间相机的运动，以及局部地图的样子。VO 又称为前端（Front End）。
3. **后端优化**（Optimization）。后端接受不同时刻视觉里程计测量的相机位姿，以及回环检测的信息，对它们进行优化，得到全局一致的轨迹和地图。由于接在 VO 之后，又称为后端（Back End）。
4. **回环检测**（Loop Closing）。回环检测判断机器人是否到达过先前的位置。如果检测到回环，它会把信息提供给后端进行处理。
5. **建图**（Mapping）。它根据估计的轨迹，建立与任务要求对应的地图。

经典的视觉 SLAM 框架是过去十几年的研究成果。这个框架本身及其所包含的算法已经基本定型，并且已经在许多视觉程序库和机器人程序库中提供。依靠这些算法，我们能够构建一个视觉 SLAM 系统，使之在正常的工作环境里实时定位与建图。因此，我们说，**如果把工作环境限定在静态、刚体，光照变化不明显、没有人为干扰的场景**，那么，这个 SLAM 系统是相当成熟的了[9]。

读者可能还没有理解上面几个模块的概念，下面就来详细介绍各个模块具体的任务。但是，准确理解其工作原理需要一些数学知识，我们将放到本书的第二部分进行。这里读者只需对各模块有一个直观的、定性的理解即可。

2.2.1 视觉里程计

视觉里程计关心**相邻图像**之间的相机运动，最简单的情况当然是两张图像之间的运动关系。例如，当看到图 2-8时，我们会自然地反应出右图应该是左图向左旋转一定角度的结果（在视频情况下感觉会更加自然）。我们不妨思考一下：自己是怎么知道“向左旋转”这件事情的呢？人类早已习惯于用眼睛探索世界，估计自己的位置，但又往往难以用理性的语言描述我们的直觉[①]。看到图 2-8时，我们会自然地认为，这个场景中离我们近的是吧台，远处是墙壁和黑板。当相机向左转动时，吧台离我们近的部分出现在视野中，而右侧远处的柜子则移出了视野。通过这些信息，我们判断相机应该是向左旋转了。

图 2-8 相机拍摄到的图片与人眼反应的运动方向。

但是，如果进一步问：能否确定旋转了多少度，平移了多少厘米？我们就很难给出一个确切的答案了。因为我们的直觉对这些具体的数字并不敏感。但是，在计算机中，又必须精确地测量这段运动信息。所以我们要问：**计算机是如何通过图像确定相机的运动的呢？**

前面也提过，在计算机视觉领域，人类在直觉上看来十分自然的事情，在计算机视觉中却非常困难。图像在计算机里只是一个数值矩阵。这个矩阵里表达着什么东西，计算机毫无概念（这也正是现在机器学习要解决的问题）。而在视觉 SLAM 中，我们只能看到一个个像素，知道它们是某些空间点在相机的成像平面上投影的结果。所以，为了定量地估计相机运动，必须先**了解相机与空间点的几何关系**。

要讲清这个几何关系以及 VO 的实现方法，需要铺垫一些背景知识。在这里我们先让读者对 VO 有个直观的概念。现在只需知道，VO 能够通过相邻帧间的图像估计相机运动，并恢复场景的空间结构。称它为“里程计”是因为它和实际的里程计一样，只计算相邻时刻的运动，而和再往前的过去的信息没有关联。在这一点上，VO 就像一种只有短时间记忆的物种（不过可以不限于两帧，数量可以更多一些，例如 5-10 帧）。

现在，假定我们已有了一个视觉里程计，估计了两张图像间的相机运动。那么，只要把相邻时刻的运动“串”起来，就构成了机器人的运动轨迹，从而解决了定位问题。另一方面，我们根据每

[①] 在很多涉及计算机视觉和机器学习的任务中，情况都是这样。拜近几年深度学习所赐，我们连机器是怎么计算的都已经看不懂了。

个时刻的相机位置，计算出各像素对应的空间点的位置，就得到了地图。这么说来，有了 VO，是不是就解决了 SLAM 问题呢？

视觉里程计确实是 SLAM 的关键，我们也会花大量的篇幅来介绍它。然而，仅通过视觉里程计来估计轨迹，将不可避免地出现**累积漂移**（Accumulating Drift）。这是由于视觉里程计（在最简单的情况下）只估计两个图像间的运动造成的。我们知道，每次估计都带有一定的误差，而由于里程计的工作方式，先前时刻的误差将会传递到下一时刻，导致经过一段时间之后，估计的轨迹将不再准确（见图 2-9）。比方说，机器人先向左转 90°，再向右转 90°。由于误差，我们把第一个 90° 估计成了 89°。那我们就会尴尬地发现，向右转之后机器人的估计位置并没有回到原点。更糟糕的是，即使之后的估计完全准确，与真实值相比，都会带上这 −1° 的误差。

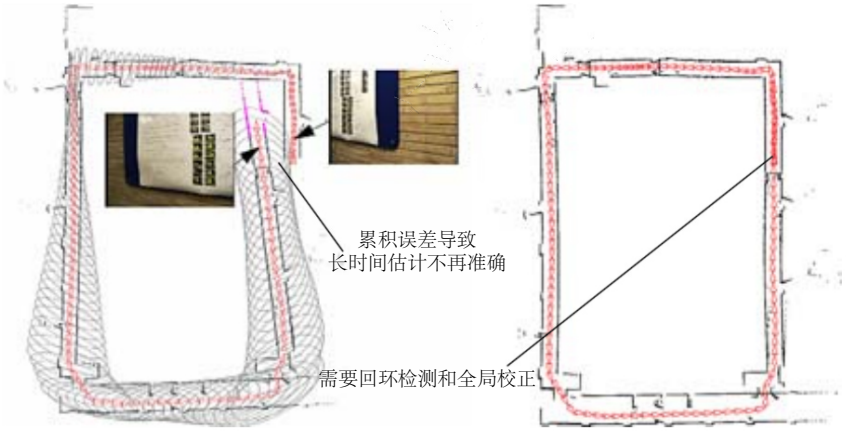


图 2-9　累积误差与回环检测的校正结果[10]。

这也就是所谓的**漂移**（Drift）。它将导致我们无法建立一致的地图。你会发现原本直的走廊变成了斜的，而原本 90° 的直角变成了歪的——这实在是一件令人难以忍受的事情！为了解决漂移问题，我们还需要两种技术：**后端优化**[①]和**回环检测**。回环检测负责把“机器人回到原始位置”的事情检测出来，而后端优化则根据该信息，校正整个轨迹的形状。

2.2.2　后端优化

笼统地说，后端优化主要指处理 SLAM 过程中**噪声**的问题。虽然我们很希望所有的数据都是准确的，然而现实中，再精确的传感器也带有一定的噪声。便宜的传感器测量误差较大，昂贵的可能会小一些，有的传感器还会受磁场、温度的影响。所以，除了解决“如何从图像估计出相机运动”之外，我们还要关心这个估计带有多大的噪声，这些噪声是如何从上一时刻传递到下一时刻的，而我们又对当前的估计有多大的自信。后端优化要考虑的问题，就是如何从这些带有噪声的数据中估计

[①] 更多时候称为后端（Back End）。由于主要使用的是优化方法，故称为后端优化。

整个系统的状态，以及这个状态估计的不确定性有多大——这称为最大后验概率估计（Maximum-a-Posteriori，MAP）。这里的状态既包括机器人自身的轨迹，也包含地图。

相对地，视觉里程计部分有时被称为“前端”。在 SLAM 框架中，前端给后端提供待优化的数据，以及这些数据的初始值。而后端负责整体的优化过程，它往往面对的只有数据，不必关心这些数据到底来自什么传感器。**在视觉 SLAM 中，前端和计算机视觉研究领域更为相关，比如图像的特征提取与匹配等，后端则主要是滤波与非线性优化算法。**

从历史意义上来说，现在我们称为后端优化的部分，很长一段时间直接被称为“SLAM 研究”。早期的 SLAM 问题是一个**状态估计**问题——正是后端优化要解决的东西。在最早提出 SLAM 的一系列论文中，当时的人们称它为“空间状态不确定性的估计”（Spatial Uncertainty）[4, 11]。虽然有一些晦涩，但也确实反映出了 SLAM 问题的本质：**对运动主体自身和周围环境空间不确定性的估计**。为了解决 SLAM 问题，我们需要状态估计理论，把定位和建图的不确定性表达出来，然后采用滤波器或非线性优化，估计状态的均值和不确定性（方差）。状态估计与非线性优化的具体内容将在第 6 讲、第 9 讲和第 10 讲介绍。让我们暂时跳过它的原理说明，继续往下介绍。

2.2.3 回环检测

回环检测，又称闭环检测（Loop Closure Detection/Loop Closingn），主要解决位置估计**随时间漂移**的问题。怎么解决呢？假设实际情况下机器人经过一段时间的运动后回到了原点，但是由于漂移，它的位置估计值却没有回到原点。怎么办呢？我们想，如果有某种手段，让机器人知道“回到了原点”这件事，或者把“原点”识别出来，我们再把位置估计值“拉”过去，就可以消除漂移了。这就是所谓的回环检测。

回环检测与“定位”和“建图”二者都有密切的关系。事实上，我们认为，地图存在的主要意义是让机器人知晓自己到过的地方。为了实现回环检测，我们需要让机器人具有**识别到过的场景**的能力。它的实现手段有很多。例如像前面说的那样，我们可以在机器人下方设置一个标志物（如一张二维码图片）。它只要看到了这个标志，就知道自己回到了原点。但是，该标志物实质上是一种环境中的传感器，对应用环境做了限制（万一不能贴二维码怎么办？）。我们更希望机器人能使用携带的传感器——也就是图像本身，来完成这一任务。例如，可以判断**图像间的相似性**来完成回环检测。这一点和人是相似的。当我们看到两张相似的图片时，容易辨认它们来自同一个地方。如果回环检测成功，可以显著地减小累积误差。所以，视觉回环检测实质上是一种计算图像数据相似性的算法。由于图像的信息非常丰富，使得正确检测回环的难度降低了不少。

在检测到回环之后，我们会把“A 与 B 是同一个点”这样的信息告诉后端优化算法。然后，后端根据这些新的信息，把轨迹和地图调整到符合回环检测结果的样子。这样，如果我们有充分而且正确的回环检测，就可以消除累积误差，得到全局一致的轨迹和地图。

2.2.4　建图

建图（Mapping）是指构建地图的过程。地图（见图 2-10）是对环境的描述，但这个描述并不是固定的，需要视 SLAM 的应用而定。

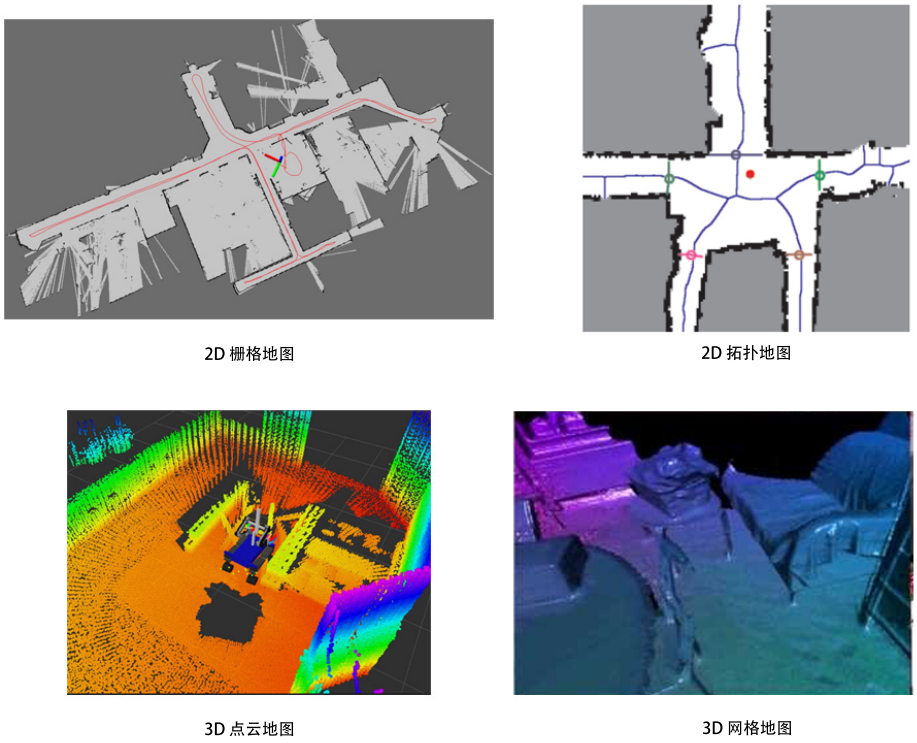


图 2-10　形形色色的地图[12]。

对于家用扫地机器人来说，这种主要在低矮平面里运动的机器人，只需要一个二维的地图，标记哪里可以通过，哪里存在障碍物，就够它在一定范围内导航了。而对于一个相机，它有 6 自由度的运动，我们至少需要一张三维的地图。有些时候，我们想要一个漂亮的重建结果，不仅是一组空间点，还需要带纹理的三角面片。另一些时候，我们又不关心地图的样子，只需要知道“A 点到 B 点可通过，而 B 点到 C 点不行”这样的事情。甚至，有时不需要地图，或者地图可以由其他人提供，例如，行驶的车辆往往可以得到已绘制好的当地地图。

对于地图，我们有太多的想法和需求。因此，相比于前面提到的视觉里程计、回环检测和后端优化，建图并没有一个固定的形式和算法。一组空间点的集合也可以称为地图，一个漂亮的 3D 模型亦是地图，一个标记着城市、村庄、铁路、河道的图片还是地图。地图的形式随 SLAM 的应用场合而定。大体上讲，可以分为**度量地图**与**拓扑地图**两种。

度量地图（Metric Map）

度量地图强调精确地表示地图中物体的位置关系，通常用稀疏（Sparse）与稠密（Dense）对其分类。稀疏地图进行了一定程度的抽象，并不需要表达所有的物体。例如，我们选择一部分具有代表意义的东西，称之为路标（Landmark），那么一张稀疏地图就是由路标组成的地图，而不是路标的部分就可以忽略掉。相对地，稠密地图着重于建模所有看到的东西。对于定位来说，稀疏路标地图就足够了。而用于导航时，则往往需要稠密的地图（否则撞上两个路标之间的墙怎么办？）。稠密地图通常按照某种分辨率，由许多个小块组成。对于二维度量地图是许多个小格子（Grid），而对于三维度量地图则是许多小方块（Voxel）。一般地，一个小块含有占据、空闲、未知三种状态，以表达该格内是否有物体。当查询某个空间位置时，地图能够给出该位置是否可以通过的信息。这样的地图可以用于各种导航算法，如 A*、D*[①]等，为机器人研究者所重视。但是我们也看到，这种地图需要存储每一个格点的状态，会耗费大量的存储空间，而且多数情况下地图的许多细节部分是无用的。另一方面，大规模度量地图有时会出现一致性问题。很小的一点转向误差，可能会导致两间屋子的墙出现重叠，使地图失效。

拓扑地图（Topological Map）

相比于度量地图的精确性，拓扑地图则更强调地图元素之间的关系。拓扑地图是一个图（Graph），由节点和边组成，只考虑节点间的连通性，例如 A、B 点是连通的，而不考虑如何从 A 点到达 B 点。它放松了地图对精确位置的需要，去掉了地图的细节问题，是一种更为紧凑的表达方式。然而，拓扑地图不擅长表达具有复杂结构的地图。如何对地图进行分割形成结点与边，又如何使用拓扑地图进行导航与路径规划，仍是有待研究的问题。

2.3 SLAM 问题的数学表述

通过前面的介绍，读者应该对 SLAM 中各个模块的组成和主要功能有了直观的了解。但仅仅靠直观印象并不能帮助我们写出可以运行的程序。我们要把它上升到理性层次，也就是用数学语言来描述 SLAM 过程。我们会用到一些变量和公式，但请读者放心，我会尽量让它保持足够地清楚。

假设小萝卜正携带着某种传感器在未知环境里运动，怎么用数学语言描述这件事呢？首先，由于相机通常是在某些时刻采集数据的，所以我们也只关心这些时刻的位置和地图。这就把一段连续时间的运动变成了离散时刻 $t = 1, \cdots, K$ 当中发生的事情。在这些时刻，用 $\boldsymbol{x}$ 表示小萝卜自身的位置。于是各时刻的位置就记为 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_K$，它们构成了小萝卜的轨迹。地图方面，我们假设地图是由许多个**路标**（Landmark）组成的，而每个时刻，传感器会测量到一部分路标点，得到它们的观测数据。不妨设路标点一共有 N 个，用 $\boldsymbol{y}_1, \cdots, \boldsymbol{y}_N$ 表示它们。

在这样的设定中，“小萝卜携带着传感器在环境中运动”，由如下两件事情描述：

[①] https://en.wikipedia.org/wiki/A*_search_algorithm

1. 什么是**运动**？我们要考察从 $k-1$ 时刻到 k 时刻，小萝卜的位置 $\boldsymbol{x}$ 是如何变化的。
2. 什么是**观测**？假设小萝卜在 k 时刻于 $\boldsymbol{x}_k$ 处探测到了某一个路标 $\boldsymbol{y}_j$，我们要考察如何用数学语言来描述这件事情。

先来看运动。通常，机器人会携带一个测量自身运动的传感器，比如说码盘或惯性传感器。这个传感器可以测量有关运动的读数，但不一定直接就是位置之差，还可能是加速度、角速度等信息。有时候我们也给小萝卜发送指令，例如“前进一米”、“左转 90 度”，或者“油门踩到底”、“刹车”等。无论是何种情况，我们都能使用一个通用的、抽象的数学模型来说明此事：

$$\boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k\right). \tag{2.1}$$

这里，$\boldsymbol{u}_k$ 是运动传感器的读数或者输入[①]，$\boldsymbol{w}_k$ 为该过程中加入的噪声。注意到，我们用一个一般函数 f 来描述这个过程，而不指明 f 具体的作用方式。这使得整个函数可以指代任意的运动传感器/输入，成为一个通用的方程，而不必限定于某个特殊的传感器上。我们把它称为**运动方程**。

噪声的存在使得这个模型变成了随机模型。换句话说，即使我们下达“前进一米”的命令，并不代表小萝卜真的前进了一米。如果所有指令都是准确的，也就没必要**估计**什么东西了。事实上，小萝卜可能某次只前进了 0.9 米，另一次前进了 1.1 米，再一次可能由于轮胎打滑，干脆没有前进。于是，每次运动过程中的噪声是随机的。如果我们不理会这个噪声，那么只根据指令来确定的位置可能与实际位置相差十万八千里了。

与运动方程相对应，还有一个**观测方程**。观测方程描述的是，当小萝卜在 $\boldsymbol{x}_k$ 位置上看到某个路标点 $\boldsymbol{y}_j$，产生了一个观测数据 $\boldsymbol{z}_{k,j}$。同样，用一个抽象的函数 h 来描述这个关系：

$$\boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}\right). \tag{2.2}$$

这里，$\boldsymbol{v}_{k,j}$ 是这次观测里的噪声。由于观测所用的传感器形式更多，这里的观测数据 $\boldsymbol{z}$ 以及观测方程 h 也有许多不同的形式。

读者或许会说，我们用的函数 f, h，似乎并没有具体地说明运动和观测是怎么回事？同时，这里的 $\boldsymbol{x}$,$\boldsymbol{y}$,$\boldsymbol{z}$ 又是什么东西呢？事实上，根据小萝卜的真实运动和传感器的种类，存在着若干种**参数化**方式（Parameterization）。什么叫参数化呢？举例来说，假设小萝卜在平面中运动，那么，它的位姿[②]由两个位置和一个转角来描述，即 $\boldsymbol{x}_k = [x_1, x_2, \theta]_k^{\mathrm{T}}$，其中 x_1, x_2 是两个轴上的位置而 θ 为转角。同时，输入的指令是两个时间间隔位置和转角的变化量 $\boldsymbol{u}_k = [\Delta x_1, \Delta x_2, \Delta\theta]_k^{\mathrm{T}}$，于是，此时运动方程就可以具体化为：

$$\begin{bmatrix} x_1 \\ x_2 \\ \theta \end{bmatrix}_k = \begin{bmatrix} x_1 \\ x_2 \\ \theta \end{bmatrix}_{k-1} + \begin{bmatrix} \Delta x_1 \\ \Delta x_2 \\ \Delta\theta \end{bmatrix}_k + \boldsymbol{w}_k. \tag{2.3}$$

[①] 有些研究者认为运动传感器的输入应该放到观测方程，但是在实践当中这两种做法基本是一样的。

[②] 在本书中，我们以“位姿”这个词表示“位置”加上“姿态”。

这是简单的线性关系。不过，并不是所有的输入指令都是位移和角度的变化量，比如“油门”或者“控制杆”的输入就是速度或加速度量，所以也存在着其他形式更加复杂的运动方程，那时我们可能需要进行动力学分析。关于观测方程，比方说小萝卜携带着一个二维激光传感器。我们知道激光传感器观测一个 2D 路标点时，能够测到两个量：路标点与小萝卜本体之间的距离 r 和夹角 ϕ。记路标点为 $\boldsymbol{y}_j = [y_1, y_2]_j^{\mathrm{T}}$，位姿为 $\boldsymbol{x}_k = [x_1, x_2]_k^{\mathrm{T}}$，观测数据为 $\boldsymbol{z}_{k,j} = [r_{k,j}, \phi_{k,j}]^{\mathrm{T}}$，那么观测方程就写为：

$$\begin{bmatrix} r_{k,j} \\ \phi_{k,j} \end{bmatrix} = \begin{bmatrix} \sqrt{(y_{1,j} - x_{1,k})^2 + (y_{2,j} - x_{2,k})^2} \\ \arctan\left(\frac{y_{2,j} - x_{2,k}}{y_{1,j} - x_{1,k}}\right) \end{bmatrix} + \boldsymbol{v}. \tag{2.4}$$

考虑视觉 SLAM 时，传感器是相机，那么观测方程就是“对路标点拍摄后，得到图像中的像素”的过程。这个过程牵涉到相机模型的描述，将在第 5 讲中详细介绍，这里暂且略过。

可见，针对不同的传感器，这两个方程有不同的参数化形式。如果我们保持通用性，把它们取成通用的抽象形式，那么 SLAM 过程可总结为两个基本方程：

$$\begin{cases} \boldsymbol{x}_k = f(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k, \boldsymbol{w}_k), \quad k = 1, \cdots, K \\ \boldsymbol{z}_{k,j} = h(\boldsymbol{y}_j, \boldsymbol{x}_k, \boldsymbol{v}_{k,j}), \quad (k,j) \in \mathcal{O} \end{cases}. \tag{2.5}$$

其中 $\mathcal{O}$ 是一个集合，记录着在哪个时刻观察到了哪个路标（通常不是每个路标在每个时刻都能看到的——我们在单个时刻很可能只看到一小部分）。这两个方程描述了最基本的 SLAM 问题：当知道运动测量的读数 $\boldsymbol{u}$，以及传感器的读数 $\boldsymbol{z}$ 时，如何求解定位问题（估计 $\boldsymbol{x}$）和建图问题（估计 $\boldsymbol{y}$）？这时，我们就把 SLAM 问题建模成了一个**状态估计问题**：如何通过带有噪声的测量数据，估计内部的、隐藏着的状态变量？

状态估计问题的求解，与两个方程的具体形式，以及噪声服从哪种分布有关。按照运动和观测方程是否为线性，噪声是否服从高斯分布进行分类，分为**线性/非线性**和**高斯/非高斯**系统。其中线性高斯系统（Linear Gaussian，LG 系统）是最简单的，它的无偏的最优估计可以由卡尔曼滤波器（Kalman Filter，KF）给出。而在复杂的非线性非高斯系统（Non-Linear Non-Gaussian，NLNG 系统）中，我们会使用以扩展卡尔曼滤波器（Extended Kalman Filter，EKF）和非线性优化两大类方法去求解。直至 21 世纪早期，以 EKF 为主的滤波器方法在 SLAM 中占据了主导地位。我们会在工作点处把系统线性化，并以预测—更新两大步骤进行求解（见第 10 讲）。最早的实时视觉 SLAM 系统即是基于 EKF[2] 开发的。随后，为了克服 EKF 的缺点（例如线性化误差和噪声高斯分布假设），人们开始使用粒子滤波器（Particle Filter）等其他滤波器，乃至使用非线性优化的方法。时至今日，主流视觉 SLAM 使用以图优化（Graph Optimization）为代表的优化技术进行状态估计[13]。我们认为优化技术已经明显优于滤波器技术，只要计算资源允许，通常都偏向于使用优化方法（见第 10 讲和第 11 讲）。

相信读者已经对 SLAM 的数学模型有了大致的了解，然而我们仍需澄清一些问题。首先，要说明机器人**位置 x 是什么**。我们方才说的**位置**是有些模糊的。也许读者能够理解，在平面中运动的小萝卜可以用两个坐标加一个转角的形式将位置参数化。然而，虽然我的漫画风格有些二次元，小萝卜在更多时候是一个三维空间里的机器人。我们知道三维空间的运动由 3 个轴构成，所以小萝卜的运动要由 3 个轴上的平移，以及绕着 3 个轴的旋转来描述，一共有 6 个自由度。那是否意味着随便用一个 $\mathbb{R}^6$ 中的向量就能描述它了呢？我们将发现事情并没有那么简单。对 6 自由度的**位姿**[①]，如何表达它，如何优化它，都需要一定篇幅来介绍，这将是第 3 讲和第 4 讲的主要内容。随后，我们要说明在视觉 SLAM 中，**观测方程**如何参数化。换句话说，空间中的路标点是如何投影到一张照片上的。这需要解释相机的成像模型，我们将在第 5 讲介绍。最后，当知道了这些信息，**怎么求解上述方程**？这需要非线性优化的知识，是第 6 讲的内容。

这些内容组成了本书数学知识的部分。在对它们进行铺垫之后，我们就能仔细讨论视觉里程计、后端优化等更详细的知识了。可以看到，本讲介绍的内容构成了本书的一个提要。如果读者还没有很好地理解上面的概念，不妨回过头再阅读一遍。下面就要开始介绍程序啦！

2.4　实践：编程基础

2.4.1　安装 Linux 操作系统

终于开始令人兴奋的实践环节啦！你是否准备好了呢？为了完成本书的实践环节，我们需要准备一台电脑。你可以使用笔记本或台式机，最好是你个人的电脑，因为我们需要在上面安装操作系统进行实验。

我们的程序以 Linux 上的 C++ 程序为主。在实验过程中，我们会使用大量程序库。大部分程序库只对 Linux 提供了较好的支持，而在 Windows 上的配置则相对（相当）麻烦。因此，我们不得不假定你已经具备关于 Linux 的基本知识了（参见上一讲的练习题），包括使用基本的命令，了解软件如何安装。当然，你不必了解如何在 Linux 下开发 C++ 程序，这正是下面要详细谈的。

我们先来搭建本书所需的实验环境。作为一本面向初学者的书，我们使用 Ubuntu 作为开发环境。在 Linux 的各大发行版中，Ubuntu 及其衍生版本一直享有对新手用户友好的美誉。Ubuntu 是一个开源操作系统，它的系统和软件可以在官方网站（`http://cn.ubuntu.com`）免费下载，并且提供了详细的安装方式说明。同时，清华、中科大等国内各大高校也提供了 Ubuntu 软件源，使软件的安装十分便捷。

本书的第一版使用 Ubuntu 14.04 作为默认开发环境。在第二版中，我们将默认版本更新至较新的 **Ubuntu 18.04**（图 2-11），以便后来研究者使用。如果你想试试其他口味，那么 Ubuntu Kylin、Debian、Deepin 和 Linux Mint 也是不错的选择。我保证书中所有代码在 Ubuntu 18.04 下经过了良好的测试，但如果你选择其他发行版，我无法确定是否会遇到问题。你可能需要花费一些时间解决问

[①] 我们以后称它为位姿（Pose），以与位置进行区别。我们说的位姿，包含了旋转（Rotation）和平移（Translation）。

题（不过也可以把它们当作锻炼自己的机会）。大体来说，Ubuntu 对各种库的支持均较为完善，软件也非常丰富。尽管我们不限制你具体使用哪种 Linux 发行版，不过在讲解中，**我们会以 Ubuntu 18.04 为例**，且主要使用 Ubuntu 下的命令（例如 apt-get），所以在其他版本的 Ubuntu 下面不会有明显的区别。一般情况下，程序在 Linux 间移植不会非常烦琐。但如果你想在 Windows 或 OS X 下使用本书中的程序，则需要有一定的移植经验。

现在，请大家在自己的 PC 上安装好 Ubuntu 18.04。关于 Ubuntu 的安装，可以在网上搜到大量教程，只要照做即可，此处略过。最简单的方式是使用虚拟机（见图 2-11），但需要占用大量内存（我们的经验是 4GB 以上）和 CPU 才能保持流畅；你也可以安装双系统，这样会快一些，但需要一个空白的 U 盘来作为启动盘。另外，虚拟机软件对外部硬件的支持往往不够好，如果希望使用实际的传感器（双目、Kinect 等），则建议你使用双系统来安装 Linux。

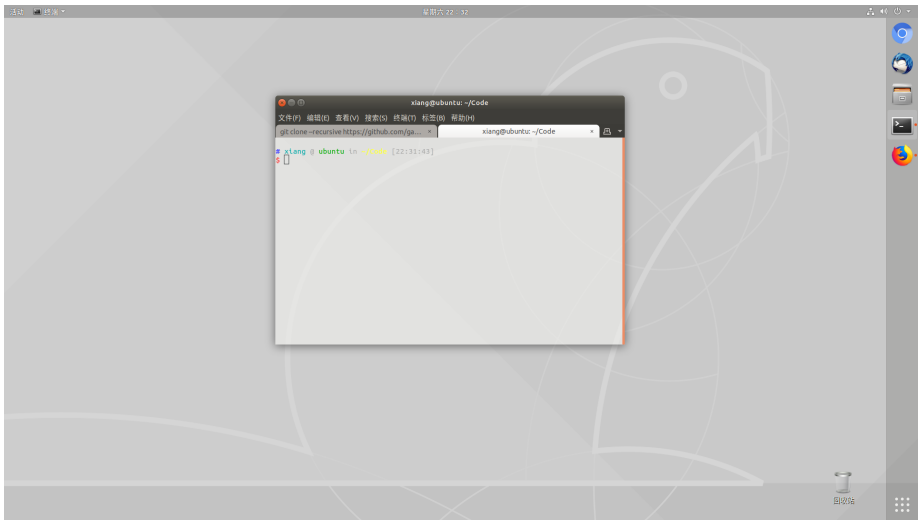


图 2-11　一个运行在虚拟机中的 Ubuntu 18.04。

关于安装的小提示：

- 安装操作系统时请不要选择“安装中下载更新”，并且断开网络连接，这样可以提高安装速度。至于更新可以在系统安装完毕后再装。如果你有 SSD 硬盘，这个过程大概用时 15 分钟。
- 安装完成后，请务必把软件源设置到离你较近的服务器上，以获得更快的下载速度。例如我使用清华的软件源通常能以 10MB/s 的速度安装软件[①]。

现在，假设你已经成功安装好 Ubuntu，无论是使用虚拟机还是双系统的方式。如果你还不熟悉 Ubuntu，可以试试它的各种软件，体验一下它的界面和交互方式[②]。不过我必须**提醒**你，特别是新手朋友：不要在 Ubuntu 的用户界面上花费太多时间！Linux 有许多可能浪费时间的地方，你可能会找

[①] 感谢 TUNA 同学们的维护！

[②] 大多数人第一次看到 Ubuntu 都觉得很漂亮。

到某些小众的软件、一些游戏，甚至会为找一张壁纸花费不少时间。但是请记住，你是用 Linux 来工作的。特别是在本书中，你是用 Linux 来学习 SLAM 的，所以要尽量把时间花在学习 SLAM 上。

好了，我们选择一个目录，放置本书中 SLAM 程序的代码。例如，可以将代码放到家目录（/home）的“slambook2”下。以后我们将把这个目录称为“**代码根目录**”。同时，可以另外选择一个目录，把本书的 Git 代码复制下来，方便做实验时随时对照。本书的代码是按章节划分的。比如，本讲的代码将在 slambook2/ch2 下，下一讲则在 slambook2/ch3 下。所以，现在请读者进入 slambook2/ch2 下（你应该会新建文件夹并进入该文件夹了吧）。

2.4.2　Hello SLAM

我们从最基本的程序开始。与许多计算机类书籍一样，我们来书写一个 HelloSLAM 程序。不过在做这件事之前，我们先来聊聊程序是什么。

在 Linux 中，程序是一个具有执行权限的文件。它可以是一个脚本，也可以是一个二进制文件，不过我们不限定它的后缀名（不像 Windows 那样需要指定成.exe 文件）。我们常用的 cd、ls 等命令，就是位于/bin 目录下的可执行文件。而对于其他地方的可执行程序，只要它有可执行权限，那么当我们在终端中输入程序名时，它就会运行。在 C++ 编程时，我们先书写一个文本文件：

slambook2/ch2/helloSLAM.cpp

```
#include <iostream>
using namespace std;

int main(int argc, char **argv) {
  cout << "Hello SLAM!" << endl;
  return 0;
}
```

然后用一个叫做**编译器**的程序，把这个文本文件编译成可执行程序。显然这是一个非常简单的程序。你应该能毫不费力地看懂它，所以这里不多加解释——如果实际情况不是这样，请你先补习一下 C++ 的基本知识。这个程序只是把一个字符串输出到屏幕上而已。你可以用文本编辑器 gedit（或 Vim，如果你在上一讲学习了 Vim）输入这些代码，并保存在上面列出的路径下。现在，我们用编译器 g++（g++ 是一个 C++ 编译器）把它编译成一个可执行文件。输入：

终端输入：

```
g++ helloSLAM.cpp
```

如果顺利，这条命令应该没有任何输出。如果机器上出现“command not found”的错误信息，说明你可能还没有安装 g++，请用如下命令进行安装：

终端输入：

```
sudo apt-get install g++
```

如果出现别的错误，请再检查一遍刚才的程序是否输入正确。

刚才这条编译命令把 helloSLAM.cpp 这个文本文件编译成了一个可执行程序。我们检查当前目录，会发现多了一个 a.out 文件，而且它具有执行权限（终端里颜色不同）。我们输入./a.out 即可运行此程序[①]：

终端输入：

```
% ./a.out
Hello SLAM!
```

如我们所想，这个程序输出“Hello SLAM!”，告诉我们它在正确运行。

请回顾一下我们之前做的事情。在这个例子中，我们用编辑器输入了 helloSLAM.cpp 的源代码，然后调用 g++ 编译器对它进行编译，得到了可执行文件。g++ 默认把源文件编译成 a.out 这个名字的程序（虽然有些古怪，但是可以接受的）。如果我们愿意，也可以指定这个输出的文件名（留作习题）。这是一个极其简单的例子，我们**使用了大量的默认参数，几乎省略了所有中间步骤**，为的是给读者一个简洁的印象（虽然你可能没有体会到）。下面我们要用 cmake 来编译这个程序。

2.4.3 使用 cmake

理论上说，任意一个 C++ 程序都可以用 g++ 来编译。但当程序规模越来越大时，一个工程可能有许多个文件夹和源文件，这时输入的编译命令将越来越长。通常一个小型 C++ 项目可能含有十几个类，各类间还存在着复杂的依赖关系。其中一部分要编译成可执行文件，另一部分编译成库文件。如果仅靠 g++ 命令，我们需要输入大量的编译指令，整个编译过程会变得异常烦琐。因此，对于 C++ 项目，使用一些工程管理工具会更加高效。在历史上工程师们曾使用 makefile 进行自动编译，但下面要谈的 cmake 比它更加方便。并且 cmake 在工程上广泛使用，我们会看到后面提到的大多数库都使用 cmake 来管理源代码。

在一个 cmake 工程中，我们会用 cmake 命令生成一个 makefile 文件，然后，用 make 命令根据这个 makefile 文件的内容编译整个工程。读者可能还不知道 makefile 是什么东西，不过没关系，我们会通过例子来学习。仍然以上面的 helloSLAM.cpp 为例，这次我们不是直接使用 g++，而是用 cmake 来制作一个工程，然后再编译它。在 slambook2/ch2/中新建一个 CMakeLists.txt 文件，内容如下：

slambook2/ch2/CMakeLists.txt

```
# 声明要求的 cmake 最低版本
cmake_minimum_required( VERSION 2.8 )

# 声明一个 cmake 工程
project( HelloSLAM )

# 添加一个可执行程序
```

[①] 前头的% 为提示符，不要连这个也打进去。

```
# 语法：add_executable( 程序名　源代码文件 )
add_executable( helloSLAM helloSLAM.cpp )
```

CMakeLists.txt 文件用于告诉 cmake 我们要对这个目录下的文件做什么事情。CMakeLists.txt 文件的内容需要遵守 cmake 的语法。这个示例中，我们演示了最基本的工程：指定一个工程名和一个可执行程序。根据注释，读者应该理解每句话做了些什么。

现在，在当前目录下（slambook2/ch2/），调用 cmake 对该工程进行 cmake 编译[①]：

终端输入：

```
cmake .
```

cmake 会输出一些编译信息，然后在当前目录下生成一些中间文件，其中最重要的就是 MakeFile[②]。由于 MakeFile 是自动生成的，我们不必修改它。现在，用 make 命令对工程进行编译：

终端输入：

```
% make
Scanning dependencies of target helloSLAM
[100%] Building CXX object CMakeFiles/helloSLAM.dir/helloSLAM.cpp.o
Linking CXX executable helloSLAM
[100%] Built target helloSLAM
```

编译过程中会输出一个编译进度。如果顺利通过，我们就可以得到在 CMakeLists.txt 中声明的那个可执行程序 helloSLAM。执行它：

终端输入：

```
% ./helloSLAM
Hello SLAM!
```

因为我们并没有修改源代码，所以得到的结果和之前是一样的。请读者想想这种做法和之前直接使用 g++ 编译的区别。这次我们用 cmake–make 的做法，cmake 过程处理了工程文件之间的关系，而 make 过程实际调用了 g++ 来编译程序。虽然这个过程中多了调用 cmake 和 make 的步骤，但我们对项目的编译管理工作，**从输入一串 g++ 命令，变成了维护若干个比较直观的 CMakeLists.txt 文件**，这将明显降低维护整个工程的难度。比如，如果想新增一个可执行文件，只需在 CMakeLists.txt 中添加一行“add_executable”命令即可，而后续的步骤是不变的。cmake 会帮我们解决代码的依赖关系，而无须输入一大串 g++ 命令。

现在这个过程中唯一让我们不满的是，cmake 生成的中间文件还留在我们的代码文件当中。当想要发布代码时，我们并不希望把这些中间文件一同发布出去。这时我们还需要把它们一个个删除，十分不便。一种更好的做法是让这些中间文件都放在一个中间目录中，在编译成功后，把这个中间目录删除即可。所以，更常见的编译 cmake 工程的做法如下：

[①] 指令最后有一个句点，请不要忘记，这表示在当前目录下进行 cmake。

[②] MakeFile 是一个自动化编译的脚本，读者现在可以将它理解成一系统自动生成的编译指令，而无须理会其内容。

终端输入：

```
mkdir build
cd build
cmake ..
make
```

我们新建了一个中间文件夹“build”，然后进入 build 文件夹，通过 cmake .. 命令对上一层文件夹，也就是代码所在的文件夹进行编译。这样，cmake 产生的中间文件就会生成在 build 文件夹中，与源代码分开。当发布源代码时，只要把 build 文件夹删掉即可。请读者自行按照这种方式对 ch2 中的代码进行编译，然后调用生成的可执行程序（请记得把上一步产生的中间文件删掉）。

2.4.4 使用库

在一个 C++ 工程中，并不是所有代码都会编译成可执行文件。只有带有 main 函数的文件才会生成可执行程序。而另一些代码，我们只想把它们打包成一个东西，供其他程序调用。这个东西叫作**库**（Library）。

一个库往往是许多算法、程序的集合，我们会在之后的练习中接触到许多库。例如，OpenCV 库提供了许多计算机视觉相关的算法，而 Eigen 库提供了矩阵代数的计算。因此，我们要学习如何用 cmake 生成库，并且使用库中的函数。现在我们演示如何自己编写一个库。书写如下的 libHelloSLAM.cpp 文件：

slambook2/ch2/libHelloSLAM.cpp

```
//这是一个库文件
#include <iostream>
using namespace std;

void printHello() {
  cout << "Hello SLAM" << endl;
}
```

这个库提供了一个 printHello 函数，调用此函数将输出一条信息。但是它没有 main 函数，这意味着这个库中没有可执行文件。我们在 CMakeLists.txt 里加上如下内容：

slambook2/ch2/CMakeLists.txt

```
add_library( hello libHelloSLAM.cpp )
```

这条命令告诉 cmake，我们想把这个文件编译成一个叫作“hello”的库。然后，和上面一样，使用 cmake 编译整个工程：

终端输入：

```
cd build
cmake ..
make
```

这时，在 build 文件夹中就会生成一个 libhello.a 文件，这就是我们得到的库。

在 Linux 中，库文件分成**静态库**和**共享库**两种[①]。静态库以.a 作为后缀名，共享库以.so 结尾。所有库都是一些函数打包后的集合，差别在于**静态库每次被调用都会生成一个副本，而共享库则只有一个副本**，更省空间。如果想生成共享库而不是静态库，只需使用以下语句即可。

slambook2/ch2/CMakeLists.txt

```
add_library( hello_shared SHARED libHelloSLAM.cpp )
```

此时得到的文件就是 libhello_shared.so 了。

库文件是一个压缩包，里面有编译好的二进制函数。不过，如果仅有.a 或.so 库文件，那么我们并不知道里面的函数到底是什么，调用的形式又是什么样。为了让别人（或者自己）使用这个库，我们需要提供一个**头文件**，说明这些库里都有些什么。因此，对于库的使用者，**只要拿到了头文件和库文件，就可以调用这个库了**。下面编写 libhello 的头文件。

slambook2/ch2/libHelloSLAM.h

```
#ifndef LIBHELLOSLAM_H_
#define LIBHELLOSLAM_H_
// 上面的宏定义是为了防止重复引用这个头文件而引起的重定义错误

// 打印一句hello的函数
void printHello();

#endif
```

这样，根据这个文件和我们刚才编译得到的库文件，就可以使用 printHello 函数了。最后，我们写一个可执行程序来调用这个简单的函数：

slambook2/ch2/useHello.cpp

```
#include "libHelloSLAM.h"

// 使用 libHelloSLAM.h 中的 printHello() 函数
int main(int argc, char **argv) {
  printHello();
  return 0;
}
```

然后，在 CMakeLists.txt 中添加一个可执行程序的生成命令，**链接**到刚才使用的库上：

slambook2/ch2/CMakeLists.txt

```
add_executable( useHello useHello.cpp )
target_link_libraries( useHello hello_shared )
```

[①] 你多半猜错了，它并不叫作动态库。

通过这两行语句，useHello 程序就能顺利使用 hello_shared 库中的代码了。这个小例子演示了如何生成并调用一个库。请注意，对于他人提供的库，我们也可用同样的方式对它们进行调用，整合到自己的程序中。

除了已经演示的功能之外，cmake 还有许多语法和选项，这里不一一列举。实际上 cmake 很像一个正常的编程语言，有变量，有条件控制语句，所以你也可以像学习编程一样来学习 cmake。习题中包含了一些 cmake 的阅读材料，感兴趣的读者可自行阅读。现在，简单回顾一下我们之前做了哪些事：

1. 首先，程序代码由头文件和源文件组成。
2. 带有 main 函数的源文件编译成可执行程序，其他的编译成库文件。
3. 如果可执行程序想调用库文件中的函数，它需要参考该库提供的头文件，以明白调用的格式。同时，要把可执行程序链接到库文件上。

这几个步骤应该是简单清楚的，但实际操作中你可能会遇上一些问题。比如说，如果代码里引用了库的函数，但忘了把程序链接到库上，会发生什么呢？请试试把 CMakeLists.txt 中的链接部分去掉，看看会发生什么情况。你能看懂 cmake 报告的错误消息吗？

2.4.5 使用 IDE

最后，我们来谈谈如何使用集成开发环境（Integrated Development Environment，IDE）。前面的编程完全可以用一个简单的文本编辑器来完成。然而，你可能需要在各个文件之间跳转，查询某个函数的声明和实现。当文件很多时，这仍然很烦琐。IDE 为开发者提供了跳转、补全、断点调试等很多方便的功能，所以，我们建议读者选择一个 IDE 进行开发。

Linux 下的 IDE 有很多种。虽然与 Windows 下的 Visual Studio 还有一些差距，不过支持 C++ 开发的也有好几种，例如：Eclipse、Qt Creator、Code::Blocks、Clion，等等。同样，我们不强制读者使用某种特定的 IDE，而仅给出我们的建议。我们使用的是 Kdevelop 和 Clion（见图 2-12和图 2-15）。其中 Kdevelop 是一个免费软件，位于 Ubuntu 的软件仓库中，意味着你可以用 apt-get 来安装它；而 Clion 则是收费软件，但持有学生邮箱可以免费使用一年。二者都是很好的 C++ 开发环境，优点列举如下：

1. 支持 cmake 工程。
2. 对 C++ 支持较好（包括 11 及之后的标准）。有高亮、跳转、补全等功能。能自动排版代码。
3. 能方便地看到各个文件和目录树。
4. 有一键编译、断点调试等功能。

基本上，它们都具备人们对 IDE 的正常功能要求，所以读者不妨尝试一下。下面我们稍微花一点篇幅介绍一下 Kdevelop 和 clion。

图 2-12　Kdevelop 界面。

Kdevelop 的使用

Kdevelop 原生支持 cmake 工程。具体做法是，在终端建立 CMakeLists.txt 后，用 Kdevelop 中的“工程 → 打开/导入工程”打开 CMakeLists.txt。软件会询问你几个问题，并且默认建立一个 build 文件夹，帮你调用刚才的 cmake 和 make 命令。只要按下快捷键 F8，这些都可以自动完成。图 2-12的下面部分就显示了编译信息。

我们把适应 IDE 的任务交给读者自己来完成。如果你是从 Windows 转过来的，会觉得它的界面与 Visual C++ 或 Visual Studio 挺相似。请用 Kdevelop 打开刚才的工程然后进行编译，看看它输出什么信息。相信你会觉得比打开终端更方便一些。

不过，本节重点想讲的是如何在 IDE 中进行调试。在 Windows 下编程的同学多半会有在 Visual Studio 下断点调试的经历。不过在 Linux 中，默认的调试工具 gdb 只提供了文本界面，对新手来讲不太方便。有些 IDE 提供了断点调试功能（底层仍旧是 gdb），Kdevelop 就是其中之一。要使用 Kdevelop 的断点调试功能，你需要完成以下几件事：

1. 在 CMakeLists.txt 中把工程调为 Debug 编译模式，同时不要使用优化选项（默认不使用）。
2. 告诉 Kdevelop 你想运行哪个程序。如果有参数，也要配置它的参数和工作目录。
3. 进入断点调试界面，就可以单步运行，看到中间变量的值了。

第一步，在 CMakeLists.txt 中加入下面的命令来设置编译模式：

slambook2/ch2/CMakeLists.txt

```
set( CMAKE_BUILD_TYPE "Debug" )
```

cmake 自带一些编译相关的内部变量，它们可以对编译过程进行更精细的控制。对于编译类型，通常有调试用的 Debug 模式与发布用的 Release 模式。在 Debug 模式中，程序运行较慢，但可以进行断点调试；而 Release 模式则速度较快，但没有调试信息。我们把程序设置成 Debug 模式，就能放置断点了。接下来，告诉 Kdevelop 你想启动哪个程序。

第二步，打开“运行 → 配置启动器”，然后单击左侧的“Add New→ 应用程序”。在这一步中，我们的任务是告诉 Kdevelop 想要启动哪一个程序。如图 2-13所示，既可以选择一个 cmake 的工程目标（也就是我们用 add_executable 指令构建的可执行程序），也可以直接指向一个二进制文件。建议使用第二种方式，根据我们的经验，这样更少出现问题。

图 2-13　启动器设置界面。

在第二栏里，可以设置程序的运行参数和工作目录。有时程序是有运行参数的，它们会作为 main 函数的参数被传入。如果没有则可以留空，对于工作目录亦是如此。配置好这两项后，单击“OK”按钮保存配置结果。

刚才这几步我们配置了一个应用程序的启动项。对于每一个启动项，我们可以单击“Execute”按钮直接启动这个程序，也可单击“Debug”按钮对它进行断点调试。读者可以试着单击“Execute”按钮，查看输出的结果。现在，为了调试这个程序，单击 printHello 那行的左侧，增加一个断点。然

后，单击“Debug”按钮，程序会停留在断点处等待，如图 2-14所示。

图 2-14　调试界面。

调试时，Kdevelop 会切换到调试模式，界面会发生一点变化。在断点处，可以用单步运行（F10 键）、单步跟进（F11 键）、单步跳出（F12 键）功能控制程序的运行。同时，可以点开左侧的界面，查看局部变量的值。或者选择“停止”按钮，结束调试。调试结束后，Kdevelop 会回到正常的开发界面。

现在你应该熟悉了整个断点调试的流程。今后，如果在程序运行阶段发生了错误，导致程序崩溃，就可以用断点调试确定出错的位置，然后加以修正[①]。

Clion 的使用

Clion 相比于 Kdevelop 来说更加完善一些，但是它需要正版帐号，同时对主机的要求也会高一些。在 Clion 中，你同样可以打开一个 CMakeLists.txt，或者指定目录。Clion 会替你完成 cmake-make 的过程。它的运行界面如图图 2-15所示。

同样的，在打开 Clion 之后，可以在界面右上角处选择你想运行或调试的程序，调整它们的启动参数和工作目录。点击该栏的小甲虫按钮可以启动断点调试模式。Clion 还有许多方便的功能，比如自动创建类、函数改名、自动调整编码风格等，请一定要试一试。

好了，如果你已经熟悉了 IDE 的使用，那么入门章节也就到此为止。你或许已经觉得我有些话唠了，所以在今后的实践部分中，我们不会再介绍怎么新建 build 文件夹，调用 cmake 和 make 命令

[①] 而不是直接给我们发邮件询问怎么处理遇到的问题。

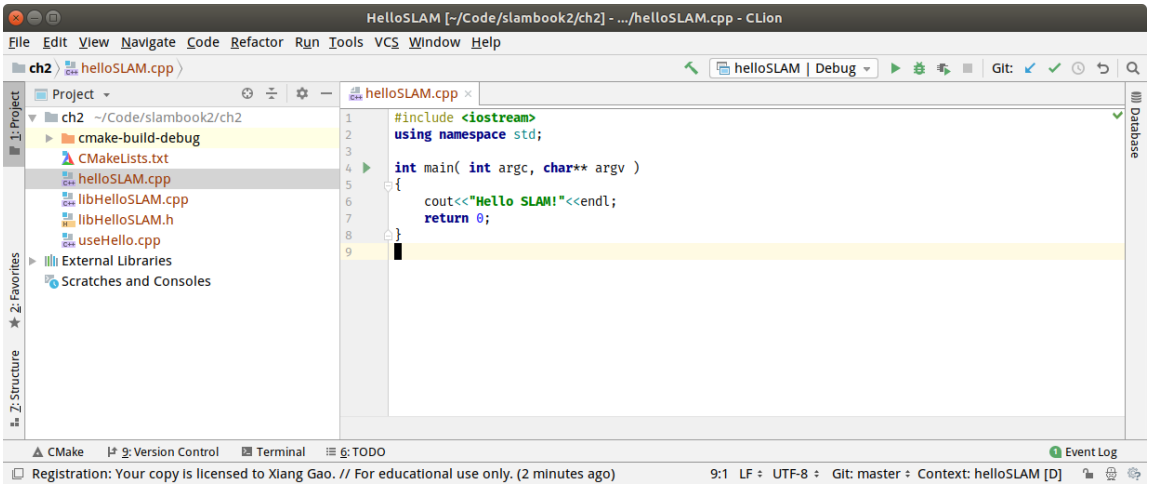


图 2-15　Clion 运行界面

来编译程序。我相信读者应该掌握了这些简单的步骤。同样的，由于本书用到的大多第三方库都是 cmake 工程，你也会不断熟悉这个编译过程。接下来我们就开始正式的章节，介绍一些相关的数学知识。

习题

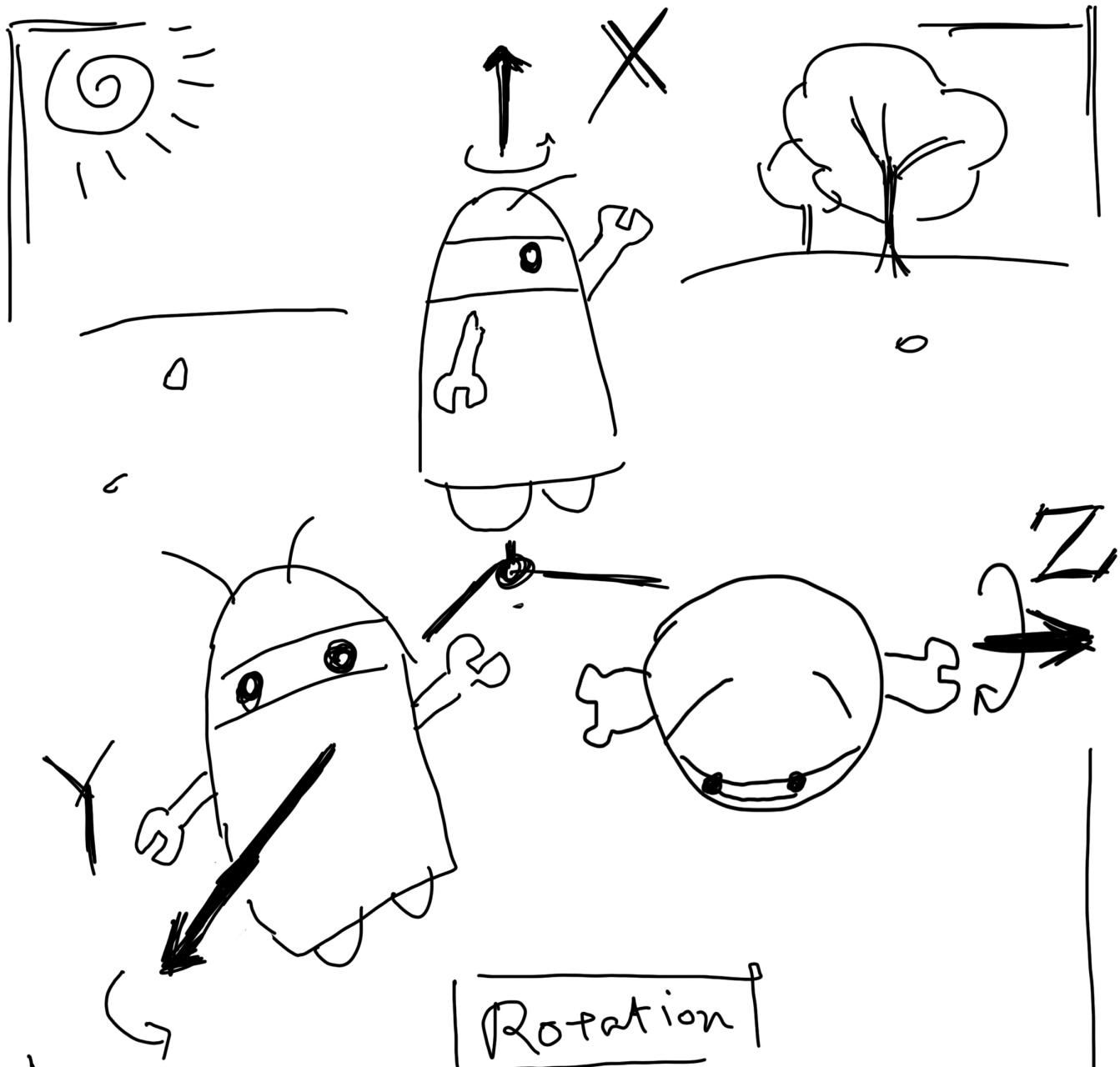
1. 阅读文献 [1] 和 [14]，你能看懂其中的内容吗？
2.* 阅读 SLAM 的综述文献，例如 [9, 15–18] 等。这些文献关于 SLAM 的看法与本书有何异同？
3. g++ 命令有哪些参数？怎么填写参数可以更改生成的程序文件名？
4. 使用 build 文件夹来编译你的 cmake 工程，然后在 Kdevelop 中试试。
5. 刻意在代码中添加一些语法错误，看看编译会生成什么样的信息。你能看懂 g++ 的错误信息吗？
6. 如果忘了把库链接到可执行程序上，编译会报错吗？报什么样的错？
7.* 阅读《cmake 实践》，了解 cmake 的其他语法。
8.* 完善 hello SLAM 小程序，把它做成一个小程序库，安装到本地硬盘中。然后，新建一个工程，使用 find_package 找这个库并调用。
9.* 阅读其他 cmake 教学材料，例如`https://github.com/TheErk/CMake-tutorial`。
10. 找到 Kdevelop 的官方网站，看看它还有哪些特性。你都用上了吗？
11. 如果在上一讲学习了 Vim，请试试 Kdevelop 的 Vim 编辑功能。

第三章　三维空间刚体运动

主要目标

1. 理解三维空间的刚体运动描述方式：旋转矩阵、变换矩阵、四元数和欧拉角。
2. 掌握 Eigen 库的矩阵、几何模块使用方法。

在上一讲中，我们讲解了视觉 SLAM 的框架与内容。本讲将介绍视觉 SLAM 的基本问题之一：**如何描述刚体在三维空间中的运动**？直观上看，我们当然知道这由一次旋转加一次平移组成。平移确实没有太大问题，但旋转的处理是件麻烦事。我们将介绍旋转矩阵、四元数、欧拉角的意义，以及它们是如何运算和转换的。在实践部分，我们将介绍线性代数库 Eigen。它提供了 C++ 中的矩阵运算，并且它的 Geometry 模块还提供了四元数等描述刚体运动的结构。Eigen 的优化非常完善，但是它的使用方法有一些特殊的地方，我们留到程序中介绍。



Rotation

$$SO(3) = \{ R \mid R^T R = I, \det(R) = 1 \}$$

roll, pitch, yaw

$$q = q_0 + q_1 i + q_2 j + q_3 k$$

$$T = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \in SE(3)$$

3.1　旋转矩阵

3.1.1　点和向量，坐标系

我们日常生活的空间是三维的，因此我们生来就习惯于三维空间的运动。三维空间由 3 个轴组成，所以一个空间点的位置可以由 3 个坐标指定。不过，我们现在要考虑**刚体**，它不光有位置，还有自身的姿态。相机也可以看成三维空间的刚体，于是位置是指相机在空间中的哪个地方，而姿态则是指相机的朝向。结合起来，我们可以说，“相机正处于空间 $(0,0,0)$ 点处，朝向正前方”这样的话。但是这种自然语言很烦琐，我们更喜欢用数学语言来描述它。

我们从最基本的内容讲起：**点**和**向量**。点就是空间当中的基本元素，没有长度，没有体积。把两个点连接起来，就构成了向量。向量可以看成从某点指向另一点的一个箭头。需要提醒读者的是，请不要把向量与它的**坐标**两个概念混淆。一个向量是空间当中的一样东西，比如说 $\boldsymbol{a}$。这里 $\boldsymbol{a}$ 并不需要和若干个实数相关联的。只有当我们指定这个三维空间中的某个**坐标系**时，才可以谈论该向量在此坐标系下的坐标，也就是找到若干个实数对应这个向量。

用线性代数的知识来说，三维空间中的某个点的坐标也可以用 $\mathbb{R}^3$ 来描述。怎么描述呢？假设在这个线性空间内，我们找到了该空间的一组**基**[①]$(\boldsymbol{e}_1,\boldsymbol{e}_2,\boldsymbol{e}_3)$，那么，任意向量 $\boldsymbol{a}$ 在这组基下就有一个**坐标**：

$$\boldsymbol{a}=[\boldsymbol{e}_1,\boldsymbol{e}_2,\boldsymbol{e}_3]\begin{bmatrix}a_1\\a_2\\a_3\end{bmatrix}=a_1\boldsymbol{e}_1+a_2\boldsymbol{e}_2+a_3\boldsymbol{e}_3. \tag{3.1}$$

这里 $(a_1,a_2,a_3)^{\mathrm{T}}$ 称为 $\boldsymbol{a}$ 在此基下的坐标[②]。坐标的具体取值，一是和向量本身有关，二是和坐标系（基）的选取有关。坐标系通常由 3 个正交的坐标轴组成（尽管也可以有非正交的，但实际中很少见）。例如，给定 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 轴时，$\boldsymbol{z}$ 轴就可以通过右手（或左手）法则由 $\boldsymbol{x}\times\boldsymbol{y}$ 定义出来。根据定义方式的不同，坐标系又分为左手系和右手系。左手系的第 3 个轴与右手系方向相反。大部分 3D 程序库使用右手系（如 OpenGL，3D Max 等），也有部分库使用左手系（如 Unity，Direct3D 等）。

根据基本的线性代数知识，我们可以谈论向量与向量，以及向量与数之间的运算，例如数乘、加法、减法、内积、外积等。数乘和加减法都是相当基本的内容，也符合直观想象。例如，两个向量相加的结果就是把它们各自坐标相加，减法亦然，等等。这里不再赘述。内外积对读者来说可能有些陌生，这里给出它们的运算方式。对于 $\boldsymbol{a},\boldsymbol{b}\in\mathbb{R}^3$，通常意义下[③]的内积可以写成：

$$\boldsymbol{a}\cdot\boldsymbol{b}=\boldsymbol{a}^{\mathrm{T}}\boldsymbol{b}=\sum_{i=1}^{3}a_ib_i=|\boldsymbol{a}|\,|\boldsymbol{b}|\cos\langle\boldsymbol{a},\boldsymbol{b}\rangle. \tag{3.2}$$

---

① 以防读者忘记，基就是张成这个空间的一组线性无关的向量，有些书里也叫**基底**。

② 本书的向量为列向量，这和一般的数学书籍类似。

③ 内积也有形式化的法则，但本书只讨论通常的内积。

其中 $\langle \boldsymbol{a}, \boldsymbol{b} \rangle$ 指向量 $\boldsymbol{a}, \boldsymbol{b}$ 的夹角。内积也可以描述向量间的投影关系。而外积则是这个样子：

$$\boldsymbol{a} \times \boldsymbol{b} = \begin{Vmatrix} \boldsymbol{e}_1 & \boldsymbol{e}_2 & \boldsymbol{e}_3 \\ a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \end{Vmatrix} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \boldsymbol{b} \triangleq \boldsymbol{a}^{\wedge} \boldsymbol{b}. \tag{3.3}$$

外积的结果是一个向量，它的方向垂直于这两个向量，大小为 $|\boldsymbol{a}|\,|\boldsymbol{b}| \sin \langle \boldsymbol{a}, \boldsymbol{b} \rangle$，是两个向量张成的四边形的有向面积。对于外积运算，我们引入 ^ 符号，把 $\boldsymbol{a}$ 写成一个矩阵。事实上是一个**反对称矩阵**（Skew-symmetric matrix）[①]，你可以将 ^ 记成一个反对称符号。这样就把外积 $\boldsymbol{a} \times \boldsymbol{b}$ 写成了矩阵与向量的乘法 $\boldsymbol{a}^{\wedge} \boldsymbol{b}$，把它变成了线性运算。这个符号将在后文经常用到，请记住它，并且此符号是一个一一映射，意味着任意向量都对应着唯一的一个反对称矩阵，反之亦然：

$$\boldsymbol{a}^{\wedge} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}. \tag{3.4}$$

同时，需要提醒读者的是，向量和加减法、内外积，即使在不谈论它们的坐标时也可以计算。例如，虽然内积在有坐标时，可以用两个向量的分量乘积之和表达，但是即使不知道它们的坐标时，也可以通过长度和夹角来计算二者的内积。所以两个向量的内积结果和坐标系的选取是无关的。

3.1.2 坐标系间的欧氏变换

我们经常会在实际场景中定义各种各样的坐标系。在机器人学中，你会给每一个连杆和关节都定义它们的坐标系；在 3D 作图时，我们也会定义每一个长方体、圆柱体的坐标系。如果考虑运动的机器人，那么常见的做法是设定一个惯性坐标系（或者叫世界坐标系），可以认为它是固定不动的，例如图 3-1中的 x_W, y_W, z_W 定义的坐标系。同时，相机或机器人则是一个移动坐标系，例如 x_C, y_C, z_C 定义的坐标系。我们可能会问：相机视野中某个向量 $\boldsymbol{p}$，它在相机坐标系下的坐标为 $\boldsymbol{p}_c$，而在世界坐标系下看，它的坐标为 $\boldsymbol{p}_w$，那么，这两个坐标之间是如何转换的呢？这时，就需要先得到该点针对机器人坐标系的坐标值，再根据机器人位姿**变换**到世界坐标系中。我们需要一种数学手段来描述这个变换关系，稍后我们会看到，可以用一个矩阵 $\boldsymbol{T}$ 来描述它。

直观上看，两个坐标系之间的运动由一个旋转加上一个平移组成，这种运动称为**刚体运动**。相机运动就是一个刚体运动。刚体运动过程中，同一个向量在各个坐标系下的长度和夹角都不会发生变化。想象你把手机抛到空中，在它落地摔碎之前[②]，只可能有空间位置和姿态的不同，而它自己的长度、各个面的角度等性质不会有任何变化。手机并不会像橡皮那样一会儿被挤扁，一会儿被拉长。此时，我们说手机坐标系到世界坐标之间，相差了一个**欧氏变换**（Euclidean Transform）。

[①] 反对称矩阵 $\boldsymbol{A}$ 满足 $\boldsymbol{A}^{\mathrm{T}} = -\boldsymbol{A}$。

[②] 请不要付诸实践。

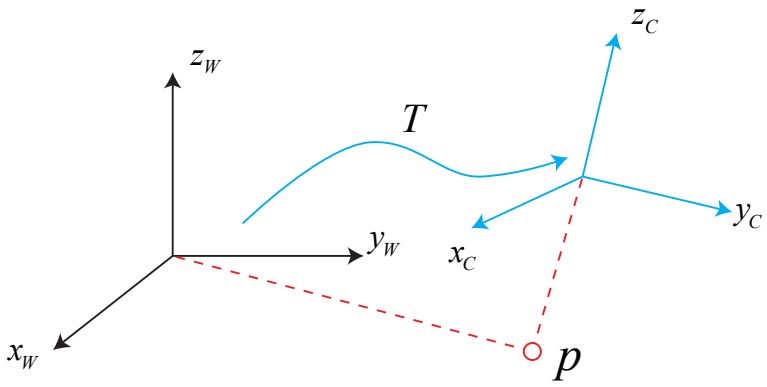


图 3-1　坐标变换。对于同一个向量 $\boldsymbol{p}$，它在世界坐标系下的坐标 $\boldsymbol{p}_w$ 和在相机坐标系下的坐标 $\boldsymbol{p}_c$ 是不同的。这个变换关系由变换矩阵 $\boldsymbol{T}$ 来描述。

欧氏变换由旋转和平移组成。我们首先考虑旋转。设某个单位正交基 $(\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3)$ 经过一次旋转变成了 $(\boldsymbol{e}'_1, \boldsymbol{e}'_2, \boldsymbol{e}'_3)$。那么，对于同一个向量 $\boldsymbol{a}$（该向量并没有随着坐标系的旋转而发生运动），它在两个坐标系下的坐标为 $[a_1, a_2, a_3]^{\mathrm{T}}$ 和 $[a'_1, a'_2, a'_3]^{\mathrm{T}}$。因为向量本身没变，根据坐标的定义，有：

$$[\boldsymbol{e}_1, \boldsymbol{e}_2, \boldsymbol{e}_3] \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = [\boldsymbol{e}'_1, \boldsymbol{e}'_2, \boldsymbol{e}'_3] \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix}. \tag{3.5}$$

为了描述两个坐标之间的关系，我们对上述等式的左右两边同时左乘 $\begin{bmatrix} \boldsymbol{e}_1^{\mathrm{T}} \\ \boldsymbol{e}_2^{\mathrm{T}} \\ \boldsymbol{e}_3^{\mathrm{T}} \end{bmatrix}$，那么左边的系数就变成了单位矩阵，所以：

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} \boldsymbol{e}_1^{\mathrm{T}}\boldsymbol{e}'_1 & \boldsymbol{e}_1^{\mathrm{T}}\boldsymbol{e}'_2 & \boldsymbol{e}_1^{\mathrm{T}}\boldsymbol{e}'_3 \\ \boldsymbol{e}_2^{\mathrm{T}}\boldsymbol{e}'_1 & \boldsymbol{e}_2^{\mathrm{T}}\boldsymbol{e}'_2 & \boldsymbol{e}_2^{\mathrm{T}}\boldsymbol{e}'_3 \\ \boldsymbol{e}_3^{\mathrm{T}}\boldsymbol{e}'_1 & \boldsymbol{e}_3^{\mathrm{T}}\boldsymbol{e}'_2 & \boldsymbol{e}_3^{\mathrm{T}}\boldsymbol{e}'_3 \end{bmatrix} \begin{bmatrix} a'_1 \\ a'_2 \\ a'_3 \end{bmatrix} \triangleq \boldsymbol{R}\boldsymbol{a}'. \tag{3.6}$$

我们把中间的矩阵拿出来，定义成一个矩阵 $\boldsymbol{R}$。这个矩阵由两组基之间的内积组成，刻画了旋转前后同一个向量的坐标变换关系。只要旋转是一样的，那么这个矩阵也是一样的。可以说，矩阵 $\boldsymbol{R}$ 描述了旋转本身。因此称为**旋转矩阵**（Rotation matrix）。同时，该矩阵各分量是两个坐标系基的内积，由于基向量的长度为 1，所以实际上是各基向量的夹角之余弦。所以这个矩阵也叫**方向余弦矩阵**（Direction Cosine matrix）。我们后文统一称呼它为旋转矩阵。

旋转矩阵有一些特别的性质。事实上，它是一个行列式为 1 的正交矩阵[①,②]。反之，行列式为 1 的正交矩阵也是一个旋转矩阵。所以，可以把 n 维旋转矩阵的集合定义如下：

$$\mathrm{SO}(n) = \{\boldsymbol{R} \in \mathbb{R}^{n\times n} | \boldsymbol{R}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{I}, \det(\boldsymbol{R}) = 1\}. \tag{3.7}$$

$\mathrm{SO}(n)$ 是**特殊正交群**（Special Orthogonal Group）的意思。我们把“群”的内容留到下一讲。这个集合由 n 维空间的旋转矩阵组成，特别地，SO(3) 就是指三维空间的旋转。通过旋转矩阵，我们可以直接谈论两个坐标系之间的旋转变换，而不用再从基开始谈起。

由于旋转矩阵为正交矩阵，它的逆（即转置）描述了一个相反的旋转。按照上面的定义方式，有：

$$\boldsymbol{a}' = \boldsymbol{R}^{-1}\boldsymbol{a} = \boldsymbol{R}^{\mathrm{T}}\boldsymbol{a}. \tag{3.8}$$

显然 $\boldsymbol{R}^{\mathrm{T}}$ 刻画了一个相反的旋转。

在欧氏变换中，除了旋转之外还有平移。考虑世界坐标系中的向量 $\boldsymbol{a}$，经过一次旋转（用 $\boldsymbol{R}$ 描述）和一次平移 $\boldsymbol{t}$ 后，得到了 $\boldsymbol{a}'$，那么把旋转和平移合到一起，有：

$$\boldsymbol{a}' = \boldsymbol{R}\boldsymbol{a} + \boldsymbol{t}. \tag{3.9}$$

其中，$\boldsymbol{t}$ 称为平移向量。相比于旋转，平移部分只需把平移向量加到旋转之后的坐标上，非常简单。通过上式，我们用一个旋转矩阵 $\boldsymbol{R}$ 和一个平移向量 $\boldsymbol{t}$ 完整地描述了一个欧氏空间的坐标变换关系。实际当中，我们会定义坐标系 1、坐标系 2，那么向量 $\boldsymbol{a}$ 在两个系下坐标为 $\boldsymbol{a}_1, \boldsymbol{a}_2$，它们之间的关系，按照完整的写法，应该是：

$$\boldsymbol{a}_1 = \boldsymbol{R}_{12}\boldsymbol{a}_2 + \boldsymbol{t}_{12}. \tag{3.10}$$

这里的 $\boldsymbol{R}_{12}$ 是指“把坐标系 2 的向量变换到坐标系 1”中。由于向量乘在这个矩阵的右边，它的下标是**从右读到左**的。这也是本书的习惯写法。坐标变换很容易搞混，特别是存在多个坐标系的情况下。同理，如果我们要表达“从 1 到 2 的旋转矩阵”时，就写成 $\boldsymbol{R}_{21}$。请读者务必清楚这边的记法，因为不同书籍里写法不同，有的会记成左上/下标，而本书写在右侧下标。

关于平移 $\boldsymbol{t}_{12}$，它实际对应的是坐标系 1 原点指向坐标系 2 原点的向量，**在坐标系 1 下取的坐标**，所以我建议读者把它记作“从 1 到 2 的向量”。但是反过来的 $\boldsymbol{t}_{21}$，即从 2 指向 1 的向量**在坐标系 2 下的坐标**，却并不等于 $-\boldsymbol{t}_{12}$，而是和两个系的旋转还有关系[③]。所以，当初学者问“我的坐标在哪里”这样的问题时，我们需要清楚地说明这句话的含义。这里“我的坐标”实际上指从世界坐标系指向自己坐标系原点的向量，在世界坐标系下取到的坐标。对应到数学符号上，应该是 $\boldsymbol{t}_{WC}$ 的取值。同理，它也不是 $-\boldsymbol{t}_{CW}$。

[①] 正交矩阵即逆为自身转置的矩阵。旋转矩阵的正交性可以直接由定义得出。

[②] 行列式为 1 是人为定义的，实际上只要求它的行列式为 ± 1，但行列式为 -1 的称为瑕旋转，即一次旋转加一次反射。

[③] 尽管从向量层面来看，它们确实是反向的关系，但这两个向量的坐标值并不是相反数。你能想清楚这是为什么吗？

3.1.3　变换矩阵与齐次坐标

式(3.9)完整地表达了欧氏空间的旋转与平移，不过还存在一个小问题：这里的变换关系不是一个线性关系。假设我们进行了两次变换：$\boldsymbol{R}_1, \boldsymbol{t}_1$ 和 $\boldsymbol{R}_2, \boldsymbol{t}_2$：

$$\boldsymbol{b} = \boldsymbol{R}_1 \boldsymbol{a} + \boldsymbol{t}_1, \quad \boldsymbol{c} = \boldsymbol{R}_2 \boldsymbol{b} + \boldsymbol{t}_2.$$

那么，从 $\boldsymbol{a}$ 到 $\boldsymbol{c}$ 的变换为：

$$\boldsymbol{c} = \boldsymbol{R}_2 \left(\boldsymbol{R}_1 \boldsymbol{a} + \boldsymbol{t}_1 \right) + \boldsymbol{t}_2.$$

这样的形式在变换多次之后会显得很啰嗦。因此，我们引入齐次坐标和变换矩阵，重写式(3.9)：

$$\begin{bmatrix} \boldsymbol{a}' \\ 1 \end{bmatrix} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix} \triangleq \boldsymbol{T} \begin{bmatrix} \boldsymbol{a} \\ 1 \end{bmatrix}. \tag{3.11}$$

这是一个数学技巧：我们在一个三维向量的末尾添加 1，将其变成了四维向量，称为**齐次坐标**。对于这个四维向量，我们可以把旋转和平移写在一个矩阵里面，使得整个关系变成线性关系。该式中，矩阵 $\boldsymbol{T}$ 称为**变换矩阵（Transform Matrix）**。

我们暂时用 $\tilde{\boldsymbol{a}}$ 表示 $\boldsymbol{a}$ 的齐次坐标。那么依靠齐次坐标和变换矩阵，两次变换的叠加就可以有很好的形式：

$$\tilde{\boldsymbol{b}} = \boldsymbol{T}_1 \tilde{\boldsymbol{a}}, \ \tilde{\boldsymbol{c}} = \boldsymbol{T}_2 \tilde{\boldsymbol{b}} \quad \Rightarrow \tilde{\boldsymbol{c}} = \boldsymbol{T}_2 \boldsymbol{T}_1 \tilde{\boldsymbol{a}}. \tag{3.12}$$

但是区分齐次和非齐次坐标的符号令我们感到厌烦，因为此处只需要在向量末尾添加 1 或者去掉 1 即可[①]。所以，在不引起歧义的情况下，以后我们就直接把它写成 $\boldsymbol{b} = \boldsymbol{T}\boldsymbol{a}$ 的样子，默认其中进行了齐次坐标的转换[②]。

关于变换矩阵 $\boldsymbol{T}$，它具有比较特别的结构：左上角为旋转矩阵，右侧为平移向量，左下角为 $\boldsymbol{0}$ 向量，右下角为 1。这种矩阵又称为特殊欧氏群（Special Euclidean Group）：

$$\mathrm{SE}(3) = \left\{ \boldsymbol{T} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4} | \boldsymbol{R} \in \mathrm{SO}(3), \boldsymbol{t} \in \mathbb{R}^3 \right\}. \tag{3.13}$$

与 SO(3) 一样，求解该矩阵的逆表示一个反向的变换：

$$\boldsymbol{T}^{-1} = \begin{bmatrix} \boldsymbol{R}^{\mathrm{T}} & -\boldsymbol{R}^{\mathrm{T}} \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}. \tag{3.14}$$

同样，我们用 $\boldsymbol{T}_{12}$ 这样的写法来表示从 2 到 1 的变换。并且，为了保持符号的简洁，在不引起歧义的情况下，以后不刻意区别齐次坐标与普通坐标的符号，**默认使用的是符合运算法则的那一种**。

[①] 但齐次坐标的用途不止于此，我们在第 7 章还会再介绍。

[②] 注意，不进行齐次坐标转换时，这边的乘法在矩阵维度上是不成立的。

例如，当我们写 $\boldsymbol{Ta}$ 时，使用的是齐次坐标（不然没法计算）。而写 $\boldsymbol{Ra}$ 时，使用的是非齐次坐标。如果写在一个等式中，就假设齐次坐标到普通坐标的转换，是已经做好了的——因为齐次坐标和非齐次坐标之间的转换事实上非常容易，而在 C++ 程序中你可以使用**运算符重载**来完成这个功能，保证在程序中看到的运算是统一的。

回顾一下：首先，我们介绍了向量及其坐标表示，并介绍了向量间的运算；然后，坐标系之间的运动由欧氏变换描述，它由平移和旋转组成。旋转可以由旋转矩阵 SO(3) 描述，而平移直接由一个 $\mathbb{R}^3$ 向量描述。最后，如果将平移和旋转放在一个矩阵中，就形成了变换矩阵 SE(3)。

3.2 实践：Eigen

本讲的实践部分有两节。第一部分中，将讲解如何使用 Eigen 来表示矩阵、向量，随后引申至旋转矩阵与变换矩阵的计算。本节的代码在 slambook2/ch3/useEigen 中。

Eigen[①]是一个 C++ 开源线性代数库。它提供了快速的有关矩阵的线性代数运算，还包括解方程等功能。许多上层的软件库也使用 Eigen 进行矩阵运算，包括 g2o、Sophus 等。照应本讲的理论部分，我们来学习一下 Eigen 的编程。

你的 PC 上可能还没有安装 Eigen。请输入以下命令进行安装：

终端输入：

```
sudo apt-get install libeigen3-dev
```

大部分常用的库都已在 Ubuntu 软件源中提供。以后，若想要安装某个库，不妨先搜索一下 Ubuntu 的软件源中是否已提供。通过 apt 命令，我们能够方便地安装 Eigen。回顾上一讲的知识，我们知道一个库由头文件和库文件组成。Eigen 头文件的默认位置在“/usr/include/eigen3/”中。如果不确定，可以输入以下命令查找：

终端输入：

```
sudo updatedb
locate eigen3
```

相比于其他库，Eigen 的特殊之处在于，它是一个纯用头文件搭建起来的库（这非常神奇！）。这意味着你只能找到它的头文件，而没有.so 或.a 那样的二进制文件。在使用时，只需引入 Eigen 的头文件即可，不需要链接库文件（因为它没有库文件）。下面写一段代码来实际练习一下 Eigen 的使用：

slambook2/ch3/useEigen/eigenMatrix.cpp

```
#include <iostream>
using namespace std;

```

[①] 官方主页：http://eigen.tuxfamily.org/index.php?title=Main_Page。

```cpp
#include <ctime>
// Eigen 核心部分
#include <Eigen/Core>
// 稠密矩阵的代数运算（逆，特征值等）
#include <Eigen/Dense>
using namespace Eigen;

#define MATRIX_SIZE 50

/****************************
* 本程序演示了 Eigen 基本类型的使用
****************************/

int main(int argc, char **argv) {
  // Eigen 中所有向量和矩阵都是Eigen::Matrix，它是一个模板类。它的前三个参数为：数据类型，行，列
  // 声明一个2*3的float矩阵
  Matrix<float, 2, 3> matrix_23;

  // 同时，Eigen 通过 typedef 提供了许多内置类型，不过底层仍是Eigen::Matrix
  // 例如 Vector3d 实质上是 Eigen::Matrix<double, 3, 1>，即三维向量
  Vector3d v_3d;
  // 这是一样的
  Matrix<float, 3, 1> vd_3d;

  // Matrix3d 实质上是 Eigen::Matrix<double, 3, 3>
  Matrix3d matrix_33 = Matrix3d::Zero(); //初始化为零
  // 如果不确定矩阵大小，可以使用动态大小的矩阵
  Matrix<double, Dynamic, Dynamic> matrix_dynamic;
  // 更简单的
  MatrixXd matrix_x;
  // 这种类型还有很多，我们不一一列举

  // 下面是对Eigen阵的操作
  // 输入数据（初始化）
  matrix_23 << 1, 2, 3, 4, 5, 6;
  // 输出
  cout << "matrix 2x3 from 1 to 6: \n" << matrix_23 << endl;

  // 用()访问矩阵中的元素
  cout << "print matrix 2x3: " << endl;
  for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) cout << matrix_23(i, j) << "\t";
    cout << endl;
  }

  // 矩阵和向量相乘（实际上仍是矩阵和矩阵）
  v_3d << 3, 2, 1;
  vd_3d << 4, 5, 6;

  // 但是在Eigen里你不能混合两种不同类型的矩阵，像这样是错的
  // Matrix<double, 2, 1> result_wrong_type = matrix_23 * v_3d;
  // 应该显式转换
  Matrix<double, 2, 1> result = matrix_23.cast<double>() * v_3d;
```

```
57  cout << "[1,2,3;4,5,6]*[3,2,1]=" << result.transpose() << endl;
58
59  Matrix<float, 2, 1> result2 = matrix_23 * vd_3d;
60  cout << "[1,2,3;4,5,6]*[4,5,6]: " << result2.transpose() << endl;
61
62  // 同样你不能搞错矩阵的维度
63  // 试着取消下面的注释，看看Eigen会报什么错
64  // Eigen::Matrix<double, 2, 3> result_wrong_dimension = matrix_23.cast<double>() * v_3d;
65
66  // 一些矩阵运算
67  // 四则运算就不演示了，直接用+-*/即可。
68  matrix_33 = Matrix3d::Random();      // 随机数矩阵
69  cout << "random matrix: \n" << matrix_33 << endl;
70  cout << "transpose: \n" << matrix_33.transpose() << endl; // 转置
71  cout << "sum: " << matrix_33.sum() << endl;               // 各元素和
72  cout << "trace: " << matrix_33.trace() << endl;           // 迹
73  cout << "times 10: \n" << 10 * matrix_33 << endl;         // 数乘
74  cout << "inverse: \n" << matrix_33.inverse() << endl;     // 逆
75  cout << "det: " << matrix_33.determinant() << endl;       // 行列式
76
77  // 特征值
78  // 实对称矩阵可以保证对角化成功
79  SelfAdjointEigenSolver<Matrix3d> eigen_solver(matrix_33.transpose() * matrix_33);
80  cout << "Eigen values = \n" << eigen_solver.eigenvalues() << endl;
81  cout << "Eigen vectors = \n" << eigen_solver.eigenvectors() << endl;
82
83  // 解方程
84  // 我们求解 matrix_NN * x = v_Nd 这个方程
85  // N的大小在前边的宏里定义，它由随机数生成
86  // 直接求逆自然是最直接的，但是求逆运算量大
87
88  Matrix<double, MATRIX_SIZE, MATRIX_SIZE> matrix_NN
89    = MatrixXd::Random(MATRIX_SIZE, MATRIX_SIZE);
90  matrix_NN = matrix_NN * matrix_NN.transpose();  // 保证半正定
91  Matrix<double, MATRIX_SIZE, 1> v_Nd = MatrixXd::Random(MATRIX_SIZE, 1);
92
93  clock_t time_stt = clock(); // 计时
94  // 直接求逆
95  Matrix<double, MATRIX_SIZE, 1> x = matrix_NN.inverse() * v_Nd;
96  cout << "time of normal inverse is "
97       << 1000 * (clock() - time_stt) / (double) CLOCKS_PER_SEC << "ms" << endl;
98  cout << "x = " << x.transpose() << endl;
99
100 // 通常用矩阵分解来求，例如QR分解，速度会快很多
101 time_stt = clock();
102 x = matrix_NN.colPivHouseholderQr().solve(v_Nd);
103 cout << "time of Qr decomposition is "
104      << 1000 * (clock() - time_stt) / (double) CLOCKS_PER_SEC << "ms" << endl;
105 cout << "x = " << x.transpose() << endl;
106
107 // 对于正定矩阵，还可以用cholesky分解来解方程
108 time_stt = clock();
109 x = matrix_NN.ldlt().solve(v_Nd);
```

```
  cout << "time of ldlt decomposition is "
       << 1000 * (clock() - time_stt) / (double) CLOCKS_PER_SEC << "ms" << endl;
  cout << "x = " << x.transpose() << endl;

  return 0;
}
```

这个例程演示了 Eigen 矩阵的基本操作与运算。要编译它，需要在 CMakeLists.txt 里指定 Eigen 的头文件目录：

slambook2/ch3/useEigen/CMakeLists.txt

```
# 添加头文件
include_directories( "/usr/include/eigen3" )
```

重复一遍，因为 Eigen 库只有头文件，所以不需要再用 target_link_libraries 语句将程序链接到库上。不过，对于其他大部分库，多数时候需要用到链接命令。这里的做法并不见得是最好的，因为其他人可能把 Eigen 安装在了不同位置，那么就必须手动修改这里的头文件目录。在之后的工作中，我们会使用 find_package 命令去搜索库，不过在本讲中暂时保持这个样子。编译好这个程序后，运行它，可以看到各矩阵的输出结果。

终端输入：

```
% build/eigenMatrix
matrix 2x3 from 1 to 6:
1 2 3
4 5 6
print matrix 2x3:
1 2 3
4 5 6
[1,2,3;4,5,6]*[3,2,1]=10 28
[1,2,3;4,5,6]*[4,5,6]: 32 77
random matrix:
0.680375   0.59688 -0.329554
-0.211234  0.823295  0.536459
0.566198 -0.604897 -0.444451
transpose:
0.680375 -0.211234  0.566198
0.59688  0.823295 -0.604897
-0.329554  0.536459 -0.444451
sum: 1.61307
trace: 1.05922
times 10:
6.80375   5.9688 -3.29554
-2.11234  8.23295  5.36459
5.66198 -6.04897 -4.44451
inverse:
-0.198521   2.22739    2.8357
1.00605 -0.555135  -1.41603
-1.62213   3.59308   3.28973
det: 0.208598
```

```
29 ……
```

由于在代码中给出了详细的注释，在此就不一一解释每行语句了。本书中，我们将仅给出几处重要地方的说明（后面的实践部分亦将保持这个风格）。

1. 读者最好亲手输入一遍上面的代码（不包括注释）。至少要编译运行一遍上面的程序。
2. Kdevelop 可能不会提示 C++ 成员运算，这是它做得不够完善导致的。请照着上面的内容输入即可，不必理会它是否提示错误。Clion 则会完整地给出提示。
3. Eigen 提供的矩阵和 MATLAB 很相似，几乎所有的数据都当作矩阵来处理。但是，为了实现更好的效率，在 Eigen 中需要指定矩阵的大小和类型。对于在编译时期就知道大小的矩阵，处理起来会比动态变化大小的矩阵更快一些。因此，像旋转矩阵、变换矩阵这样的数据，完全可在编译时期确定它们的大小和数据类型。
4. Eigen 内部的矩阵实现比较复杂，这里不做介绍，我们希望你像使用 float、double 等内置数据类型那样使用 Eigen 的矩阵。这应该是符合其设计初衷的。
5. Eigen 矩阵不支持自动类型提升，这和 C++ 的内建数据类型有较大差异。在 C++ 程序中，我们可以把一个 float 数据和 double 数据相加、相乘，**编译器会自动把数据类型转换为最合适的那种**。而在 Eigen 中，出于性能的考虑，必须**显式地**对矩阵类型进行转换。而如果忘了这样做，Eigen 会（不太友好地）提示你一个“YOU MIXED DIFFERENT NUMERIC TYPES ...”的编译错误。你可以尝试找一下这条信息出现在错误提示的哪个部分。如果错误信息太长最好保存到一个文件里再找。
6. 同理，在计算过程中也需要保证矩阵维数的正确性，否则会出现“YOU MIXED MATRICES OF DIFFERENT SIZES”错误。请不要抱怨这种错误提示方式，对于 C++ 模板元编程，能够提示出可以阅读的信息已经是很幸运的了。以后，若发现 Eigen 出错，你可以直接寻找大写的部分，推测出了什么问题。
7. 我们的例程只介绍了基本的矩阵运算。你可以阅读 Eigen 官网教程：`http://eigen.tuxfamily.org/dox-devel/modules.html`学习更多的 Eigen 知识。这里只演示了最简单的部分，能看懂演示程序不等于你已经能够熟练操作 Eigen。

最后一段代码中比较了求逆与求 QR 分解的运行效率，你可以看看自己机器上的时间差异，两种方法是否有明显的差异？

3.3 旋转向量和欧拉角

3.3.1 旋转向量

我们重新回到理论部分。有了旋转矩阵来描述旋转，有了变换矩阵描述一个 6 自由度的三维刚体运动，是不是已经足够了呢？矩阵表示方式至少有以下几个缺点：

1. SO(3) 的旋转矩阵有 9 个量，但一次旋转只有 3 个自由度。因此这种表达方式是冗余的。同

理，变换矩阵用 16 个量表达了 6 自由度的变换。那么，是否有更紧凑的表示呢？

2. 旋转矩阵自身带有约束：它必须是个正交矩阵，且行列式为 1。变换矩阵也是如此。当想要估计或优化一个旋转矩阵/变换矩阵时，这些约束会使得求解变得更困难。

因此，我们希望有一种方式能够紧凑地描述旋转和平移。例如，用一个三维向量表达旋转，用六维向量表达变换，可行吗？事实上，任意旋转都可以用**一个旋转轴和一个旋转角**来刻画。于是，我们可以使用一个向量，其方向与旋转轴一致，而长度等于旋转角。这种向量称为**旋转向量**（或轴角/角轴，Axis-Angle），只需一个三维向量即可描述旋转。同样，对于变换矩阵，我们使用一个旋转向量和一个平移向量即可表达一次变换。这时的变量维数正好是六维。

考虑某个用 $\boldsymbol{R}$ 表示的旋转。如果用旋转向量来描述，假设旋转轴为一个单位长度的向量 $\boldsymbol{n}$，角度为 θ，那么向量 $\theta\boldsymbol{n}$ 也可以描述这个旋转。于是，我们要问，两种表达方式之间有什么联系吗？事实上推导它们的转换关系并不难。从旋转向量到旋转矩阵的转换过程由**罗德里格斯公式**（Rodrigues's Formula）表明，由于推导过程比较复杂，这里不作描述，只给出转换的结果[①]：

$$\boldsymbol{R} = \cos\theta\boldsymbol{I} + (1-\cos\theta)\,\boldsymbol{n}\boldsymbol{n}^{\mathrm{T}} + \sin\theta\boldsymbol{n}^{\wedge}. \tag{3.15}$$

符号 $^{\wedge}$ 是向量到反对称的转换符，见式(3.3)。反之，我们也可以计算从一个旋转矩阵到旋转向量的转换。对于转角 θ，取两边的**迹**[②]，有：

$$\begin{aligned}\operatorname{tr}(\boldsymbol{R}) &= \cos\theta\operatorname{tr}(\boldsymbol{I}) + (1-\cos\theta)\operatorname{tr}\left(\boldsymbol{n}\boldsymbol{n}^{\mathrm{T}}\right) + \sin\theta\operatorname{tr}(\boldsymbol{n}^{\wedge}) \\ &= 3\cos\theta + (1-\cos\theta) \\ &= 1 + 2\cos\theta.\end{aligned} \tag{3.16}$$

因此：

$$\theta = \arccos\left(\frac{\operatorname{tr}(\boldsymbol{R})-1}{2}\right). \tag{3.17}$$

关于转轴 $\boldsymbol{n}$，由于旋转轴上的向量在旋转后不发生改变，说明：

$$\boldsymbol{R}\boldsymbol{n} = \boldsymbol{n}. \tag{3.18}$$

因此，转轴 $\boldsymbol{n}$ 是矩阵 $\boldsymbol{R}$ 特征值 1 对应的特征向量。求解此方程，再归一化，就得到了旋转轴。读者也可以从“旋转轴经过旋转之后不变”的几何角度看待这个方程。顺便提一下，这里的两个转换公式在下一讲仍将出现，你会发现它们正是 SO(3) 上李群与李代数的对应关系。

3.3.2　欧拉角

下面来说说欧拉角。

[①] 感兴趣的读者请参见https://en.wikipedia.org/wiki/Rodrigues%27_rotation_formula，事实上下一章会从李代数层面给出一个证明。

[②] 求**迹**（trace）即是求矩阵的对角线元素之和。

无论是旋转矩阵、旋转向量，它们虽然能描述旋转，但对我们人类是非常不直观的。当我们看到一个旋转矩阵或旋转向量时，很难想象出这个旋转究竟是什么样的。当它们变换时，我们也不知道物体是向哪个方向在转动。而欧拉角则提供了一种非常直观的方式来描述旋转——它使用了**3 个分离的转角**，把一个旋转分解成 3 次绕不同轴的旋转。而人类很容易理解绕单个轴旋转的过程。但是，由于分解方式有许多种，所以欧拉角也存在着众多不同的、易于混淆的定义方法。比如说，先绕 X 轴旋转，再绕 Y 轴，最后绕 Z 轴，就得到了一个 XYZ 轴的旋转。同理，可以定义 ZYZ、ZYX 等旋转方式。如果讨论得更细一些，还需要区分每次是绕**固定轴**旋转的，还是绕**旋转之后的轴**旋转的，这也会给出不一样的定义方式。

这种定义方式上的不确定性带来了很多实用当中的困难，所幸在特定领域内，欧拉角通常有统一的定义方式。你或许在航空、航模中听说过“俯仰角”“偏航角”这些词。欧拉角当中比较常用的一种，便是用“偏航－俯仰－滚转”（yaw-pitch-roll）3 个角度来描述一个旋转。由于它等价于 ZYX 轴的旋转，因此就以 ZYX 为例。假设一个刚体的前方（朝向我们的方向）为 X 轴，右侧为 Y 轴，上方为 Z 轴，如图 3-2所示。那么，ZYX 转角相当于把任意旋转分解成以下 3 个轴上的转角：

1. 绕物体的 Z 轴旋转，得到偏航角 yaw；
2. 绕**旋转之后**的 Y 轴旋转，得到俯仰角 pitch；
3. 绕**旋转之后**的 X 轴旋转，得到滚转角 roll。

此时，可以使用 $[r, p, y]^{\mathrm{T}}$ 这样一个三维的向量描述任意旋转。这个向量十分直观，我们可以从这个向量想象出旋转的过程。其他的欧拉角亦是通过这种方式，把旋转分解到 3 个轴上，得到一个三维的向量，只不过选用的轴及顺序不一样。这里介绍的 rpy 角是比较常用的一种，只有很少的欧拉角种类会有 rpy 这样脍炙人口的名字。不同的欧拉角是按照旋转轴的顺序来称呼的。例如，rpy 角的旋转顺序是 ZYX。同样，也有 XYZ, ZYZ 这样的欧拉角——但是它们就没有专门的名字了。值得一提的是，大部分领域在使用欧拉角时都有各自的坐标方向和顺序上的习惯，不一定和我们这里说的相同。

欧拉角的一个重大缺点是会碰到著名的**万向锁问题**（Gimbal Lock[①]）：在俯仰角为 $\pm 90°$ 时，第一次旋转与第三次旋转将使用同一个轴，使得系统丢失了一个自由度（由 3 次旋转变成了 2 次旋转）。这被称为奇异性问题，在其他形式的欧拉角中也同样存在。理论上可以证明，只要想用 3 个实数来表达三维旋转时，都会不可避免地碰到奇异性问题[②]。由于这种原理，欧拉角不适于插值和迭代，往往只用于人机交互中。我们也很少在 SLAM 程序中直接使用欧拉角表达姿态，同样不会在滤波或优化中使用欧拉角表达旋转（因为它具有奇异性）。不过，若你想验证自己的算法是否有错，转换成欧拉角能够帮你快速分辨结果是否正确。在某些主体主要为 2D 运动的场合（例如扫地机、自动驾驶车辆），我们也可以把旋转分解成三个欧拉角，然后把其中一个（例如偏航角）拿出来作为定位信息输出。

---

① https://en.wikipedia.org/wiki/Gimbal_lock。

② 旋转向量也有奇异性，发生在转角 θ 超过 2π 而产生周期性时。

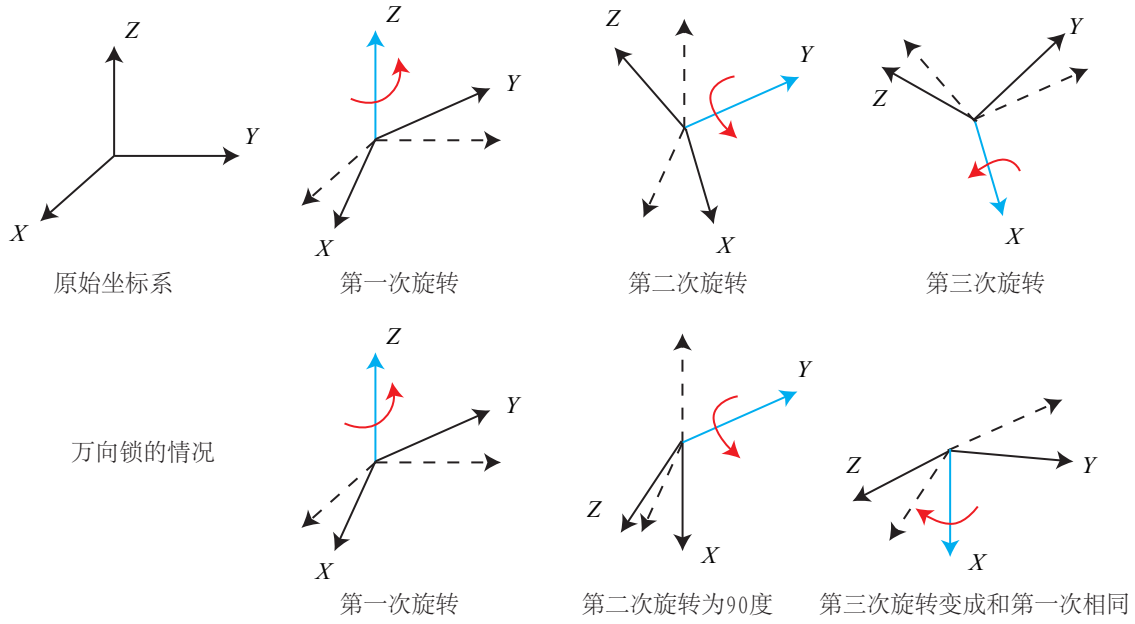


图 3-2　欧拉角的旋转示意图。上方为 ZYX 角定义。下方为 pitch=90° 时，第三次旋转与第一次滚转角相同，使得系统丢失了一个自由度。如果你还没有理解万向锁，可以看看相关视频，理解起来会更方便。

3.4　四元数

3.4.1　四元数的定义

旋转矩阵用 9 个量描述 3 自由度的旋转，具有冗余性；欧拉角和旋转向量是紧凑的，但具有奇异性。事实上，我们**找不到不带奇异性的三维向量描述方式**[19]。这有点类似于用两个坐标表示地球表面（如经度和纬度），将必定存在奇异性（纬度为 ±90° 时经度无意义）。

回忆以前学习过的复数。我们用复数集 $\mathbb{C}$ 表示复平面上的向量，而复数的乘法则表示复平面上的旋转：例如，乘上复数 i 相当于逆时针把一个复向量旋转 90°。类似地，在表达三维空间旋转时，也有一种类似于复数的代数：**四元数**（Quaternion）。四元数是 Hamilton 找到的一种扩展的复数。它**既是紧凑的，也没有奇异性**。如果说缺点，四元数不够直观，其运算稍复杂些。

把四元数与复数类比可以帮助你更快地理解四元数。例如，当我们想要将复平面的向量旋转 θ 角时，可以给这个复向量乘以 $\mathrm{e}^{i\theta}$。这是极坐标表示的复数，它也可以写成普通的形式，只要使用欧拉公式即可：

$$\mathrm{e}^{i\theta} = \cos\theta + i\sin\theta. \tag{3.19}$$

这正是一个单位长度的复数。所以，在二维情况下，旋转可以由**单位复数**来描述。类似地，我们会看到，三维旋转则可以由**单位四元数**来描述。

一个四元数 $\boldsymbol{q}$ 拥有一个实部和三个虚部。本书把实部写在前面（也有地方把实部写在后面），像下面这样：

$$\boldsymbol{q} = q_0 + q_1 i + q_2 j + q_3 k, \tag{3.20}$$

其中 i, j, k 为四元数的三个虚部。这三个虚部满足以下关系式：

$$\begin{cases} i^2 = j^2 = k^2 = -1 \\ ij = k, ji = -k \\ jk = i, kj = -i \\ ki = j, ik = -j \end{cases}. \tag{3.21}$$

如果把 i, j, k 看成三个坐标轴，那么它们与自己的乘法和复数一样，相互之间的乘法和外积一样。有时人们也用一个标量和一个向量来表达四元数：

$$\boldsymbol{q} = [s, \boldsymbol{v}]^{\mathrm{T}}, \quad s = q_0 \in \mathbb{R}, \quad \boldsymbol{v} = [q_1, q_2, q_3]^{\mathrm{T}} \in \mathbb{R}^3,$$

这里，s 称为四元数的实部，而 $\boldsymbol{v}$ 称为它的虚部。如果一个四元数的虚部为 $\mathbf{0}$，称之为**实四元数**。反之，若它的实部为 0，则称之为**虚四元数**。

可以用**单位四元数**表示三维空间中任意一个旋转，不过这种表达方式和复数有着微妙的不同。在复数中，乘以 i 意味着旋转 90°。这是否意味着四元数中，乘 i 就是绕 i 轴旋转 90°？那么，$ij = k$ 是否意味着，先绕 i 转 90°，再绕 j 转 90°，就等于绕 k 转 90°？读者可以找一部手机比划一下——然后你会发现情况并不是这样。正确的情形应该是，乘以 i 对应着旋转 180°，这样才能保证 $ij = k$ 的性质。而 $i^2 = -1$，意味着绕 i 轴旋转 360° 后得到一个相反的东西。这个东西要旋转两周才会和它原先的样子相等。

这似乎有些玄妙了，完整的解释需要引入太多额外的东西，我们还是冷静一下回到眼前。至少，我们知道单位四元数能够表达三维空间的旋转。那么四元数本身有些什么性质，它们互相之间又可以做哪些运算呢？下面我们先考察四元数之间的运算法则。

3.4.2 四元数的运算

四元数和通常复数一样，可以进行一系列的运算。常见的有四则运算、数乘、求逆、共轭等。下面分别介绍。

现有两个四元数 $\boldsymbol{q}_a, \boldsymbol{q}_b$，它们的向量表示为 $[s_a, \boldsymbol{v}_a]^{\mathrm{T}}, [s_b, \boldsymbol{v}_b]^{\mathrm{T}}$，或者原始四元数表示为：

$$\boldsymbol{q}_a = s_a + x_a i + y_a j + z_a k, \quad \boldsymbol{q}_b = s_b + x_b i + y_b j + z_b k.$$

那么，其运算可表示如下。

1. 加法和减法

   四元数 $\boldsymbol{q}_a, \boldsymbol{q}_b$ 的加减运算为：

$$\boldsymbol{q}_a \pm \boldsymbol{q}_b = [s_a \pm s_b, \boldsymbol{v}_a \pm \boldsymbol{v}_b]^{\mathrm{T}}. \tag{3.22}$$

2. 乘法

   乘法是把 $\boldsymbol{q}_a$ 的每一项与 $\boldsymbol{q}_b$ 的每项相乘，最后相加，虚部要按照式(3.21)进行。整理可得：

$$\begin{aligned} \boldsymbol{q}_a \boldsymbol{q}_b = & s_a s_b - x_a x_b - y_a y_b - z_a z_b \\ & + (s_a x_b + x_a s_b + y_a z_b - z_a y_b) i \\ & + (s_a y_b - x_a z_b + y_a s_b + z_a x_b) j \\ & + (s_a z_b + x_a y_b - y_a x_b + z_a s_b) k. \end{aligned} \tag{3.23}$$

   虽然稍为复杂，但形式上是整齐有序的。如果写成向量形式并利用内外积运算，该表达会更加简洁：

$$\boldsymbol{q}_a \boldsymbol{q}_b = \left[s_a s_b - \boldsymbol{v}_a^{\mathrm{T}} \boldsymbol{v}_b, s_a \boldsymbol{v}_b + s_b \boldsymbol{v}_a + \boldsymbol{v}_a \times \boldsymbol{v}_b \right]^{\mathrm{T}}. \tag{3.24}$$

   在该乘法定义下，两个实的四元数乘积仍是实的，这与复数也是一致的。然而，注意到，由于最后一项外积的存在，四元数乘法通常是不可交换的，除非 $\boldsymbol{v}_a$ 和 $\boldsymbol{v}_b$ 在 $\mathbb{R}^3$ 中共线，此时外积项为零。

3. 模长

   四元数的模长定义为

$$\|\boldsymbol{q}_a\| = \sqrt{s_a^2 + x_a^2 + y_a^2 + z_a^2}. \tag{3.25}$$

   可以验证，两个四元数乘积的模即为模的乘积。这使得单位四元数相乘后仍是单位四元数。

$$\|\boldsymbol{q}_a \boldsymbol{q}_b\| = \|\boldsymbol{q}_a\| \|\boldsymbol{q}_b\|. \tag{3.26}$$

4. 共轭

   四元数的共轭是把虚部取成相反数：

$$\boldsymbol{q}_a^* = s_a - x_a i - y_a j - z_a k = [s_a, -\boldsymbol{v}_a]^{\mathrm{T}}. \tag{3.27}$$

   四元数共轭与其本身相乘，会得到一个实四元数，其实部为模长的平方：

$$\boldsymbol{q}^* \boldsymbol{q} = \boldsymbol{q} \boldsymbol{q}^* = [s_a^2 + \boldsymbol{v}^{\mathrm{T}} \boldsymbol{v}, \mathbf{0}]^{\mathrm{T}}. \tag{3.28}$$

5. 逆

   一个四元数的逆为

$$\boldsymbol{q}^{-1} = \boldsymbol{q}^* / \|\boldsymbol{q}\|^2. \tag{3.29}$$

按此定义，四元数和自己的逆的乘积为实四元数 $\mathbf{1}$：

$$\boldsymbol{q}\boldsymbol{q}^{-1} = \boldsymbol{q}^{-1}\boldsymbol{q} = \mathbf{1}. \tag{3.30}$$

如果 $\boldsymbol{q}$ 为单位四元数，其逆和共轭就是同一个量。同时，乘积的逆有和矩阵相似的性质：

$$(\boldsymbol{q}_a\boldsymbol{q}_b)^{-1} = \boldsymbol{q}_b^{-1}\boldsymbol{q}_a^{-1}. \tag{3.31}$$

6. 数乘

和向量相似，四元数可以与数相乘：

$$k\boldsymbol{q} = [ks, k\boldsymbol{v}]^{\mathrm{T}}. \tag{3.32}$$

3.4.3 用四元数表示旋转

我们可以用四元数表达对一个点的旋转。假设一个空间三维点 $\boldsymbol{p} = [x, y, z] \in \mathbb{R}^3$，以及一个由单位四元数 $\boldsymbol{q}$ 指定的旋转。三维点 $\boldsymbol{p}$ 经过旋转之后变为 $\boldsymbol{p}'$。如果使用矩阵描述，那么有 $\boldsymbol{p}' = \boldsymbol{R}\boldsymbol{p}$。而如果用四元数描述旋转，它们的关系又如何来表达呢？

首先，把三维空间点用一个虚四元数来描述：

$$\boldsymbol{p} = [0, x, y, z]^{\mathrm{T}} = [0, \boldsymbol{v}]^{\mathrm{T}}.$$

相当于把四元数的 3 个虚部与空间中的 3 个轴相对应。那么，旋转后的点 $\boldsymbol{p}'$ 即可表示为这样的乘积：

$$\boldsymbol{p}' = \boldsymbol{q}\boldsymbol{p}\boldsymbol{q}^{-1}. \tag{3.33}$$

这里的乘法均为四元数乘法，结果也是四元数。最后把 $\boldsymbol{p}'$ 的虚部取出，即得旋转之后点的坐标。并且，可以验证（留作习题），计算结果的实部为 0，故为纯虚四元数。

3.4.4 四元数到其他旋转表示的转换

任意单位四元数描述了一个旋转，该旋转亦可用旋转矩阵或旋转向量描述。现在来考察四元数与旋转向量、旋转矩阵之间的转换关系。在此之前，我们要说，四元数乘法也可以写成一种矩阵的乘法。设 $\boldsymbol{q} = [s, \boldsymbol{v}]^{\mathrm{T}}$，那么，定义如下的符号 $^+$ 和 $^\oplus$ 为[20]：

$$\boldsymbol{q}^+ = \begin{bmatrix} s & -\boldsymbol{v}^{\mathrm{T}} \\ \boldsymbol{v} & s\boldsymbol{I} + \boldsymbol{v}^\wedge \end{bmatrix}, \quad \boldsymbol{q}^\oplus = \begin{bmatrix} s & -\boldsymbol{v}^{\mathrm{T}} \\ \boldsymbol{v} & s\boldsymbol{I} - \boldsymbol{v}^\wedge \end{bmatrix}, \tag{3.34}$$

这两个符号将四元数映射成为一个 4×4 的矩阵。于是四元数乘法可以写成矩阵的形式：

$$\boldsymbol{q}_1^+\boldsymbol{q}_2 = \begin{bmatrix} s_1 & -\boldsymbol{v}_1^T \\ \boldsymbol{v}_1 & s_1\boldsymbol{I} + \boldsymbol{v}_1^\wedge \end{bmatrix} \begin{bmatrix} s_2 \\ \boldsymbol{v}_2 \end{bmatrix} = \begin{bmatrix} -\boldsymbol{v}_1^T\boldsymbol{v}_2 + s_1 s_2 \\ s_1\boldsymbol{v}_2 + s_2\boldsymbol{v}_1 + \boldsymbol{v}_1^\wedge\boldsymbol{v}_2 \end{bmatrix} = \boldsymbol{q}_1\boldsymbol{q}_2 \tag{3.35}$$

同理亦可证：

$$\boldsymbol{q}_1\boldsymbol{q}_2 = \boldsymbol{q}_1^{+}\boldsymbol{q}_2 = \boldsymbol{q}_2^{\oplus}\boldsymbol{q}_1. \tag{3.36}$$

然后，考虑使用四元数对空间点进行旋转的问题。根据前面的说法，有：

$$\begin{aligned} \boldsymbol{p}' &= \boldsymbol{q}\boldsymbol{p}\boldsymbol{q}^{-1} = \boldsymbol{q}^{+}\boldsymbol{p}^{+}\boldsymbol{q}^{-1} \\ &= \boldsymbol{q}^{+}\boldsymbol{q}^{-1^{\oplus}}\boldsymbol{p}. \end{aligned} \tag{3.37}$$

代入两个符号对应的矩阵，得：

$$\boldsymbol{q}^{+}\left(\boldsymbol{q}^{-1}\right)^{\oplus} = \begin{bmatrix} s & -\boldsymbol{v}^T \\ \boldsymbol{v} & s\boldsymbol{I} + \boldsymbol{v}^{\wedge} \end{bmatrix} \begin{bmatrix} s & \boldsymbol{v}^T \\ -\boldsymbol{v} & s\boldsymbol{I} + \boldsymbol{v}^{\wedge} \end{bmatrix} = \begin{bmatrix} 1 & \boldsymbol{0} \\ \boldsymbol{0}^{\mathrm{T}} & \boldsymbol{v}\boldsymbol{v}^{\mathrm{T}} + s^2\boldsymbol{I} + 2s\boldsymbol{v}^{\wedge} + \left(\boldsymbol{v}^{\wedge}\right)^2 \end{bmatrix}. \tag{3.38}$$

因为 $\boldsymbol{p}'$ 和 $\boldsymbol{p}$ 都是虚四元数，那么事实上该矩阵的右下角即给出了**从四元数到旋转矩阵**的变换关系：

$$\boldsymbol{R} = \boldsymbol{v}\boldsymbol{v}^{\mathrm{T}} + s^2\boldsymbol{I} + 2s\boldsymbol{v}^{\wedge} + \left(\boldsymbol{v}^{\wedge}\right)^2. \tag{3.39}$$

为了得到四元数到旋转向量的转换公式，对上式两侧求迹，得：

$$\begin{aligned} \mathrm{tr}(\boldsymbol{R}) &= \mathrm{tr}(\boldsymbol{v}\boldsymbol{v}^{\mathrm{T}} + 3s^2 + 2s\cdot 0 + \mathrm{tr}((\boldsymbol{v}^{\wedge})^2) \\ &= v_1^2 + v_2^2 + v_3 s^2 + 3s^2 - 2(v_1^2 + v_2^2 + v_3^2) \\ &= (1 - s^2) + 3s^2 - 2(1 - s^2) \\ &= 4s^2 - 1. \end{aligned} \tag{3.40}$$

又由式(3.17)得：

$$\begin{aligned} \theta &= \arccos\left(\frac{\mathrm{tr}(\boldsymbol{R} - 1)}{2}\right) \\ &= \arccos(2s^2 - 1). \end{aligned} \tag{3.41}$$

即

$$\cos\theta = 2s^2 - 1 = 2\cos^2\frac{\theta}{2} - 1, \tag{3.42}$$

所以：

$$\theta = 2\arccos s. \tag{3.43}$$

至于旋转轴，如果在式(3.38)中用 $\boldsymbol{q}$ 的虚部代替 $\boldsymbol{p}$，易知 $\boldsymbol{q}$ 的虚部组成的向量在旋转时是不动的，即构成旋转轴。于是只要将它除掉它的模长，即得。总而言之，四元数到旋转向量的转换公式可列写如下：

$$\begin{cases} \theta = 2\arccos q_0 \\ [n_x, n_y, n_z]^{\mathrm{T}} = [q_1, q_2, q_3]^{\mathrm{T}} / \sin\frac{\theta}{2} \end{cases}. \tag{3.44}$$

至于如何从其他方式转换到四元数，只须把上述步骤倒过来处理即可。在实际编程中，程序库通常会为我们准备好各种形式之间的转换。无论是四元数、旋转矩阵还是轴角，它们都可以用来描述同一个旋转。我们应该在实际中选择最为方便的形式，而不必拘泥于某种特定的形式。在随后的实践和习题中，我们会演示各种表达方式之间的转换，以加深读者的印象。

3.5 * 相似、仿射、射影变换

除了欧氏变换之外，3D 空间还存在其他几种变换方式，只不过欧氏变换是最简单的。它们一部分和测量几何有关，因为在之后的讲解中可能会提到，所以先罗列出来。欧氏变换保持了向量的长度和夹角，相当于我们把一个刚体原封不动地进行了移动或旋转，不改变它自身的样子。其他几种变换则会改变它的外形。它们都拥有类似的矩阵表示。

1. 相似变换

   相似变换比欧氏变换多了一个自由度，它允许物体进行均匀缩放，其矩阵表示为

$$\boldsymbol{T}_S = \begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}. \tag{3.45}$$

   注意到旋转部分多了一个缩放因子 s，表示我们在对向量旋转之后，可以在 x, y, z 三个坐标上进行均匀缩放。由于含有缩放，相似变换不再保持图形的面积不变。你可以想象一个边长为 1 的立方体通过相似变换后，变成边长为 10 的样子（但仍然是立方体）。三维相似变换的集合也叫做**相似变换群**，记作 $\mathrm{Sim}(3)$。

2. 仿射变换

   仿射变换的矩阵形式如下：

$$\boldsymbol{T}_A = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}. \tag{3.46}$$

   与欧氏变换不同的是，仿射变换只要求 $\boldsymbol{A}$ 是一个可逆矩阵，而不必是正交矩阵。仿射变换也叫正交投影。经过仿射变换之后，立方体就不再是方的了，但是各个面仍然是平行四边形。

3. 射影变换

   射影变换是最一般的变换，它的矩阵形式为

$$\boldsymbol{T}_P = \begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{a}^{\mathrm{T}} & v \end{bmatrix}. \tag{3.47}$$

   它的左上角为可逆矩阵 $\boldsymbol{A}$，右上角为平移 $\boldsymbol{t}$，左下角为缩放 $\boldsymbol{a}^{\mathrm{T}}$。由于采用了齐次坐标，当 $v \neq 0$ 时，我们可以对整个矩阵除以 v 得到一个右下角为 1 的矩阵；否则得到右下角为 0 的矩阵。因此，2D 的射影变换一共有 8 个自由度，3D 则共有 15 个自由度。射影变换是现在讲过的变换中，形式最为一般的。从真实世界到相机照片的变换可以看成一个射影变换。读者可以想象一个原本方形的地板砖，在照片当中是什么样子：首先，它不再是方形的。由于近大远小的关系，它甚至不是平行四边形，而是一个不规则的四边形。

表 3-1总结了目前讲到的几种变换的性质。注意在“不变性质”中，从上到下是有包含关系的。例如，欧氏变换除了保体积之外，也具有保平行、相交等性质。

表 3-1　常见变换性质比较

变换名称	矩阵形式	自由度	不变性质
欧氏变换	$\begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}$	6	长度、夹角、体积
相似变换	$\begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}$	7	体积比
仿射变换	$\begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix}$	12	平行性、体积比
射影变换	$\begin{bmatrix} \boldsymbol{A} & \boldsymbol{t} \\ \boldsymbol{a}^{\mathrm{T}} & v \end{bmatrix}$	15	接触平面的相交和相切

我们之后会说到，从真实世界到相机照片的变换是一个射影变换。如果相机的焦距为无穷远，那么这个变换为仿射变换。不过，在详细讲述相机模型之前，我们只要对它们有个大致的印象即可。

3.6　实践：Eigen 几何模块

3.6.1　Eigen 几何模块的数据演示

现在，我们来实际演练一下前面讲到的各种旋转表达方式。我们将在 Eigen 中使用四元数、欧拉角和旋转矩阵，演示它们之间的变换方式。我们还会给出一个可视化程序，帮助读者理解这几个变换的关系。

slambook2/ch3/useGeometry/useGeometry.cpp

```
#include <iostream>
#include <cmath>
using namespace std;

#include <Eigen/Core>
#include <Eigen/Geometry>

using namespace Eigen;
// 本程序演示了 Eigen 几何模块的使用方法

int main(int argc, char **argv) {
    // Eigen/Geometry 模块提供了各种旋转和平移的表示
    // 3D 旋转矩阵直接使用 Matrix3d 或 Matrix3f
    Matrix3d rotation_matrix = Matrix3d::Identity();
    // 旋转向量使用 AngleAxis, 它底层不直接是Matrix，但运算可以当作矩阵（因为重载了运算符）
```

```
    AngleAxisd rotation_vector(M_PI / 4, Vector3d(0, 0, 1));     //沿 Z 轴旋转 45 度
    cout.precision(3);
    cout << "rotation matrix =\n" << rotation_vector.matrix() << endl;   //用matrix()转换成矩阵
    // 也可以直接赋值
    rotation_matrix = rotation_vector.toRotationMatrix();
    // 用 AngleAxis 可以进行坐标变换
    Vector3d v(1, 0, 0);
    Vector3d v_rotated = rotation_vector * v;
    cout << "(1,0,0) after rotation (by angle axis) = " << v_rotated.transpose() << endl;
    // 或者用旋转矩阵
    v_rotated = rotation_matrix * v;
    cout << "(1,0,0) after rotation (by matrix) = " << v_rotated.transpose() << endl;

    // 欧拉角: 可以将旋转矩阵直接转换成欧拉角
    Vector3d euler_angles = rotation_matrix.eulerAngles(2, 1, 0); // ZYX顺序，即roll pitch yaw顺序
    cout << "yaw pitch roll = " << euler_angles.transpose() << endl;

    // 欧氏变换矩阵使用 Eigen::Isometry
    Isometry3d T = Isometry3d::Identity();       // 虽然称为3d，实质上是4*4的矩阵
    T.rotate(rotation_vector);                   // 按照rotation_vector进行旋转
    T.pretranslate(Vector3d(1, 3, 4));           // 把平移向量设成(1,3,4)
    cout << "Transform matrix = \n" << T.matrix() << endl;

    // 用变换矩阵进行坐标变换
    Vector3d v_transformed = T * v;                              // 相当于R*v+t
    cout << "v tranformed = " << v_transformed.transpose() << endl;

    // 对于仿射和射影变换，使用 Eigen::Affine3d 和 Eigen::Projective3d 即可，略

    // 四元数
    // 可以直接把AngleAxis赋值给四元数，反之亦然
    Quaterniond q = Quaterniond(rotation_vector);
    cout << "quaternion from rotation vector = " << q.coeffs().transpose()
    << endl;   // 请注意coeffs的顺序是(x,y,z,w),w为实部，前三者为虚部
    // 也可以把旋转矩阵赋给它
    q = Quaterniond(rotation_matrix);
    cout << "quaternion from rotation matrix = " << q.coeffs().transpose() << endl;
    // 使用四元数旋转一个向量，使用重载的乘法即可
    v_rotated = q * v; // 注意数学上是qvq^{-1}
    cout << "(1,0,0) after rotation = " << v_rotated.transpose() << endl;
    // 用常规向量乘法表示，则应该如下计算
    cout << "should be equal to " << (q * Quaterniond(0, 1, 0, 0) * q.inverse()).coeffs().transpose()
         << endl;

    return 0;
}
```

Eigen 中对各种形式的表达方式总结如下。请注意每种类型都有单精度和双精度两种数据类型，而且和之前一样，不能由编译器自动转换。下面以双精度为例，你可以把最后的 d 改成 f，即得到单精度的数据结构。

- 旋转矩阵（3×3）：Eigen::Matrix3d。

- 旋转向量（3×1）：Eigen::AngleAxisd。
- 欧拉角（3×1）：Eigen::Vector3d。
- 四元数（4×1）：Eigen::Quaterniond。
- 欧氏变换矩阵（4×4）：Eigen::Isometry3d。
- 仿射变换（4×4）：Eigen::Affine3d。
- 射影变换（4×4）：Eigen::Projective3d。

参考代码中对应的 CMakeLists 即可编译此程序。在这个程序中，演示了如何使用 Eigen 中的旋转矩阵、旋转向量（AngleAxis）、欧拉角和四元数。我们用这几种旋转方式去旋转一个向量 $\boldsymbol{v}$，发现结果是一样的（不一样就真是见鬼了）。同时，也演示了如何在程序中转换这几种表达方式。想进一步了解 Eigen 的几何模块的读者可以参考（`http://eigen.tuxfamily.org/dox/group__TutorialGeometry.html`）。

请读者注意，**程序代码通过和数学表示有一些细微的差别**。例如，通过运算符重载，四元数和三维向量可以直接计算乘法，但在数学上则需要先把向量转成虚四元数，再利用四元数乘法进行计算，同样的情况也适用于变换矩阵乘三维向量的情况。总体而言，程序中的用法会比数学公式更灵活一些。

3.6.2　实际的坐标变换例子

下面我们举一个小例子来演示坐标变换。

例子　设有小萝卜一号和小萝卜二号位于世界坐标系中。记世界坐标系为 W，小萝卜们的坐标系为 R_1 和 R_2。小萝卜一号的位姿为 $\boldsymbol{q}_1 = [0.35, 0.2, 0.3, 0.1]^{\mathrm{T}}, \boldsymbol{t}_1 = [0.3, 0.1, 0.1]^{\mathrm{T}}$。小萝卜二号的位姿为 $\boldsymbol{q}_2 = [-0.5, 0.4, -0.1, 0.2]^{\mathrm{T}}, \boldsymbol{t}_2 = [-0.1, 0.5, 0.3]^{\mathrm{T}}$。这里的 $\boldsymbol{q}$ 和 $\boldsymbol{t}$ 表达的是 $\boldsymbol{T}_{R_k,W}, k = 1, 2$，也就是世界坐标系到相机坐标系的变换关系。现在，小萝卜一号看到某个点在自身的坐标系下坐标为 $\boldsymbol{p}_{R_1} = [0.5, 0, 0.2]^{\mathrm{T}}$，求该向量在小萝卜二号坐标系下的坐标。

这是一个非常简单，但又具有代表性的例子。在实际场景中你经常需要在同一个机器人的不同部分，或者不同机器人之间转换坐标。下面我们书写一段程序来演示这个计算。

slambook2/ch3/examples/coordinateTransform.cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <Eigen/Core>
#include <Eigen/Geometry>

using namespace std;
using namespace Eigen;

int main(int argc, char** argv) {
    Quaterniond q1(0.35, 0.2, 0.3, 0.1), q2(-0.5, 0.4, -0.1, 0.2);
    q1.normalize();
```

```
    q2.normalize();
    Vector3d t1(0.3, 0.1, 0.1), t2(-0.1, 0.5, 0.3);
    Vector3d p1(0.5, 0, 0.2);

    Isometry3d T1w(q1), T2w(q2);
    T1w.pretranslate(t1);
    T2w.pretranslate(t2);

    Vector3d p2 = T2w * T1w.inverse() * p1;
    cout << endl << p2.transpose() << endl;
    return 0;
}
```

程序输出的答案是 $[-0.0309731, 0.73499, 0.296108]^{\mathrm{T}}$，计算过程也十分简单，只需计算

$$\boldsymbol{p}_{R_2} = \boldsymbol{T}_{R_2,W}\boldsymbol{T}_{W,R_1}\boldsymbol{p}_{R_1}$$

即可。注意四元数使用之前需要归一化。

3.7 可视化演示

3.7.1 显示运动轨迹

如果你是第一次接触旋转和平移这些概念，可能会觉得它们形式看起来很复杂，因为毕竟每种表达方式都可以与其他方式互相转换，而转换公式有时还比较长。不过，虽然旋转矩阵、变换矩阵的数值可能不够直观，但我们可以很容易地把它们画在窗口里面。

本节我们演示两个可视化例子。首先，假设我们通过某种方式记录了一个机器人的运动轨迹，现在想把它画到一个窗口中。假设轨迹文件存储于 trajectory.txt，每一行用下面的格式存储：

$$\text{time}, t_x, t_y, t_z, q_x, q_y, q_z, q_w,$$

其中 time 指该位姿的记录时间，$\boldsymbol{t}$ 为平移，$\boldsymbol{q}$ 为旋转四元数，均是以世界坐标系到机器人坐标系记录。下面我们从文件中读取这些轨迹，并显示到一个窗口中。原则上，如果只是谈论“机器人的位姿”，那么你可以使用 $\boldsymbol{T}_{WR}$ 或者 $\boldsymbol{T}_{RW}$，事实上它们也只差一个逆而已，意味着知道其中一个就可以很轻松地得到另一个。如果你想要存储**机器人的轨迹**，那么你可以存储所有时刻的 $\boldsymbol{T}_{WR}$ 或者 $\boldsymbol{T}_{RW}$，这并没有太大的差别。

在画轨迹的时候，我们可以把“轨迹”画成一系列点组成的序列，这和我们想象中的“轨迹”比较相似。严格说来，这其实是**机器人（相机）坐标系的原点在世界坐标系中的坐标**。考虑机器人坐标系的原点，即 $\boldsymbol{O}_R$，那么，此时的 $\boldsymbol{O}_W$ 就是这个原点在世界坐标系下的坐标：

$$\boldsymbol{O}_W = \boldsymbol{T}_{WR}\boldsymbol{O}_R = \boldsymbol{t}_{WR}. \tag{3.48}$$

这正是 $\boldsymbol{T}_{WR}$ 的平移部分。因此，可以从 $\boldsymbol{T}_{WR}$ 中直接看到**相机在何处**，这也是我们说 $\boldsymbol{T}_{WR}$ 更为直观的原因。因此，在可视化程序里，轨迹文件存储了 $\boldsymbol{T}_{WR}$ 而不是 $\boldsymbol{T}_{RW}$。

最后，我们需要一个支持 3D 绘图的程序库。有许多库都支持 3D 绘图，比如大家熟悉的 matlab，python 的 matplotlib，OpenGL 等。在 linux 中，一个常见的库是基于 OpenGL 的 Pangolin 库[①]，它在支持 OpenGL 的绘图操作基础之上还提供了一些 GUI 的功能。在第二版的书籍中，我们使用 git 的 submodule 功能来管理本书依赖的第三方库。读者可以进入 3rdparty 文件夹中直接安装所需的库，git 保证了我和你使用的版本是一致的。

slambook2/ch3/examples/plotTrajectory.cpp

```cpp
#include <pangolin/pangolin.h>
#include <Eigen/Core>
#include <unistd.h>

using namespace std;
using namespace Eigen;

// path to trajectory file
string trajectory_file = "./examples/trajectory.txt";

void DrawTrajectory(vector<Isometry3d, Eigen::aligned_allocator<Isometry3d>>);

int main(int argc, char **argv) {
    vector<Isometry3d, Eigen::aligned_allocator<Isometry3d>> poses;
    ifstream fin(trajectory_file);
    if (!fin) {
        cout << "cannot find trajectory file at " << trajectory_file << endl;
        return 1;
    }

    while (!fin.eof()) {
        double time, tx, ty, tz, qx, qy, qz, qw;
        fin >> time >> tx >> ty >> tz >> qx >> qy >> qz >> qw;
        Isometry3d Twr(Quaterniond(qw, qx, qy, qz));
        Twr.pretranslate(Vector3d(tx, ty, tz));
        poses.push_back(Twr);
    }
    cout << "read total " << poses.size() << " pose entries" << endl;

    // draw trajectory in pangolin
    DrawTrajectory(poses);
    return 0;
}

void DrawTrajectory(vector<Isometry3d, Eigen::aligned_allocator<Isometry3d>> poses) {
    // create pangolin window and plot the trajectory
    pangolin::CreateWindowAndBind("Trajectory Viewer", 1024, 768);
    glEnable(GL_DEPTH_TEST);
```

[①] https://github.com/stevenlovegrove/Pangolin

```
    glEnable(GL_BLEND);
    glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);

    pangolin::OpenGlRenderState s_cam(
    pangolin::ProjectionMatrix(1024, 768, 500, 500, 512, 389, 0.1, 1000),
    pangolin::ModelViewLookAt(0, -0.1, -1.8, 0, 0, 0, 0.0, -1.0, 0.0)
    );

    pangolin::View &d_cam = pangolin::CreateDisplay()
        .SetBounds(0.0, 1.0, 0.0, 1.0, -1024.0f / 768.0f)
        .SetHandler(new pangolin::Handler3D(s_cam));

    while (pangolin::ShouldQuit() == false) {
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        d_cam.Activate(s_cam);
        glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
        glLineWidth(2);
        for (size_t i = 0; i < poses.size(); i++) {
            // 画每个位姿的三个坐标轴
            Vector3d Ow = poses[i].translation();
            Vector3d Xw = poses[i] * (0.1 * Vector3d(1, 0, 0));
            Vector3d Yw = poses[i] * (0.1 * Vector3d(0, 1, 0));
            Vector3d Zw = poses[i] * (0.1 * Vector3d(0, 0, 1));
            glBegin(GL_LINES);
            glColor3f(1.0, 0.0, 0.0);
            glVertex3d(Ow[0], Ow[1], Ow[2]);
            glVertex3d(Xw[0], Xw[1], Xw[2]);
            glColor3f(0.0, 1.0, 0.0);
            glVertex3d(Ow[0], Ow[1], Ow[2]);
            glVertex3d(Yw[0], Yw[1], Yw[2]);
            glColor3f(0.0, 0.0, 1.0);
            glVertex3d(Ow[0], Ow[1], Ow[2]);
            glVertex3d(Zw[0], Zw[1], Zw[2]);
            glEnd();
        }
        // 画出连线
        for (size_t i = 0; i < poses.size(); i++) {
            glColor3f(0.0, 0.0, 0.0);
            glBegin(GL_LINES);
            auto p1 = poses[i], p2 = poses[i + 1];
            glVertex3d(p1.translation()[0], p1.translation()[1], p1.translation()[2]);
            glVertex3d(p2.translation()[0], p2.translation()[1], p2.translation()[2]);
            glEnd();
        }
        pangolin::FinishFrame();
        usleep(5000);   // sleep 5 ms
    }
}
```

该程序演示了如何在 Panglin 中画出 3D 的位姿。我们用红、绿、蓝三种颜色画出每个位姿的三个坐标轴（实际上我们计算了各坐标轴的世界坐标），然后用黑色线将轨迹连起来。程序运行结果如图 3-3所示。

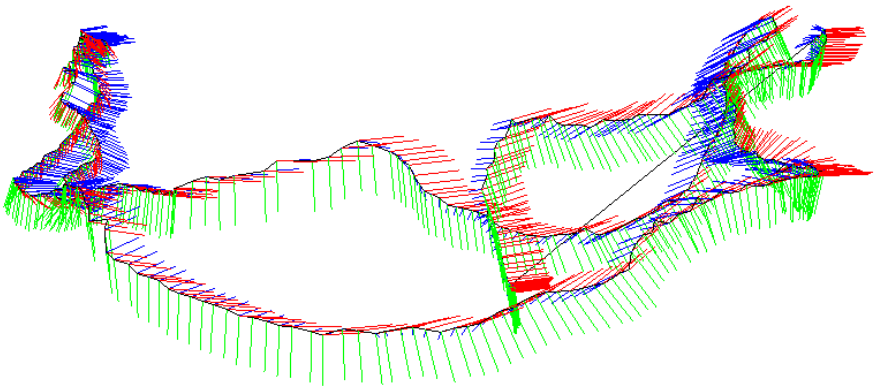


图 3-3　位姿可视化的结果

3.7.2　显示相机的位姿

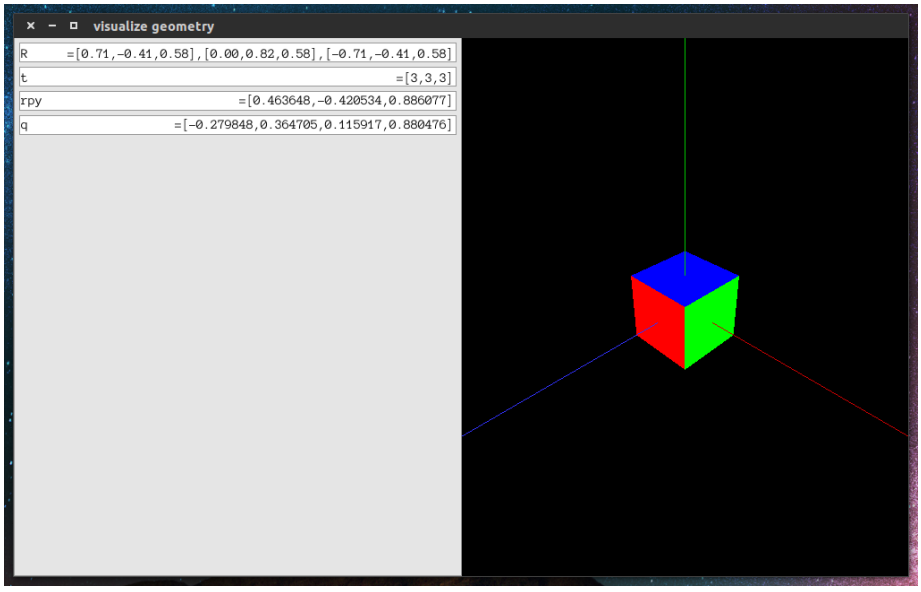


图 3-4　旋转矩阵、欧拉角、四元数的可视化程序。

除了显示轨迹之外，我们也可以显示 3D 窗口中相机的位姿。在 slambook2/ch3/visualizeGeometry 中，我们以可视化的形式演示了相机位姿的各种表达方式（见图 3-4）。当读者用鼠标操作相机时，左侧的方框里会实时显示相机位姿对应的旋转矩阵、平移、欧拉角和四元数，你可以看看数据是如何变化的。根据我们的经验，除了欧拉角之外，你应该看不出它们直观的含义。然而，尽管旋转矩

阵或变换矩阵并不直观，但是将它们可视化地显示出来并没有什么困难。该程序使用 Pangolin 库作为 3D 显示库，请参考 Readme.txt 来编译该程序。

习题

1. 验证旋转矩阵是正交矩阵。
2.* 寻找罗德里格斯公式的推导过程并加以理解。
3. 验证四元数旋转某个点后，结果是一个虚四元数（实部为零），所以仍然对应到一个三维空间点，见式(3.33)。
4. 画表总结旋转矩阵、轴角、欧拉角、四元数的转换关系。
5. 假设有一个大的 Eigen 矩阵，想把它的左上角 3×3 的块取出来，然后赋值为 $\boldsymbol{I}_{3\times 3}$。请编程实现。
6.* 一般线性方程 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 有哪几种做法？你能在 Eigen 中实现吗？

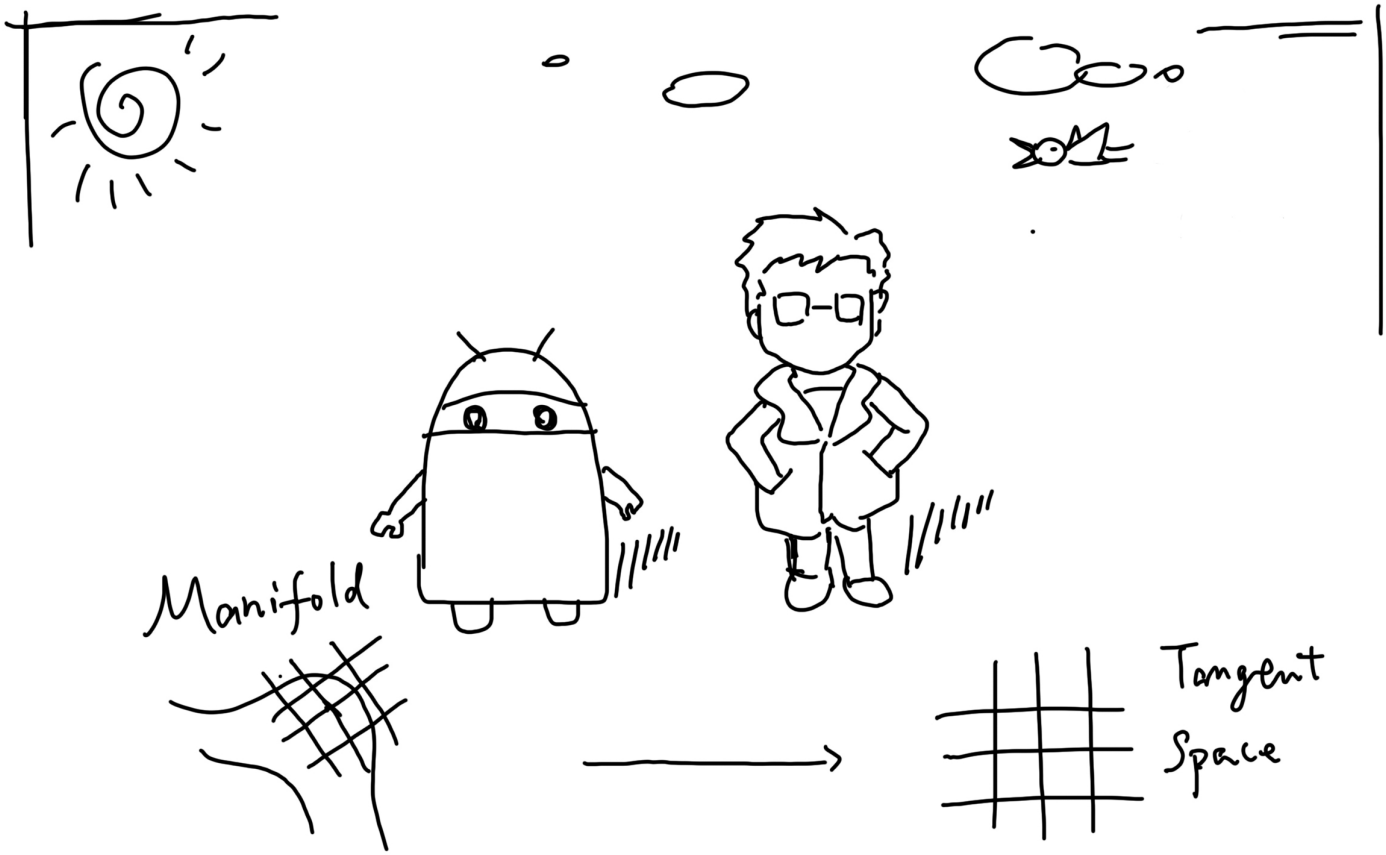
第四章　李群与李代数

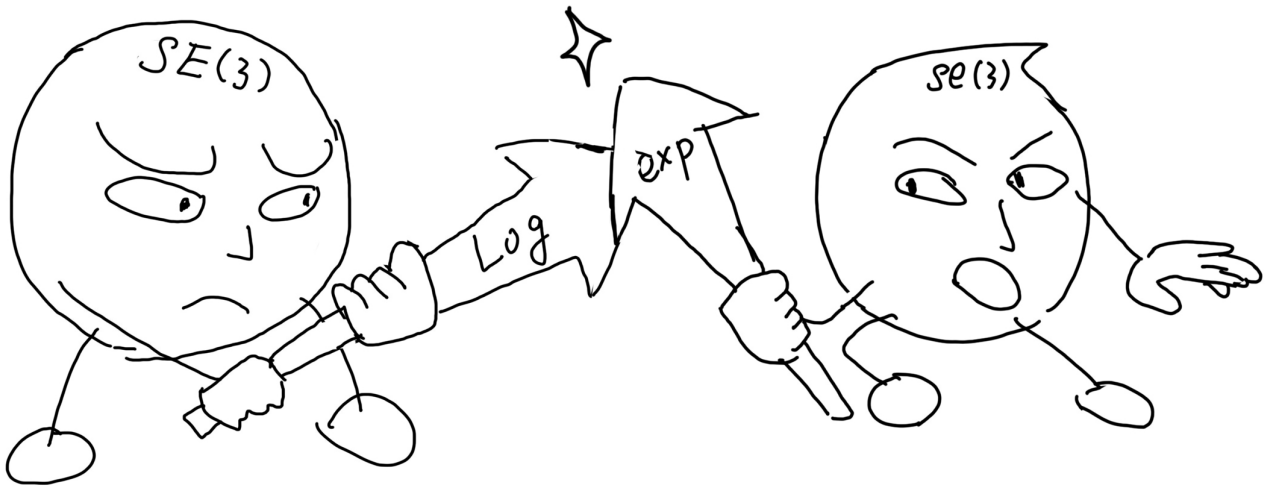
主要目标

1. 理解李群与李代数的概念，掌握 SO(3), SE(3) 与对应李代数的表示方式。
2. 理解 BCH 近似的意义。
3. 学会在李代数上的扰动模型。
4. 使用 Sophus 对李代数进行运算。

上一讲，我们介绍了三维世界中刚体运动的描述方式，包括旋转矩阵、旋转向量、欧拉角、四元数等若干种方式。我们重点介绍了旋转的表示，但是在 SLAM 中，除了表示之外，我们还要对它们进行估计和优化。因为在 SLAM 中位姿是未知的，而我们需要解决形如“**什么样的相机位姿最符合当前观测数据**”这样的问题。一种典型的方式是把它构建成一个优化问题，求解最优的 $\boldsymbol{R}, \boldsymbol{t}$，使得误差最小化。

如前所言，旋转矩阵自身是带有约束的（正交且行列式为 1）。它们作为优化变量时，会引入额外的约束，使优化变得困难。通过李群—李代数间的转换关系，我们希望把位姿估计变成无约束的优化问题，简化求解方式。考虑到读者可能还没有李群李代数的基本知识，我们将从最基本的知识开始讲起。





$$T = \begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix} \in SE(3) \qquad \xi = [\rho, \phi]^T \in se(3)$$

$$\log(T)^\vee = \xi$$

$$\exp(\xi^\wedge) = T$$

4×4 $\qquad$ 6×1

4.1　李群与李代数基础

上一讲，我们介绍了旋转矩阵和变换矩阵的定义。当时，我们说三维旋转矩阵构成了**特殊正交群** SO(3)，而变换矩阵构成了**特殊欧氏群** SE(3)。它们写起来像这样：

$$\mathrm{SO}(3) = \{\boldsymbol{R} \in \mathbb{R}^{3\times 3} | \boldsymbol{R}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{I}, \det(\boldsymbol{R}) = 1\}. \tag{4.1}$$

$$\mathrm{SE}(3) = \left\{ \boldsymbol{T} = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4} | \boldsymbol{R} \in \mathrm{SO}(3), \boldsymbol{t} \in \mathbb{R}^3 \right\}. \tag{4.2}$$

不过，当时我们并未详细解释**群**的含义。细心的读者应该会注意到，旋转矩阵也好，变换矩阵也好，**它们对加法是不封闭的**。换句话说，对于任意两个旋转矩阵 $\boldsymbol{R}_1, \boldsymbol{R}_2$，按照矩阵加法的定义，和不再是一个旋转矩阵：

$$\boldsymbol{R}_1 + \boldsymbol{R}_2 \notin \mathrm{SO}(3), \quad \boldsymbol{T}_1 + \boldsymbol{T}_2 \notin \mathrm{SE}(3). \tag{4.3}$$

你也可以说两种矩阵并没有良好定义的加法，或者通常矩阵加法对这两个集合不封闭。相对地，它们只有一种较好的运算：乘法。SO(3) 和 SE(3) 关于乘法是封闭的:

$$\boldsymbol{R}_1\boldsymbol{R}_2 \in \mathrm{SO}(3), \quad \boldsymbol{T}_1\boldsymbol{T}_2 \in \mathrm{SE}(3). \tag{4.4}$$

同时我们也可以对任何一个旋转或变换矩阵（在乘法的意义上）求逆。我们知道，乘法对应着旋转或变换的复合，两个旋转矩阵相乘表示做了两次旋转。对于这种只有一个（良好的）运算的集合，我们称之为**群**。

4.1.1　群

接下来我们要稍微涉及一些抽象代数方面的知识。我觉得这是讨论李群李代数的必要条件，但实际上除了数学、物理系的同学之外，大部分同学在本科学习中并不会接触到这方面的知识。所以我们先来看一些基本的知识。

群（Group）是**一种集合**加上**一种运算**的代数结构。我们把集合记作 A，运算记作 $\cdot$，那么群可以记作 $G = (A, \cdot)$。群要求这个运算满足以下几个条件：

1. 封闭性：　$\forall a_1, a_2 \in A, \quad a_1 \cdot a_2 \in A.$
2. 结合律：　$\forall a_1, a_2, a_3 \in A, \quad (a_1 \cdot a_2) \cdot a_3 = a_1 \cdot (a_2 \cdot a_3).$
3. 幺元：　$\exists a_0 \in A, \quad \text{s.t.} \quad \forall a \in A, \quad a_0 \cdot a = a \cdot a_0 = a.$
4. 逆：　$\forall a \in A, \quad \exists a^{-1} \in A, \quad s.t. \quad a \cdot a^{-1} = a_0.$

读者可以记作“封结幺逆”[①]。容易验证，旋转矩阵集合和矩阵乘法构成群，同样变换矩阵和矩阵乘法也构成群（因此才能称它们为旋转矩阵群和变换矩阵群）。其他常见的群包括整数的加法 $(\mathbb{Z}, +)$，去掉 0 后的有理数的乘法（幺元为 1）$(\mathbb{Q}\backslash 0, \cdot)$，等等。矩阵中常见的群有：

[①] 谐音“凤姐咬你”。

- 一般线性群 $\mathrm{GL}(n)$　指 $n \times n$ 的可逆矩阵，它们对矩阵乘法成群。
- 特殊正交群 $\mathrm{SO}(n)$　也就是所谓的旋转矩阵群，其中 SO(2) 和 SO(3) 最为常见。
- 特殊欧氏群 $\mathrm{SE}(n)$　也就是前面提到的 n 维欧氏变换，如 SE(2) 和 SE(3)。

群结构保证了在群上的运算具有良好的性质，群论则是研究群的各种结构和性质的理论。对群论感兴趣的读者可以参考任意一本近世代数教材。**李群**是指具有连续（光滑）性质的群。像整数群 $\mathbb{Z}$ 那样离散的群没有连续性质，所以不是李群。而 $\mathrm{SO}(n)$ 和 $\mathrm{SE}(n)$ 在实数空间上是连续的。我们能够直观地想象一个刚体能够连续地在空间中运动，所以它们都是李群。由于 SO(3) 和 SE(3) 对于相机姿态估计尤其重要，所以我们主要讨论这两个李群。然而，严格地讨论“连续”、“光滑”这些概念需要具备分析和拓扑学的知识，但我们不是数学书，所以只介绍一些重要的、与 SLAM 直接相关的结论。如果读者对李群的理论性质感兴趣，请参考文献 [21]。

李群与李代数通常有两种思路来介绍。一是直接引入李群和李代数，然后告诉读者每个李群对应着一个李代数之类的事实，但这样的话，读者会觉得李代数似乎是一个从天而降的符号，不知道它有什么物理意义。所以，我准备稍微花一点时间从旋转矩阵引出李代数，类似于文献 [22] 的做法。我们先从较简单的 SO(3) 开始讨论，引出 SO(3) 上面的李代数 $\mathfrak{so}(3)$。

4.1.2 李代数的引出

考虑任意旋转矩阵 $\boldsymbol{R}$，我们知道它满足：

$$\boldsymbol{R}\boldsymbol{R}^{\mathrm{T}} = \boldsymbol{I}. \tag{4.5}$$

现在，我们说，$\boldsymbol{R}$ 是某个相机的旋转，它会随时间连续地变化，即为时间的函数：$\boldsymbol{R}(t)$。由于它仍是旋转矩阵，有

$$\boldsymbol{R}(t)\boldsymbol{R}(t)^{T} = \boldsymbol{I}.$$

在等式两边对时间求导，得到：

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}} + \boldsymbol{R}(t)\dot{\boldsymbol{R}}(t)^{\mathrm{T}} = 0.$$

整理得：

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}} = -\left(\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}}\right)^{\mathrm{T}}. \tag{4.6}$$

可以看出 $\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}}$ 是一个**反对称**矩阵。回忆一下，我们在式(3.3)介绍叉积时，引入了 $\wedge$ 符号，将一个向量变成了反对称矩阵。同理，对于任意反对称矩阵，我们亦能找到唯一一个与之对应的向量。把这个运算用符号 $\vee$ 表示：

$$\boldsymbol{a}^{\wedge} = \boldsymbol{A} = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix}, \quad \boldsymbol{A}^{\vee} = \boldsymbol{a}. \tag{4.7}$$

于是，由于 $\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}}$ 是一个反对称矩阵，我们可以找到一个三维向量 $\boldsymbol{\phi}(t) \in \mathbb{R}^3$ 与之对应：

$$\dot{\boldsymbol{R}}(t)\boldsymbol{R}(t)^{\mathrm{T}} = \boldsymbol{\phi}(t)^{\wedge}.$$

等式两边右乘 $\boldsymbol{R}(t)$，由于 $\boldsymbol{R}$ 为正交阵，有：

$$\dot{\boldsymbol{R}}(t) = \boldsymbol{\phi}(t)^{\wedge}\boldsymbol{R}(t) = \begin{bmatrix} 0 & -\phi_3 & \phi_2 \\ \phi_3 & 0 & -\phi_1 \\ -\phi_2 & \phi_1 & 0 \end{bmatrix} \boldsymbol{R}(t). \tag{4.8}$$

可以看到，每对旋转矩阵求一次导数，只需左乘一个 $\boldsymbol{\phi}^{\wedge}(t)$ 矩阵即可。考虑 $t_0 = 0$ 时刻，设此时旋转矩阵为 $\boldsymbol{R}(0) = \boldsymbol{I}$。按照导数定义，可以把 $\boldsymbol{R}(t)$ 在 $t = 0$ 附近进行一阶泰勒展开：

$$\begin{aligned} \boldsymbol{R}(t) &\approx \boldsymbol{R}(t_0) + \dot{\boldsymbol{R}}(t_0)(t - t_0) \\ &= \boldsymbol{I} + \boldsymbol{\phi}(t_0)^{\wedge}(t). \end{aligned} \tag{4.9}$$

我们看到 $\boldsymbol{\phi}$ 反映了 $\boldsymbol{R}$ 的导数性质，故称它在 SO(3) 原点附近的正切空间 (Tangent Space) 上。同时在 t_0 附近，设 $\boldsymbol{\phi}$ 保持为常数 $\boldsymbol{\phi}(t_0) = \boldsymbol{\phi}_0$。那么根据式(4.8)，有：

$$\dot{\boldsymbol{R}}(t) = \boldsymbol{\phi}(t_0)^{\wedge}\boldsymbol{R}(t) = \boldsymbol{\phi}_0^{\wedge}\boldsymbol{R}(t).$$

上式是一个关于 $\boldsymbol{R}$ 的微分方程，而且有初始值 $\boldsymbol{R}(0) = \boldsymbol{I}$，解之，得：

$$\boldsymbol{R}(t) = \exp(\boldsymbol{\phi}_0^{\wedge}t). \tag{4.10}$$

读者可以验证上式对微分方程和初始值均成立。这说明在 $t = 0$ 附近，旋转矩阵可以由 $\exp(\boldsymbol{\phi}_0^{\wedge}t)$ 计算出来[①]。我们看到，旋转矩阵 $\boldsymbol{R}$ 与另一个反对称矩阵 $\boldsymbol{\phi}_0^{\wedge}t$ 通过指数关系发生了联系。但是矩阵的指数是什么呢？这里我们有两个问题需要澄清：

1. 给定某时刻的 $\boldsymbol{R}$，我们就能求得一个 $\boldsymbol{\phi}$，它描述了 $\boldsymbol{R}$ 在局部的导数关系。与 $\boldsymbol{R}$ 对应的 $\boldsymbol{\phi}$ 有什么含义呢？我们说，$\boldsymbol{\phi}$ 正是对应到 SO(3) 上的李代数 $\mathfrak{so}(3)$；
2. 其次，给定某个向量 $\boldsymbol{\phi}$ 时，矩阵指数 $\exp(\boldsymbol{\phi}^{\wedge})$ 如何计算？反之，给定 $\boldsymbol{R}$ 时，能否有相反的运算来计算 $\boldsymbol{\phi}$？事实上，这正是李群与李代数间的指数/对数映射。

下面我们来解决这两个问题。

4.1.3　李代数的定义

每个李群都有与之对应的李代数。李代数描述了李群的局部性质，准确地说，是单位元附近的正切空间。一般的李代数的定义如下：

[①] 此时我们还没有说明 exp 是如何作用的。我们马上会看到它的定义和计算过程。

李代数由一个集合 $\mathbb{V}$，一个数域 $\mathbb{F}$ 和一个二元运算 $[,]$ 组成。如果它们满足以下几条性质，则称 $(\mathbb{V}, \mathbb{F}, [,])$ 为一个李代数，记作 $\mathfrak{g}$。

1. 封闭性　$\forall \boldsymbol{X}, \boldsymbol{Y} \in \mathbb{V}, [\boldsymbol{X}, \boldsymbol{Y}] \in \mathbb{V}$.
2. 双线性　$\forall \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{Z} \in \mathbb{V}, a, b \in \mathbb{F}$, 有：

$$[a\boldsymbol{X} + b\boldsymbol{Y}, \boldsymbol{Z}] = a[\boldsymbol{X}, \boldsymbol{Z}] + b[\boldsymbol{Y}, \boldsymbol{Z}], \quad [\boldsymbol{Z}, a\boldsymbol{X} + b\boldsymbol{Y}] = a[\boldsymbol{Z}, \boldsymbol{X}] + b[\boldsymbol{Z}, \boldsymbol{Y}].$$

3. 自反性[①]　$\forall \boldsymbol{X} \in \mathbb{V}, [\boldsymbol{X}, \boldsymbol{X}] = \boldsymbol{0}$.
4. 雅可比等价　$\forall \boldsymbol{X}, \boldsymbol{Y}, \boldsymbol{Z} \in \mathbb{V}, [\boldsymbol{X}, [\boldsymbol{Y}, \boldsymbol{Z}]] + [\boldsymbol{Z}, [\boldsymbol{X}, \boldsymbol{Y}]] + [\boldsymbol{Y}, [\boldsymbol{Z}, \boldsymbol{X}]] = \boldsymbol{0}$.

其中二元运算被称为**李括号**。从表面上来看，李代数所需要的性质还是挺多的。相比于群中的较为简单的二元运算，李括号表达了两个元素的差异。它不要求结合律，而要求元素和自己做李括号之后为零的性质。作为例子，三维向量 $\mathbb{R}^3$ 上定义的叉积 $\times$ 是一种李括号，因此 $\mathfrak{g} = (\mathbb{R}^3, \mathbb{R}, \times)$ 构成了一个李代数。读者可以尝试将叉积的性质代入到上面四条性质中。

4.1.4 李代数 $\mathfrak{so}(3)$

之前提到的 $\boldsymbol{\phi}$，事实上是一种李代数。SO(3) 对应的李代数是定义在 $\mathbb{R}^3$ 上的向量，我们记作 $\boldsymbol{\phi}$。根据前面的推导，每个 $\boldsymbol{\phi}$ 都可以生成一个反对称矩阵：

$$\boldsymbol{\Phi} = \boldsymbol{\phi}^\wedge = \begin{bmatrix} 0 & -\phi_3 & \phi_2 \\ \phi_3 & 0 & -\phi_1 \\ -\phi_2 & \phi_1 & 0 \end{bmatrix} \in \mathbb{R}^{3\times 3}. \tag{4.11}$$

在此定义下，两个向量 $\boldsymbol{\phi}_1, \boldsymbol{\phi}_2$ 的李括号为

$$[\boldsymbol{\phi}_1, \boldsymbol{\phi}_2] = (\boldsymbol{\Phi}_1\boldsymbol{\Phi}_2 - \boldsymbol{\Phi}_2\boldsymbol{\Phi}_1)^\vee. \tag{4.12}$$

读者可以验证该定义下的李括号满足上面的几条性质。由于向量 $\boldsymbol{\phi}$ 与反对称矩阵是一一对应的，在不引起歧义的情况下，就说 $\mathfrak{so}(3)$ 的元素是三维向量或者三维反对称矩阵，不加区别：

$$\mathfrak{so}(3) = \left\{ \boldsymbol{\phi} \in \mathbb{R}^3, \boldsymbol{\Phi} = \boldsymbol{\phi}^\wedge \in \mathbb{R}^{3\times 3} \right\}. \tag{4.13}$$

有些书里也会用 $\widehat{\boldsymbol{\phi}}$ 这样的符号表示反对称，但意义是一样的。至此，我们已清楚了 $\mathfrak{so}(3)$ 的内容。它们是一个由**三维向量**组成的集合，每个向量对应到一个反对称矩阵，可以用于表达旋转矩阵的导数。它与 SO(3) 的关系由指数映射给定：

$$\boldsymbol{R} = \exp(\boldsymbol{\phi}^\wedge). \tag{4.14}$$

指数映射会在稍后介绍。由于已经介绍了 $\mathfrak{so}(3)$，我们顺带先来看 SE(3) 上对应的李代数。

[①] 自反性是指自己与自己的运算为零。

4.1.5　李代数 $\mathfrak{se}(3)$

对于 SE(3)，它也有对应的李代数 $\mathfrak{se}(3)$。为节省篇幅，这里就不介绍如何引出 $\mathfrak{se}(3)$ 了。与 $\mathfrak{so}(3)$ 相似，$\mathfrak{se}(3)$ 位于 $\mathbb{R}^6$ 空间中：

$$\mathfrak{se}(3)=\left\{\boldsymbol{\xi}=\begin{bmatrix}\boldsymbol{\rho}\\ \boldsymbol{\phi}\end{bmatrix}\in\mathbb{R}^6,\boldsymbol{\rho}\in\mathbb{R}^3,\boldsymbol{\phi}\in\mathfrak{so}(3),\boldsymbol{\xi}^\wedge=\begin{bmatrix}\boldsymbol{\phi}^\wedge & \boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 0\end{bmatrix}\in\mathbb{R}^{4\times4}\right\}. \tag{4.15}$$

我们把每个 $\mathfrak{se}(3)$ 元素记作 $\boldsymbol{\xi}$，它是一个六维向量。前三维为平移（但含义与变换矩阵中的平移不同，分析见后），记作 $\boldsymbol{\rho}$；后三维为旋转，记作 $\boldsymbol{\phi}$，实质上是 $\mathfrak{so}(3)$ 元素[①]。同时，我们拓展了 $\wedge$ 符号的含义。在 $\mathfrak{se}(3)$ 中，同样使用 $\wedge$ 符号，将一个六维向量转换成四维矩阵，但这里不再表示反对称：

$$\boldsymbol{\xi}^\wedge=\begin{bmatrix}\boldsymbol{\phi}^\wedge & \boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 0\end{bmatrix}\in\mathbb{R}^{4\times4}. \tag{4.16}$$

我们仍使用 $\wedge$ 和 $\vee$ 符号来指代“从向量到矩阵”和“从矩阵到向量”的关系，以保持和 $\mathfrak{so}(3)$ 上的一致性。它们依旧是一一对应的。读者可以简单地把 $\mathfrak{se}(3)$ 理解成“由一个平移加上一个 $\mathfrak{so}(3)$ 元素构成的向量”（尽管这里的 $\boldsymbol{\rho}$ 还不直接是平移）。同样，李代数 $\mathfrak{se}(3)$ 亦有类似于 $\mathfrak{so}(3)$ 的李括号：

$$[\boldsymbol{\xi}_1,\boldsymbol{\xi}_2]=(\boldsymbol{\xi}_1^\wedge\boldsymbol{\xi}_2^\wedge-\boldsymbol{\xi}_2^\wedge\boldsymbol{\xi}_1^\wedge)^\vee. \tag{4.17}$$

读者可以验证它是否满足李代数的定义（留作习题）。至此我们已经见过两种重要的李代数 $\mathfrak{so}(3)$ 和 $\mathfrak{se}(3)$ 了。

4.2　指数与对数映射

4.2.1　SO(3) 上的指数映射

现在来考虑第二个问题：如何计算 $\exp(\boldsymbol{\phi}^\wedge)$？显然它是一个矩阵的指数，在李群和李代数中，称为指数映射（Exponential Map）。同样，我们会先讨论 $\mathfrak{so}(3)$ 的指数映射，再讨论 $\mathfrak{se}(3)$ 的情形。

任意矩阵的指数映射可以写成一个泰勒展开，但是只有在收敛的情况下才会有结果，其结果仍是一个矩阵：

$$\exp(\boldsymbol{A})=\sum_{n=0}^{\infty}\frac{1}{n!}\boldsymbol{A}^n. \tag{4.18}$$

同样地，对 $\mathfrak{so}(3)$ 中任意元素 $\boldsymbol{\phi}$，我们亦可按此方式定义它的指数映射：

$$\exp(\boldsymbol{\phi}^\wedge)=\sum_{n=0}^{\infty}\frac{1}{n!}(\boldsymbol{\phi}^\wedge)^n. \tag{4.19}$$

[①] 请注意有些地方把旋转放前面，平移放后面，也是可行的。在程序里则无所谓前后，它们都存储在一个结构体中。

但这个定义没法直接计算，因为我们不想计算矩阵的无穷次幂。下面我们推导一种计算指数映射的简便方法。由于 $\boldsymbol{\phi}$ 是三维向量，我们可以定义它的模长和它的方向，分别记作 θ 和 $\boldsymbol{a}$，于是有 $\boldsymbol{\phi} = \theta \boldsymbol{a}$。这里 $\boldsymbol{a}$ 是一个长度为 1 的方向向量，即 $\|\boldsymbol{a}\| = 1$。首先，对于 $\boldsymbol{a}^\wedge$，有以下两条性质：

$$\boldsymbol{a}^\wedge \boldsymbol{a}^\wedge = \begin{bmatrix} -a_2^2 - a_3^2 & a_1 a_2 & a_1 a_3 \\ a_1 a_2 & -a_1^2 - a_3^2 & a_2 a_3 \\ a_1 a_3 & a_2 a_3 & -a_1^2 - a_2^2 \end{bmatrix} = \boldsymbol{a}\boldsymbol{a}^{\mathrm{T}} - \boldsymbol{I}, \tag{4.20}$$

以及

$$\boldsymbol{a}^\wedge \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge = \boldsymbol{a}^\wedge (\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}} - \boldsymbol{I}) = -\boldsymbol{a}^\wedge. \tag{4.21}$$

这两个式子提供了处理 $\boldsymbol{a}^\wedge$ 高阶项的方法。我们可以把指数映射写成：

$$\begin{aligned} \exp(\boldsymbol{\phi}^\wedge) = \exp(\theta \boldsymbol{a}^\wedge) &= \sum_{n=0}^{\infty} \frac{1}{n!} (\theta \boldsymbol{a}^\wedge)^n \\ &= \boldsymbol{I} + \theta \boldsymbol{a}^\wedge + \frac{1}{2!}\theta^2 \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \frac{1}{3!}\theta^3 \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \frac{1}{4!}\theta^4 (\boldsymbol{a}^\wedge)^4 + \ldots \\ &= \boldsymbol{a}\boldsymbol{a}^{\mathrm{T}} - \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \theta \boldsymbol{a}^\wedge + \frac{1}{2!}\theta^2 \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge - \frac{1}{3!}\theta^3 \boldsymbol{a}^\wedge - \frac{1}{4!}\theta^4 (\boldsymbol{a}^\wedge)^2 + \ldots \\ &= \boldsymbol{a}\boldsymbol{a}^{\mathrm{T}} + \underbrace{\left(\theta - \frac{1}{3!}\theta^3 + \frac{1}{5!}\theta^5 - \ldots\right)}_{\sin\theta} \boldsymbol{a}^\wedge - \underbrace{\left(1 - \frac{1}{2!}\theta^2 + \frac{1}{4!}\theta^4 - \ldots\right)}_{\cos\theta} \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge \\ &= \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \boldsymbol{I} + \sin\theta \boldsymbol{a}^\wedge - \cos\theta \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge \\ &= (1 - \cos\theta) \boldsymbol{a}^\wedge \boldsymbol{a}^\wedge + \boldsymbol{I} + \sin\theta \boldsymbol{a}^\wedge \\ &= \cos\theta \boldsymbol{I} + (1 - \cos\theta) \boldsymbol{a}\boldsymbol{a}^{\mathrm{T}} + \sin\theta \boldsymbol{a}^\wedge. \end{aligned}$$

最后得到一个似曾相识的式子：

$$\exp(\theta \boldsymbol{a}^\wedge) = \cos\theta \boldsymbol{I} + (1 - \cos\theta) \boldsymbol{a}\boldsymbol{a}^{\mathrm{T}} + \sin\theta \boldsymbol{a}^\wedge. \tag{4.22}$$

回想前一讲内容，它和罗德里格斯公式，即式(3.15)如出一辙。这表明，$\mathfrak{so}(3)$ 实际上就是由所谓的**旋转向量**组成的空间，而指数映射即罗德里格斯公式。通过它们，我们把 $\mathfrak{so}(3)$ 中任意一个向量对应到了一个位于 SO(3) 中的旋转矩阵。反之，如果定义对数映射，也能把 SO(3) 中的元素对应到 $\mathfrak{so}(3)$ 中：

$$\boldsymbol{\phi} = \ln(\boldsymbol{R})^\vee = \left(\sum_{n=0}^{\infty} \frac{(-1)^n}{n+1} (\boldsymbol{R} - \boldsymbol{I})^{n+1} \right)^\vee. \tag{4.23}$$

和指数映射一样，我们没必要直接用泰勒展开计算对数映射。在第 3 讲中，我们已经介绍过如何根据旋转矩阵计算对应的李代数，即使用式(3.17)，利用迹的性质分别求解转角和转轴，采用那种方式更加省事一些。

现在，我们介绍了指数映射的计算方法。读者可能会问，指数映射性质如何呢？是否对于任意的 $\boldsymbol{R}$ 都能找到一个唯一的 $\boldsymbol{\phi}$？很遗憾，指数映射只是一个满射，并不是单射。这意味着每个 SO(3) 中的元素，都可以找到一个 $\mathfrak{so}(3)$ 元素与之对应；但是可能存在多个 $\mathfrak{so}(3)$ 中的元素，对应到同一个 SO(3)。至少对于旋转角 θ，我们知道多转 360° 和没有转是一样的——它具有周期性。但是，如果我们把旋转角度固定在 $\pm\pi$ 之间，那么李群和李代数元素是一一对应的。

SO(3) 与 $\mathfrak{so}(3)$ 的结论似乎在我们的意料之中。它和我们前面讲的旋转向量与旋转矩阵很相似，而指数映射即是罗德里格斯公式。旋转矩阵的导数可以由旋转向量指定，指导着如何在旋转矩阵中进行微积分运算。

4.2.2　SE(3) 上的指数映射

下面介绍 $\mathfrak{se}(3)$ 上的指数映射。为了节省篇幅，我们不再像 $\mathfrak{so}(3)$ 那样详细推导指数映射。$\mathfrak{se}(3)$ 上的指数映射形式如下：

$$\exp\left(\boldsymbol{\xi}^{\wedge}\right)=\begin{bmatrix}\sum\limits_{n=0}^{\infty}\frac{1}{n!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n} & \sum\limits_{n=0}^{\infty}\frac{1}{(n+1)!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n}\boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 1\end{bmatrix} \tag{4.24}$$

$$\triangleq\begin{bmatrix}\boldsymbol{R} & \boldsymbol{J}\boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 1\end{bmatrix}=\boldsymbol{T}. \tag{4.25}$$

只要有一点耐心，可以照着 $\mathfrak{so}(3)$ 上的做法推导，把 exp 进行泰勒展开推导此式。令 $\boldsymbol{\phi}=\theta\boldsymbol{a}$，其中 $\boldsymbol{a}$ 为单位向量，则：

$$\begin{aligned}\sum_{n=0}^{\infty}\frac{1}{(n+1)!}\left(\boldsymbol{\phi}^{\wedge}\right)^{n}&=\boldsymbol{I}+\frac{1}{2!}\theta\boldsymbol{a}^{\wedge}+\frac{1}{3!}\theta^{2}\left(\boldsymbol{a}^{\wedge}\right)^{2}+\frac{1}{4!}\theta^{3}\left(\boldsymbol{a}^{\wedge}\right)^{3}+\frac{1}{5!}\theta^{4}\left(\boldsymbol{a}^{\wedge}\right)^{4}\cdots\\&=\frac{1}{\theta}\left(\frac{1}{2!}\theta^{2}-\frac{1}{4!}\theta^{4}+\cdots\right)\left(\boldsymbol{a}^{\wedge}\right)+\frac{1}{\theta}\left(\frac{1}{3!}\theta^{3}-\frac{1}{5}\theta^{5}+\cdots\right)\left(\boldsymbol{a}^{\wedge}\right)^{2}+\boldsymbol{I}\\&=\frac{1}{\theta}\left(1-\cos\theta\right)\left(\boldsymbol{a}^{\wedge}\right)+\frac{\theta-\sin\theta}{\theta}\left(\boldsymbol{a}\boldsymbol{a}^{T}-\boldsymbol{I}\right)+\boldsymbol{I}\\&=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{T}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^{\wedge}\triangleq\boldsymbol{J}.\end{aligned} \tag{4.26}$$

从结果上看，$\boldsymbol{\xi}$ 的指数映射左上角的 $\boldsymbol{R}$ 是我们熟知的 SO(3) 中的元素，与 $\mathfrak{se}(3)$ 当中的旋转部分 $\boldsymbol{\phi}$ 对应。而右上角的 $\boldsymbol{J}$ 由上面的推导给出：

$$\boldsymbol{J}=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^{\wedge}. \tag{4.27}$$

该式与罗德里格斯公式有些相似，但不完全一样。我们看到，平移部分经过指数映射之后，发生了一次以 $\boldsymbol{J}$ 为系数矩阵的线性变换。请读者重视这里的 $\boldsymbol{J}$，因为后面还要用到。

同样地，虽然我们也可以类比推得对数映射，不过根据变换矩阵 $\boldsymbol{T}$ 求 $\mathfrak{so}(3)$ 上的对应向量也有更省事的方式：从左上角的 $\boldsymbol{R}$ 计算旋转向量，而右上角的 $\boldsymbol{t}$ 满足：

$$\boldsymbol{t} = \boldsymbol{J}\rho. \tag{4.28}$$

由于 $\boldsymbol{J}$ 可以由 ϕ 得到，所以这里的 ρ 亦可由此线性方程解得。现在，我们已经弄清了李群、李代数的定义与相互的转换关系，总结如图 4-1 所示。如果读者有哪里不明白，可以翻回去几页看看公式推导。

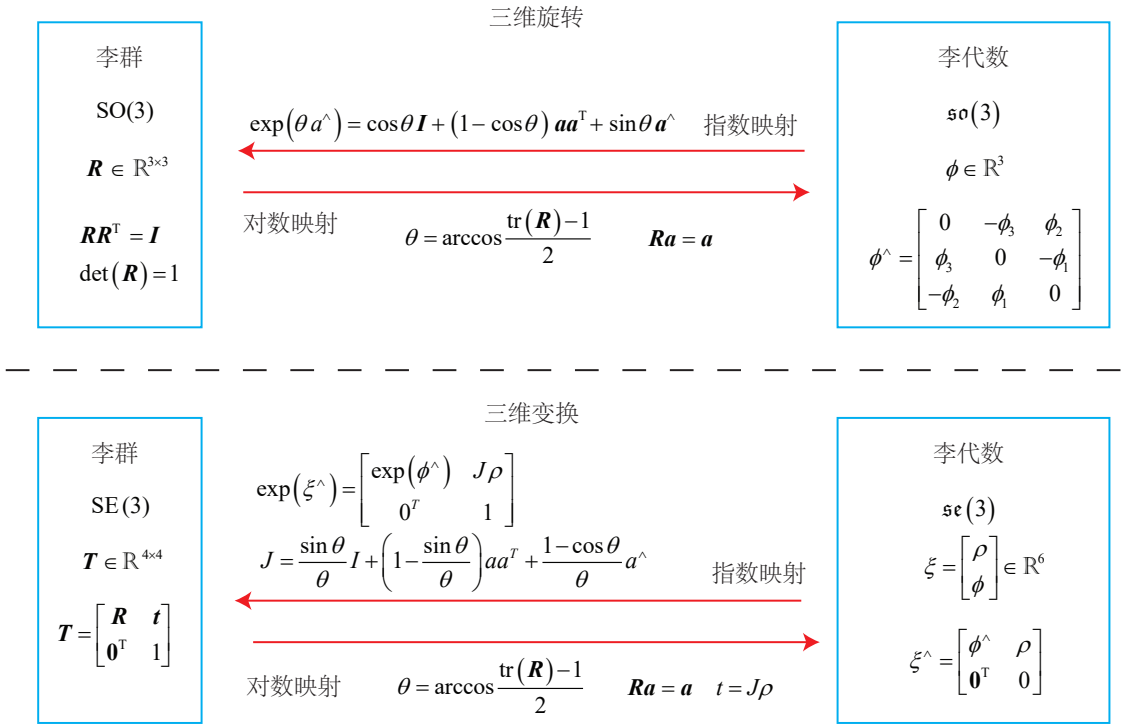


图 4-1　$\mathrm{SO}(3), \mathrm{SE}(3), \mathfrak{so}(3), \mathfrak{se}(3)$ 的对应关系。

4.3　李代数求导与扰动模型

4.3.1　BCH 公式与近似形式

使用李代数的一大动机是进行优化，而在优化过程中导数是非常必要的信息（我们会在第 6 讲详细介绍）。下面来考虑一个问题。虽然我们已经清楚了 SO(3) 和 SE(3) 上的李群与李代数关系，但是，当在 SO(3) 中完成两个矩阵乘法时，李代数中 $\mathfrak{so}(3)$ 上发生了什么改变呢？反过来说，当 $\mathfrak{so}(3)$ 上做两个李代数的加法时，SO(3) 上是否对应着两个矩阵的乘积？如果成立，相当于：

$$\exp\left(\boldsymbol{\phi}_1^\wedge\right)\exp\left(\boldsymbol{\phi}_2^\wedge\right)=\exp\left(\left(\boldsymbol{\phi}_1+\boldsymbol{\phi}_2\right)^\wedge\right)?$$

如果 $\boldsymbol{\phi}_1,\boldsymbol{\phi}_2$ 为标量，那显然该式成立；但此处我们计算的是**矩阵**的指数函数，而非标量的指数。换言之，我们在研究下式是否成立：

$$\ln\left(\exp\left(\boldsymbol{A}\right)\exp\left(\boldsymbol{B}\right)\right)=\boldsymbol{A}+\boldsymbol{B}\ ?$$

很遗憾，该式在矩阵时并不成立。两个李代数指数映射乘积的完整形式，由 Baker-Campbell-Hausdorff 公式（BCH 公式）[①]给出。由于其完整形式较复杂，我们只给出其展开式的前几项：

$$\ln\left(\exp\left(\boldsymbol{A}\right)\exp\left(\boldsymbol{B}\right)\right)=\boldsymbol{A}+\boldsymbol{B}+\frac{1}{2}\left[\boldsymbol{A},\boldsymbol{B}\right]+\frac{1}{12}\left[\boldsymbol{A},\left[\boldsymbol{A},\boldsymbol{B}\right]\right]-\frac{1}{12}\left[\boldsymbol{B},\left[\boldsymbol{A},\boldsymbol{B}\right]\right]+\cdots \tag{4.29}$$

其中 [] 为李括号。BCH 公式告诉我们，当处理两个矩阵指数之积时，它们会产生一些由李括号组成的余项。特别地，考虑 SO(3) 上的李代数 $\ln\left(\exp\left(\boldsymbol{\phi}_1^\wedge\right)\exp\left(\boldsymbol{\phi}_2^\wedge\right)\right)^\vee$，当 $\boldsymbol{\phi_1}$ 或 $\boldsymbol{\phi_2}$ 为小量时，小量二次以上的项都可以被忽略掉。此时，BCH 拥有线性近似表达[②]：

$$\ln\left(\exp\left(\boldsymbol{\phi}_1^\wedge\right)\exp\left(\boldsymbol{\phi}_2^\wedge\right)\right)^\vee\approx\begin{cases}\boldsymbol{J}_l(\boldsymbol{\phi}_2)^{-1}\boldsymbol{\phi}_1+\boldsymbol{\phi}_2 & \text{当}\boldsymbol{\phi}_1\text{为小量,}\\ \boldsymbol{J}_r(\boldsymbol{\phi}_1)^{-1}\boldsymbol{\phi}_2+\boldsymbol{\phi}_1 & \text{当}\boldsymbol{\phi}_2\text{为小量.}\end{cases} \tag{4.30}$$

以第一个近似为例。该式告诉我们，当对一个旋转矩阵 $\boldsymbol{R}_2$（李代数为 $\boldsymbol{\phi}_2$）左乘一个微小旋转矩阵 $\boldsymbol{R}_1$（李代数为 $\boldsymbol{\phi}_1$）时，可以近似地看作，在原有的李代数 $\boldsymbol{\phi}_2$ 上加上了一项 $\boldsymbol{J}_l(\boldsymbol{\phi}_2)^{-1}\boldsymbol{\phi}_1$。同理，第二个近似描述了右乘一个微小位移的情况。于是，李代数在 BCH 近似下，分成了左乘近似和右乘近似两种，在使用时我们须注意使用的是左乘模型还是右乘模型。

本书以左乘为例。左乘 BCH 近似雅可比 $\boldsymbol{J}_l$ 事实上就是式 (4.27) 的内容：

$$\boldsymbol{J}_l=\boldsymbol{J}=\frac{\sin\theta}{\theta}\boldsymbol{I}+\left(1-\frac{\sin\theta}{\theta}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}+\frac{1-\cos\theta}{\theta}\boldsymbol{a}^\wedge. \tag{4.31}$$

它的逆为：

$$\boldsymbol{J}_l^{-1}=\frac{\theta}{2}\cot\frac{\theta}{2}\boldsymbol{I}+\left(1-\frac{\theta}{2}\cot\frac{\theta}{2}\right)\boldsymbol{a}\boldsymbol{a}^{\mathrm{T}}-\frac{\theta}{2}\boldsymbol{a}^\wedge. \tag{4.32}$$

[①] 参见https://en.wikipedia.org/wiki/Baker-Campbell-Hausdorff_formula。

[②] 对 BCH 具体形式和近似表达的具体推导，本书不作讨论，请参考文献 [6]。

而右乘雅可比仅需要对自变量取负号即可：

$$\boldsymbol{J}_r(\boldsymbol{\phi}) = \boldsymbol{J}_l(-\boldsymbol{\phi}). \tag{4.33}$$

这样，我们就可以谈论李群乘法与李代数加法的关系了。

为了方便读者理解，我们重新叙述一下 BCH 近似的意义。假定对某个旋转 $\boldsymbol{R}$，对应的李代数为 $\boldsymbol{\phi}$。我们给它左乘一个微小旋转，记作 $\Delta\boldsymbol{R}$，对应的李代数为 $\Delta\boldsymbol{\phi}$。那么，在李群上，得到的结果就是 $\Delta\boldsymbol{R}\cdot\boldsymbol{R}$，而在李代数上，根据 BCH 近似，为 $\boldsymbol{J}_l^{-1}(\boldsymbol{\phi})\Delta\boldsymbol{\phi}+\boldsymbol{\phi}$。合并起来，可以简单地写成：

$$\exp\left(\Delta\boldsymbol{\phi}^\wedge\right)\exp\left(\boldsymbol{\phi}^\wedge\right) = \exp\left(\left(\boldsymbol{\phi}+\boldsymbol{J}_l^{-1}\left(\boldsymbol{\phi}\right)\Delta\boldsymbol{\phi}\right)^\wedge\right). \tag{4.34}$$

反之，如果我们在李代数上进行加法，让一个 $\boldsymbol{\phi}$ 加上 $\Delta\boldsymbol{\phi}$，那么可以近似为李群上带左右雅可比的乘法：

$$\exp\left((\boldsymbol{\phi}+\Delta\boldsymbol{\phi})^\wedge\right) = \exp\left((\boldsymbol{J}_l\Delta\boldsymbol{\phi})^\wedge\right)\exp\left(\boldsymbol{\phi}^\wedge\right) = \exp\left(\boldsymbol{\phi}^\wedge\right)\exp\left((\boldsymbol{J}_r\Delta\boldsymbol{\phi})^\wedge\right). \tag{4.35}$$

这就为之后李代数上做微积分提供了理论基础。同样地，对于 SE(3)，亦有类似的 BCH 近似：

$$\exp\left(\Delta\boldsymbol{\xi}^\wedge\right)\exp\left(\boldsymbol{\xi}^\wedge\right) \approx \exp\left(\left(\boldsymbol{\mathcal{J}}_l^{-1}\Delta\boldsymbol{\xi}+\boldsymbol{\xi}\right)^\wedge\right), \tag{4.36}$$

$$\exp\left(\boldsymbol{\xi}^\wedge\right)\exp\left(\Delta\boldsymbol{\xi}^\wedge\right) \approx \exp\left(\left(\boldsymbol{\mathcal{J}}_r^{-1}\Delta\boldsymbol{\xi}+\boldsymbol{\xi}\right)^\wedge\right). \tag{4.37}$$

这里 $\boldsymbol{\mathcal{J}}_l$ 形式比较复杂，它是一个 6×6 的矩阵，读者可以参考文献 [6] 中式 (7.82) 和 (7.83) 的内容。由于我们在计算中没有用到该雅可比，故这里略去它的实际形式。

4.3.2 SO(3) 李代数上的求导

下面来讨论一个带有李代数的函数，如何关于该李代数求导的问题。该问题有很强的实际背景。在 SLAM 中，我们要估计一个相机的位置和姿态，该位姿是由 SO(3) 上的旋转矩阵或 SE(3) 上的变换矩阵描述的。不妨设某个时刻小萝卜的位姿为 $\boldsymbol{T}$。它观察到了一个世界坐标位于 $\boldsymbol{p}$ 的点，产生了一个观测数据 $\boldsymbol{z}$。那么，由坐标变换关系知：

$$\boldsymbol{z} = \boldsymbol{T}\boldsymbol{p}+\boldsymbol{w}. \tag{4.38}$$

其中 $\boldsymbol{w}$ 为随机噪声。由于它的存在，$\boldsymbol{z}$ 往往不可能精确地满足 $\boldsymbol{z}=\boldsymbol{T}\boldsymbol{p}$ 的关系。所以，我们通常会计算理想的观测与实际数据的误差：

$$\boldsymbol{e} = \boldsymbol{z}-\boldsymbol{T}\boldsymbol{p}. \tag{4.39}$$

假设一共有 N 个这样的路标点和观测，于是就有 N 个上式。那么，对小萝卜的位姿估计，相当于是寻找一个最优的 $\boldsymbol{T}$，使得整体误差最小化：

$$\min_{\boldsymbol{T}} J(\boldsymbol{T}) = \sum_{i=1}^{N}\left\|\boldsymbol{z}_i-\boldsymbol{T}\boldsymbol{p}_i\right\|_2^2. \tag{4.40}$$

求解此问题，需要计算目标函数 J 关于变换矩阵 $\boldsymbol{T}$ 的导数。我们把具体的算法留到后面再讲。这里的重点是，**我们经常会构建与位姿有关的函数，然后讨论该函数关于位姿的导数，以调整当前的估计值**。然而，SO(3), SE(3) 上并没有良好定义的加法，它们只是群。如果我们把 $\boldsymbol{T}$ 当成一个普通矩阵来处理优化，那就必须对它加以约束。而从李代数角度来说，由于李代数由向量组成，具有良好的加法运算。因此，使用李代数解决求导问题的思路分为两种：

1. 用李代数表示姿态，然后根据李代数加法来对**李代数求导**。
2. 对李群**左乘**或**右乘**微小扰动，然后对该**扰动求导**，称为左扰动和右扰动模型。

第一种方式对应到李代数的求导模型，而第二种则对应到扰动模型。下面来讨论这两种思路的异同。

4.3.3　李代数求导

首先，考虑 SO(3) 上的情况。假设我们对一个空间点 $\boldsymbol{p}$ 进行了旋转，得到了 $\boldsymbol{Rp}$。现在，要计算旋转之后点的坐标相对于旋转的导数，我们非正式地记为[①]：

$$\frac{\partial (\boldsymbol{Rp})}{\partial \boldsymbol{R}}.$$

由于 SO(3) 没有加法，所以该导数无法按照导数的定义进行计算。设 $\boldsymbol{R}$ 对应的李代数为 $\boldsymbol{\phi}$，我们转而计算[②]：

$$\frac{\partial \left(\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}\right)}{\partial \boldsymbol{\phi}}.$$

按照导数的定义，有：

$$\begin{aligned}
\frac{\partial \left(\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}\right)}{\partial \boldsymbol{\phi}} &= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{\exp\left((\boldsymbol{\phi}+\delta\boldsymbol{\phi})^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{\exp\left((\boldsymbol{J}_l\delta\boldsymbol{\phi})^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{\left(\boldsymbol{I}+(\boldsymbol{J}_l\delta\boldsymbol{\phi})^{\wedge}\right)\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p} - \exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{(\boldsymbol{J}_l\delta\boldsymbol{\phi})^{\wedge}\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p}}{\delta\boldsymbol{\phi}} \\
&= \lim_{\delta\boldsymbol{\phi}\to\boldsymbol{0}} \frac{-(\exp\left(\boldsymbol{\phi}^{\wedge}\right)\boldsymbol{p})^{\wedge}\boldsymbol{J}_l\delta\boldsymbol{\phi}}{\delta\boldsymbol{\phi}} = -(\boldsymbol{Rp})^{\wedge}\boldsymbol{J}_l.
\end{aligned}$$

第 2 行的近似为 BCH 线性近似，第 3 行为泰勒展开舍去高阶项后的近似（但由于取了极限，可以写等号），第 4 行至第 5 行将反对称符号看作叉积，交换之后变号。于是，我们推导出了旋转后的

[①] 请注意这里并不能按照矩阵微分来定义导数，这只是一个记号。

[②] 严格说来，在矩阵微分中，只能求行向量关于列向量的导数，所得结果是一个矩阵。但本书写成列向量对列向量的导数，读者可以认为先对分子进行转置，再对最后结果进行转置。这使得式子变得简洁，不然我们就不得不给每一行的分子加一个转置符号。在这种意义下，可以认为 $\mathrm{d}(\boldsymbol{Ax})/\mathrm{d}\boldsymbol{x} = \boldsymbol{A}$。

点相对于李代数的导数：

$$\frac{\partial (\boldsymbol{Rp})}{\partial \boldsymbol{\phi}} = (-\boldsymbol{Rp})^{\wedge} \boldsymbol{J}_l. \tag{4.41}$$

不过，由于这里仍然含有形式比较复杂的 $\boldsymbol{J}_l$，我们不太希望计算它。而下面要讲的扰动模型则提供了更简单的导数计算方式。

4.3.4 扰动模型（左乘）

另一种求导方式是对 $\boldsymbol{R}$ 进行一次扰动 $\Delta\boldsymbol{R}$，看结果相对于扰动的变化率。这个扰动可以乘在左边也可以乘在右边，最后结果会有一点儿微小的差异，我们以左扰动为例。设左扰动 $\Delta\boldsymbol{R}$ 对应的李代数为 $\boldsymbol{\varphi}$。然后，对 $\boldsymbol{\varphi}$ 求导，即：

$$\frac{\partial (\boldsymbol{Rp})}{\partial \boldsymbol{\varphi}} = \lim_{\boldsymbol{\varphi} \to \mathbf{0}} \frac{\exp(\boldsymbol{\varphi}^{\wedge}) \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p} - \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p}}{\boldsymbol{\varphi}}. \tag{4.42}$$

该式的求导比上面更为简单：

$$\begin{aligned} \frac{\partial (\boldsymbol{Rp})}{\partial \boldsymbol{\varphi}} &= \lim_{\boldsymbol{\varphi} \to \mathbf{0}} \frac{\exp(\boldsymbol{\varphi}^{\wedge}) \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p} - \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p}}{\boldsymbol{\varphi}} \\ &= \lim_{\boldsymbol{\varphi} \to \mathbf{0}} \frac{(\boldsymbol{I} + \boldsymbol{\varphi}^{\wedge}) \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p} - \exp(\boldsymbol{\phi}^{\wedge}) \boldsymbol{p}}{\boldsymbol{\varphi}} \\ &= \lim_{\boldsymbol{\varphi} \to \mathbf{0}} \frac{\boldsymbol{\varphi}^{\wedge} \boldsymbol{Rp}}{\boldsymbol{\varphi}} = \lim_{\boldsymbol{\varphi} \to \mathbf{0}} \frac{-(\boldsymbol{Rp})^{\wedge} \boldsymbol{\varphi}}{\boldsymbol{\varphi}} = -(\boldsymbol{Rp})^{\wedge}. \end{aligned}$$

可见，相比于直接对李代数求导，省去了一个雅可比 $\boldsymbol{J}_l$ 的计算。这使得扰动模型更为实用。请读者务必理解这里的求导运算，这在位姿估计当中具有重要的意义。

4.3.5 SE(3) 上的李代数求导

最后，我们给出 SE(3) 上的扰动模型，而直接李代数上的求导就不再介绍了。假设某空间点 $\boldsymbol{p}$ 经过一次变换 $\boldsymbol{T}$（对应李代数为 $\boldsymbol{\xi}$），得到 $\boldsymbol{Tp}$[①]。现在，给 $\boldsymbol{T}$ 左乘一个扰动 $\Delta\boldsymbol{T} = \exp(\delta\boldsymbol{\xi}^{\wedge})$，我们

[①] 请注意为了使乘法成立，$\boldsymbol{p}$ 必须使用齐次坐标。

设扰动项的李代数为 $\delta\boldsymbol{\xi}=[\delta\boldsymbol{\rho},\delta\boldsymbol{\phi}]^{\mathrm{T}}$，那么：

$$\begin{aligned}\frac{\partial(\boldsymbol{T}\boldsymbol{p})}{\partial\delta\boldsymbol{\xi}}&=\lim_{\delta\boldsymbol{\xi}\to\mathbf{0}}\frac{\exp(\delta\boldsymbol{\xi}^{\wedge})\exp(\boldsymbol{\xi}^{\wedge})\boldsymbol{p}-\exp(\boldsymbol{\xi}^{\wedge})\boldsymbol{p}}{\delta\boldsymbol{\xi}}\\&=\lim_{\delta\boldsymbol{\xi}\to\mathbf{0}}\frac{(\boldsymbol{I}+\delta\boldsymbol{\xi}^{\wedge})\exp(\boldsymbol{\xi}^{\wedge})\boldsymbol{p}-\exp(\boldsymbol{\xi}^{\wedge})\boldsymbol{p}}{\delta\boldsymbol{\xi}}\\&=\lim_{\delta\boldsymbol{\xi}\to\mathbf{0}}\frac{\delta\boldsymbol{\xi}^{\wedge}\exp(\boldsymbol{\xi}^{\wedge})\boldsymbol{p}}{\delta\boldsymbol{\xi}}\\&=\lim_{\delta\boldsymbol{\xi}\to\mathbf{0}}\frac{\begin{bmatrix}\delta\boldsymbol{\phi}^{\wedge}&\delta\boldsymbol{\rho}\\\mathbf{0}^{\mathrm{T}}&0\end{bmatrix}\begin{bmatrix}\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t}\\1\end{bmatrix}}{\delta\boldsymbol{\xi}}\\&=\lim_{\delta\boldsymbol{\xi}\to\mathbf{0}}\frac{\begin{bmatrix}\delta\boldsymbol{\phi}^{\wedge}(\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t})+\delta\boldsymbol{\rho}\\\mathbf{0}^{\mathrm{T}}\end{bmatrix}}{[\delta\boldsymbol{\rho},\delta\boldsymbol{\phi}]^{\mathrm{T}}}=\begin{bmatrix}\boldsymbol{I}&-(\boldsymbol{R}\boldsymbol{p}+\boldsymbol{t})^{\wedge}\\\mathbf{0}^{\mathrm{T}}&\mathbf{0}^{\mathrm{T}}\end{bmatrix}\triangleq(\boldsymbol{T}\boldsymbol{p})^{\odot}.\end{aligned}$$

我们把最后的结果定义成一个算符 $\odot$[①]，它把一个齐次坐标的空间点变换成一个 4×6 的矩阵。此式稍微需要解释的是矩阵求导方面的顺序，假设 $\boldsymbol{a},\boldsymbol{b},\boldsymbol{x},\boldsymbol{y}$ 都是列向量，那么在我们的符号写法下，有如下的规则：

$$\frac{\mathrm{d}\begin{bmatrix}\boldsymbol{a}\\\boldsymbol{b}\end{bmatrix}}{\mathrm{d}\begin{bmatrix}\boldsymbol{x}\\\boldsymbol{y}\end{bmatrix}}=\left(\frac{\mathrm{d}[\boldsymbol{a},\boldsymbol{b}]^{\mathrm{T}}}{\mathrm{d}\begin{bmatrix}\boldsymbol{x}\\\boldsymbol{y}\end{bmatrix}}\right)^{\mathrm{T}}=\begin{bmatrix}\frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}\boldsymbol{x}}&\frac{\mathrm{d}\boldsymbol{b}}{\mathrm{d}\boldsymbol{x}}\\\frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}\boldsymbol{y}}&\frac{\mathrm{d}\boldsymbol{b}}{\mathrm{d}\boldsymbol{y}}\end{bmatrix}^{\mathrm{T}}=\begin{bmatrix}\frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}\boldsymbol{x}}&\frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}\boldsymbol{y}}\\\frac{\mathrm{d}\boldsymbol{b}}{\mathrm{d}\boldsymbol{x}}&\frac{\mathrm{d}\boldsymbol{b}}{\mathrm{d}\boldsymbol{y}}\end{bmatrix}\tag{4.43}$$

至此，我们已经介绍了李群李代数上的微分运算。之后的章节中，我们将应用这些知识去解决实际问题。关于李群李代数的某些重要数学性质，我们作为习题留给读者。

4.4　实践：Sophus

4.4.1　Sophus 的基本使用方法

我们已经介绍了李代数的入门知识，现在是通过实践演练巩固一下所学知识的时候了。我们来讨论如何在程序中操作李代数。在第 3 讲中，我们看到 Eigen 提供了几何模块，但没有提供李代数的支持。一个较好的李代数库是 Strasdat 维护的 Sophus 库[②]。Sophus 库支持本章主要讨论的 SO(3) 和 SE(3)，此外还含有二维运动 SO(2), SE(2) 以及相似变换 Sim(3) 的内容。它是直接在 Eigen 基础上开

[①] 我会读作“咚”，像一个石子掉在井里的声音。

[②] 最早提出李代数的是 Sophus Lie，这个库就以他的名字命名了。

发的，我们不需要安装额外的依赖库。读者可以直接从 GitHub 上获取 Sophus，或者，在本书的代码目录 slambook/3rdparty 下也提供了 Sophus 源代码。由于历史原因，Sophus 早期版本只提供了双精度的李群/李代数类。后续版本改写成了模板类。模板类的 Sophus 中可以使用不同精度的李群/李代数，但同时增加了使用难度。在本书第二版中，我们使用**带模板**的 Sophus 库。本书的 3rdparty 中提供的 Sophus 是**模板**版本，它应该在你下载本书代码的时候就已经复制下来了。Sophus 本身亦是一个 cmake 工程。想必你已经了解如何编译 cmake 工程了，这里不再赘述。Sophus 库只需编译即可，无须安装。

下面来演示一下 Sophus 库中的 SO(3) 和 SE(3) 运算：

slambook/ch4/useSophus.cpp

```cpp
#include <iostream>
#include <cmath>
#include <Eigen/Core>
#include <Eigen/Geometry>
#include "sophus/se3.hpp"

using namespace std;
using namespace Eigen;

/// 本程序演示sophus的基本用法
int main(int argc, char **argv) {
  // 沿Z轴转90度的旋转矩阵
  Matrix3d R = AngleAxisd(M_PI / 2, Vector3d(0, 0, 1)).toRotationMatrix();
  // 或者四元数
  Quaterniond q(R);
  Sophus::SO3d SO3_R(R);              // Sophus::SO3d可以直接从旋转矩阵构造
  Sophus::SO3d SO3_q(q);              // 也可以通过四元数构造
  // 二者是等价的
  cout << "SO(3) from matrix:\n" << SO3_R.matrix() << endl;
  cout << "SO(3) from quaternion:\n" << SO3_q.matrix() << endl;
  cout << "they are equal" << endl;

  // 使用对数映射获得它的李代数
  Vector3d so3 = SO3_R.log();
  cout << "so3 = " << so3.transpose() << endl;
  // hat 为向量到反对称矩阵
  cout << "so3 hat=\n" << Sophus::SO3d::hat(so3) << endl;
  // 相对的，vee为反对称到向量
  cout << "so3 hat vee= " << Sophus::SO3d::vee(Sophus::SO3d::hat(so3)).transpose() << endl;

  // 增量扰动模型的更新
  Vector3d update_so3(1e-4, 0, 0); //假设更新量为这么多
  Sophus::SO3d SO3_updated = Sophus::SO3d::exp(update_so3) * SO3_R;
  cout << "SO3 updated = \n" << SO3_updated.matrix() << endl;

  cout << "*******************************" << endl;
  // 对SE(3)操作大同小异
  Vector3d t(1, 0, 0);           // 沿X轴平移1
  Sophus::SE3d SE3_Rt(R, t);           // 从R,t构造SE(3)
```

```
  Sophus::SE3d SE3_qt(q, t);            // 从q,t构造SE(3)
  cout << "SE3 from R,t= \n" << SE3_Rt.matrix() << endl;
  cout << "SE3 from q,t= \n" << SE3_qt.matrix() << endl;
  // 李代数se(3) 是一个六维向量，方便起见先typedef一下
  typedef Eigen::Matrix<double, 6, 1> Vector6d;
  Vector6d se3 = SE3_Rt.log();
  cout << "se3 = " << se3.transpose() << endl;
  // 观察输出，会发现在Sophus中，se(3)的平移在前，旋转在后.
  // 同样的，有hat和vee两个算符
  cout << "se3 hat = \n" << Sophus::SE3d::hat(se3) << endl;
  cout << "se3 hat vee = " << Sophus::SE3d::vee(Sophus::SE3d::hat(se3)).transpose() << endl;

  // 最后，演示一下更新
  Vector6d update_se3; //更新量
  update_se3.setZero();
  update_se3(0, 0) = 1e-4d;
  Sophus::SE3d SE3_updated = Sophus::SE3d::exp(update_se3) * SE3_Rt;
  cout << "SE3 updated = " << endl << SE3_updated.matrix() << endl;

  return 0;
}
```

该演示程序分为两部分。前半部分介绍 SO(3) 上的操作，后半部分则为 SE(3)。我们演示了如何构造 SO(3), SE(3) 对象，对它们进行指数、对数映射，以及当知道更新量后，如何对李群元素进行更新。如果读者切实理解了本讲内容，那么这个程序对你来说应该没有什么难度。为了编译它，请在 CMakeLists.txt 里添加以下几行：

slambook/ch4/useSophus/CMakeLists.txt

```
# 为使用 sophus，需要使用 find_package 命令找到它
find_package( Sophus REQUIRED )
include_directories( ${Sophus_INCLUDE_DIRS} )

add_executable( useSophus useSophus.cpp )
```

find_package 命令是 cmake 提供的寻找某个库的头文件与库文件的指令。如果 cmake 能够找到它，就会提供头文件和库文件所在的目录的变量。在 Sophus 这个例子中，就是 Sophus_INCLUDE_DIRS。基于模板的 Sophus 库和 Eigen 一样，是仅含头文件而没有源文件的。根据它们，我们就能将 Sophus 库引入自己的 cmake 工程了。请读者自行查看此程序的输出信息，它与我们之前的推导是一致的。

4.4.2　例子：评估轨迹的误差

在实际工程中，我们经常需要评估一个算法的估计轨迹与真实轨迹的差异，来评价算法的精度。真实轨迹往往通过某些更高精度的系统获得，而估计估计则是待评价的算法计算得到。上一章我们演示了如何显示存储在文件中的某条轨迹，本节我们来考虑如何计算两条轨迹的误差。考虑一条估

计轨迹 $\boldsymbol{T}_{\text{esti},i}$ 和真实轨迹 $\boldsymbol{T}_{\text{gt},i}$，其中 $i=1,\cdots,N$，那么我们可以定义一些误差指标来描述它们之间的差别。

误差指标可以有很多种，常见的有**绝对轨迹误差**（Absolute Trajectory Error, ATE），形如：

$$\text{ATE}_{\text{all}}=\sqrt{\frac{1}{N}\sum_{i=1}^{N}\|\log(\boldsymbol{T}_{\text{gt},i}^{-1}\boldsymbol{T}_{\text{esti},i})^{\vee}\|_2^2}, \tag{4.44}$$

这实际上是每个位姿李代数的**均方根误差**（Root-Mean-Squared Error, RMSE）。这种误差可以刻画两条轨迹的旋转和平移误差。同时，也有的文献仅考虑平移误差 [23]，从而可以定义**绝对平移误差**（Average Translational Error）：

$$\text{ATE}_{\text{trans}}=\sqrt{\frac{1}{N}\sum_{i=1}^{N}\|\text{trans}(\boldsymbol{T}_{\text{gt},i}^{-1}\boldsymbol{T}_{\text{esti},i})\|_2^2}, \tag{4.45}$$

其中 trans 表示取括号内部变量的平移部分。因为从整条轨迹上来看，旋转出现误差后，随后的轨迹在平移上也会出现误差，所以两种指标在实际当中都适用。

除此之外，也可以定义相对的误差。例如考虑 i 时刻到 $i+\Delta t$ 时刻的运动，那么相对位姿误差（Relative Pose Error, RPE）可定义为：

$$\text{RPE}_{\text{all}}=\sqrt{\frac{1}{N-\Delta t}\sum_{i=1}^{N-\Delta t}\|\log\left(\left(\boldsymbol{T}_{\text{gt},i}^{-1}\boldsymbol{T}_{\text{gt},i+\Delta t}\right)\right)^{-1}\left(\boldsymbol{T}_{\text{esti},i}^{-1}\boldsymbol{T}_{\text{esti},i+\Delta t}\right)\Big)^{\vee}\|_2^2}, \tag{4.46}$$

同样地，亦可只取平移部分：

$$\text{RPE}_{\text{trans}}=\sqrt{\frac{1}{N-\Delta t}\sum_{i=1}^{N-\Delta t}\|\text{trans}\left(\left(\boldsymbol{T}_{\text{gt},i}^{-1}\boldsymbol{T}_{\text{gt},i+\Delta t}\right)\right)^{-1}\left(\boldsymbol{T}_{\text{esti},i}^{-1}\boldsymbol{T}_{\text{esti},i+\Delta t}\right)\Big)\|_2^2}. \tag{4.47}$$

利用 Sophus 库，很容易实现这部分计算。下面我们演示一下绝对轨迹误差的计算。在这个例子中，我们有 groundtruth.txt 和 estimated.txt 两条轨迹，下面的代码将读取这两条轨迹，计算误差，然后显示到 3D 窗口中。为简洁起见，省略了画轨迹部分的代码，在上一讲中我们已经做过类似的工作。

slambook/ch4/example/trajectoryError.cpp（部分）

```
#include <iostream>
#include <fstream>
#include <unistd.h>
#include <pangolin/pangolin.h>
#include <sophus/se3.hpp>

using namespace Sophus;
using namespace std;
```

```
9
10 string groundtruth_file = "./example/groundtruth.txt";
11 string estimated_file = "./example/estimated.txt";
12
13 typedef vector<Sophus::SE3d, Eigen::aligned_allocator<Sophus::SE3d>> TrajectoryType;
14
15 void DrawTrajectory(const TrajectoryType &gt, const TrajectoryType &esti);
16
17 TrajectoryType ReadTrajectory(const string &path);
18
19 int main(int argc, char **argv) {
20   TrajectoryType groundtruth = ReadTrajectory(groundtruth_file);
21   TrajectoryType estimated = ReadTrajectory(estimated_file);
22   assert(!groundtruth.empty() && !estimated.empty());
23   assert(groundtruth.size() == estimated.size());
24
25   // compute rmse
26   double rmse = 0;
27   for (size_t i = 0; i < estimated.size(); i++) {
28     Sophus::SE3d p1 = estimated[i], p2 = groundtruth[i];
29     double error = (p2.inverse() * p1).log().norm();
30     rmse += error * error;
31   }
32   rmse = rmse / double(estimated.size());
33   rmse = sqrt(rmse);
34   cout << "RMSE = " << rmse << endl;
35
36   DrawTrajectory(groundtruth, estimated);
37   return 0;
38 }
39
40 TrajectoryType ReadTrajectory(const string &path) {
41   ifstream fin(path);
42   TrajectoryType trajectory;
43   if (!fin) {
44     cerr << "trajectory " << path << " not found." << endl;
45     return trajectory;
46   }
47
48   while (!fin.eof()) {
49     double time, tx, ty, tz, qx, qy, qz, qw;
50     fin >> time >> tx >> ty >> tz >> qx >> qy >> qz >> qw;
51     Sophus::SE3d p1(Eigen::Quaterniond(qx, qy, qz, qw), Eigen::Vector3d(tx, ty, tz));
52     trajectory.push_back(p1);
53   }
54   return trajectory;
55 }
```

该程序输出的结果为 2.207，图像如图 4-2所示。读者也以尝试将旋转部分去掉，仅计算平移部分的误差。就这个例子来说，我们事实上已经帮助读者做了一些预处理任务，包括轨迹的时间对齐、外参预估，这些内容现在还没有讲到，我们将在以后的学习中谈论。

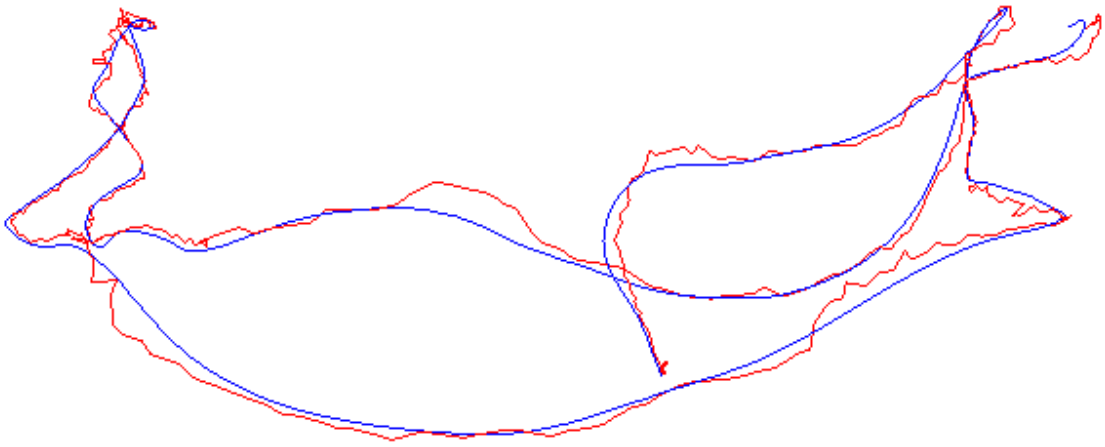


图 4-2 计算估计轨迹与真实轨迹之间的误差。

4.5 * 相似变换群与李代数

最后，我们要提一下在单目视觉中使用的相似变换群 Sim(3)，以及对应的李代数 $\mathfrak{sim}(3)$。如果你只对双目 SLAM 或 RGB-D SLAM 感兴趣，可以跳过本节。

我们已经介绍过单目的尺度不确定性。如果在单目 SLAM 中使用 SE(3) 表示位姿，那么由于尺度不确定性与尺度漂移，整个 SLAM 过程中的尺度会发生变化，这在 SE(3) 中未能体现出来。因此，在单目情况下我们一般会显式地把尺度因子表达出来。用数学语言来说，对于位于空间的点 $\boldsymbol{p}$，在相机坐标系下要经过一个**相似变换**，而非欧氏变换：

$$\boldsymbol{p}' = \begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \boldsymbol{p} = s\boldsymbol{R}\boldsymbol{p} + \boldsymbol{t}. \tag{4.48}$$

在相似变换中，我们把尺度 s 表达了出来。它同时作用在 $\boldsymbol{p}$ 的 3 个坐标之上，对 $\boldsymbol{p}$ 进行了一次缩放。与 SO(3)、SE(3) 相似，相似变换亦对矩阵乘法构成群，称为相似变换群 Sim(3)：

$$\mathrm{Sim}(3) = \left\{ \boldsymbol{S} = \begin{bmatrix} s\boldsymbol{R} & \boldsymbol{t} \\ \boldsymbol{0}^{\mathrm{T}} & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4} \right\}. \tag{4.49}$$

同样地，Sim(3) 也有对应的李代数、指数映射、对数映射等。李代数 $\mathfrak{sim}(3)$ 元素是一个 7 维向

量 $\boldsymbol{\zeta}$。它的前 6 维与 $\mathfrak{se}(3)$ 相同，最后多了一项 σ。

$$\mathfrak{sim}(3)=\left\{\boldsymbol{\zeta}|\boldsymbol{\zeta}=\begin{bmatrix}\boldsymbol{\rho}\\ \boldsymbol{\phi}\\ \sigma\end{bmatrix}\in\mathbb{R}^7,\boldsymbol{\zeta}^\wedge=\begin{bmatrix}\sigma\boldsymbol{I}+\boldsymbol{\phi}^\wedge & \boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 0\end{bmatrix}\in\mathbb{R}^{4\times4}\right\}. \tag{4.50}$$

它比 $\mathfrak{se}(3)$ 多了一项 σ。关联 Sim(3) 和 $\mathfrak{sim}(3)$ 的仍是指数映射和对数映射。指数映射为

$$\exp\left(\boldsymbol{\zeta}^\wedge\right)=\begin{bmatrix}\mathrm{e}^\sigma\exp\left(\boldsymbol{\phi}^\wedge\right) & \boldsymbol{J}_s\boldsymbol{\rho}\\ \mathbf{0}^{\mathrm{T}} & 1\end{bmatrix}. \tag{4.51}$$

其中 $\boldsymbol{J}_s$ 形式为

$$\begin{aligned}\boldsymbol{J}_s=&\frac{\mathrm{e}^\sigma-1}{\sigma}\boldsymbol{I}+\frac{\sigma\mathrm{e}^\sigma\sin\theta+(1-\mathrm{e}^\sigma\cos\theta)\theta}{\sigma^2+\theta^2}\boldsymbol{a}^\wedge\\&+\left(\frac{\mathrm{e}^\sigma-1}{\sigma}-\frac{(\mathrm{e}^\sigma\cos\theta-1)\sigma+(\mathrm{e}^\sigma\sin\theta)\theta}{\sigma^2+\theta^2}\right)\boldsymbol{a}^\wedge\boldsymbol{a}^\wedge.\end{aligned}$$

通过指数映射，我们能够找到李代数与李群的关系。对于李代数 $\boldsymbol{\zeta}$，它与李群的对应关系为：

$$s=\mathrm{e}^\sigma,\ \boldsymbol{R}=\exp(\boldsymbol{\phi}^\wedge),\ \boldsymbol{t}=\boldsymbol{J}_s\boldsymbol{\rho}. \tag{4.52}$$

旋转部分和 SO(3) 是一致的。平移部分，在 $\mathfrak{se}(3)$ 中需要乘一个雅可比 $\boldsymbol{\mathcal{J}}$，而相似变换的雅可比更复杂一些。对于尺度因子，可以看到李群中的 s 即为李代数中 σ 的指数函数。

Sim(3) 的 BCH 近似与 SE(3) 是类似的。我们可以讨论一个点 $\boldsymbol{p}$ 经过相似变换 $\boldsymbol{Sp}$ 后，相对于 $\boldsymbol{S}$ 的导数。同样地，存在微分模型和扰动模型两种方式，而扰动模型较为简单。我们省略推导过程，直接给出扰动模型的结果。设给予 $\boldsymbol{Sp}$ 左侧一个小扰动 $\exp(\boldsymbol{\zeta}^\wedge)$，并求 $\boldsymbol{Sp}$ 对于扰动的导数。因为 $\boldsymbol{Sp}$ 是 4 维的齐次坐标，$\boldsymbol{\zeta}$ 是 7 维向量，该导数应该是 4×7 的雅可比。为了方便起见，记 $\boldsymbol{Sp}$ 的前 3 维组成向量 $\boldsymbol{q}$，那么：

$$\frac{\partial\boldsymbol{Sp}}{\partial\boldsymbol{\zeta}}=\begin{bmatrix}\boldsymbol{I} & -\boldsymbol{q}^\wedge & \boldsymbol{q}\\ \mathbf{0}^{\mathrm{T}} & \mathbf{0}^{\mathrm{T}} & 0\end{bmatrix}. \tag{4.53}$$

关于 Sim(3)，我们就介绍到这里。更详细的关于 Sim(3) 的资料，建议读者参见文献 [24]。

4.6　小结

本讲引入了李群 SO(3) 和 SE(3)，以及它们对应的李代数 $\mathfrak{so}(3)$ 和 $\mathfrak{se}(3)$。我们介绍了位姿在它们上面的表达和转换，然后通过 BCH 的线性近似，就可以对位姿进行扰动并求导了。这给之后讲解

位姿的优化打下了理论基础，因为我们需要经常地对某一个位姿的估计值进行调整，使它对应的误差减小。只有在弄清楚如何对位姿进行调整和更新之后，我们才能继续下一步的内容。

本讲的内容可能比较偏理论化，毕竟它不像计算机视觉那样经常有好看的图片可以展示。相比于讲解李群李代数的数学教科书，由于我们只关心实用的内容，所以讲的过程非常精简，速度也相对快了一些。请读者务必理解本章内容，它是解决后续许多问题的基础，特别是位姿估计部分。

需要一提的是，除了李代数之外，同样也可以用四元数、欧拉角等方式表示旋转，只是后续的处理要麻烦一些。在实际应用中，也可以使用 SO(3) 加上平移的方式来代替 SE(3)，从而回避一些雅可比的计算。

习题

1. 验证 SO(3)、SE(3) 和 Sim(3) 关于乘法成群。
2. 验证 $(\mathbb{R}^3, \mathbb{R}, \times)$ 构成李代数。
3. 验证 $\mathfrak{so}(3)$ 和 $\mathfrak{se}(3)$ 满足李代数要求的性质。
4. 验证性质（4.20）和（4.21）。
5. 证明：
$$\boldsymbol{R}\boldsymbol{p}^{\wedge}\boldsymbol{R}^{\mathrm{T}} = (\boldsymbol{R}\boldsymbol{p})^{\wedge}.$$
6. 证明：
$$\boldsymbol{R}\exp(\boldsymbol{p}^{\wedge})\boldsymbol{R}^{\mathrm{T}} = \exp((\boldsymbol{R}\boldsymbol{p})^{\wedge}).$$
该式称为 SO(3) 上的**伴随**性质。同样地，在 SE(3) 上亦有伴随性质：
$$\boldsymbol{T}\exp(\boldsymbol{\xi}^{\wedge})\boldsymbol{T}^{-1} = \exp\left((\mathrm{Ad}(\boldsymbol{T})\boldsymbol{\xi})^{\wedge}\right), \tag{4.54}$$
其中：
$$\mathrm{Ad}(\boldsymbol{T}) = \begin{bmatrix} \boldsymbol{R} & \boldsymbol{t}^{\wedge}\boldsymbol{R} \\ \boldsymbol{0} & \boldsymbol{R} \end{bmatrix}. \tag{4.55}$$
7. 仿照左扰动的推导，推导 SO(3) 和 SE(3) 在右扰动下的导数。
8. 搜索 cmake 的 find_package 指令是如何运作的。它有哪些可选的参数？为了让 cmake 找到某个库，需要哪些先决条件？

第五章　相机与图像

主要目标

1. 理解针孔相机的模型、内参与径向畸变参数。
2. 理解一个空间点是如何投影到相机成像平面的。
3. 掌握 OpenCV 的图像存储与表达方式。
4. 学会基本的摄像头标定方法。

前面两讲中，我们介绍了“机器人如何表示自身位姿”的问题，部分地解释了 SLAM 经典模型中变量的含义和运动方程部分。本讲将讨论“机器人如何观测外部世界”，也就是观测方程部分。而在以相机为主的视觉 SLAM 中，观测主要是指**相机成像**的过程。

我们在现实生活中能看到大量的照片。在计算机中，一张照片由很多个像素组成，每个像素记录了色彩或亮度的信息。三维世界中的一个物体反射或发出的光线，穿过相机光心后，投影在相机的成像平面上。相机的感光器件接收到光线后，产生测量值，就得到了像素，形成了我们见到的照片。这个过程能否用数学原理来描述呢？本讲将首先讨论相机模型，说明投影关系具体如何描述，相机的内参是什么。同时，简单介绍双目成像与 RGB-D 相机的原理。然后，介绍二维照片像素的基本操作。最后，根据内外参数的含义，演示一个点云拼接的实验。

$$s\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K(Rp+t)$$

5.1　相机模型

相机将三维世界中的坐标点（单位为米）映射到二维图像平面（单位为像素）的过程能够用一个几何模型进行描述。这个模型有很多种，其中最简单的称为**针孔模型**。针孔模型是很常用而且有效的模型，它描述了一束光线通过针孔之后，在针孔背面投影成像的关系。在本书中我们用一个简单的针孔相机模型来对这种映射关系进行建模。同时，由于相机镜头上的透镜的存在，使得光线投影到成像平面的过程中会产生**畸变**。因此，我们使用针孔和畸变两个模型来描述整个投影过程。

在本节先给出相机的针孔模型，再对透镜的畸变模型进行讲解。这两个模型能够把外部的三维点投影到相机内部成像平面，构成相机的**内参数**（Intrinsics）。

5.1.1　针孔相机模型

在初中物理课堂上，我们可能都见过一个蜡烛投影实验：在一个暗箱的前方放着一支点燃的蜡烛，蜡烛的光透过暗箱上的一个小孔投影在暗箱的后方平面上，并在这个平面上形成一个倒立的蜡烛图像。在这个过程中，小孔模型能够把三维世界中的蜡烛投影到一个二维成像平面。同理，我们可以用这个简单的模型来解释相机的成像过程，如图 5-1所示。

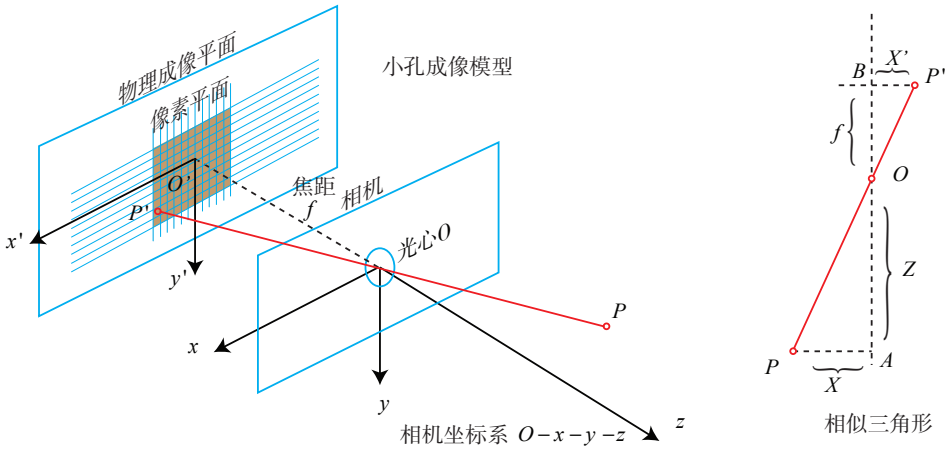


图 5-1　针孔相机模型。

现在来对这个简单的针孔模型进行几何建模。设 $O-x-y-z$ 为相机坐标系，习惯上我们让 z 轴指向相机前方，x 向右，y 向下（此图我们应该站在左侧看右侧）。O 为摄像机的**光心**，也是针孔模型中的针孔。现实世界的空间点 P，经过小孔 O 投影之后，落在物理成像平面 $O'-x'-y'$ 上，成像点为 P'。设 P 的坐标为 $[X,Y,Z]^{\mathrm{T}}$，P' 为 $[X',Y',Z']^{\mathrm{T}}$，并且设物理成像平面到小孔的距离为 f（焦距）。那么，根据三角形相似关系，有：

$$\frac{Z}{f} = -\frac{X}{X'} = -\frac{Y}{Y'}. \tag{5.1}$$

其中负号表示成的像是倒立的。不过，实际相机得到的图像并不是倒像（否则相机的使用会非常不方便）。为了让模型更符合实际，我们可以等价地把成像平面对称地放到相机前方，和三维空间点一起放在摄像机坐标系的同一侧，如图 5-2所示。这样做可以把公式中的负号去掉，使式子更加简洁：

$$\frac{Z}{f} = \frac{X}{X'} = \frac{Y}{Y'}. \tag{5.2}$$

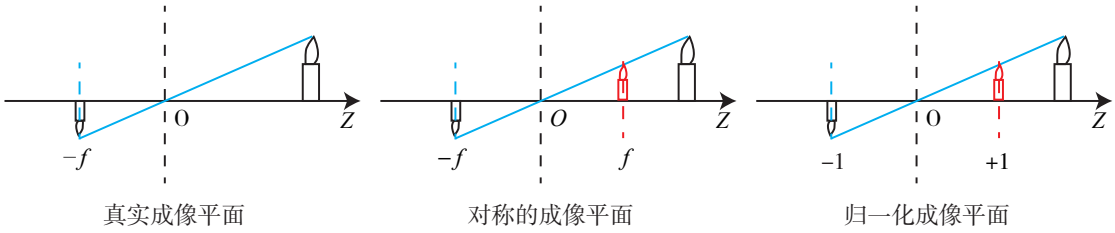


图 5-2 真实成像平面，对称成像平面，归一化成像平面的图示。

把 X', Y' 放到等式左侧，整理得：

$$\begin{aligned} X' &= f\frac{X}{Z} \\ Y' &= f\frac{Y}{Z} \end{aligned}. \tag{5.3}$$

读者可能要问，为什么我们可以看似随意地把成像平面挪到前方呢？这只是我们处理真实世界与相机投影的数学手段，并且，大多数相机输出的图像并不是倒像——相机自身的软件会帮你翻转这张图像，所以我们实际得到的是正像，也就是对称的成像平面上的像。所以，尽管从物理原理来说，小孔成像应该是倒像，但由于我们对图像作了预处理，所以理解成在对称平面上的像并不会带来什么坏处。于是，在不引起歧义的情况下，我们也不加限制地称后一种情况为针孔模型。

式(5.3)描述了点 P 和它的像之间的空间关系，这里所有点的单位都可理解成米，比如焦距是 0.2 米，X' 是 0.14 米。不过，在相机中，我们最终获得的是一个个的像素，这还需要在成像平面上对像进行采样和量化。为了描述传感器将感受到的光线转换成图像像素的过程，我们设在物理成像平面上固定着一个像素平面 $o-u-v$。我们在像素平面得到了 P' 的**像素坐标**：$[u, v]^{\mathrm{T}}$。

像素坐标系[①]通常的定义方式是：原点 o' 位于图像的左上角，u 轴向右与 x 轴平行，v 轴向下与 y 轴平行。像素坐标系与成像平面之间，相差了一个**缩放**和一个**原点的平移**。我们设像素坐标在

[①] 或图像坐标系，见本讲第 2 节。

u 轴上缩放了 α 倍，在 v 上缩放了 β 倍。同时，原点平移了 $[c_x, c_y]^{\mathrm{T}}$。那么，P' 的坐标与像素坐标 $[u, v]^{\mathrm{T}}$ 的关系为：

$$\begin{cases} u = \alpha X' + c_x \\ v = \beta Y' + c_y \end{cases}. \tag{5.4}$$

代入式(5.3)并把 αf 合并成 f_x，把 βf 合并成 f_y，得：

$$\begin{cases} u = f_x \frac{X}{Z} + c_x \\ v = f_y \frac{Y}{Z} + c_y \end{cases}. \tag{5.5}$$

其中，f 的单位为米，α, β 的单位为像素/米，所以 f_x, f_y 和 c_x, c_y 的单位为像素。把该式写成矩阵形式会更加简洁，不过左侧需要用到齐次坐标，右侧则是非齐次坐标：

$$\begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \frac{1}{Z} \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq \frac{1}{Z} \boldsymbol{K} \boldsymbol{P}. \tag{5.6}$$

我们按照传统的习惯把 Z 挪到左侧：

$$Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \end{pmatrix} \triangleq \boldsymbol{K} \boldsymbol{P}. \tag{5.7}$$

该式中，我们把中间的量组成的矩阵称为相机的**内参数矩阵**（Camera Intrinsics）$\boldsymbol{K}$。通常认为，相机的内参在出厂之后是固定的，不会在使用过程中发生变化。有的相机生产厂商会告诉你相机的内参，而有时需要你自己确定相机的内参，也就是所谓的**标定**。鉴于标定算法业已成熟（如著名的单目棋盘格张正友标定法[25]），这里就不介绍了。

有内参，自然也有相对的外参。考虑到在式 (5.6) 中我们使用的是 P 在相机坐标系下的坐标。由于相机在运动，所以 P 的相机坐标应该是它的世界坐标（记为 $\boldsymbol{P}_w$）根据相机的当前位姿变换到相机坐标系下的结果。相机的位姿由它的旋转矩阵 $\boldsymbol{R}$ 和平移向量 $\boldsymbol{t}$ 来描述。那么有：

$$Z \boldsymbol{P}_{uv} = Z \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \boldsymbol{K} (\boldsymbol{R} \boldsymbol{P}_w + \boldsymbol{t}) = \boldsymbol{K} \boldsymbol{T} \boldsymbol{P}_w. \tag{5.8}$$

注意后一个式子隐含了一次齐次坐标到非齐次坐标的转换（你能看出来吗？）[①]。它描述了 P 的世

[①] 即，在 $\boldsymbol{TP}$ 中使用齐次坐标，再转化为非齐次坐标，再与 $\boldsymbol{K}$ 相乘。

界坐标到像素坐标的投影关系。其中，相机的位姿 $\boldsymbol{R}, \boldsymbol{t}$ 又称为相机的**外参数**（Camera Extrinsics）[①]。相比于不变的内参，外参会随着相机运动发生改变，同时也是 SLAM 中待估计的目标，代表着机器人的轨迹。

投影过程还可以从另一个角度来看。式(5.8)表明，我们可以把一个世界坐标点先转换到相机坐标系，再除掉它最后一维的数值（即该点距离相机成像平面的深度），这相当于把最后一维进行**归一化处理**，得到点 P 在相机**归一化平面**上的投影：

$$(\boldsymbol{R}\boldsymbol{P}_w + \boldsymbol{t}) = \underbrace{[X, Y, Z]^{\mathrm{T}}}_{\text{相机坐标}} \rightarrow \underbrace{[X/Z, Y/Z, 1]^{\mathrm{T}}}_{\text{归一化坐标}}. \tag{5.9}$$

归一化坐标可看成相机前方[②]$z = 1$ 处的平面上的一个点，这个 $z = 1$ 平面也称为**归一化平面**。归一化坐标再左乘内参就得到了像素坐标，所以我们可以把像素坐标 $[u, v]^{\mathrm{T}}$ 看成对归一化平面上的点进行量化测量的结果。从这个模型中也可以看出，如果对相机坐标同时乘以任意非零常数，归一化坐标都是一样的，这说明**点的深度在投影过程中被丢失了**，所以单目视觉中没法得到像素点的深度值。

5.1.2 畸变

为了获得好的成像效果，我们在相机的前方加了透镜。透镜的加入对成像过程中光线的传播会产生新的影响：一是透镜自身的形状对光线传播的影响，二是在机械组装过程中，透镜和成像平面不可能完全平行，这也会使得光线穿过透镜投影到成像面时的位置发生变化。

由透镜形状引起的**畸变**（Distortion，也叫失真）称为**径向畸变**。在针孔模型中，一条直线投影到像素平面上还是一条直线。可是，在实际拍摄的照片中，摄像机的透镜往往使得真实环境中的一条直线在图片中变成了曲线[③]。越靠近图像的边缘，这种现象越明显。由于实际加工制作的透镜往往是中心对称的，这使得不规则的畸变通常径向对称。它们主要分为两大类：**桶形畸变**和**枕形畸变**，如图 5-3所示。

桶形畸变是由于图像放大率随着与光轴之间的距离增加而减小，而枕形畸变则恰好相反。在这两种畸变中，穿过图像中心和光轴有交点的直线还能保持形状不变。

除了透镜的形状会引入径向畸变外，在相机的组装过程中由于不能使透镜和成像面严格平行也会引入**切向畸变**，如图 5-4所示。

为更好地理解径向畸变和切向畸变，我们用更严格的数学形式对两者进行描述。考虑**归一化平面**上的任意一点 $\boldsymbol{p}$，它的坐标为 $[x, y]^{\mathrm{T}}$，也可写成极坐标的形式 $[r, \theta]^{\mathrm{T}}$，其中 r 表示点 $\boldsymbol{p}$ 与坐标系原点之间的距离，θ 表示与水平轴的夹角。径向畸变可看成坐标点沿着长度方向发生了变化，也就是其距离原点的长度发生了变化。切向畸变可以看成坐标点沿着切线方向发生了变化，也就是水平夹

[①] 在机器人或自动驾驶车辆中，外参有时也解释成相机坐标系到机器人本体坐标系之间的变换，描述“相机安装在什么地方”。

[②] 注意在实际计算中需要检查 Z 是否为正，因为负数 Z 也可以通过这个方式得到归一化平面上的一个点，但相机并不会拍到成像平面后方的景物。

[③] 是的，它不再直了，而是变成弯的。如果往里弯，称为桶形失真；往外弯则是枕形失真。

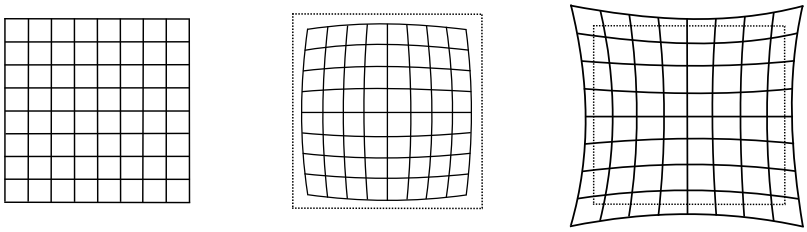


图 5-3　径向畸变的两种类型。

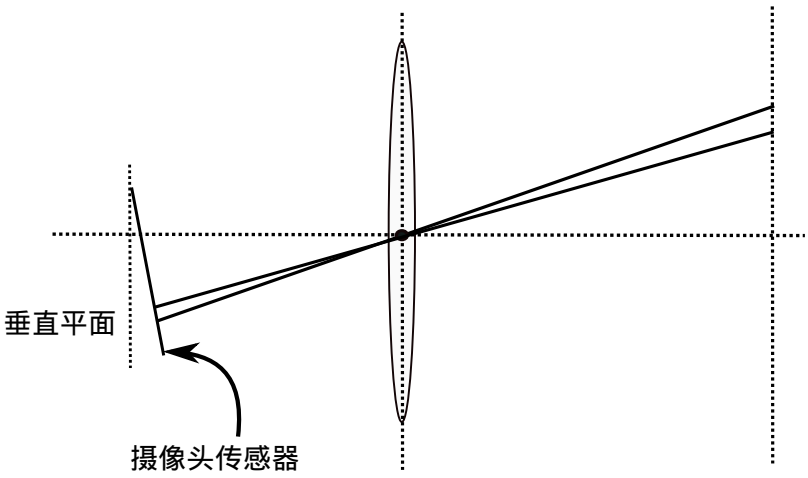


图 5-4　切向畸变来源示意图。

角发生了变化。通常假设这些畸变呈多项式关系，即：

$$\begin{aligned} x_{\text{distorted}} &= x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \\ y_{\text{distorted}} &= y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) \end{aligned}. \tag{5.10}$$

其中 $[x_{\text{distorted}}, y_{\text{distorted}}]^{\mathrm{T}}$ 是畸变后点的**归一化坐标**。另一方面，对于**切向畸变**，可以使用另外的两个参数 p_1, p_2 来进行纠正：

$$\begin{aligned} x_{\text{distorted}} &= x + 2p_1 xy + p_2(r^2 + 2x^2) \\ y_{\text{distorted}} &= y + p_1(r^2 + 2y^2) + 2p_2 xy \end{aligned}. \tag{5.11}$$

因此，联合式(5.10)和式(5.11)，对于相机坐标系中的一点 $\boldsymbol{P}$，我们能够通过 5 个畸变系数找到这个点在像素平面上的正确位置：

1. 将三维空间点投影到归一化图像平面。设它的归一化坐标为 $[x, y]^{\mathrm{T}}$。

2. 对归一化平面上的点计算径向畸变和切向畸变。

$$\begin{cases} x_{\text{distorted}} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2) \\ y_{\text{distorted}} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy \end{cases}. \tag{5.12}$$

3. 将畸变后的点通过内参数矩阵投影到像素平面，得到该点在图像上的正确位置。

$$\begin{cases} u = f_x x_{\text{distorted}} + c_x \\ v = f_y y_{\text{distorted}} + c_y \end{cases}. \tag{5.13}$$

在上面的纠正畸变的过程中，我们使用了 5 个畸变项。实际应用中，可以灵活选择纠正模型，比如只选择 k_1, p_1, p_2 这 3 项等。

在这一节中，我们对相机的成像过程使用针孔模型进行了建模，也对透镜引起的径向畸变和切向畸变进行了描述。实际的图像系统中，学者们提出了很多其他的模型，比如相机的仿射模型和透视模型等，同时也存在很多其他类型的畸变。考虑到视觉 SLAM 中一般都使用普通的摄像头，针孔模型及径向畸变和切向畸变模型已经足够，因此，我们不再对其他模型进行描述。

值得一提的是，存在两种去畸变处理（Undistort，或称畸变校正）做法。我们可以选择先对整张图像进行去畸变，得到去畸变后的图像，然后讨论此图像上的点的空间位置。或者，也可以从畸变图像上的某个点出发，按照畸变方程，讨论其畸变前的空间位置。二者都是可行的，不过前者在视觉 SLAM 中似乎更加常见一些。所以，当一个图像去畸变之后，我们就可以直接用针孔模型建立投影关系，而不用考虑畸变了。因此，在后文的讨论中，我们可以直接假设图像已经进行了去畸变处理。

最后，我们小结一下单目相机的成像过程：

1. 首先，世界坐标系下有一个固定的点 P，世界坐标为 $\boldsymbol{P}_w$。
2. 由于相机在运动，它的运动由 $\boldsymbol{R}, \boldsymbol{t}$ 或变换矩阵 $\boldsymbol{T} \in \text{SE(3)}$ 描述。P 的相机坐标为 $\tilde{\boldsymbol{P}}_c = \boldsymbol{R}\boldsymbol{P}_w + \boldsymbol{t}$。
3. 这时的 $\tilde{\boldsymbol{P}}_c$ 的分量为 X, Y, Z，把它们投影到归一化平面 $Z = 1$ 上，得到 P 的归一化坐标：$\boldsymbol{P}_c = [X/Z, Y/Z, 1]^{\text{T}}$[①]。
4. 有畸变时，根据畸变参数计算 $\boldsymbol{P}_c$ 发生畸变后的坐标。
5. 最后，P 的归一化坐标经过内参后，对应到它的像素坐标：$\boldsymbol{P}_{uv} = \boldsymbol{K}\boldsymbol{P}_c$。

综上所述，我们一共谈到了四种坐标：世界坐标、相机坐标、归一化坐标和像素坐标。请读者厘清它们的关系，它反映了整个成像的过程。

[①] 注意到 Z 可能小于 1，说明该点位于归一化平面后面，它可能不会在相机平面上成像，实践当中要检查一次。

5.1.3　双目相机模型

针孔相机模型描述了单个相机的成像模型。然而，仅根据一个像素，我们无法确定这个空间点的具体位置。这是因为，从相机光心到归一化平面连线上的所有点，都可以投影至该像素上。只有当 P 的深度确定时（比如通过双目或 RGB-D 相机），我们才能确切地知道它的空间位置。如图 5-5 所示。

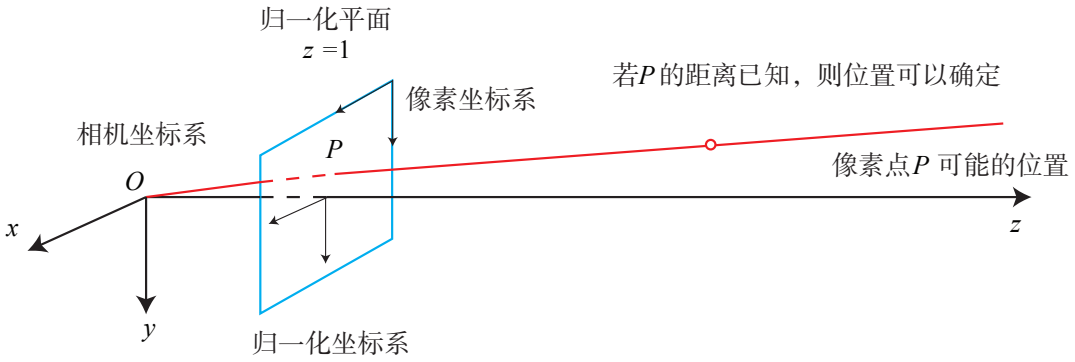


图 5-5　像素点可能存在的位置。

测量像素距离（或深度）的方式有很多种，比如人眼就可以根据左右眼看到的景物差异（或称视差）来判断物体离我们的距离。双目相机的原理亦是如此：通过同步采集左右相机的图像，计算图像间视差，来估计每一个像素的深度。下面简单讲讲双目相机的成像原理（如图 5-6 所示）。

双目相机一般由左眼相机和右眼相机两个水平放置的相机组成。当然也可以做成上下两个目[①]，不过我们见到的主流双目都是做成左右形式的。在左右双目相机中，我们可以把两个相机都看作针孔相机。它们是水平放置的，意味着两个相机的光圈中心都位于 x 轴上。两者之间的距离称为双目相机的**基线**（Baseline，记作 b），是双目相机的重要参数。

现在，考虑一个空间点 P，它在左眼相机和右眼相机各成一像，记作 P_L, P_R。由于相机基线的存在，这两个成像位置是不同的。理想情况下，由于左右相机只在 x 轴上有位移，因此 P 的像也只在 x 轴（对应图像的 u 轴）上有差异。记它的左侧坐标为 u_L，右侧坐标为 u_R，几何关系如图 5-6右侧所示。根据 $\triangle PP_LP_R$ 和 $\triangle PO_LO_R$ 的相似关系，有：

$$\frac{z-f}{z} = \frac{b-u_L+u_R}{b}. \tag{5.14}$$

稍加整理，得：

$$z = \frac{fb}{d}, \quad d \triangleq u_L - u_R. \tag{5.15}$$

[①] 那样的话外观会有些奇特。

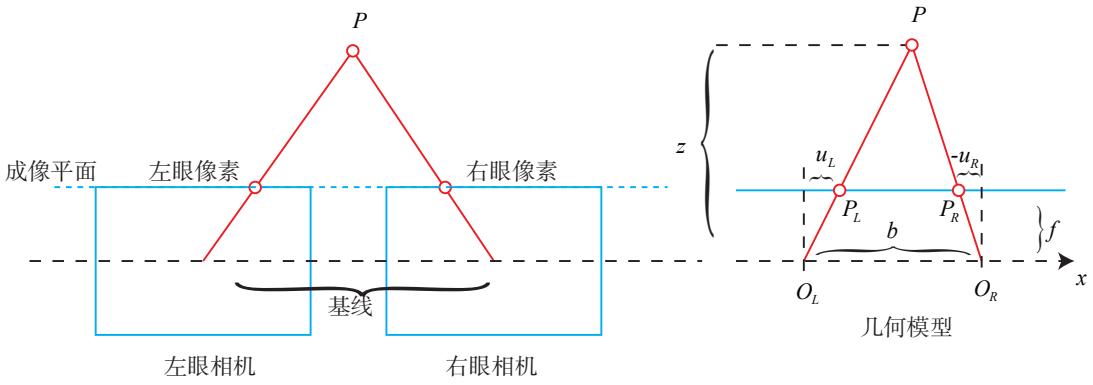


图 5-6 双目相机的成像模型。O_L, O_R 为左右光圈中心，方框为成像平面，f 为焦距。u_L 和 u_R 为成像平面的坐标。请注意，按照图中坐标定义，u_R 应该是负数，所以图中标出的距离为 $-u_R$。

其中 d 定义为左右图的横坐标之差，称为**视差**（Disparity）。根据视差，我们可以估计一个像素与相机之间的距离。视差与距离成反比：视差越大，距离越近[①]。同时，由于视差最小为一个像素，于是双目的深度存在一个理论上的最大值，由 fb 确定。我们看到，当基线越长时，双目能测到的最大距离就会越远；反之，小型双目器件则只能测量很近的距离。类比地，我们人眼在看非常远的物体时（如很远的飞机），通常不能准确判断它的距离。

虽然由视差计算深度的公式很简洁，但视差 d 本身的计算却比较困难。我们需要确切地知道左眼图像某个像素出现在右眼图像的哪一个位置（即对应关系），这件事亦属于“人类觉得容易而计算机觉得困难”的任务。当我们想计算每个像素的深度时，其计算量与精度都将成为问题，而且只有在图像纹理变化丰富的地方才能计算视差。由于计算量的原因，双目深度估计仍需要使用 GPU 或 FPGA 来实时计算。这将在第 13 讲中提到。

5.1.4 RGB-D 相机模型

相比于双目相机通过视差计算深度的方式，RGB-D 相机的做法更为“主动”一些，它能够主动测量每个像素的深度。目前的 RGB-D 相机按原理可分为两大类（见图 5-7）：

1. 通过**红外结构光**（Structured Light）来测量像素距离的。例子有 Kinect 1 代、Project Tango 1 代、Intel RealSense 等。
2. 通过**飞行时间法**（Time-of-flight，ToF）原理测量像素距离的。例子有 Kinect 2 代和一些现有的 ToF 传感器等。

[①] 读者可以自己用眼睛模拟一下。

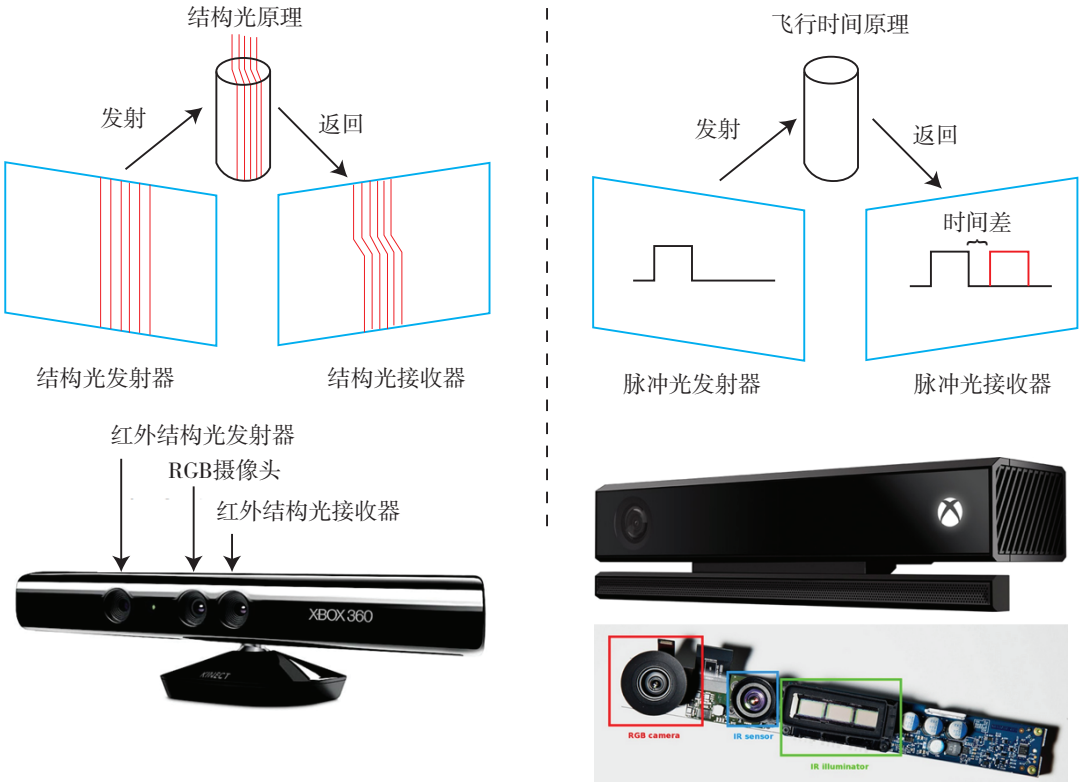


图 5-7　RGB-D 相机原理示意图。

无论是哪种类型，RGB-D 相机都需要向探测目标发射一束光线（通常是红外光）。在结构光原理中，相机根据返回的结构光图案，计算物体与自身之间的距离。而在 ToF 原理中，相机向目标发射脉冲光，然后根据发送到返回之间的光束飞行时间，确定物体与自身之间的距离。ToF 原理和激光传感器十分相似，只不过激光是通过逐点扫描来获取距离，而 ToF 相机则可以获得整个图像的像素深度，这也正是 RGB-D 相机的特点。所以，如果你把一个 RGB-D 相机拆开，通常会发现除了普通的摄像头之外，至少会有一个发射器和一个接收器。

在测量深度之后，RGB-D 相机通常按照生产时的各相机摆放位置，自己完成深度与彩色图像素之间的配对，输出一一对应的彩色图和深度图。我们可以在同一个图像位置，读取到色彩信息和距离信息，计算像素的 3D 相机坐标，生成点云（Point Cloud）。对 RGB-D 数据，既可以在图像层面进行处理，亦可在点云层面处理。本讲的第二个实验将演示 RGB-D 相机的点云构建过程。

RGB-D 相机能够实时地测量每个像素点的距离。但是，由于这种发射－接收的测量方式，其使用范围比较受限。用红外光进行深度值测量的 RGB-D 相机，容易受到日光或其他传感器发射的红

外光干扰，因此不能在室外使用。在没有调制的情况下，同时使用多个 RGB-D 相机时也会相互干扰。对于透射材质的物体，因为接收不到反射光，所以无法测量这些点的位置。此外，RGB-D 相机在成本、功耗方面，都有一些劣势。

5.2 图像

相机加上镜头，把三维世界中的信息转换成了一张由像素组成的照片，随后存储在计算机中，作为后续处理的数据来源。在数学中，图像可以用一个矩阵来描述；而在计算机中，它们占据一段连续的磁盘或内存空间，可以用二维数组来表示。这样一来，程序就不必区别它们处理的是一个数值矩阵，还是有实际意义的图像了。

本节，我们将介绍计算机图像处理的一些基本操作。特别地，通过 OpenCV 中图像数据的处理，理解计算机中处理图像的常见步骤，为后续章节打下基础。我们从最简单的图像——灰度图说起。在一张灰度图中，每个像素位置 (x, y) 对应一个灰度值 I，所以，一张宽度为 w、高度为 h 的图像，数学上可以记为一个函数：

$$I(x, y) : \mathbb{R}^2 \mapsto \mathbb{R}.$$

其中 (x, y) 是像素的坐标。然而，计算机并不能表达实数空间，所以我们需要对下标和图像读数在某个范围内进行量化。例如 x, y 通常是 0 开始的整数。在常见的灰度图中，用 0~255 的整数（即一个 unsigned char，1 个字节）来表达图像的灰度读数。那么，一张宽度为 640 像素、高度为 480 像素分辨率的灰度图就可以表示为：

二维数组表达图像

```
unsigned char image[480][640];
```

为什么这里的二维数组是 480×640 呢？因为在程序中，图像以二维数组形式存储。它的第一个下标是指数组的行，而第二个下标则是列。在图像中，数组的行数对应图像的高度，而列数对应图像的宽度。

下面考察这幅图像的内容。图像自然是由像素组成的。当访问某一个像素时，需要指明它所处的坐标，如图 5-8 所示。该图左边显示了传统像素坐标系的定义方式。像素坐标系原点位于图像的左上角，X 轴向右，Y 轴向下（也就是前面所说的 u, v 坐标）。如果它还有第三个轴——Z 轴，那么根据右手法则，Z 轴应该是向前的。这种定义方式是与相机坐标系一致的。我们平时说的图像的宽度或列数，对应着 X 轴；而图像的行数或高度，则对应着它的 Y 轴。

根据这种定义方式，如果我们讨论一个位于 x, y 处的像素，那么它在程序中的访问方式应该是：

访问图像像素

```
unsigned char pixel = image[y][x];
```

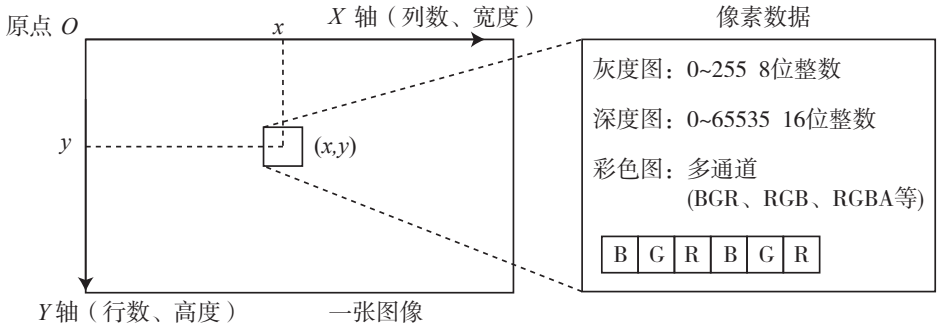


图 5-8　图像坐标示意图。

它对应着灰度值 $I(x,y)$ 的读数。请注意这里的 x 和 y 的顺序。虽然我们不厌其烦地讨论坐标系的问题，但是像这种下标顺序的错误，仍会是新手在调试过程中经常碰到的，且具有一定隐蔽性的错误之一。如果你在写程序时不慎调换了 x,y 的坐标，编译器无法提供任何信息，而你所能看到的只是程序运行中的一个越界错误而已。

一个像素的灰度可以用 8 位整数记录，也就是一个 0~255 的值。当我们要记录的信息更多时，一个字节恐怕就不够了。例如，在 RGB-D 相机的深度图中，记录了各个像素与相机之间的距离。这个距离通常是以毫米为单位，而 RGB-D 相机的量程通常在十几米左右，超过了 255。这时，人们会采用 16 位整数（C++ 中的 unsigned short）来记录深度图的信息，也就是位于 0~65535 的值。换算成米的话，最大可以表示 65 米，足够 RGB-D 相机使用了。

彩色图像的表示则需要通道（channel）的概念。在计算机中，我们用红色、绿色和蓝色这三种颜色的组合来表达任意一种色彩。于是对于每一个像素，就要记录其 R、G、B 三个数值，每一个数值就称为一个通道。例如，最常见的彩色图像有三个通道，每个通道都由 8 位整数表示。在这种规定下，一个像素占据 24 位空间。

通道的数量、顺序都是可以自由定义的。在 OpenCV 的彩色图像中，通道的默认顺序是 B、G、R。也就是说，当我们得到一个 24 位的像素时，前 8 位表示蓝色数值，中间 8 位为绿色，最后 8 位为红色。同理，亦可使用 R、G、B 的顺序表示一个彩色图。如果还想表达图像的透明度，就使用 R、G、B、A 四个通道。

5.3　实践：计算机中的图像

5.3.1　OpenCV 的基础使用方法

下面通过一个演示程序来理解，在 OpenCV 中图像是如何存取，而我们又是如何访问其中的像素的。

安装 OpenCV

OpenCV[①]提供了大量的开源图像算法，是计算机视觉中使用极广的图像处理算法库。本书也使用 OpenCV 做基本的图像处理。在使用之前，建议读者从源代码安装它。在 Ubuntu 下，有**从源代码安装**和**只安装库文件**两种方式可以选择：

1. 从源代码安装，是指从 OpenCV 网站下载所有的 OpenCV 源代码，并在机器上编译安装，以便使用。好处是可以选择的版本比较丰富，而且能看到源代码，不过需要花费一些编译时间。
2. 只安装库文件，是指通过 Ubuntu 来安装由 Ubuntu 社区人员已经编译好的库文件，这样就无须重新编译一遍。

由于我们使用较新版本的 OpenCV，所以必须从源代码来安装它。一来，可以调整一些编译选项，匹配编程环境（例如，需不需要 GPU 加速等）；再者，源代码安装可以使用一些额外的功能。OpenCV 目前维护了两个主要版本，分为 OpenCV 2.4 系列和 OpenCV 3 系列。本书使用 OpenCV **3** 系列。

由于 OpenCV 工程比较大，就不放在本书的 3rdparty 下了。请读者从 `http://opencv.org/downloads.html` 下载，选择 OpenCV for Linux 版本即可。你会获得一个像 opencv-3.1.0.zip 这样的压缩包。将它解压到任意目录下，我们发现 OpenCV 亦是一个 cmake 工程。

在编译之前，先来安装 OpenCV 的依赖项：

终端输入：

```
sudo apt-get install build-essential libgtk2.0-dev libvtk5-dev libjpeg-dev libtiff4-dev libjasper-dev
    libopenexr-dev libtbb-dev
```

事实上，OpenCV 的依赖项很多，缺少某些编译项会影响它的部分功能（不过我们也不会用到所有功能）。OpenCV 会在 cmake 阶段检查依赖项是否会安装，并调整自己的功能。如果你的电脑上有 GPU 并且安装了相关依赖项，OpenCV 也会把 GPU 加速打开。不过对于本书，上面那些依赖项就足够了。

随后的编译安装和普通的 cmake 工程一样，请在 make 之后，调用 sudo make install 将 OpenCV 安装到你的机器上（而不是仅仅编译它）。视机器配置，这个编译过程大概需要二十分钟到一个小时不等。如果你的 CPU 比较强，可以使用“make -j4”这样的命令，调用多个线程进行编译（-j 后面的参数就是使用的线程数量）。安装后，OpenCV 默认存储在/usr/local 目录下。你可以去寻找 OpenCV 头文件与库文件的安装位置，看看它们都在哪里。另外，如果之前已经安装了 OpenCV 2 系列，那么建议你把 OpenCV 3 安装到别的地方（想想这应该如何操作）。

操作 OpenCV 图像

接下来通过一个例程熟悉一下 OpenCV 对图像的操作。

[①] 官方主页：`http://opencv.org`。

slambook/ch5/imageBasics/imageBasics.cpp

```cpp
#include <iostream>
#include <chrono>

using namespace std;

#include <opencv2/core/core.hpp>
#include <opencv2/highgui/highgui.hpp>

int main(int argc, char **argv) {
  // 读取argv[1]指定的图像
  cv::Mat image;
  image = cv::imread(argv[1]); //cv::imread函数读取指定路径下的图像

  // 判断图像文件是否正确读取
  if (image.data == nullptr) { //数据不存在,可能是文件不存在
    cerr << "文件" << argv[1] << "不存在." << endl;
    return 0;
  }

  // 文件顺利读取, 首先输出一些基本信息
  cout << "图像宽为" << image.cols << ",高为" << image.rows
       << ",通道数为" << image.channels() << endl;
  cv::imshow("image", image);      // 用cv::imshow显示图像
  cv::waitKey(0);                  // 暂停程序,等待一个按键输入

  // 判断image的类型
  if (image.type() != CV_8UC1 && image.type() != CV_8UC3) {
    // 图像类型不符合要求
    cout << "请输入一张彩色图或灰度图." << endl;
    return 0;
  }

  // 遍历图像, 请注意以下遍历方式亦可使用于随机像素访问
  // 使用 std::chrono 来给算法计时
  chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
  for (size_t y = 0; y < image.rows; y++) {
    // 用cv::Mat::ptr获得图像的行指针
    unsigned char *row_ptr = image.ptr<unsigned char>(y);  // row_ptr是第y行的头指针
    for (size_t x = 0; x < image.cols; x++) {
      // 访问位于 x,y 处的像素
      unsigned char *data_ptr = &row_ptr[x * image.channels()]; // data_ptr 指向待访问的像素数据
      // 输出该像素的每个通道,如果是灰度图就只有一个通道
      for (int c = 0; c != image.channels(); c++) {
        unsigned char data = data_ptr[c]; // data为I(x,y)第c个通道的值
      }
    }
  }
  chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
  chrono::duration<double> time_used = chrono::duration_cast < chrono::duration < double >> (t2 - t1);
  cout << "遍历图像用时：" << time_used.count() << " 秒。" << endl;

  // 关于 cv::Mat 的拷贝
```

```
  // 直接赋值并不会拷贝数据
  cv::Mat image_another = image;
  // 修改 image_another 会导致 image 发生变化
  image_another(cv::Rect(0, 0, 100, 100)).setTo(0); // 将左上角100*100的块置零
  cv::imshow("image", image);
  cv::waitKey(0);

  // 使用clone函数来拷贝数据
  cv::Mat image_clone = image.clone();
  image_clone(cv::Rect(0, 0, 100, 100)).setTo(255);
  cv::imshow("image", image);
  cv::imshow("image_clone", image_clone);
  cv::waitKey(0);

  // 对于图像还有很多基本的操作,如剪切,旋转,缩放等,限于篇幅就不一一介绍了,请参看OpenCV官方文档查询每个
       函数的调用方法.
  cv::destroyAllWindows();
  return 0;
}
```

在该例程中，我们演示了如下几个操作：图像读取、显示、像素遍历、复制、赋值等。大部分的注解已写在代码里面。编译该程序时，你需要在 CMakeLists.txt 中添加 OpenCV 的头文件，然后把程序链接到库文件上。同时，由于使用了 C++ 11 标准（如 nullptr 和 chrono），还需要设置一下编译器：

slambook/ch5/imageBasics/CMakeLists.txt

```
# 添加 C++ 11 标准支持
set( CMAKE_CXX_FLAGS "-std=c++11" )

# 寻找 OpenCV 库
find_package( OpenCV REQUIRED )
# 添加头文件
include_directories( ${OpenCV_INCLUDE_DIRS} )

add_executable( imageBasics imageBasics.cpp )
# 链接 OpenCV 库
target_link_libraries( imageBasics ${OpenCV_LIBS} )
```

关于代码，我们给出几点说明：

1. 程序从 argv[1]，也就是命令行的第一个参数中读取图像位置。我们为读者准备了一张图像（ubuntu.png，一张 Ubuntu 的壁纸，希望你喜欢）供测试使用。因此，编译之后，使用如下命令调用此程序：

   终端输入：

   ```
   build/imageBasics ubuntu.png
   ```

   如果在 IDE 中调用此程序，请务必确保把参数同时给它。这可以在启动项中配置。

2. 程序的 10~18 行，使用 cv::imread 函数读取图像，并把图像和基本信息显示出来。
3. 在 35~47 行，遍历了图像中的所有像素，并计算了整个循环所用的时间。请注意像素的遍历方式并不是唯一的，而且例程给出的方式也不是最高效的。OpenCV 提供了迭代器，你可以通过迭代器遍历图像的像素。或者，cv::Mat::data 提供了指向图像数据开头的指针，你可以直接通过该指针自行计算偏移量，然后得到像素的实际内存位置。例程所用的方式是为了便于读者理解图像的结构。

   在笔者的机器上（虚拟机），遍历这张图像用时大约 12.74ms。你可以对比一下自己机器上的速度。不过，我们使用的是 cmake 默认的 debug 模式，如果使用 release 模式会快很多。
4. OpenCV 提供了许多对图像进行操作的函数，我们在此不一一列举，否则本书就会变成 OpenCV 操作手册了。例程给出了较为常见的读取、显示操作，以及复制图像中可能陷入的深拷贝误区。在编程过程中，读者还会碰到图像的旋转、插值等操作，这时你应该自行查阅函数对应的文档，以了解它们的原理与使用方式。

应该指出，OpenCV 并不是唯一的图像库，它只是许多图像库里使用范围较广泛的一个。不过，多数图像库对图像的表达是大同小异的。我们希望读者了解了 OpenCV 对图像的表示后，能够理解其他库中图像的表达，从而在需要数据格式时能够自己处理。另外，由于 cv::Mat 亦是矩阵类，除了表示图像之外，我们也可以用它来存储位姿等矩阵数据。一般认为，Eigen 对于固定大小的矩阵使用起来效率更高一些。

5.3.2　图像去畸变

在理论部分我们介绍了径向和切向畸变，下面来演示一个去畸变过程。OpenCV 提供了去畸变函数 cv::Undistort()，但本例我们从公式出发来计算一下畸变前后的图像坐标。

slambook/ch5/imageBasics/undistortImage.cpp

```
#include <opencv2/opencv.hpp>
#include <string>
using namespace std;
string image_file = "./distorted.png";   // 请确保路径正确

int main(int argc, char **argv) {
  // 本程序实现去畸变部分的代码。尽管我们可以调用OpenCV的去畸变，但自己实现一遍有助于理解。
  // 畸变参数
  double k1 = -0.28340811, k2 = 0.07395907, p1 = 0.00019359, p2 = 1.76187114e-05;
  // 内参
  double fx = 458.654, fy = 457.296, cx = 367.215, cy = 248.375;

  cv::Mat image = cv::imread(image_file, 0);   // 图像是灰度图，CV_8UC1
  int rows = image.rows, cols = image.cols;
  cv::Mat image_undistort = cv::Mat(rows, cols, CV_8UC1);   // 去畸变以后的图

  // 计算去畸变后图像的内容
  for (int v = 0; v < rows; v++) {
```

```
    for (int u = 0; u < cols; u++) {
      // 按照公式，计算点(u,v)对应到畸变图像中的坐标(u_distorted, v_distorted)
      double x = (u - cx) / fx, y = (v - cy) / fy;
      double r = sqrt(x * x + y * y);
      double x_distorted = x * (1 + k1 * r * r + k2 * r * r * r * r) + 2 * p1 * x * y + p2 * (r * r +
          2 * x * x);
      double y_distorted = y * (1 + k1 * r * r + k2 * r * r * r * r) + p1 * (r * r + 2 * y * y) + 2 *
          p2 * x * y;
      double u_distorted = fx * x_distorted + cx;
      double v_distorted = fy * y_distorted + cy;

      // 赋值（最近邻插值）
      if (u_distorted >= 0 && v_distorted >= 0 && u_distorted < cols && v_distorted < rows) {
        image_undistort.at<uchar>(v, u) = image.at<uchar>((int) v_distorted, (int) u_distorted);
      } else {
        image_undistort.at<uchar>(v, u) = 0;
      }
    }
  }

  // 画图去畸变后图像
  cv::imshow("distorted", image);
  cv::imshow("undistorted", image_undistort);
  cv::waitKey();
  return 0;
}
```

5.4 实践：3D 视觉

5.4.1 双目视觉

我们已经介绍了双目视觉的成像原理。现在我们从双目的左右图像出发，计算图像对应的视差图，然后再计算各像素在相机坐标系下的坐标，它们将构成**点云**。在第五章代码目录的 stereo 文件夹中，我们准备了左右目的图像，如图 5-9所示。下面的代码演示了计算视差图和点云部分：

slambook/ch5/stereoVision/stereoVision.cpp（部分）

```
int main(int argc, char **argv) {
    // 内参
    double fx = 718.856, fy = 718.856, cx = 607.1928, cy = 185.2157;
    // 基线
    double b = 0.573;

    // 读取图像
    cv::Mat left = cv::imread(left_file, 0);
    cv::Mat right = cv::imread(right_file, 0);
    cv::Ptr<cv::StereoSGBM> sgbm = cv::StereoSGBM::create(
        0, 96, 9, 8 * 9 * 9, 32 * 9 * 9, 1, 63, 10, 100, 32);    // 神奇的参数
    cv::Mat disparity_sgbm, disparity;
```

```
    sgbm->compute(left, right, disparity_sgbm);
    disparity_sgbm.convertTo(disparity, CV_32F, 1.0 / 16.0f);

    // 生成点云
    vector<Vector4d, Eigen::aligned_allocator<Vector4d>> pointcloud;

    // 如果你的机器慢，请把后面的v++和u++改成v+=2, u+=2
    for (int v = 0; v < left.rows; v++)
        for (int u = 0; u < left.cols; u++) {
            if (disparity.at<float>(v, u) <= 10.0 || disparity.at<float>(v, u) >= 96.0) continue;

            Vector4d point(0, 0, 0, left.at<uchar>(v, u) / 255.0); // 前三维为xyz,第四维为颜色

            // 根据双目模型计算 point 的位置
            double x = (u - cx) / fx;
            double y = (v - cy) / fy;
            double depth = fx * b / (disparity.at<float>(v, u));
            point[0] = x * depth;
            point[1] = y * depth;
            point[2] = depth;

            pointcloud.push_back(point);
    }

    cv::imshow("disparity", disparity / 96.0);
    cv::waitKey(0);
    // 画出点云
    showPointCloud(pointcloud);
    return 0;
}
```

这个例子中我们调用了 OpenCV 实现的 SGBM（Semi-global Batch Matching）[26] 算法计算左右图像的视差，然后通过双目相机的几何模型把它变换到相机的 3D 空间中。SGBM 使用了来自网络的经典参数配置，我们主要调整了最大和最小视差。视差数据结合相机的内参、基线，即能确定各点在三维空间中的位置。为了节省版面，我们省略了显示点云相关的代码。

本书不准备展开介绍双目相机的视差计算算法，感兴趣的读者可以阅读相关的参考文献[27, 28]。除了 OpenCV 实现的双目算法以外，还有许多其他的库专注于实现高效的视差计算。它是一个复杂又实用的课题。

5.4.2　RGB-D 视觉

最后，我们演示一个 RGB-D 视觉的例子。RGB-D 相机的方便之处在于能通过物理方法获得像素深度信息。如果已知相机的内外参，我们可以计算任何一个像素在世界坐标系下的位置，从而建立一张点云地图。现在我们就来演示一下。

我们准备了 5 对图像，位于 slambook/ch5/rgbd 文件夹中。在 color/下有 1.png 到 5.png 共 5 张 RGB 图，而在 depth/下有 5 张对应的深度图。同时，pose.txt 文件给出了 5 张图像的相机外参位姿

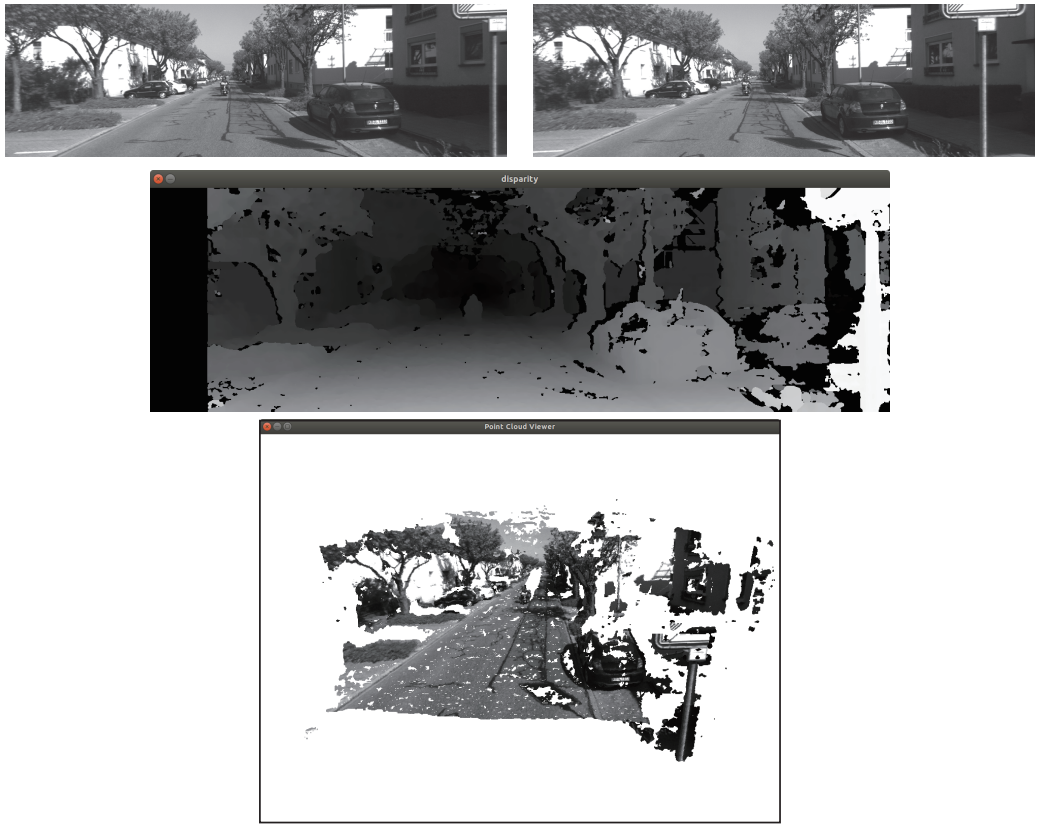


图 5-9 双目视觉的例子，左上：左眼图像，右上：右眼图像，中间：SGBM 的视差图，下方：点云图。注意左侧相机看到了一部分右侧相机没有看到的内容，所以对应的视差是空的。

（以 $\boldsymbol{T}_{\mathrm{wc}}$ 形式）。位姿记录的形式和之前一样，为平移向量加旋转四元数：

$$[x, y, z, q_x, q_y, q_z, q_w],$$

其中 q_w 是四元数的实部。例如，第一对图的外参为：

$$[-0.228993, 0.00645704, 0.0287837, -0.0004327, -0.113131, -0.0326832, 0.993042].$$

下面我们写一段程序，完成两件事：(1). 根据内参计算一对 RGB-D 图像对应的点云；(2). 根据各张图的相机位姿（也就是外参），把点云加起来，组成地图。

slambook/ch5/rgbd/jointMap.cpp（部分）

```
int main(int argc, char **argv) {
    vector<cv::Mat> colorImgs, depthImgs;    // 彩色图和深度图
```

```
    TrajectoryType poses;         // 相机位姿

    ifstream fin("./pose.txt");
    if (!fin) {
        cerr << "请在有pose.txt的目录下运行此程序" << endl;
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        boost::format fmt("./%s/%d.%s"); //图像文件格式
        colorImgs.push_back(cv::imread((fmt % "color" % (i + 1) % "png").str()));
        depthImgs.push_back(cv::imread((fmt % "depth" % (i + 1) % "pgm").str(), -1)); // 使用-1读取原
            始图像

        double data[7] = {0};
        for (auto &d:data) fin >> d;
        Sophus::SE3d pose(Eigen::Quaterniond(data[6], data[3], data[4], data[5]),
        Eigen::Vector3d(data[0], data[1], data[2]));
        poses.push_back(pose);
    }

    // 计算点云并拼接
    // 相机内参
    double cx = 325.5;
    double cy = 253.5;
    double fx = 518.0;
    double fy = 519.0;
    double depthScale = 1000.0;
    vector<Vector6d, Eigen::aligned_allocator<Vector6d>> pointcloud;
    pointcloud.reserve(1000000);

    for (int i = 0; i < 5; i++) {
        cout << "转换图像中: " << i + 1 << endl;
        cv::Mat color = colorImgs[i];
        cv::Mat depth = depthImgs[i];
        Sophus::SE3d T = poses[i];
        for (int v = 0; v < color.rows; v++)
            for (int u = 0; u < color.cols; u++) {
                unsigned int d = depth.ptr<unsigned short>(v)[u]; // 深度值
                if (d == 0) continue; // 为0表示没有测量到
                Eigen::Vector3d point;
                point[2] = double(d) / depthScale;
                point[0] = (u - cx) * point[2] / fx;
                point[1] = (v - cy) * point[2] / fy;
                Eigen::Vector3d pointWorld = T * point;

                Vector6d p;
                p.head<3>() = pointWorld;
                p[5] = color.data[v * color.step + u * color.channels()];   // blue
                p[4] = color.data[v * color.step + u * color.channels() + 1]; // green
                p[3] = color.data[v * color.step + u * color.channels() + 2]; // red
                pointcloud.push_back(p);
            }
```

```
    }

    cout << "点云共有" << pointcloud.size() << "个点." << endl;
    showPointCloud(pointcloud);
    return 0;
}
```

运行程序后即可在 Pangolin 窗口中看到拼合的点云地图了（见图 5-10）。你可以拖动鼠标查看。

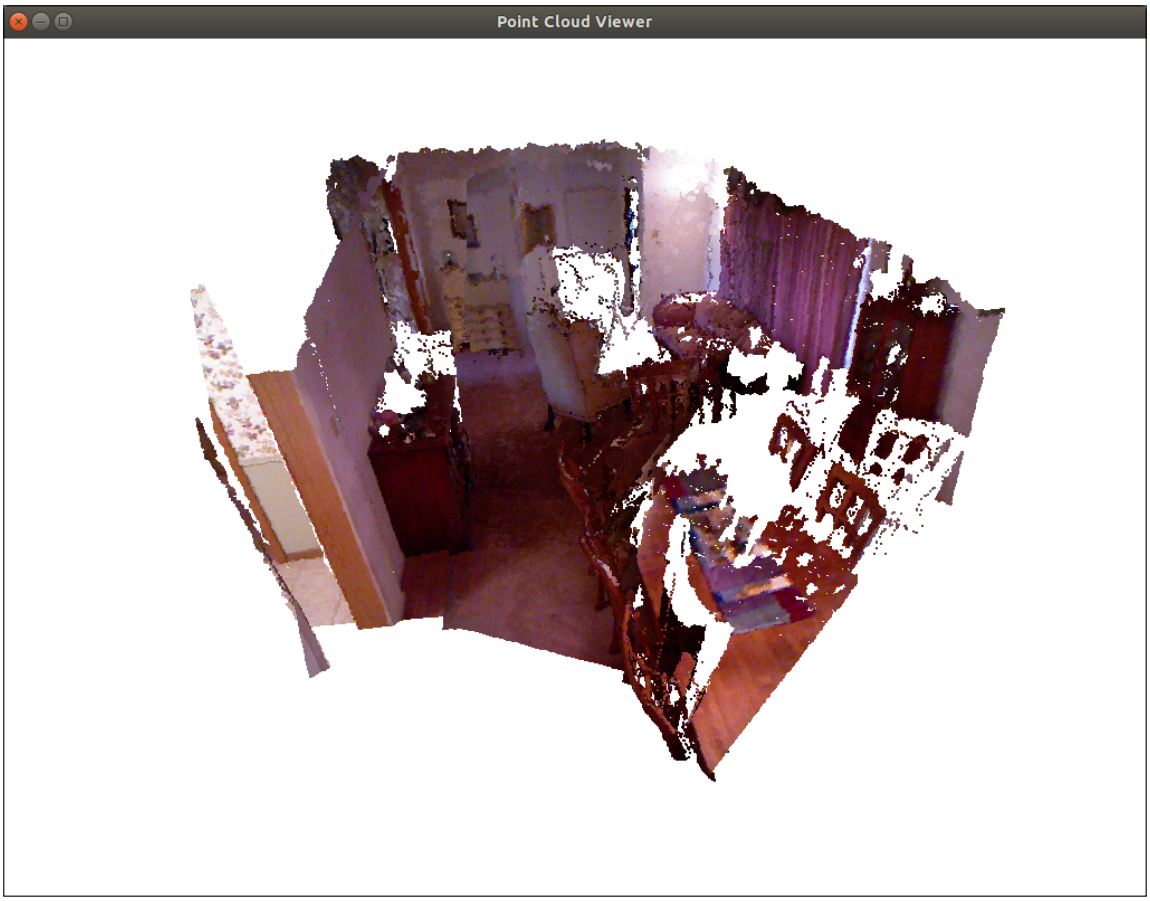


图 5-10　查看拼合的点云地图。

通过这些例子，我们演示了计算机视觉中一些常见的单目、双目和深度相机算法。希望读者通过这些简单的例子，实际地体会相机内外参、畸变参数的含义。

习题

1.* 寻找一部相机（你的手机或笔记本的摄像头即可），标定它的内参。你可能会用到标定板，或者自己打印一张标定用的棋盘格。
2. 叙述相机内参的物理意义。如果一部相机的分辨率变为原来的两倍而其他地方不变，它的内参如何变化？
3. 搜索特殊相机（鱼眼或全景相机）的标定方法。它们与普通的针孔模型有何不同？
4. 调研全局快门相机（global shutter）和卷帘快门相机（rolling shutter）的异同。它们在 SLAM 中有何优缺点？
5. RGB-D 相机是如何标定的？以 Kinect 为例，需要标定哪些参数？（参照`https://github.com/code-iai/iai_kinect2`。）
6. 除了示例程序演示的遍历图像的方式，你还能举出哪些遍历图像的方法？
7.* 阅读 OpenCV 官方教程，学习它的基本用法。

第六章　非线性优化

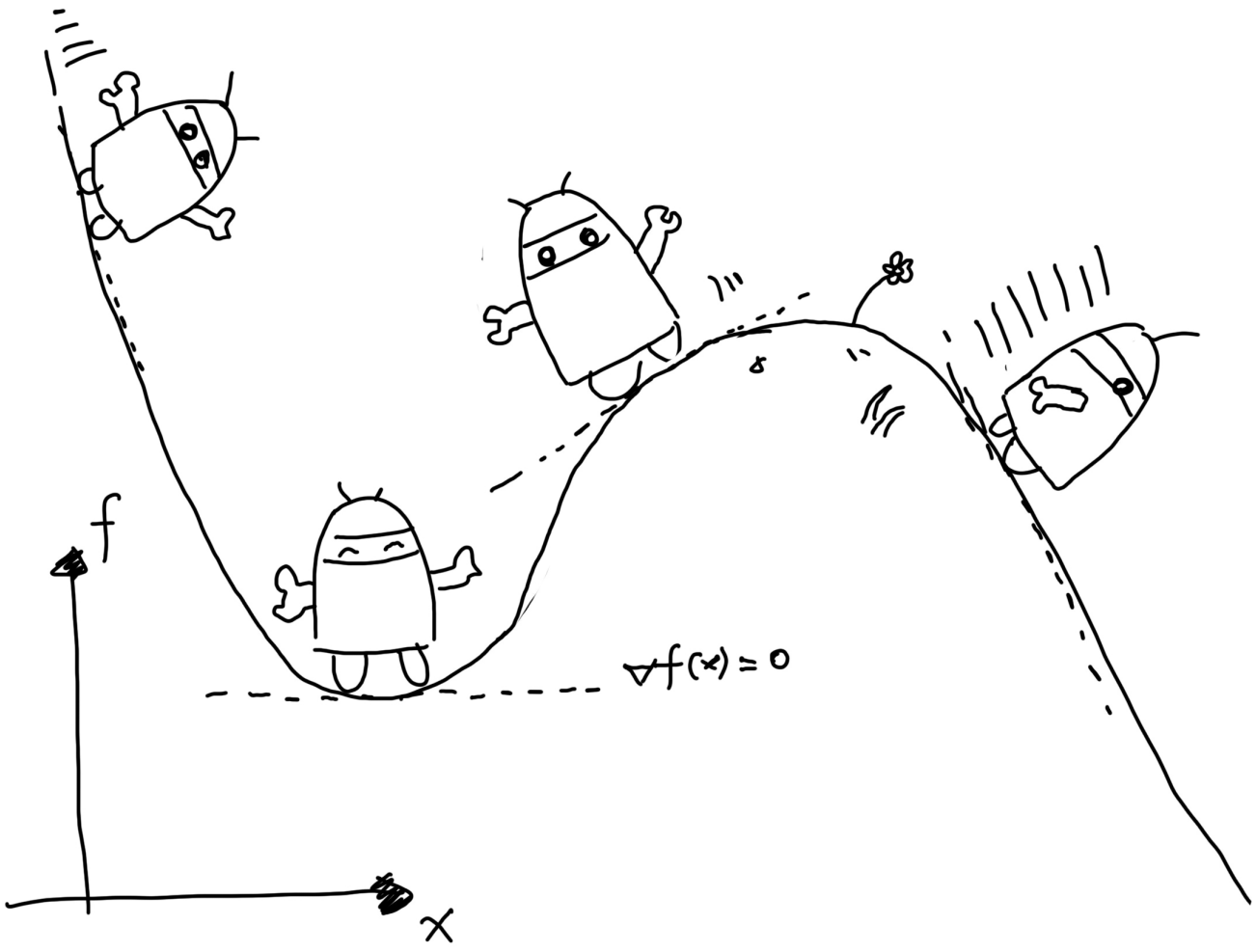
主要目标

1. 理解最小二乘法的含义和处理方式。
2. 理解高斯牛顿法（Gauss-Newton）、列文伯格—马夸尔特方法（Levenburg-Marquadt）等下降策略。
3. 学习 Ceres 库和 g2o 库的基本使用方法。

在前面几讲，我们介绍了经典 SLAM 模型的运动方程和观测方程。现在我们已经知道，方程中的位姿可以由变换矩阵来描述，然后用李代数进行优化。观测方程由相机成像模型给出，其中内参是随相机固定的，而外参则是相机的位姿。于是，我们已经弄清了经典 SLAM 模型在视觉情况下的具体表达。

然而，由于噪声的存在，运动方程和观测方程的等式必定不是精确成立的。尽管相机可以非常好地符合针孔模型，但遗憾的是，我们得到的数据通常是受各种未知噪声影响的。即使我们有高精度的相机，运动方程和观测方程也只能近似成立。所以，与其假设数据必须符合方程，不如来讨论如何在有噪声的数据中进行准确的状态估计。

解决状态估计问题需要一定程度的最优化背景知识。本节将介绍基本的无约束非线性优化方法，同时介绍优化库 g2o 和 Ceres 的使用方式。



$$f(x+\Delta x) \approx f(x) + \nabla f(x)\Delta x$$

$$+ \frac{1}{2} \Delta x^T H(x) \Delta x$$

$$+ \cdots$$

6.1　状态估计问题

6.1.1　批量状态估计与最大后验估计

接着前面几讲的内容，我们回顾一下第 2 讲讨论的经典 SLAM 模型。它由一个运动方程和一个观测方程构成，如式(2.5)所示：

$$\begin{cases} \boldsymbol{x}_k = f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k\right) + \boldsymbol{w}_k \\ \boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k\right) + \boldsymbol{v}_{k,j} \end{cases}. \tag{6.1}$$

通过第 4 讲的知识，我们了解到这里的 $\boldsymbol{x}_k$ 乃是相机的位姿，可以用 SE(3) 来描述。至于观测方程，第 5 讲已经说明，即针孔相机模型。为了让读者对它们有更深的印象，我们不妨讨论一下其具体参数化形式。首先，位姿变量 $\boldsymbol{x}_k$ 可以由 $\boldsymbol{T}_k \in \mathrm{SE}(3)$ 表达。其次，运动方程与输入的具体形式有关，但在视觉 SLAM 中没有特殊性（和普通的机器人、车辆的情况一样），我们暂且不谈。观测方程则由针孔模型给定。假设在 $\boldsymbol{x}_k$ 处对路标 $\boldsymbol{y}_j$ 进行了一次观测，对应到图像上的像素位置 $\boldsymbol{z}_{k,j}$，那么，观测方程可以表示成

$$s\boldsymbol{z}_{k,j} = \boldsymbol{K}(\boldsymbol{R}_k\boldsymbol{y}_j + \boldsymbol{t}_k). \tag{6.2}$$

其中 $\boldsymbol{K}$ 为相机内参，s 为像素点的距离，也是 $(\boldsymbol{R}_k\boldsymbol{y}_j + \boldsymbol{t}_k)$ 的第三个分量。如果使用变换矩阵 $\boldsymbol{T}_k$ 描述位姿，那么路标点 $\boldsymbol{y}_j$ 必须以齐次坐标来描述，计算完成后要转换为非齐次坐标。如果你还不熟悉这个过程，请到上一讲回顾一遍。

现在，考虑数据受噪声影响后会发生什么改变。在运动和观测方程中，我们**通常**假设两个噪声项 $\boldsymbol{w}_k, \boldsymbol{v}_{k,j}$ 满足零均值的高斯分布，像这样：

$$\boldsymbol{w}_k \sim \mathcal{N}\left(\boldsymbol{0}, \boldsymbol{R}_k\right), \boldsymbol{v}_k \sim \mathcal{N}\left(\boldsymbol{0}, \boldsymbol{Q}_{k,j}\right). \tag{6.3}$$

其中 $\mathcal{N}$ 表示高斯分布，$\boldsymbol{0}$ 表示零均值，$\boldsymbol{R}_k, \boldsymbol{Q}_{k,j}$ 为协方差矩阵。在这些噪声的影响下，我们希望通过带噪声的数据 $\boldsymbol{z}$ 和 $\boldsymbol{u}$ 推断位姿 $\boldsymbol{x}$ 和地图 $\boldsymbol{y}$（以及它们的概率分布），这构成了一个状态估计问题。

处理这个状态估计问题的方法大致分成两种。由于在 SLAM 过程中，这些数据是随时间逐渐到来的，所以在直观上，我们应该持有一个当前时刻的估计状态，然后用新的数据来更新它。这种方式称为**增量/渐进**（incremental）的方法，或者叫**滤波器**。在历史上很长一段时间内，研究者们使用滤波器，尤其是扩展卡尔曼滤波器（EKF）及其衍生方法求解它。另一种方式，则是把数据“攒”起来一并处理，这种方式称为**批量**（batch）的方法。例如，我们可以把 0 到 k 时刻所有的输入和观测数据都放在一起，问，在这样的输入和观测下，如何估计整个 0 到 k 时刻的轨迹与地图？

这两种不同的处理方式引出了很多不同的估计手段。大体来说，增量方法仅关心**当前时刻**的状态估计 $\boldsymbol{x}_k$，而对之前的状态则不多考虑；相对地，批量方法可以在**更大的范围**达到最优化，被认为优于传统的滤波器[13]，而成为当前视觉 SLAM 的主流方法。极端情况下，我们可以让机器人或无人机收集所有时刻的数据，再带回计算中心统一处理，这也正是 SfM（Structure from Motion）的主流

做法。当然，这种极端情况显然是不**实时**的，不符合 SLAM 的运用场景。所以在 SLAM 中，实用的方法通常是一些折衷的手段。比如，我们固定一些历史轨迹，仅对当前时刻附近的一些轨迹进行优化，这是后面要讲到的**滑动窗口估计**法。

在理论上，批量方法更加容易介绍。同时，理解了批量方法也更容易理解增量的方法。所以本节我们重点介绍以非线性优化为主的批量优化方法，对卡尔曼滤波器以及更深入的知识则留到后端章节再进行讨论。由于讨论的是批量方法，考虑从 1 到 N 的所有时刻，并假设有 M 个路标点。定义所有时刻的机器人位姿和路标点坐标为：

$$\boldsymbol{x} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}, \quad \boldsymbol{y} = \{\boldsymbol{y}_1, \ldots, \boldsymbol{y}_M\}.$$

同样地，用不带下标的 $\boldsymbol{u}$ 表示所有时刻的输入，$\boldsymbol{z}$ 表示所有时刻的观测数据。那么我们说，对机器人状态的估计，从概率学的观点来看，就是已知输入数据 $\boldsymbol{u}$ 和观测数据 $\boldsymbol{z}$ 的条件下，求状态 $\boldsymbol{x}, \boldsymbol{y}$ 的条件概率分布：

$$P(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{z}, \boldsymbol{u}). \tag{6.4}$$

特别地，当我们不知道控制输入，只有一张张的图像时，即只考虑观测方程带来的数据时，相当于估计 $P(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{z})$ 的条件概率分布，此问题也称为 Structure from Motion（SfM），即如何从许多图像中重建三维空间结构[29]。

为了估计状态变量的条件分布，利用贝叶斯法则，有：

$$P(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{z}, \boldsymbol{u}) = \frac{P(\boldsymbol{z}, \boldsymbol{u}|\boldsymbol{x}, \boldsymbol{y}) P(\boldsymbol{x}, \boldsymbol{y})}{P(\boldsymbol{z}, \boldsymbol{u})} \propto \underbrace{P(\boldsymbol{z}, \boldsymbol{u}|\boldsymbol{x}, \boldsymbol{y})}_{\text{似然}} \underbrace{P(\boldsymbol{x}, \boldsymbol{y})}_{\text{先验}}. \tag{6.5}$$

贝叶斯法则左侧称为**后验概率**，右侧的 $P(\boldsymbol{z}|\boldsymbol{x})$ 称为**似然**（Likehood），另一部分 $P(\boldsymbol{x})$ 称为**先验**（Prior）。**直接求后验分布是困难的，但是求一个状态最优估计，使得在该状态下后验概率最大化**（Maximize a Posterior，MAP），则是可行的：

$$(\boldsymbol{x}, \boldsymbol{y})^*_{\text{MAP}} = \arg\max P(\boldsymbol{x}, \boldsymbol{y}|\boldsymbol{z}, \boldsymbol{u}) = \arg\max P(\boldsymbol{z}, \boldsymbol{u}|\boldsymbol{x}, \boldsymbol{y}) P(\boldsymbol{x}, \boldsymbol{y}). \tag{6.6}$$

请注意贝叶斯法则的分母部分与待估计的状态 $\boldsymbol{x}, \boldsymbol{y}$ 无关，因而可以忽略。贝叶斯法则告诉我们，求解最大后验概率**等价于最大化似然和先验的乘积**。进一步，我们当然也可以说，对不起，我不知道机器人位姿或路标大概在什么地方，此时就没有了**先验**。那么，可以求解**最大似然估计**（Maximize Likelihood Estimation，MLE）：

$$(\boldsymbol{x}, \boldsymbol{y})^*_{\text{MLE}} = \arg\max P(\boldsymbol{z}, \boldsymbol{u}|\boldsymbol{x}, \boldsymbol{y}). \tag{6.7}$$

直观讲，似然是指“在现在的位姿下，可能产生怎样的观测数据”。由于我们知道观测数据，所以最大似然估计可以理解成：**“在什么样的状态下，最可能产生现在观测到的数据”**。这就是最大似然估计的直观意义。

6.1.2　最小二乘的引出

那么如何求最大似然估计呢？我们说，在高斯分布的假设下，最大似然能够有较简单的形式。回顾观测模型，对于某一次观测：

$$\boldsymbol{z}_{k,j} = h\left(\boldsymbol{y}_j, \boldsymbol{x}_k\right) + \boldsymbol{v}_{k,j},$$

由于我们假设了噪声项 $\boldsymbol{v}_k \sim \mathcal{N}\left(\boldsymbol{0}, \boldsymbol{Q}_{k,j}\right)$，所以观测数据的条件概率为：

$$P(\boldsymbol{z}_{j,k}|\boldsymbol{x}_k, \boldsymbol{y}_j) = N\left(h(\boldsymbol{y}_j, \boldsymbol{x}_k), \boldsymbol{Q}_{k,j}\right).$$

它依然是一个高斯分布。考虑单次观测的最大似然估计，可以使用**最小化负对数**来求一个高斯分布的最大似然。

我们知道高斯分布在负对数下有较好的数学形式。考虑任意高维高斯分布 $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$，它的概率密度函数展开形式为：

$$P\left(\boldsymbol{x}\right) = \frac{1}{\sqrt{(2\pi)^N \det(\boldsymbol{\Sigma})}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \left(\boldsymbol{x}-\boldsymbol{\mu}\right)\right). \tag{6.8}$$

对其取负对数，则变为：

$$-\ln\left(P\left(\boldsymbol{x}\right)\right) = \frac{1}{2}\ln\left((2\pi)^N \det\left(\boldsymbol{\Sigma}\right)\right) + \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \left(\boldsymbol{x}-\boldsymbol{\mu}\right). \tag{6.9}$$

因为对数函数是单调递增的，所以对原函数求最大化相当于对负对数求最小化。在最小化上式的 $\boldsymbol{x}$ 时，第一项与 $\boldsymbol{x}$ 无关，可以略去。于是，只要最小化右侧的二次型项，就得到了对状态的最大似然估计。代入 SLAM 的观测模型，相当于在求：

$$\begin{aligned}\left(\boldsymbol{x}_k, \boldsymbol{y}_j\right)^* &= \arg\max \mathcal{N}(h(\boldsymbol{y}_j, \boldsymbol{x}_k), \boldsymbol{Q}_{k,j}) \\ &= \arg\min\left(\left(\boldsymbol{z}_{k,j} - h\left(\boldsymbol{x}_k, \boldsymbol{y}_j\right)\right)^{\mathrm{T}} \boldsymbol{Q}_{k,j}^{-1} \left(\boldsymbol{z}_{k,j} - h\left(\boldsymbol{x}_k, \boldsymbol{y}_j\right)\right)\right).\end{aligned} \tag{6.10}$$

我们发现，该式等价于最小化噪声项（即误差）的一个二次型。这个二次型称为**马哈拉诺比斯距离**（Mahalanobis distance），又叫**马氏距离**。它也可以看成是由 $\boldsymbol{Q}_{k,j}^{-1}$ 加权之后的欧氏距离（二范数），这里 $\boldsymbol{Q}_{k,j}^{-1}$ 也叫做**信息矩阵**，即高斯分布协方差矩阵之逆。

现在我们考虑批量时刻的数据。通常假设各个时刻的输入和观测是相互独立的，这意味着各个输入之间是独立的，各个观测之间是独立的，并且输入和观测也是独立的。于是我们可以对联合分布进行因式分解：

$$P\left(\boldsymbol{z}, \boldsymbol{u}|\boldsymbol{x}, \boldsymbol{y}\right) = \prod_k P\left(\boldsymbol{u}_k|\boldsymbol{x}_{k-1}, \boldsymbol{x}_k\right) \prod_{k,j} P\left(\boldsymbol{z}_{k,j}|\boldsymbol{x}_k, \boldsymbol{y}_j\right), \tag{6.11}$$

这说明我们可以独立地处理各时刻的运动和观测。定义各次输入和观测数据与模型之间的误差：

$$\begin{aligned}\boldsymbol{e}_{u,k} &= \boldsymbol{x}_k - f\left(\boldsymbol{x}_{k-1}, \boldsymbol{u}_k\right) \\ \boldsymbol{e}_{z,j,k} &= \boldsymbol{z}_{k,j} - h\left(\boldsymbol{x}_k, \boldsymbol{y}_j\right),\end{aligned} \tag{6.12}$$

那么，最小化所有时刻估计值与真实读数之间马氏距离，等价于求最大似然估计。负对数允许我们把乘积变成求和：

$$\min J(\boldsymbol{x}, \boldsymbol{y}) = \sum_k \boldsymbol{e}_{u,k}^{\mathrm{T}} \boldsymbol{R}_k^{-1} \boldsymbol{e}_{u,k} + \sum_k \sum_j \boldsymbol{e}_{z,k,j}^{\mathrm{T}} \boldsymbol{Q}_{k,j}^{-1} \boldsymbol{e}_{z,k,j}. \tag{6.13}$$

这样就得到了一个**最小二乘问题**（Least Square Problem），它的解等价于状态的最大似然估计。直观上看，由于噪声的存在，当我们把估计的轨迹与地图代入 SLAM 的运动、观测方程中时，它们并不会完美地成立。这时怎么办呢？我们对状态的估计值进行**微调**，使得整体的误差下降一些。当然这个下降也有限度，它一般会到达一个**极小值**。这就是一个典型非线性优化的过程。

仔细观察式(6.13)，我们发现 SLAM 中的最小二乘问题具有一些特定的结构：

- 首先，整个问题的目标函数由许多个误差的（加权的）二次型组成。虽然总体的状态变量维数很高，但每个误差项都是简单的，仅与一两个状态变量有关。例如，运动误差只与 $\boldsymbol{x}_{k-1}, \boldsymbol{x}_k$ 有关，观测误差只与 $\boldsymbol{x}_k, \boldsymbol{y}_j$ 有关。这种关系会让整个问题有一种**稀疏**的形式，我们将在后端章节中看到。
- 其次，如果使用李代数表示增量，则该问题是**无约束**的最小二乘问题。但如果用旋转矩阵/变换矩阵描述位姿，则会引入旋转矩阵自身的约束，即需在问题中加入 s.t. $\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R} = \boldsymbol{I}$ 且 $\det(\boldsymbol{R}) = 1$ 这样令人头大的条件。额外的约束会使优化变得更困难。这体现了李代数的优势。
- 最后，我们使用了二次型度量误差。误差的分布将影响此项在整个问题中的权重。例如，某次的观测非常准确，那么协方差矩阵就会“小”，而信息矩阵就会“大”，所以这个误差项会在整个问题中占有较高的权重。我们之后也会看到它存在一些问题，但是目前先不讨论。

现在，我们介绍如何求解这个最小二乘问题，这需要一些**非线性优化的基本知识**。特别地，我们要针对这样一个通用的无约束非线性最小二乘问题，探讨它是如何求解的。在后续几讲中，我们会大量使用本讲的结果，详细讨论它在 SLAM 前端、后端中的应用。

6.1.3 例子：批量状态估计

我发现在这里举一个简单的例子会更好一些。考虑一个非常简单的离散时间系统：

$$\begin{aligned} x_k &= x_{k-1} + u_k + w_k, & w_k &\sim \mathcal{N}(0, Q_k) \\ z_k &= x_k + n_k, & n_k &\sim \mathcal{N}(0, R_k) \end{aligned} \tag{6.14}$$

这可以表达一辆沿 x 轴前进或后退的汽车。第一个公式为运动方程，u_k 为输入，w_k 为噪声；第二个公式为观测方程，z_k 为对汽车位置的测量。取时间 $k = 1, \ldots, 3$，现希望根据已有的 v, y 进行状态估计。设初始状态 x_0 已知。下面来推导批量（batch）状态的最大似然估计。

首先，令批量状态变量为 $\boldsymbol{x} = [x_0, x_1, x_2, x_3]^{\mathrm{T}}$，令批量观测为 $\boldsymbol{z} = [z_1, z_2, z_3]^{\mathrm{T}}$，按同样方式定

义 $\boldsymbol{u} = [u_1, u_2, u_3]^{\mathrm{T}}$。按照先前的推导，我们知道最大似然估计为：

$$\begin{aligned} \boldsymbol{x}^*_{\mathrm{map}} &= \arg\max P(\boldsymbol{x}|\boldsymbol{u}, \boldsymbol{z}) = \arg\max P(\boldsymbol{u}, \boldsymbol{z}|\boldsymbol{x}) \\ &= \prod_{k=1}^{3} P(u_k|x_{k-1}, x_k) \prod_{k=1}^{3} P\left(z_k|x_k\right), \end{aligned} \tag{6.15}$$

对于具体的每一项，比如运动方程，我们知道：

$$P(u_k|x_{k-1}, x_k) = \mathcal{N}(x_k - x_{k-1}, Q_k), \tag{6.16}$$

观测方程也是类似的：

$$P\left(z_k|x_k\right) = \mathcal{N}\left(x_k, R_k\right). \tag{6.17}$$

根据这些方法，我们就能够实际地解决上面的批量状态估计问题。根据之前的叙述，可以构建误差变量：

$$e_{u,k} = x_k - x_{k-1} - u_k, \quad e_{z,k} = z_k - x_k, \tag{6.18}$$

于是最小二乘的目标函数为：

$$\min \sum_{k=1}^{3} e_{u,k}^{\mathrm{T}} Q_k^{-1} e_{u,k} + \sum_{k=1}^{3} e_{z,k}^{\mathrm{T}} R_k^{-1} e_{z,k}. \tag{6.19}$$

此外，这个系统是线性系统，我们可以很容易地将它写成向量形式。定义向量 $\boldsymbol{y} = [\boldsymbol{u}, \boldsymbol{z}]^{\mathrm{T}}$，那么可以写出矩阵 $\boldsymbol{H}$，使得：

$$\boldsymbol{y} - \boldsymbol{H}\boldsymbol{x} = \boldsymbol{e} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma}). \tag{6.20}$$

那么：

$$\boldsymbol{H} = \left[\begin{array}{cccc} 1 & -1 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & 0 & 1 & -1 \\ \hline 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{array}\right], \tag{6.21}$$

且 $\boldsymbol{\Sigma} = \mathrm{diag}(Q_1, Q_2, Q_3, R_1, R_2, R_3)$。整个问题可以写成：

$$\boldsymbol{x}^*_{\mathrm{map}} = \arg\min \boldsymbol{e}^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \boldsymbol{e}, \tag{6.22}$$

之后我们将看到，这个问题有唯一的解：

$$\boldsymbol{x}^*_{\mathrm{map}} = (\boldsymbol{H}^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \boldsymbol{H})^{-1} \boldsymbol{H}^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} \boldsymbol{y}. \tag{6.23}$$

6.2 非线性最小二乘

先来考虑一个简单的最小二乘问题：

$$\min_{\boldsymbol{x}} F(\boldsymbol{x}) = \frac{1}{2}\|f(\boldsymbol{x})\|_2^2. \tag{6.24}$$

其中，自变量 $\boldsymbol{x} \in \mathbb{R}^n$，$f$ 是任意标量非线性函数 $f(\boldsymbol{x}) : \mathbb{R}^n \mapsto \mathbb{R}$。注意这里的系数 $\frac{1}{2}$ 是无关紧要的，有些文献上带有这个系数，有些文献则不带，它也不会影响之后的结论。下面讨论如何求解这样一个优化问题。显然，如果 f 是个数学形式上很简单的函数，那么该问题可以用解析形式来求。令目标函数的导数为零，然后求解 $\boldsymbol{x}$ 的最优值，就和求二元函数的极值一样：

$$\frac{\mathrm{d}F}{\mathrm{d}\boldsymbol{x}} = \boldsymbol{0}. \tag{6.25}$$

解此方程，就得到了导数为零处的极值。它们可能是极大、极小或鞍点处的值，只要逐个比较它们的函数值大小即可。但是，这个方程是否容易求解呢？这取决于 f 导函数的形式。如果 f 为简单的线性函数，那么这个问题就是简单的线性最小二乘问题，但是有些导函数可能形式复杂，使得该方程可能不容易求解。求解这个方程需要我们知道关于目标函数的**全局性质**，而通常这是不大可能的。对于不方便直接求解的最小二乘问题，我们可以用**迭代**的方式，从一个初始值出发，不断地更新当前的优化变量，使目标函数下降。具体步骤可列写如下：

1. 给定某个初始值 $\boldsymbol{x}_0$。
2. 对于第 k 次迭代，寻找一个增量 $\Delta\boldsymbol{x}_k$，使得 $\|f(\boldsymbol{x}_k + \Delta\boldsymbol{x}_k)\|_2^2$ 达到极小值。
3. 若 $\Delta\boldsymbol{x}_k$ 足够小，则停止。
4. 否则，令 $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta\boldsymbol{x}_k$，返回第 2 步。

这让求解**导函数为零**的问题变成了一个不断**寻找下降增量** $\Delta\boldsymbol{x}_k$ 的问题，我们将看到，由于可以对 f 进行线性化，增量的计算将简单很多。当函数下降直到增量非常小的时候，就认为算法收敛，目标函数达到了一个极小值。在这个过程中，问题在于如何找到每次迭代点的增量，而这是一个局部的问题，我们只需要关心 f 在迭代值处的局部性质而非全局性质。这类方法在最优化、机器学习等领域应用非常广泛。

接下来我们考察如何寻找这个增量 $\Delta\boldsymbol{x}_k$。这部分知识实际属于数值优化的领域，我们来看一些广泛使用的结果。

6.2.1 一阶和二阶梯度法

现在考虑第 k 次迭代，假设我们在 $\boldsymbol{x}_k$ 处，想要寻到增量 $\Delta\boldsymbol{x}_k$，那么最直观的方式是将目标函数在 $\boldsymbol{x}_k$ 附近进行泰勒展开：

$$F(\boldsymbol{x}_k + \Delta\boldsymbol{x}_k) \approx F(\boldsymbol{x}_k) + \boldsymbol{J}(\boldsymbol{x}_k)^{\mathrm{T}}\Delta\boldsymbol{x}_k + \frac{1}{2}\Delta\boldsymbol{x}_k^{\mathrm{T}}\boldsymbol{H}(\boldsymbol{x}_k)\Delta\boldsymbol{x}_k. \tag{6.26}$$

其中 $\boldsymbol{J}(\boldsymbol{x}_k)$ 是 $F(\boldsymbol{x})$ 关于 $\boldsymbol{x}$ 的一阶导数（也叫梯度、**雅可比**矩阵〔Jacobian〕）[①]，$\boldsymbol{H}$ 则是二阶导数（**海塞**〔Hessian〕矩阵），它们都在 $\boldsymbol{x}_k$ 处取值，读者应该在大学本科多元微积分课程中学习过。我们可以选择保留泰勒展开的一阶或二阶项，那么对应的求解方法则称为一阶梯度或二阶梯度法。如果保留一阶梯度，那么取增量为反向的梯度，即可保证函数下降：

$$\Delta \boldsymbol{x}^* = -\boldsymbol{J}(\boldsymbol{x}_k). \tag{6.27}$$

当然这只是个方向，通常我们还要再指定一个步长 λ。步长可以根据一定的条件来计算[30]，在机器学习当中也有一些经验性质的方法，但我们不展开谈。这种方法被称为**最速下降法**。它的直观意义非常简单，只要我们沿着反向梯度方向前进，在一阶（线性）的近似下，目标函数必定会下降。

注意到以上讨论都是在第 k 次迭代时进行的，并不涉及其他的迭代信息。所以为了简化符号起见，后面我们省略下标 k，并认为这些讨论对任意一次迭代都成立。

另一方面，我们可选择保留二阶梯度信息，此时增量方程为：

$$\Delta \boldsymbol{x}^* = \arg\min \left(F(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta \boldsymbol{x} + \frac{1}{2} \Delta \boldsymbol{x}^{\mathrm{T}} \boldsymbol{H} \Delta \boldsymbol{x} \right). \tag{6.28}$$

右侧只含 $\Delta \boldsymbol{x}$ 的零次、一次和二次项。求右侧等式关于 $\Delta \boldsymbol{x}$ 的导数并令它为零[②]，得到：

$$\boldsymbol{J} + \boldsymbol{H} \Delta \boldsymbol{x} = \boldsymbol{0} \Rightarrow \boldsymbol{H} \Delta \boldsymbol{x} = -\boldsymbol{J}. \tag{6.29}$$

求解这个线性方程，就得到了增量。该方法又称为**牛顿法**。

我们看到，一阶和二阶梯度法都十分直观，只要把函数在迭代点附近进行泰勒展开，并针对更新量做最小化即可。事实上，我们用一个一次或二次的函数近似了原函数，然后用近似函数的最小值来猜测原函数的极小值。只要原目标函数局部看起来像一次或二次函数，这类算法就是成立的（这也是现实当中的情形）。不过，这两种方法也存在它们自身的问题。最速下降法过于贪心，容易走出锯齿路线，反而增加了迭代次数。而牛顿法则需要计算目标函数的 $\boldsymbol{H}$ 矩阵，这在问题规模较大时非常困难，我们通常倾向于避免 $\boldsymbol{H}$ 的计算。对于一般的问题，一些拟牛顿法可以得到较好的结果，而对于最小二乘问题，还有几类更加实用的方法：**高斯牛顿法**（Gauss-Newton's method）和**列文伯格—马夸尔特方法**（Levernburg-Marquardt's method）。

6.2.2　高斯牛顿法

高斯牛顿法是最优化算法中最简单的方法之一。它的思想是将 $f(\boldsymbol{x})$ 进行一阶的泰勒展开。请注意这里不是目标函数 $F(\boldsymbol{x})$ 而是 $f(\boldsymbol{x})$，否则就变成牛顿法了。

$$f(\boldsymbol{x} + \Delta \boldsymbol{x}) \approx f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta \boldsymbol{x}. \tag{6.30}$$

[①] 我们把 $\boldsymbol{J}(\boldsymbol{x})$ 写成列向量，那么它可以和 $\Delta \boldsymbol{x}$ 进行内积，得到一个标量。

[②] 对矩阵求导不熟悉的同学请参考附录 B。

这里 $\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}$ 为 $f(\boldsymbol{x})$ 关于 $\boldsymbol{x}$ 的导数，为 $n \times 1$ 的列向量。根据前面的框架，当前的目标是寻找增量 $\Delta\boldsymbol{x}$，使得 $\|f(\boldsymbol{x}+\Delta\boldsymbol{x})\|^2$ 达到最小。为了求 $\Delta\boldsymbol{x}$，我们需要解一个线性的最小二乘问题：

$$\Delta\boldsymbol{x}^* = \arg\min_{\Delta\boldsymbol{x}} \frac{1}{2}\left\| f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta\boldsymbol{x} \right\|^2. \tag{6.31}$$

这个方程与之前有什么不一样呢？根据极值条件，将上述目标函数对 $\Delta\boldsymbol{x}$ 求导，并令导数为零。为此，先展开目标函数的平方项：

$$\begin{aligned} \frac{1}{2}\left\| f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta\boldsymbol{x} \right\|^2 &= \frac{1}{2}\left(f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta\boldsymbol{x} \right)^{\mathrm{T}} \left(f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}} \Delta\boldsymbol{x} \right) \\ &= \frac{1}{2}\left(\|f(\boldsymbol{x})\|_2^2 + 2f(\boldsymbol{x})\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}\Delta\boldsymbol{x} + \Delta\boldsymbol{x}^{\mathrm{T}}\boldsymbol{J}(\boldsymbol{x})\boldsymbol{J}(\boldsymbol{x})^{\mathrm{T}}\Delta\boldsymbol{x} \right). \end{aligned}$$

求上式关于 $\Delta\boldsymbol{x}$ 的导数，并令其为零：

$$\boldsymbol{J}(\boldsymbol{x})f(\boldsymbol{x}) + \boldsymbol{J}(\boldsymbol{x})\boldsymbol{J}^{\mathrm{T}}(\boldsymbol{x})\Delta\boldsymbol{x} = \boldsymbol{0}.$$

可以得到如下方程组：

$$\underbrace{\boldsymbol{J}(\boldsymbol{x})\boldsymbol{J}^{\mathrm{T}}}_{\boldsymbol{H}(\boldsymbol{x})}(\boldsymbol{x})\Delta\boldsymbol{x} = \underbrace{-\boldsymbol{J}(\boldsymbol{x})f(\boldsymbol{x})}_{\boldsymbol{g}(\boldsymbol{x})}. \tag{6.32}$$

这个方程是关于变量 $\Delta\boldsymbol{x}$ 的**线性方程组**，我们称它为**增量方程**，也可以称为**高斯牛顿方程**（Gauss-Newton equation）或者**正规方程**（Normal equation）。我们把左边的系数定义为 $\boldsymbol{H}$，右边定义为 $\boldsymbol{g}$，那么上式变为：

$$\boldsymbol{H}\Delta\boldsymbol{x} = \boldsymbol{g}. \tag{6.33}$$

这里把左侧记作 $\boldsymbol{H}$ 是有意义的。对比牛顿法可见，高斯牛顿法用 $\boldsymbol{J}\boldsymbol{J}^{\mathrm{T}}$ **作为牛顿法中二阶 Hessian 矩阵的近似**，从而省略了计算 $\boldsymbol{H}$ 的过程。**求解增量方程是整个优化问题的核心所在**。如果我们能够顺利解出该方程，那么高斯牛顿法的算法步骤可以写成：

1. 给定初始值 $\boldsymbol{x}_0$。
2. 对于第 k 次迭代，求出当前的雅可比矩阵 $\boldsymbol{J}(\boldsymbol{x}_k)$ 和误差 $f(\boldsymbol{x}_k)$。
3. 求解增量方程：$\boldsymbol{H}\Delta\boldsymbol{x}_k = \boldsymbol{g}$。
4. 若 $\Delta\boldsymbol{x}_k$ 足够小，则停止。否则，令 $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta\boldsymbol{x}_k$，返回第 2 步。

从算法步骤中可以看到，增量方程的求解占据着主要地位。只要我们能够顺利解出增量，就能保证目标函数能够正确地下降。

为了求解增量方程，我们需要求解 $\boldsymbol{H}^{-1}$，这需要 $\boldsymbol{H}$ 矩阵可逆，但实际数据中计算得到的 $\boldsymbol{J}\boldsymbol{J}^{\mathrm{T}}$ 却只有半正定性。也就是说，在使用高斯牛顿法时，可能出现 $\boldsymbol{J}\boldsymbol{J}^{\mathrm{T}}$ 为奇异矩阵或者病态（ill-condition）的情况，此时增量的稳定性较差，导致算法不收敛。直观地说，原函数在这个点的局部近似不像一个二次函数。更严重的是，就算我们假设 $\boldsymbol{H}$ 非奇异也非病态，如果我们求出来的步长 $\Delta\boldsymbol{x}$ 太大，也

会导致我们采用的局部近似式(6.30)不够准确，这样一来我们甚至无法保证它的迭代收敛，哪怕是让目标函数变得更大都是有可能的。

尽管高斯牛顿法有这些缺点，但它依然算是非线性优化方面一种简单有效的方法，值得我们去学习。在非线性优化领域，相当多的算法都可以归结为高斯牛顿法的变种。这些算法都借助了高斯牛顿法的思想并且通过自己的改进修正其缺点。例如一些**线搜索方法** (line search method) 加入了一个步长 α，在确定了 $\Delta \boldsymbol{x}$ 后进一步找到 α 使得 $\|f(\boldsymbol{x}+\alpha\Delta\boldsymbol{x})\|^2$ 达到最小，而不是简单地令 $\alpha=1$。

列文伯格—马夸尔特方法在一定程度上修正了这些问题。一般认为它比高斯牛顿法更为健壮，但它的收敛速度可能会比高斯牛顿法更慢，被称为**阻尼牛顿法**（Damped Newton Method）。

6.2.3　列文伯格—马夸尔特方法

高斯牛顿法中采用的近似二阶泰勒展开只能在展开点附近有较好的近似效果，所以我们很自然地想到应该给 $\Delta \boldsymbol{x}$ 添加一个范围，称为**信赖区域**（Trust Region）。这个范围定义了在什么情况下二阶近似是有效的，这类方法也称为**信赖区域方法**（Trust Region Method）。在信赖区域里边，我们认为近似是有效的；出了这个区域，近似可能会出问题。

那么如何确定这个信赖区域的范围呢？一个比较好的方法是根据我们的近似模型跟实际函数之间的差异来确定：如果差异小，说明近似效果好，我们扩大近似的范围；反之，如果差异大，就缩小近似的范围。我们定义一个指标 ρ 来刻画近似的好坏程度：

$$\rho=\frac{f\left(\boldsymbol{x}+\Delta \boldsymbol{x}\right)-f\left(\boldsymbol{x}\right)}{\boldsymbol{J}\left(\boldsymbol{x}\right)^{\mathrm{T}}\Delta \boldsymbol{x}}. \tag{6.34}$$

ρ 的分子是实际函数下降的值，分母是近似模型下降的值。如果 ρ 接近于 1，则近似是好的。如果 ρ 太小，说明实际减小的值远少于近似减小的值，则认为近似比较差，需要缩小近似范围。反之，如果 ρ 比较大，则说明实际下降的比预计的更大，我们可以放大近似范围。

于是，我们构建一个改良版的非线性优化框架，该框架会比高斯牛顿法有更好的效果：

1. 给定初始值 $\boldsymbol{x}_0$，以及初始优化半径 μ。
2. 对于第 k 次迭代，在高斯牛顿法的基础上加上信赖区域，求解：

$$\min_{\Delta \boldsymbol{x}_k}\frac{1}{2}\left\|f\left(\boldsymbol{x}_k\right)+\boldsymbol{J}\left(\boldsymbol{x}_k\right)^{\mathrm{T}}\Delta \boldsymbol{x}_k\right\|^2,\quad \text{s.t.}\quad \|\boldsymbol{D}\Delta \boldsymbol{x}_k\|^2\leqslant\mu, \tag{6.35}$$

   其中 μ 是信赖区域的半径，$\boldsymbol{D}$ 为系数矩阵，将在后文说明。
3. 按式(6.34)计算 ρ。
4. 若 $\rho>\frac{3}{4}$，则设置 $\mu=2\mu$。
5. 若 $\rho<\frac{1}{4}$，则设置 $\mu=0.5\mu$。

6. 如果 ρ 大于某阈值，则认为近似可行。令 $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta\boldsymbol{x}_k$。
7. 判断算法是否收敛。如不收敛则返回第 2 步，否则结束。

这里近似范围扩大的倍数和阈值都是经验值，可以替换成别的数值。在式(6.35)中，我们把增量限定于一个半径为 μ 的球中，认为只在这个球内才是有效的。带上 $\boldsymbol{D}$ 之后，这个球可以看成一个椭球。在列文伯格提出的优化方法中，把 $\boldsymbol{D}$ 取成单位阵 $\boldsymbol{I}$，相当于直接把 $\Delta\boldsymbol{x}_k$ 约束在一个球中。随后，马夸尔特提出将 $\boldsymbol{D}$ 取成非负数对角阵——实际中通常用 $\boldsymbol{J}^T\boldsymbol{J}$ 的对角元素平方根，使得在梯度小的维度上约束范围更大一些。

不论如何，在列文伯格—马夸尔特优化中，我们都需要解式(6.35)那样一个子问题来获得梯度。这个子问题是带不等式约束的优化问题，我们用拉格朗日乘子把约束项放到目标函数中，构成拉格朗日函数：

$$\mathcal{L}(\Delta\boldsymbol{x}_k, \lambda) = \frac{1}{2}\left\| f(\boldsymbol{x}_k) + \boldsymbol{J}(\boldsymbol{x}_k)^{\mathrm{T}} \Delta\boldsymbol{x}_k \right\|^2 + \frac{\lambda}{2}\left(\|\boldsymbol{D}\Delta\boldsymbol{x}_k\|^2 - \mu \right). \tag{6.36}$$

这里 λ 为拉格朗日乘子。类似于高斯牛顿法中的做法，令该拉格朗日函数关于 $\Delta\boldsymbol{x}$ 的导数为零，它的核心仍是计算增量的线性方程：

$$\left(\boldsymbol{H} + \lambda\boldsymbol{D}^{\mathrm{T}}\boldsymbol{D}\right)\Delta\boldsymbol{x}_k = \boldsymbol{g}. \tag{6.37}$$

可以看到，增量方程相比于高斯牛顿法，多了一项 $\lambda\boldsymbol{D}^T\boldsymbol{D}$。如果考虑它的简化形式，即 $\boldsymbol{D} = \boldsymbol{I}$，那么相当于求解[①]：

$$(\boldsymbol{H} + \lambda\boldsymbol{I})\Delta\boldsymbol{x}_k = \boldsymbol{g}.$$

我们看到，当参数 λ 比较小时，$\boldsymbol{H}$ 占主要地位，这说明二次近似模型在该范围内是比较好的，列文伯格—马夸尔特方法更接近于高斯牛顿法。另一方面，当 λ 比较大时，$\lambda\boldsymbol{I}$ 占据主要地位，列文伯格—马夸尔特方法更接近于一阶梯度下降法（即最速下降），这说明附近的二次近似不够好。列文伯格—马夸尔特方法的求解方式，可在一定程度上避免线性方程组的系数矩阵的非奇异和病态问题，提供更稳定、更准确的增量 $\Delta\boldsymbol{x}$。

在实际中，还存在许多其他的方式来求解增量，例如 Dog-Leg[31] 等方法。我们在这里所介绍的，只是最常见而且最基本的方法，也是视觉 SLAM 中用得最多的方法。实际问题中，我们通常选择高斯牛顿法或列文伯格—马夸尔特方法其中之一作为梯度下降策略。当问题性质较好时，用高斯牛顿。如果问题接近病态，则用列文伯格—马夸尔特方法。

[①] 严谨的读者可能不满意此处的叙述。信赖域原问题的约束条件除了拉格朗日函数求导为零以外，KKT 条件还会有一些别的约束：$\lambda > 0$，且 $\lambda(\|\boldsymbol{D}\Delta\boldsymbol{x}\|^2 - \mu) = 0$。但是在 L-M 迭代中，我们不妨把它看成在原问题的目标函数上，以 λ 为权重的惩罚项。在每一步迭代后，若发现信赖域条件不满足，或者目标函数增加，就增加 λ 的权重，直到最终满足信赖域条件。所以，理论上对 L-M 算法存在不同的解释，但实际当中我们只关心它是否顺利工作。

6.2.4　小结

由于不希望这本书变成一本让人觉得头疼的数学教科书，所以这里只罗列了最常见的两种非线性优化方案——高斯牛顿法和列文伯格—马夸尔特方法。我们避开了许多数学性质上的讨论。如果读者对优化感兴趣，可以进一步阅读专门介绍数值优化的书籍（这是一个很大的课题）[31]。以高斯牛顿法和列文伯格—马夸尔特方法为代表的优化方法，在很多开源的优化库中都已经实现并提供给用户，我们会在下文进行实验。最优化是处理许多实际问题的基本数学工具，不光在视觉 SLAM 中起着核心作用，在类似于深度学习等其他领域，它也是求解问题的核心方法之一（深度学习数据量很大，以一阶方法为主）。我们希望读者能够根据自身能力，去了解更多的最优化算法。

也许你发现了，无论是高斯牛顿法还是列文伯格—马夸尔特方法，在做最优化计算时，都需要提供变量的初始值。你也许会问到，这个初始值能否随意设置？当然不是。实际上非线性优化的所有迭代求解方案，都需要用户来提供一个良好的初始值。由于目标函数太复杂，导致在求解空间上的变化难以预测，对问题提供不同的初始值往往会导致不同的计算结果。这种情况是非线性优化的通病：大多数算法都容易陷入局部极小值。因此，无论是哪类科学问题，我们提供初始值都应该有科学依据，例如视觉 SLAM 问题中，我们会用 ICP、PnP 之类的算法提供优化初始值。总之，一个良好的初始值对最优化问题非常重要！

也许读者还会对上面提到的最优化产生疑问：如何求解线性增量方程组呢？我们只讲到了增量方程是一个线性方程，但是直接对系数矩阵进行求逆岂不是要进行大量的计算？当然不是。在视觉 SLAM 算法里，经常遇到 $\Delta \boldsymbol{x}$ 的维度大到好几百或者上千，如果你是要做大规模的视觉三维重建，就会经常发现这个维度可以轻易达到几十万甚至更高的级别。要对那么大个矩阵进行求逆是大多数处理器无法负担的，因此存在着许多针对线性方程组的数值求解方法。在不同的领域有不同的求解方式，但几乎没有一种方式是直接求系数矩阵的逆，我们会采用矩阵分解的方法来解线性方程，例如 QR、Cholesky 等分解方法。这些方法通常在矩阵论等教科书中可以找到，我们不多加介绍。

幸运的是，视觉 SLAM 里这个矩阵往往有特定的稀疏形式，这为实时求解优化问题提供了可能性。我们将在第 9 讲中详细介绍它的原理。利用稀疏形式的消元、分解，最后再进行求解增量，会让求解的效率大大提高。在很多开源的优化库上，维度为一万多的变量在一般的 PC 上就可以在几秒甚至更短的时间内被求解出来，其原因也是用了更加高级的数学工具。视觉 SLAM 算法现在能够实时地实现，也多亏了系数矩阵是稀疏的，如果矩阵是稠密的，恐怕优化这类视觉 SLAM 算法就不会被学界广泛采纳了[32–34]。

6.3　实践：曲线拟合问题

6.3.1　手写高斯牛顿法

接下来我们用一个简单的例子来说明如何求解最小二乘问题。我们将演示如何手写高斯牛顿法，然后再介绍如何使用优化库求解此问题。对于同一个问题，这些实现方式会得到同样的结果，因为

它们的核心算法是一样的。

考虑一条满足以下方程的曲线：

$$y = \exp(ax^2 + bx + c) + w,$$

其中 a, b, c 为曲线的参数，w 为高斯噪声，满足 $w \sim (0, \sigma^2)$。我们故意选择了这样一个非线性模型，使问题不至于太简单。现在，假设我们有 N 个关于 x, y 的观测数据点，想根据这些数据点求出曲线的参数。那么，可以求解下面的最小二乘问题以估计曲线参数：

$$\min_{a,b,c} \frac{1}{2} \sum_{i=1}^{N} \left\| y_i - \exp\left(ax_i^2 + bx_i + c\right) \right\|^2. \tag{6.38}$$

请注意，在这个问题中，待估计的变量是 a, b, c，而不是 x。我们的程序里先根据模型生成 x, y 的真值，然后在真值中添加高斯分布的噪声。随后，使用高斯牛顿法来从带噪声的数据拟合参数模型。定义误差为:

$$e_i = y_i - \exp\left(ax_i^2 + bx_i + c\right), \tag{6.39}$$

那么可以求出每个误差项对于状态变量的导数：

$$\begin{aligned} \frac{\partial e_i}{\partial a} &= -x_i^2 \exp\left(ax_i^2 + bx_i + c\right) \\ \frac{\partial e_i}{\partial b} &= -x_i \exp\left(ax_i^2 + bx_i + c\right) \\ \frac{\partial e_i}{\partial c} &= -\exp\left(ax_i^2 + bx_i + c\right) \end{aligned} \tag{6.40}$$

于是 $\boldsymbol{J}_i = \left[\frac{\partial e_i}{\partial a}, \frac{\partial e_i}{\partial b}, \frac{\partial e_i}{\partial c}\right]^{\mathrm{T}}$，高斯牛顿法的增量方程为：

$$\left(\sum_{i=1}^{100} \boldsymbol{J}_i (\sigma^2)^{-1} \boldsymbol{J}_i^{\mathrm{T}} \right) \Delta \boldsymbol{x}_k = \sum_{i=1}^{100} -\boldsymbol{J}_i (\sigma^2)^{-1} e_i, \tag{6.41}$$

当然我们也可以选择把所有的 $\boldsymbol{J}_i$ 排成一列，将这个方程写成矩阵形式，不过它的含义与求和形式是一致的。下面的代码演示了这个过程是如何进行的。

slambook2/ch6/gaussNewton.cpp

```
#include <iostream>
#include <opencv2/opencv.hpp>
#include <Eigen/Core>
#include <Eigen/Dense>

using namespace std;
using namespace Eigen;

int main(int argc, char **argv) {
    double ar = 1.0, br = 2.0, cr = 1.0;         // 真实参数值
```

```
    double ae = 2.0, be = -1.0, ce = 5.0;        // 估计参数值
    int N = 100;                                 // 数据点
    double w_sigma = 1.0;                        // 噪声Sigma值
    double inv_sigma = 1.0 / w_sigma;
    cv::RNG rng;                                 // OpenCV随机数产生器

    vector<double> x_data, y_data;      // 数据
    for (int i = 0; i < N; i++) {
        double x = i / 100.0;
        x_data.push_back(x);
        y_data.push_back(exp(ar * x * x + br * x + cr) + rng.gaussian(w_sigma * w_sigma));
    }

    // 开始Gauss-Newton迭代
    int iterations = 100;    // 迭代次数
    double cost = 0, lastCost = 0;  // 本次迭代的cost和上一次迭代的cost

    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    for (int iter = 0; iter < iterations; iter++) {

        Matrix3d H = Matrix3d::Zero();             // Hessian = J^T W^{-1} J in Gauss-Newton
        Vector3d b = Vector3d::Zero();             // bias
        cost = 0;

        for (int i = 0; i < N; i++) {
            double xi = x_data[i], yi = y_data[i];  // 第i个数据点
            double error = yi - exp(ae * xi * xi + be * xi + ce);
            Vector3d J; // 雅可比矩阵
            J[0] = -xi * xi * exp(ae * xi * xi + be * xi + ce);  // de/da
            J[1] = -xi * exp(ae * xi * xi + be * xi + ce);  // de/db
            J[2] = -exp(ae * xi * xi + be * xi + ce);  // de/dc

            H += inv_sigma * inv_sigma * J * J.transpose();
            b += -inv_sigma * inv_sigma * error * J;

            cost += error * error;
        }

        // 求解线性方程 Hx=b
        Vector3d dx = H.ldlt().solve(b);
        if (isnan(dx[0])) {
            cout << "result is nan!" << endl;
            break;
        }

        if (iter > 0 && cost >= lastCost) {
            cout << "cost: " << cost << ">= last cost: " << lastCost << ", break." << endl;
            break;
        }

        ae += dx[0];
        be += dx[1];
        ce += dx[2];
```

```
64
65          lastCost = cost;
66
67          cout << "total cost: " << cost << ", \t\tupdate: " << dx.transpose() <<
68            "\t\testimated params: " << ae << "," << be << "," << ce << endl;
69      }
70
71      chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
72      chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 - t1);
73      cout << "solve time cost = " << time_used.count() << " seconds. " << endl;
74      cout << "estimated abc = " << ae << ", " << be << ", " << ce << endl;
75      return 0;
76  }
```

在这个例子中，我们演示了如何对一个简单的拟合问题进行迭代优化。通过自己手写的代码，很容易看清楚整个优化的流程。该程序输出每一步迭代的目标函数值和更新量，如下：

终端输出：

```
1  /home/xiang/Code/slambook2/ch6/cmake-build-debug/gaussNewton
2  total cost: 3.19575e+06,    update: 0.0455771  0.078164 -0.985329   estimated params:
       2.04558,-0.921836,4.01467
3  total cost: 376785,     update:  0.065762  0.224972 -0.962521   estimated params:
       2.11134,-0.696864,3.05215
4  total cost: 35673.6,    update: -0.0670241   0.617616  -0.907497    estimated params:
       2.04432,-0.0792484,2.14465
5  total cost: 2195.01,    update: -0.522767   1.19192 -0.756452   estimated params:
       1.52155,1.11267,1.3882
6  total cost: 174.853,    update: -0.537502  0.909933 -0.386395   estimated params:
       0.984045,2.0226,1.00181
7  total cost: 102.78,     update: -0.0919666   0.147331 -0.0573675    estimated params:
       0.892079,2.16994,0.944438
8  total cost: 101.937,    update: -0.00117081  0.00196749 -0.00081055   estimated params:
       0.890908,2.1719,0.943628
9  total cost: 101.937,    update:   3.4312e-06 -4.28555e-06  1.08348e-06    estimated params:
       0.890912,2.1719,0.943629
10 total cost: 101.937,    update: -2.01204e-08  2.68928e-08 -7.86602e-09    estimated params:
       0.890912,2.1719,0.943629
11 cost: 101.937>= last cost: 101.937, break.
12 solve time cost = 0.000212903 seconds.
13 estimated abc = 0.890912, 2.1719, 0.943629
```

易见整个问题的目标函数在迭代 9 次之后趋近收敛，更新量趋近于零。最终估计的值与真值接近，函数图像如图 6-1所示。在我的机器上（我的 CPU 是 i7-8700），优化用时约 0.2 个毫秒。下面我们尝试使用优化库来完成同样的任务。

6.3.2 使用 Ceres 进行曲线拟合

本节向大家介绍两个 C++ 的优化库：来自谷歌的 Ceres 库[35] 以及基于图优化的 g2o 库[36]。由于 g2o 的使用还需要介绍一点图优化的相关知识，所以我们先来介绍 Ceres，然后介绍一些图优化

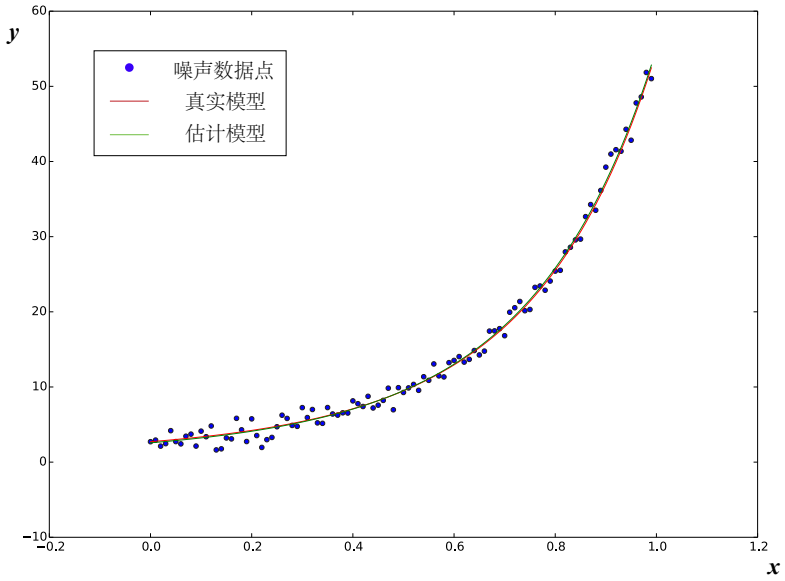


图 6-1　噪声 $\sigma = 1$ 时的曲线拟合结果。真实模型和估计模型非常接近。

理论，最后来讲 g2o。由于优化算法在之后的“视觉里程计”和“后端”中都会出现，所以请读者务必掌握优化算法的意义，理解程序的内容。

Ceres 简介

Google Ceres 是一个广泛使用的最小二乘问题求解库。在 Ceres 中，我们作为用户，只需按照一定步骤定义待解的优化问题，然后交给求解器计算即可。Ceres 求解的最小二乘问题最一般的形式如下（带边界的核函数最小二乘）：

$$\begin{aligned} \min_{x} \quad & \frac{1}{2} \sum_{i} \rho_i \left(\left\| f_i \left(x_{i_1}, \cdots x_{i_n} \right) \right\|^2 \right) \\ \text{s.t.} \quad & l_j \leqslant x_j \leqslant u_j. \end{aligned} \tag{6.42}$$

在这个问题中，$x_1, \cdots, x_n$ 为优化变量，又称**参数块**（Parameter blocks），f_i 称为**代价函数**（Cost function），也称为残差块（Residual blocks），在 SLAM 中亦可理解为误差项。l_j 和 u_j 为第 j 个优化变量的上限和下限。在最简单的情况下，取 $l_j = -\infty, u_j = \infty$（不限制优化变量的边界）。此时，目标函数由许多平方项经过一个**核函数** $\rho(\cdot)$ 之后求和组成[①]。同样，可以取 ρ 为恒等函数，那么目标函数即为许多项的平方和，我们就得到了无约束的最小二乘问题，和先前介绍的理论是一致的。

[①] 核函数的详细讨论见第 9 讲。

为了让 Ceres 帮我们求解这个问题，我们需要做以下几件事：

1. 定义每个参数块。参数块通常为平凡的向量，但是在 SLAM 里也可以定义成四元数、李代数这种特殊的结构。如果是向量，那么我们需要为每个参数块分配一个 double 数组，来存储变量的值。
2. 然后，定义残差块的计算方式。残差块通常关联若干个参数块，对它们进行一些自定义的计算，然后返回残差值。Ceres 对它们求平方和之后，作为目标函数的值。
3. 残差块往往也需要定义雅可比的计算方式。在 Ceres 中，你可以使用它提供的“自动求导”功能，也可以手动指定雅可比的计算过程。如果要使用自动求导，那么残差块需要按照特定的写法来书写：残差的计算过程应该是一个带模板的括号运算符。这一点我们通过例子来说明。
4. 最后，把所有的参数块和残差块加入 Ceres 定义的 Problem 对象中，调用 Solve 函数求解即可。求解之前，我们可以传入一些配置信息，例如迭代次数、终止条件等，也可以使用默认的配置。

下面，我们来实际操作一下 Ceres 求解曲线拟合问题，理解优化的过程。

安装 Ceres

为了使用 Ceres，我们需要对它进行编译安装。Ceres 的 github 地址为：`https://github.com/ceres-solver/ceres-solver`，不过你也可以直接使用本书代码 3rdparty 目录里的 Ceres，这样你将和我使用完全一样的版本。

与之前碰到的库一样，Ceres 是一个 cmake 工程。先来安装它的依赖项，在 Ubuntu 中可以用 apt-get 安装，主要是谷歌自己使用的一些日志和测试工具：

终端输入：

```
sudo apt-get install liblapack-dev libsuitesparse-dev libcxsparse3 libgflags-dev libgoogle-glog-dev
    libgtest-dev
```

然后，进入 Ceres 库目录下，使用 cmake 编译并安装它。这个过程我们已经做过很多遍了，此处不再赘述。安装完成后，在/usr/local/include/ceres 下找到 Ceres 的头文件，并在/usr/local/lib/下找到名为 libceres.a 的库文件。有了这些文件，就可以使用 Ceres 进行优化计算了。

使用 Ceres 拟合曲线

下面的代码演示了如何使用 Ceres 求解同样的问题。

slambook/ch6/ceresCurveFitting.cpp

```
#include <iostream>
#include <opencv2/core/core.hpp>
#include <ceres/ceres.h>
```

```cpp
#include <chrono>

using namespace std;

// 代价函数的计算模型
struct CURVE_FITTING_COST {
    CURVE_FITTING_COST(double x, double y) : _x(x), _y(y) {}

    // 残差的计算
    template<typename T>
    bool operator()(
        const T *const abc, // 模型参数，有3维
        T *residual) const {
        // y-exp(ax^2+bx+c)
        residual[0] = T(_y) - ceres::exp(abc[0] * T(_x) * T(_x) + abc[1] * T(_x) + abc[2]);
        return true;
    }

    const double _x, _y;    // x,y数据
};

int main(int argc, char **argv) {
    double ar = 1.0, br = 2.0, cr = 1.0;         // 真实参数值
    double ae = 2.0, be = -1.0, ce = 5.0;        // 估计参数值
    int N = 100;                                 // 数据点
    double w_sigma = 1.0;                        // 噪声Sigma值
    double inv_sigma = 1.0 / w_sigma;
    cv::RNG rng;                                 // OpenCV随机数产生器

    vector<double> x_data, y_data;      // 数据
    for (int i = 0; i < N; i++) {
        double x = i / 100.0;
        x_data.push_back(x);
        y_data.push_back(exp(ar * x * x + br * x + cr) + rng.gaussian(w_sigma * w_sigma));
    }

    double abc[3] = {ae, be, ce};

    // 构建最小二乘问题
    ceres::Problem problem;
    for (int i = 0; i < N; i++) {
        problem.AddResidualBlock(     // 向问题中添加误差项
            // 使用自动求导，模板参数：误差类型，输出维度，输入维度，维数要与前面struct中一致
            new ceres::AutoDiffCostFunction<CURVE_FITTING_COST, 1, 3>(
                new CURVE_FITTING_COST(x_data[i], y_data[i])
            ),
            nullptr,            // 核函数，这里不使用，为空
            abc                 // 待估计参数
        );
    }

    // 配置求解器
    ceres::Solver::Options options;     // 这里有很多配置项可以填
```

```
    options.linear_solver_type = ceres::DENSE_NORMAL_CHOLESKY;  // 增量方程如何求解
    options.minimizer_progress_to_stdout = true;   // 输出到cout

    ceres::Solver::Summary summary;                // 优化信息
    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    ceres::Solve(options, &problem, &summary);  // 开始优化
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 - t1);
    cout << "solve time cost = " << time_used.count() << " seconds. " << endl;

    // 输出结果
    cout << summary.BriefReport() << endl;
    cout << "estimated a,b,c = ";
    for (auto a:abc) cout << a << " ";
    cout << endl;

    return 0;
}
```

程序中需要说明的地方均已加注释。可以看到，我们利用 OpenCV 的噪声生成器生成了 100 个带高斯噪声的数据，随后利用 Ceres 进行拟合。这里演示的 Ceres 用法有如下几项：

1. 定义残差块的类。方法是书写一个类（或结构体），并在类中定义带模板参数的 () 运算符，这样该类就成为了一个**拟函数**（Functor）[①]。这种定义方式使得 Ceres 可以像调用函数一样，对该类的某个对象（比如 a）调用 a<double>() 方法。事实上，Ceres 会把雅可比矩阵作为类型参数传入此函数，从而实现自动求导的功能。
2. 程序中的 double abc[3] 即为参数块，而对于残差块，我们对每一个数据构造 CURVE_FITTING_COST 对象，然后调用 AddResidualBlock 将误差项添加到目标函数中。由于优化需要梯度，我们有若干种选择：（1）使用 Ceres 的自动求导（Auto Diff）；（2）使用数值求导（Numeric Diff）[②]；（3）自行推导解析的导数形式，提供给 Ceres。因为自动求导在编码上是最方便的，于是我们使用自动求导。
3. 自动求导需要指定误差项和优化变量的维度。这里的误差是标量，维度为 1；优化的是 a, b, c 三个量，维度为 3。于是，在自动求导类 AutoDiffCostFunction 的模板参数中设定变量维度为 1、3。
4. 设定好问题后，调用 Solve 函数进行求解。你可以在 options 里配置（非常详细的）优化选项。例如，可以选择使用 Line Search 还是 Trust Region、迭代次数、步长，等等。读者可以查看 Options 的定义，看看有哪些优化方法可选，当然默认的配置已经可用于很广泛的问题了。

最后，我们来看看实验结果。调用 build/ceresCurveFitting 查看优化结果：

[①] C++ 术语，因为带有括号运算符的类在使用括号算符时，就仿佛是一个函数一样。

[②] 自动求导也是用数值导数实现的，但由于是模板运算，所以运行更快一些。

```
iter      cost      cost_change  |gradient|   |step|    tr_ratio  tr_radius  ls_iter  iter_time
    total_time
0  1.597873e+06    0.00e+00    3.52e+06   0.00e+00   0.00e+00  1.00e+04        0    2.10e-05    7.92e
    -05
1  1.884440e+05    1.41e+06    4.86e+05   9.88e-01   8.82e-01  1.81e+04        1    5.60e-05    1.05e
    -03
2  1.784821e+04    1.71e+05    6.78e+04   9.89e-01   9.06e-01  3.87e+04        1    2.00e-05    1.09e
    -03
3  1.099631e+03    1.67e+04    8.58e+03   1.10e+00   9.41e-01  1.16e+05        1    6.70e-05    1.16e
    -03
4  8.784938e+01    1.01e+03    6.53e+02   1.51e+00   9.67e-01  3.48e+05        1    1.88e-05    1.19e
    -03
5  5.141230e+01    3.64e+01    2.72e+01   1.13e+00   9.90e-01  1.05e+06        1    1.81e-05    1.22e
    -03
6  5.096862e+01    4.44e-01    4.27e-01   1.89e-01   9.98e-01  3.14e+06        1    1.79e-05    1.25e
    -03
7  5.096851e+01    1.10e-04    9.53e-04   2.84e-03   9.99e-01  9.41e+06        1    1.81e-05    1.28e
    -03
solve time cost = 0.00130755 seconds.
Ceres Solver Report: Iterations: 8, Initial cost: 1.597873e+06, Final cost: 5.096851e+01, Termination:
     CONVERGENCE
estimated a,b,c = 0.890908 2.1719 0.943628
```

最终的优化值和我们上一节的实验结果基本相同，但运行速度上 Ceres 要相对慢一些。在我的机器上 Ceres 约使用了 1.3 个毫秒，这比手写高斯牛顿法慢了约六倍左右。

希望读者通过这个简单的例子对 Ceres 的使用方法有一个大致了解。它的优点是提供了自动求导工具，使得不必去计算很麻烦的雅可比矩阵。Ceres 的自动求导是通过模板元实现的，在编译时期就可以完成自动求导工作，不过仍然是数值导数。本书大部分时候仍然会介绍雅可比矩阵的计算，因为那样对理解问题更有帮助，而且在优化中更少出现问题。此外，Ceres 的优化过程配置也很丰富，使其适合很广泛的最小二乘优化问题，包括 SLAM 之外的各种问题。

6.3.3　使用 g2o 进行曲线拟合

本讲的第 2 个实践部分将介绍另一个（主要在 SLAM 领域）广为使用的优化库：g2o（General Graphic Optimization，G^2O）。它是一个基于**图优化**的库。图优化是一种将非线性优化与图论结合起来的理论，因此在使用它之前，我们花一点篇幅介绍一下图优化理论。

图优化理论简介

我们已经介绍了非线性最小二乘的求解方式。它们是由很多个误差项之和组成的。然而，仅有一组优化变量和许多个误差项，我们并不清楚它们之间的**关联**。比如，某个优化变量 x_j 存在于多少个误差项中呢？我们能保证对它的优化是有意义的吗？进一步，我们希望能够直观地看到该优化问题**长什么样**。于是，就牵涉到了图优化。

图优化，是把优化问题表现成**图（Graph）**的一种方式。这里的**图**是图论意义上的图。一个图由若干个**顶点（Vertex）**，以及连接着这些顶点的**边（Edge）**组成。进而，用**顶点**表示**优化变量**，用**边**表示**误差项**。于是，对任意一个上述形式的非线性最小二乘问题，我们可以构建与之对应的一个**图**。我们可以简单地称它为**图**，也可以用概率图里的定义，称之为**贝叶斯图**或**因子图**。

图 6-2 是一个简单的图优化例子。我们用三角形表示相机位姿节点，用圆形表示路标点，它们构成了图优化的顶点；同时，实线表示相机的运动模型，虚线表示观测模型，它们构成了图优化的边。此时，虽然整个问题的数学形式仍是式(6.13)那样，但现在我们可以直观地看到问题的**结构**了。如果希望，也可以做**去掉孤立顶点**或**优先优化边数较多（或按图论的术语，度数较大）的顶点**这样的改进。但是最基本的图优化是用图模型来表达一个非线性最小二乘的优化问题。而我们可以利用图模型的某些性质做更好的优化。

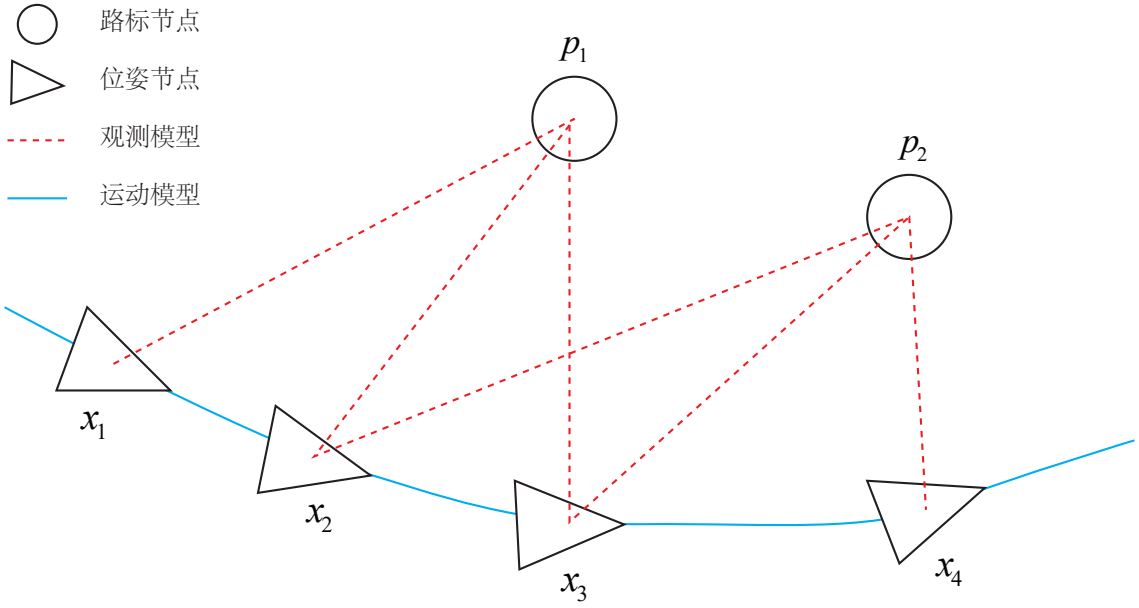


图 6-2　图优化的例子。

g2o 是一个通用的图优化库。“通用”意味着你可以在 g2o 里求解任何能够表示为图优化的最小二乘问题，显然包括上面谈的曲线拟合问题。下面我们来演示这个过程。

6.3.4　g2o 的编译与安装

在使用一个库之前，我们需要对它进行编译和安装。读者应该已经体验过很多次这种过程了，它们基本大同小异。关于 g2o，读者可以从 GitHub 下载它：`https://github.com/RainerKuemmerle/g2o`，或从本书提供的第三方代码库中获得。由于 g2o 还在继续更新，所以我建议你使用 3rdparty 下

的 g2o 以保证版本与我的相同。

g2o 也是一个 cmake 工程。我们先来安装它的依赖项（部分依赖项与 Ceres 重合）：

终端输入：

```
sudo apt-get install qt5-qmake qt5-default libqglviewer-dev-qt5 libsuitesparse-dev libcxsparse3
    libcholmod3
```

然后，按照 cmake 的方式对 g2o 进行编译安装即可，这里略去对该过程的说明。安装完成后，g2o 的头文件将位于/usr/local/g2o 下，库文件位于/usr/local/lib/下。现在，我们重新考虑 Ceres 例程中的曲线拟合实验，在 g2o 中实验一遍。

6.3.5　使用 g2o 拟合曲线

为了使用 g2o，首先要将曲线拟合问题抽象成图优化。这个过程中，只要记住**节点为优化变量，边为误差项**即可。曲线拟合的图优化问题可以画成图 6-3 的形式。

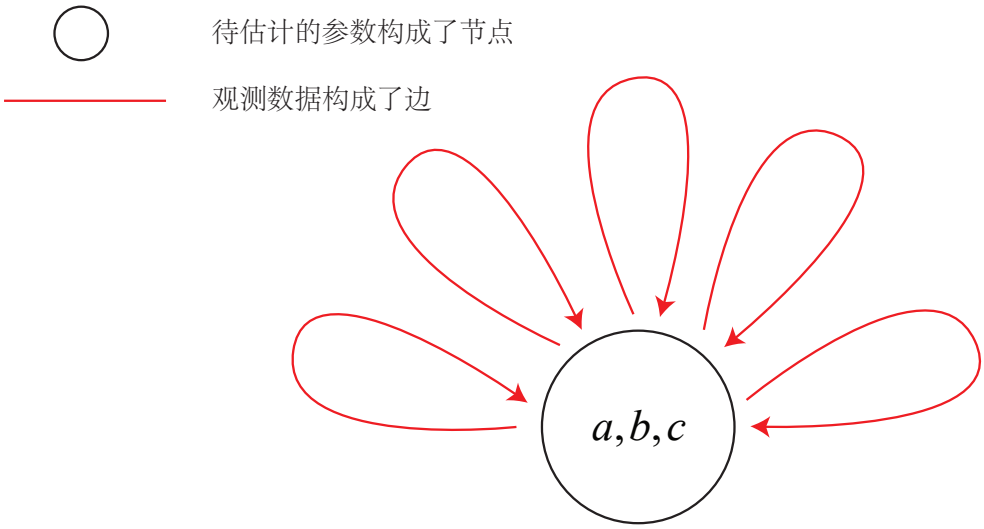


图 6-3　曲线拟合对应的图优化模型。（莫明其妙地有些像华为的标志）

在曲线拟合问题中，整个问题只有一个顶点：曲线模型的参数 a, b, c；而各个带噪声的数据点，构成了一个个误差项，也就是图优化的边。但这里的边与我们平时想的边不太一样，它们是**一元边**（Unary Edge），即**只连接一个顶点**——因为整个图只有一个顶点。所以在图 6-3 中，我们只能把它画成自己连到自己的样子。事实上，图优化中一条边可以连接一个、两个或多个顶点，这主要反映每个误差与多少个优化变量有关。在稍有些玄妙的说法中，我们把它叫作**超边**（Hyper Edge），整个

图叫作**超图**（Hyper Graph）[①]。

弄清了这个图模型之后，接下来就是在 g2o 中建立该模型进行优化了。作为 g2o 的用户，我们要做的事主要包含以下步骤：

1. 定义顶点和边的类型。
2. 构建图。
3. 选择优化算法。
4. 调用 g2o 进行优化，返回结果。

这部分和 Ceres 是非常相似的，当然程序在书写上会有一些不同。下面演示一下程序。

slambook/ch6/g2oCurveFitting.cpp

```cpp
#include <iostream>
#include <g2o/core/g2o_core_api.h>
#include <g2o/core/base_vertex.h>
#include <g2o/core/base_unary_edge.h>
#include <g2o/core/block_solver.h>
#include <g2o/core/optimization_algorithm_levenberg.h>
#include <g2o/core/optimization_algorithm_gauss_newton.h>
#include <g2o/core/optimization_algorithm_dogleg.h>
#include <g2o/solvers/dense/linear_solver_dense.h>
#include <Eigen/Core>
#include <opencv2/core/core.hpp>
#include <cmath>
#include <chrono>

using namespace std;

// 曲线模型的顶点，模板参数：优化变量维度和数据类型
class CurveFittingVertex : public g2o::BaseVertex<3, Eigen::Vector3d> {
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW

    // 重置
    virtual void setToOriginImpl() override {
        _estimate << 0, 0, 0;
    }

    // 更新
    virtual void oplusImpl(const double *update) override {
        _estimate += Eigen::Vector3d(update);
    }

    // 存盘和读盘：留空
    virtual bool read(istream &in) {}
    virtual bool write(ostream &out) const {}
};

```

[①] 显然我个人并不太喜欢有些故弄玄虚的说法，我是个自然主义者。

```cpp
// 误差模型 模板参数：观测值维度，类型，连接顶点类型
class CurveFittingEdge : public g2o::BaseUnaryEdge<1, double, CurveFittingVertex> {
public:
    EIGEN_MAKE_ALIGNED_OPERATOR_NEW

    CurveFittingEdge(double x) : BaseUnaryEdge(), _x(x) {}

    // 计算曲线模型误差
    virtual void computeError() override {
        const CurveFittingVertex *v = static_cast<const CurveFittingVertex *> (_vertices[0]);
        const Eigen::Vector3d abc = v->estimate();
        _error(0, 0) = _measurement - std::exp(abc(0, 0) * _x * _x + abc(1, 0) * _x + abc(2, 0));
    }

    // 计算雅可比矩阵
    virtual void linearizeOplus() override {
        const CurveFittingVertex *v = static_cast<const CurveFittingVertex *> (_vertices[0]);
        const Eigen::Vector3d abc = v->estimate();
        double y = exp(abc[0] * _x * _x + abc[1] * _x + abc[2]);
        _jacobianOplusXi[0] = -_x * _x * y;
        _jacobianOplusXi[1] = -_x * y;
        _jacobianOplusXi[2] = -y;
    }

    virtual bool read(istream &in) {}
    virtual bool write(ostream &out) const {}
public:
    double _x;  // x 值， y 值为 _measurement
};

int main(int argc, char **argv) {
    // 省略数据生成部分代码
    // 构建图优化，先设定g2o
    typedef g2o::BlockSolver<g2o::BlockSolverTraits<3, 1>> BlockSolverType;  // 每个误差项优化变量维度为3，误差值维度为1
    typedef g2o::LinearSolverDense<BlockSolverType::PoseMatrixType> LinearSolverType; // 线性求解器类型

    // 梯度下降方法，可以从GN, LM, DogLeg 中选
    auto solver = new g2o::OptimizationAlgorithmGaussNewton(
        g2o::make_unique<BlockSolverType>(g2o::make_unique<LinearSolverType>()));
    g2o::SparseOptimizer optimizer;     // 图模型
    optimizer.setAlgorithm(solver);   // 设置求解器
    optimizer.setVerbose(true);       // 打开调试输出

    // 往图中增加顶点
    CurveFittingVertex *v = new CurveFittingVertex();
    v->setEstimate(Eigen::Vector3d(ae, be, ce));
    v->setId(0);
    optimizer.addVertex(v);

    // 往图中增加边
    for (int i = 0; i < N; i++) {
```

```
        CurveFittingEdge *edge = new CurveFittingEdge(x_data[i]);
        edge->setId(i);
        edge->setVertex(0, v);                // 设置连接的顶点
        edge->setMeasurement(y_data[i]);      // 观测数值
        edge->setInformation(Eigen::Matrix<double, 1, 1>::Identity() * 1 / (w_sigma * w_sigma)); // 信息矩阵：协方差矩阵之逆
        optimizer.addEdge(edge);
    }

    // 执行优化
    cout << "start optimization" << endl;
    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    optimizer.initializeOptimization();
    optimizer.optimize(10);
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 - t1);
    cout << "solve time cost = " << time_used.count() << " seconds. " << endl;

    // 输出优化值
    Eigen::Vector3d abc_estimate = v->estimate();
    cout << "estimated model: " << abc_estimate.transpose() << endl;

    return 0;
}
```

在这个程序中，我们从 g2o 派生出了用于曲线拟合的图优化顶点和边：CurveFittingVertex 和 CurveFittingEdge，这实质上扩展了 g2o 的使用方式。这两个类分别派生自 BaseVertex 和 BaseUnaryEdge 类。在派生类中，我们重写了重要的虚函数：

1. 顶点的更新函数：oplusImpl。我们知道优化过程最重要的是增量 $\Delta\boldsymbol{x}$ 的计算，而该函数处理的是 $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta\boldsymbol{x}$ 的过程。

   读者也许觉得这并不是什么值得一提的事情，因为仅仅是个简单的加法而已，为什么 g2o 不帮我们完成呢？在曲线拟合过程中，由于优化变量（曲线参数）本身位于**向量空间**中，这个更新计算确实就是简单的加法。但是，当优化变量不在向量空间中时，比如说 $\boldsymbol{x}$ 是相机位姿，它本身不一定有加法运算。这时，就需要重新定义**增量如何加到现有的估计上**的行为了。按照第 4 讲的解释，我们可能使用左乘更新或右乘更新，而不是直接的加法。
2. 顶点的重置函数：setToOriginImpl。这是平凡的，我们把估计值置零即可。
3. 边的误差计算函数：computeError。该函数需要取出边所连接的顶点的当前估计值，根据曲线模型，与它的观测值进行比较。这和最小二乘问题中的误差模型是一致的。
4. 边的雅可比计算函数：linearizeOplus。这个函数里我们计算了每条边相对于顶点的雅可比。
5. 存盘和读盘函数：read、write。由于我们并不想进行读/写操作，所以留空。

定义了顶点和边之后，我们在 main 函数里声明了一个图模型，然后按照生成的噪声数据，往图模型中添加顶点和边，最后调用优化函数进行优化。g2o 会给出优化的结果：

终端输出：

```
start optimization
iteration= 0   chi2= 376785.128234   time= 3.3299e-05  cumTime= 3.3299e-05   edges= 100  schur= 0
iteration= 1   chi2= 35673.566018  time= 1.3789e-05  cumTime= 4.7088e-05   edges= 100  schur= 0
iteration= 2   chi2= 2195.012304   time= 1.2323e-05  cumTime= 5.9411e-05   edges= 100  schur= 0
iteration= 3   chi2= 174.853126  time= 1.3302e-05  cumTime= 7.2713e-05   edges= 100  schur= 0
iteration= 4   chi2= 102.779695  time= 1.2424e-05  cumTime= 8.5137e-05   edges= 100  schur= 0
iteration= 5   chi2= 101.937194  time= 1.2523e-05  cumTime= 9.766e-05  edges= 100  schur= 0
iteration= 6   chi2= 101.937020  time= 1.2268e-05  cumTime= 0.000109928  edges= 100  schur= 0
iteration= 7   chi2= 101.937020  time= 1.2612e-05  cumTime= 0.00012254   edges= 100  schur= 0
iteration= 8   chi2= 101.937020  time= 1.2159e-05  cumTime= 0.000134699  edges= 100  schur= 0
iteration= 9   chi2= 101.937020  time= 1.2688e-05  cumTime= 0.000147387  edges= 100  schur= 0
solve time cost = 0.000919301 seconds.
estimated model: 0.890912   2.1719 0.943629
```

我们使用高斯—牛顿方法进行梯度下降，在迭代了 9 次后得到优化结果，与 Ceres 和手写高斯牛顿法相差无几。从运行速度来看，我们实验结论是手写快于 g2o，而 g2o 快于 Ceres。这是一个大体符合直觉的经验，通用性和高效性往往是互相矛盾的。但是本实验中 Ceres 使用了自动求导，且求解器配置与高斯牛顿还不完全一致，所以看起来慢一些。

6.4　小结

本节介绍了 SLAM 中经常碰到的一种非线性优化问题：由许多个误差项平方和组成的最小二乘问题。我们介绍了它的定义和求解，并且讨论了两种主要的梯度下降方式：高斯牛顿法和列文伯格—马夸尔特方法。在实践部分中，分别使用了手写高斯牛顿法、Ceres 和 g2o 两种优化库求解同一个曲线拟合问题，发现它们给出了相似的结果。

由于还没有详细谈 Bundle Adjustment，所以我们在实践部分选择了曲线拟合这样一个简单但有代表性的例子，以演示一般的非线性最小二乘求解方式。特别地，如果用 g2o 来拟合曲线，必须先把问题转换为图优化，定义新的顶点和边，这种做法是有一些迂回的——g2o 的主要目的并不在此。相比之下，Ceres 定义误差项求曲线拟合问题则自然了很多，因为它本身即是一个优化库。然而，在 SLAM 中更多的问题是，一个带有许多个相机位姿和许多个空间点的优化问题如何求解。特别地，当相机位姿以李代数表示时，误差项关于相机位姿的导数如何计算，将是一件值得详细讨论的事。我们将在后续内容发现，g2o 提供了大量现成的顶点和边，非常便于相机位姿估计问题。而在 Ceres 中，我们不得不自己实现每一个 Cost Function，有一些不便。

在实践部分的两个程序中，我们没有去计算曲线模型关于三个参数的导数，而是利用了优化库的数值求导，这使得理论和代码都会简洁一些。Ceres 库提供了基于模板元的自动求导和运行时的数值求导，而 g2o 只提供了运行时数值求导这一种方式。但是，对于大多数问题，如果能够推导出雅可比矩阵的解析形式并告诉优化库，就可以避免数值求导中的诸多问题。

最后，希望读者能够适应 Ceres 和 g2o 这些大量使用模板编程的方式。也许一开始会看上去比较吓人（特别是 Ceres 设置残差块的括号运算符，以及 g2o 初始化部分的代码），但是熟悉之后，就

会觉得这样的方式是自然的，而且容易扩展。我们将在 SLAM 后端一讲中继续讨论稀疏性、核函数、位姿图（Pose Graph）等问题。

习题

1. 证明线性方程 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 当系数矩阵 $\boldsymbol{A}$ 超定时，最小二乘解为 $\boldsymbol{x} = (\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A})^{-1}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{b}$。
2. 调研最速下降法、牛顿法、高斯牛顿法和列文伯格—马夸尔特方法各有什么优缺点。除了我们举的 Ceres 库和 g2o 库，还有哪些常用的优化库？（你可能会找到一些 MATLAB 上的库。）
3. 为什么高斯牛顿法的增量方程系数矩阵可能不正定？不正定有什么几何含义？为什么在这种情况下解就不稳定了？
4. DogLeg 是什么？它与高斯牛顿法和列文伯格—马夸尔特方法有何异同？请搜索相关的材料[①]。
5. 阅读 Ceres 的教学材料（`http://ceres-solver.org/tutorial.html`）以更好地掌握其用法。
6. 阅读 g2o 自带的文档，你能看懂它吗？如果还不能完全看懂，请在第 10 讲和第 11 讲之后回来再看。
7.* 请更改曲线拟合实验中的曲线模型，并用 Ceres 和 g2o 进行优化实验。例如，可以使用更多的参数和更复杂的模型。

[①] 例如，`http://www.numerical.rl.ac.uk/people/nimg/course/lectures/raphael/lectures/lec7slides.pdf`。

第二部分

实践应用

第七章　视觉里程计 1

主要目标

1. 理解图像特征点的意义，并掌握在单幅图像中提取出特征点及多幅图像中匹配特征点的方法。
2. 理解对极几何的原理，利用对极几何的约束，恢复出图像之间的摄像机的三维运动。
3. 理解 PNP 问题，以及利用已知三维结构与图像的对应关系求解摄像机的三维运动。
4. 理解 ICP 问题，以及利用点云的匹配关系求解摄像机的三维运动。
5. 理解如何通过三角化获得二维图像上对应点的三维结构。

本书前面介绍了运动方程和观测方程的具体形式，并讲解了以非线性优化为主的求解方法。从本讲开始，我们结束基础知识的铺垫而步入正题：按照第 2 讲的顺序，分别介绍视觉里程计、优化后端、回环检测和地图构建 4 个模块。本讲和下一讲主要介绍两类视觉里程计里常用的方法：特征点法和光流法。本讲中，我们将介绍什么是特征点、如何提取和匹配特征点，以及如何根据配对的特征点估计相机运动。

Image 219
d = 7.05
u = 312
v = 219
desc = [1001011...]
R = [ ]
t = [ ]

7.1　特征点法

在第二讲中，我们说，一个 SLAM 系统分为前端和后端，其中前端也称为视觉里程计（VO）。VO 根据相邻图像的信息估计出粗略的相机运动，给后端提供较好的初始值。VO 的算法主要分为两个大类：**特征点法**和**直接法**。基于特征点法的前端，长久以来（直到现在）被认为是视觉里程计的主流方法。它具有稳定，对光照、动态物体不敏感的优势，是目前比较成熟的解决方案。在本讲中，我们将从特征点法入手，学习如何提取、匹配图像特征点，然后估计两帧之间的相机运动和场景结构，从而实现一个两帧间视觉里程计。这类算法有时也称为两视图几何（Two-view geometry）。

7.1.1　特征点

VO 的核心问题是**如何根据图像来估计相机运动**。然而，图像本身是一个由亮度和色彩组成的矩阵，如果直接从矩阵层面考虑运动估计，将会非常困难。所以，比较方便的做法是：首先，从图像中选取比较**有代表性**的**点**。这些点在相机视角发生少量变化后会保持不变，于是我们能在各个图像中找到相同的点。然后，在这些点的基础上，讨论相机位姿估计问题，以及这些点的定位问题。在经典 SLAM 模型中，我们称这些点为**路标**（Landmark）。而在视觉 SLAM 中，路标则是指图像特征（Feature）。

根据维基百科的定义，图像特征是一组与计算任务相关的信息，计算任务取决于具体的应用[37]。简而言之，**特征是图像信息的另一种数字表达形式**。一组好的特征对于在指定任务上的最终表现至关重要，所以多年来研究者们花费了大量的精力对特征进行研究。数字图像在计算机中以灰度值矩阵的方式存储，所以最简单的，单个图像像素也是一种“特征”。但是，在视觉里程计中，我们希望**特征点在相机运动之后保持稳定**，而灰度值受光照、形变、物体材质的影响严重，在不同图像间变化非常大，不够稳定。理想的情况是，当场景和相机视角发生少量改变时，算法还能从图像中判断哪些地方是同一个点。所以，仅凭灰度值是不够的，我们需要对图像提取特征点。

特征点是图像里一些**特别的地方**。以图 7-1 为例。我们可以把图像中的角点、边缘和区块都当成图像中有代表性的地方。不过，我们更容易精确地指出，某两幅图像中出现了同一个角点；同一个边缘则稍微困难一些，因为沿着该边缘前进，图像局部是相似的；同一个区块则是最困难的。我们发现，图像中的角点、边缘相比于像素区块而言更加“特别”，在不同图像之间的辨识度更强。所以，一种直观的提取特征的方式就是在不同图像间辨认角点，确定它们的对应关系。在这种做法中，角点就是所谓的特征。角点的提取算法有很多，例如 Harris 角点[38]、FAST 角点[39]、GFTT 角点[40]，等等。它们大部分是 2000 年以前提出的算法。

然而，在大多数应用中，单纯的角点依然不能满足我们的很多需求。例如，从远处看上去是角点的地方，当相机走近之后，可能就不显示为角点了。或者，当旋转相机时，角点的外观会发生变化，我们也就不容易辨认出那是同一个角点。为此，计算机视觉领域的研究者们在长年的研究中设计了许多更加稳定的局部图像特征，如著名的 SIFT[41]、SURF[42]、ORB[43]，等等。相比于朴素的角点，这些人工设计的特征点能够拥有如下的性质：

1. 可重复性（Repeatability）：相同的特征可以在不同的图像中找到。
2. 可区别性（Distinctiveness）：不同的特征有不同的表达。
3. 高效率（Efficiency）：同一图像中，特征点的数量应远小于像素的数量。
4. 本地性（Locality）：特征仅与一小片图像区域相关。

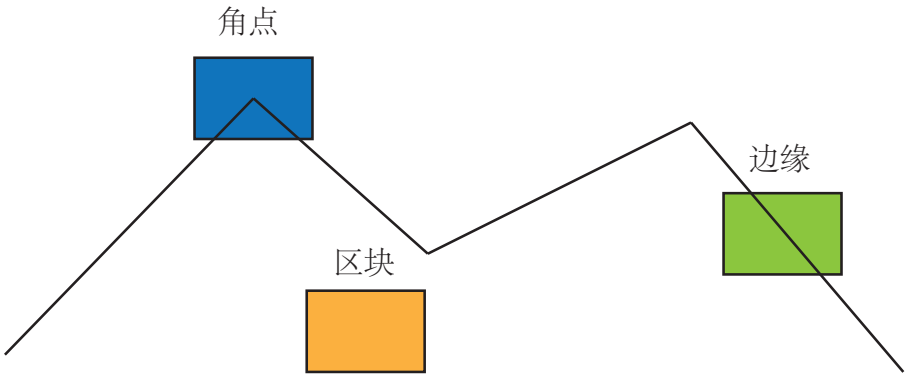


图 7-1　可以作为图像特征的部分：角点、边缘、区块。

特征点由**关键点**（Key-point）和**描述子**（Descriptor）两部分组成。比如，当我们说“在一张图像中计算 SIFT 特征点”，是指“提取 SIFT 关键点，并计算 SIFT 描述子”两件事情。关键点是指该特征点在图像里的位置，有些特征点还具有朝向、大小等信息。描述子通常是一个向量，按照某种人为设计的方式，描述了该关键点周围像素的信息。描述子是按照“**外观相似的特征应该有相似的描述子**”的原则设计的。因此，只要两个特征点的描述子在向量空间上的距离相近，就可以认为它们是同样的特征点。

历史上，研究者们提出过许多图像特征。它们有些很精确，在相机的运动和光照变化下仍具有相似的表达，但相应地需要较大的计算量。其中，SIFT（尺度不变特征变换，Scale-Invariant Feature Transform）当属最为经典的一种。它充分考虑了在图像变换过程中出现的光照、尺度、旋转等变化，但随之而来的是极大的计算量。由于整个 SLAM 过程中图像特征的提取与匹配仅仅是诸多环节中的一个，到目前（2016 年）为止，普通 PC 的 CPU 还无法实时地计算 SIFT 特征，进行定位与建图[①]。所以在 SLAM 中我们甚少使用这种“奢侈”的图像特征。

另一些特征，则考虑适当降低精度和鲁棒性，以提升计算的速度。例如，FAST 关键点属于计算特别快的一种特征点（注意这里“关键点”的表述，说明它没有描述子），而 ORB（Oriented FAST and Rotated BRIEF）特征则是目前看来非常具有代表性的实时图像特征。它改进了 FAST 检测子[39]不具有方向性的问题，并采用速度极快的二进制描述子 BRIEF[44]，使整个图像特征提取的环节大大加速。根据作者在论文中所述测试，在同一幅图像中同时提取约 1000 个特征点的情况下，ORB 约要花费 15.3ms，SURF 约花费 217.3ms，SIFT 约花费 5228.7ms。由此可以看出，ORB 在保持了特征

[①] 这里是指 30Hz 的实时速度。

子具有旋转、尺度不变性的同时，速度方面提升明显，对于实时性要求很高的 SLAM 来说是一个很好的选择。

大部分特征提取都具有较好的并行性，可以通过 GPU 等设备来加速计算。经过 GPU 加速后的 SIFT，就可以满足实时计算要求。但是，引入 GPU 将带来整个 SLAM 成本的提升。由此带来的性能提升是否足以抵去付出的计算成本，需要系统的设计人员仔细考量。

显然，计算机视觉领域存在大量的特征点种类，我们不可能在书中一一介绍。在目前的 SLAM 方案中，ORB 是质量与性能之间较好的折中，因此，我们以 ORB 为代表介绍提取特征的整个过程。如果读者对特征提取和匹配算法感兴趣，我们建议阅读这方面的相关书籍 [45]。

7.1.2　ORB 特征

ORB 特征亦由**关键点**和**描述子**两部分组成。它的关键点称为“Oriented FAST”，是一种改进的 FAST 角点，关于什么是 FAST 角点我们将在下文介绍。它的描述子称为 BRIEF（Binary Robust Independent Elementary Feature）。因此，提取 ORB 特征分为如下两个步骤：

1. FAST 角点提取：找出图像中的“角点”。相较于原版的 FAST，ORB 中计算了特征点的主方向，为后续的 BRIEF 描述子增加了旋转不变特性。
2. BRIEF 描述子：对前一步提取出特征点的周围图像区域进行描述。ORB 对 BRIEF 进行了一些改进，主要是指在 BRIEF 中使用了先前计算的方向信息。

下面分别介绍 FAST 和 BRIEF。

FAST 关键点

FAST 是一种角点，主要检测局部像素灰度变化明显的地方，以速度快著称。它的思想是：如果一个像素与邻域的像素差别较大（过亮或过暗），那么它更可能是角点。相比于其他角点检测算法，FAST 只需比较像素亮度的大小，十分快捷。它的检测过程如下（见图 7-2 ）：

1. 在图像中选取像素 p，假设它的亮度为 I_p。
2. 设置一个阈值 T（比如，I_p 的 20%）。
3. 以像素 p 为中心，选取半径为 3 的圆上的 16 个像素点。
4. 假如选取的圆上有连续的 N 个点的亮度大于 $I_p + T$ 或小于 $I_p - T$，那么像素 p 可以被认为是特征点（N 通常取 12，即为 FAST-12。其他常用的 N 取值为 9 和 11，它们分别被称为 FAST-9 和 FAST-11 ）。
5. 循环以上四步，对每一个像素执行相同的操作。

在 FAST-12 算法中，为了更高效，可以添加一项预测试操作，以快速地排除绝大多数不是角点的像素。具体操作为，对于每个像素，直接检测邻域圆上的第 1, 5, 9, 13 个像素的亮度。只有当这 4 个像素中有 3 个同时大于 $I_p + T$ 或小于 $I_p - T$ 时，当前像素才有可能是一个角点，否则应该直接排除。这样的预测试操作大大加速了角点检测。此外，原始的 FAST 角点经常出现“扎堆”的现象。

所以在第一遍检测之后，还需要用非极大值抑制（Non-maximal suppression），在一定区域内仅保留响应极大值的角点，避免角点集中的问题。

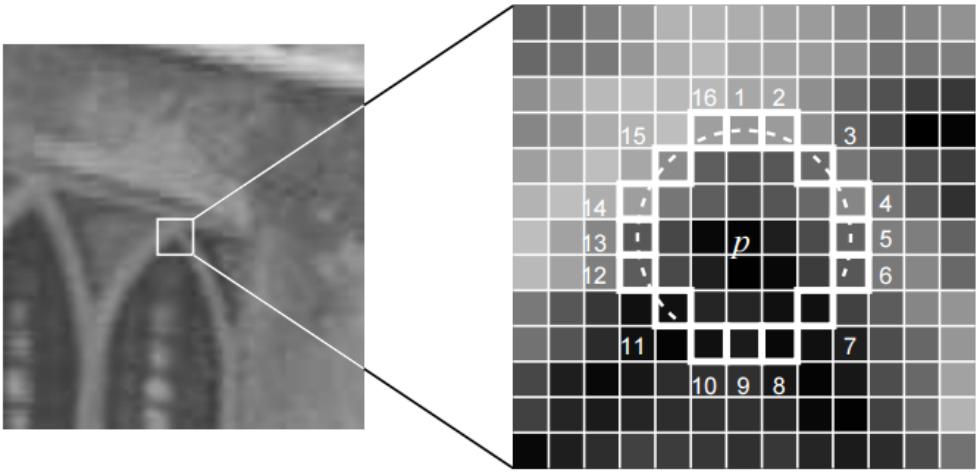


图 7-2　FAST 特征点[39]。

FAST 特征点的计算仅仅是比较像素间亮度的差异所所以速度非常快，但它也有重复性不强，分布不均匀的缺点。此外，FAST 角点不具有方向信息。同时，由于它固定取半径为 3 的圆，存在尺度问题：远处看着像是角点的地方，接近后看可能就不是角点了。针对 FAST 角点不具有方向性和尺度的弱点，ORB 添加了尺度和旋转的描述。尺度不变性由构建图像金字塔①，并在金字塔的每一层上检测角点来实现。而特征的旋转是由灰度质心法（Intensity Centroid）实现的。

金字塔时计算图视觉中常用的一种处理方法，示意图见图 7-3。金字塔底层是原始图像。每往上一层，就对图像进行一个固定倍率的缩放，这样我们就有了不同分辨率的图像。较小的图像可以看成是远处看过来的场景。在特征匹配算法中，我们可以匹配不同层上的图像，从而实现尺度不变性。例如，如果相机在后退，那么我们应该能够在上一个图像金字塔的上层和下一个图像的下层中找到匹配。

在旋转方面，我们计算特征点附近的图像灰度质心。所谓质心是指以图像块灰度值作为权重的中心。其具体操作步骤如下[46]：

1. 在一个小的图像块 B 中，定义图像块的矩为

$$m_{pq} = \sum_{x,y \in B} x^p y^q I(x,y), \quad p,q = \{0,1\}.$$

① 金字塔是指对图像进行不同层次的降采样，以获得不同分辨率的图像。

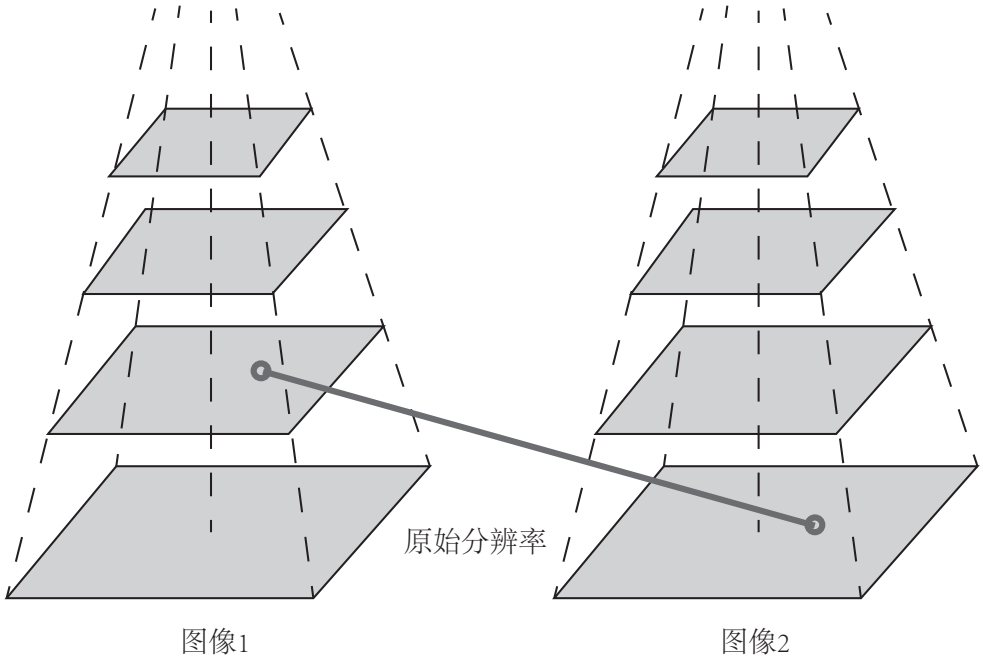


图 7-3　使用金字塔可以匹配不同缩放倍率下的图像。

2. 通过矩可以找到图像块的质心：

$$C = (\frac{m_{10}}{m_{00}}, \frac{m_{01}}{m_{00}}).$$

3. 连接图像块的几何中心 O 与质心 C，得到一个方向向量 $\overrightarrow{OC}$，于是特征点的方向可以定义为

$$\theta = \arctan(m_{01}/m_{10}).$$

通过以上方法，FAST 角点便具有了尺度与旋转的描述，从而大大提升了其在不同图像之间表述的鲁棒性。所以在 ORB 中，把这种改进后的 FAST 称为 Oriented FAST。

BRIEF 描述子

在提取 Oriented FAST 关键点后，我们对每个点计算其描述子。ORB 使用改进的 BRIEF 特征描述。我们先来介绍一下 BRIEF 是什么。

BRIEF 是一种**二进制**描述子，其描述向量由许多个 0 和 1 组成，这里的 0 和 1 编码了关键点附近两个随机像素（比如 p 和 q）的大小关系：如果 p 比 q 大，则取 1，反之就取 0。如果我们取了 128 个这样的 p, q，最后就得到 128 维由 0、1 组成的向量[44]。BRIEF 使用了随机选点的比较，速度非常

快，而且由于使用了二进制表达，存储起来也十分方便，适用于实时的图像匹配。原始的 BRIEF 描述子不具有旋转不变性，因此在图像发生旋转时容易丢失。而 ORB 在 FAST 特征点提取阶段计算了关键点的方向，所以可以利用方向信息，计算了旋转之后的“Steer BRIEF”特征使 ORB 的描述子具有较好的旋转不变性。

由于考虑到了旋转和缩放，使得 ORB 在平移、旋转和缩放的变换下仍有良好的表现。同时，FAST 和 BREIF 的组合也非常高效，使得 ORB 特征在实时 SLAM 中非常受欢迎。我们在图 7-4 中展示了一张图像提取 ORB 之后的结果，下面来介绍如何在不同的图像之间进行特征匹配。

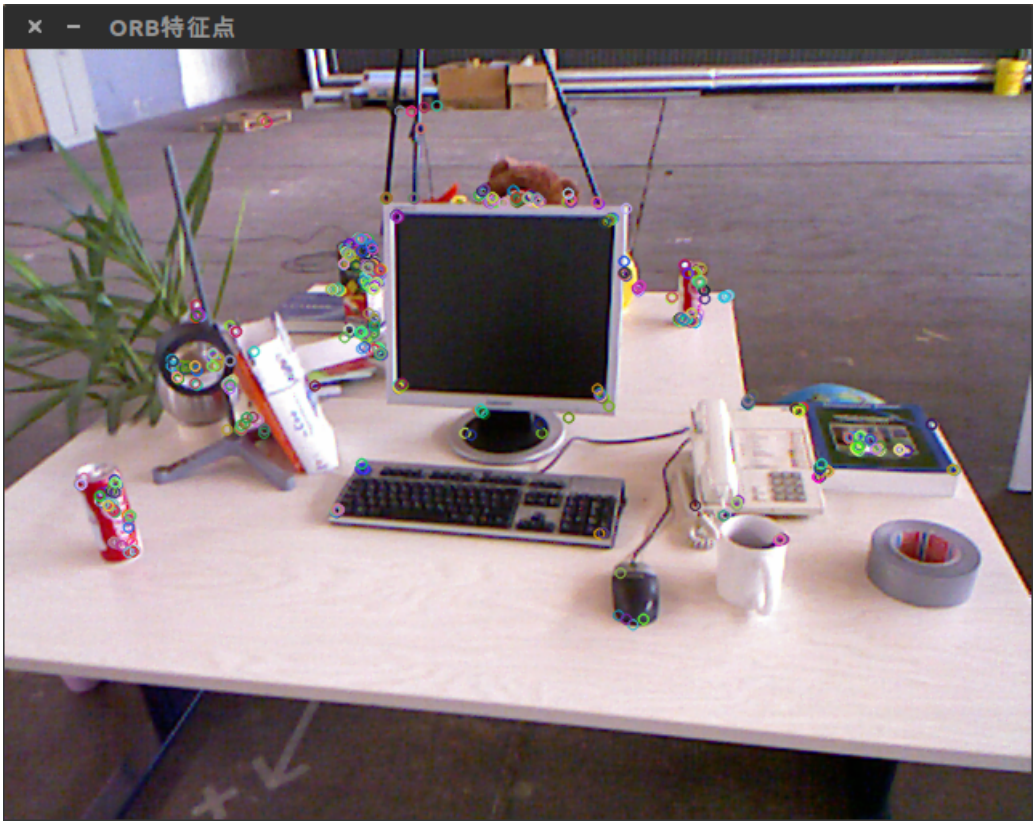


图 7-4　OpenCV 提供的 ORB 特征点检测结果。

7.1.3　特征匹配

特征匹配（如图 7-5 所示）是视觉 SLAM 中极为关键的一步，宽泛地说，特征匹配解决了 SLAM 中的数据关联问题（data association），即确定当前看到的路标与之前看到的路标之间的对应关系。通过对图像与图像或者图像与地图之间的描述子进行准确匹配，我们可以为后续的姿态估计、优化等操作减轻大量负担。然而，由于图像特征的局部特性，误匹配的情况广泛存在，而且长期以来一直

没有得到有效解决，目前已经成为视觉 SLAM 中制约性能提升的一大瓶颈。部分原因是场景中经常存在大量的重复纹理，使得特征描述非常相似。在这种情况下，仅利用局部特征解决误匹配是非常困难的。

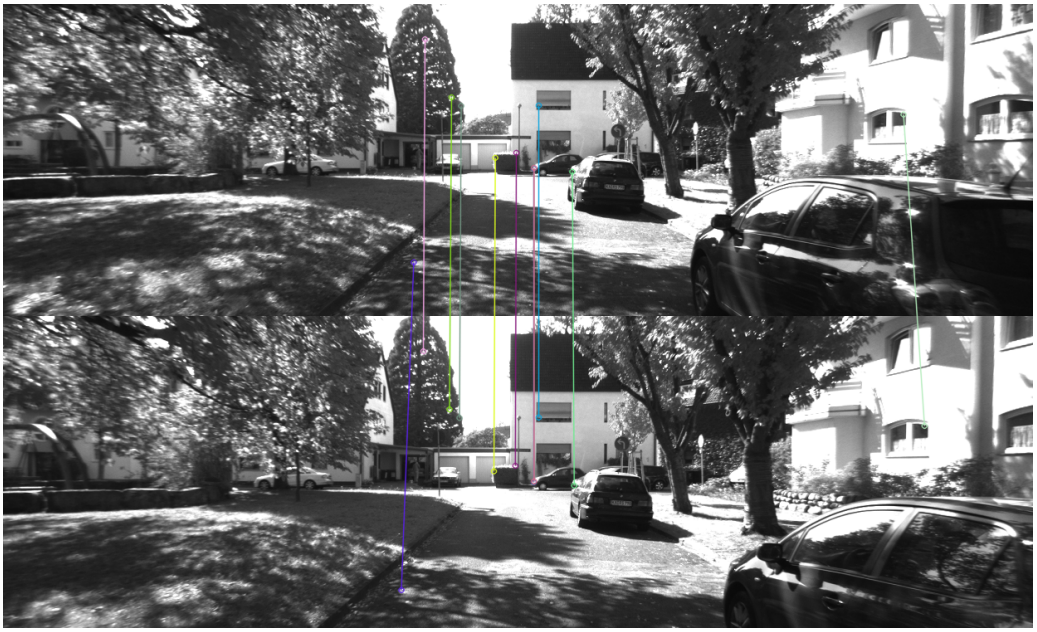


图 7-5　两帧图像间的特征匹配。

不过，让我们先来看正确匹配的情况，等做完实验再回头去讨论误匹配问题。考虑两个时刻的图像。如果在图像 I_t 中提取到特征点 $x_t^m, m = 1, 2, ..., M$，在图像 I_{t+1} 中提取到特征点 $x_{t+1}^n, n = 1, 2, ..., N$，如何寻找这两个集合元素的对应关系呢？最简单的特征匹配方法就是**暴力匹配（Brute-Force Matcher）**。即对每一个特征点 x_t^m 与所有的 x_{t+1}^n 测量描述子的距离，然后排序，取最近的一个作为匹配点。描述子距离表示了两个特征之间的**相似程度**，不过在实际运用中还可以取不同的距离度量范数。对于浮点类型的描述子，使用欧氏距离进行度量即可。而对于二进制的描述子（比如 BRIEF 这样的），我们往往使用汉明距离（Hamming distance）作为度量——两个二进制串之间的汉明距离，指的是其**不同位数的个数**。

然而，当特征点数量很大时，暴力匹配法的运算量将变得很大，特别是当想要匹配某个帧和一张地图的时候。这不符合我们在 SLAM 中的实时性需求。此时**快速近似最近邻（FLANN）**算法更加适合于匹配点数量极多的情况。由于这些匹配算法理论已经成熟，而且实现上也已集成到 OpenCV，所以这里就不再描述它的技术细节了。感兴趣的读者可以参考阅读文献 [47]。

7.2 实践：特征提取和匹配

图 7-6 实验使用的两帧图像。

OpenCV 已经集成了多数主流的图像特征，我们可以很方便地进行调用。下面我们来完成两个实验：第一个实验中，我们演示使用 OpenCV 进行 ORB 的特征匹配；第二个实验中，我们演示如何根据前面介绍的原理，手写一个简单的 ORB 特征。通过手写的过程，读者可以更加清楚地理解 ORB 的计算过程，并类推到其他特征上去。

7.2.1 OpenCV 的 ORB 特征

首先我们调用 OpenCV 来提取和匹配 ORB。我为此实验准备了两张图像，位于 slambook2/ch7/下的 1.png 和 2.png，如图 7-6 所示。它们是来自公开数据集 [23] 中的两张图像，我们看到相机发生了微小的运动。本节程序演示如何提取 ORB 特征并进行匹配。下一小节中，我们将演示如何用匹配结果来估计相机运动。

下面程序演示了 ORB 的使用方法：

slambook/ch7/orb_cv.cpp

```
#include <iostream>
#include <opencv2/core/core.hpp>
#include <opencv2/features2d/features2d.hpp>
#include <opencv2/highgui/highgui.hpp>
#include <chrono>

using namespace std;
using namespace cv;

int main(int argc, char **argv) {
    if (argc != 3) {
        cout << "usage: feature_extraction img1 img2" << endl;
        return 1;
```

```
    }
    //—— 读取图像
    Mat img_1 = imread(argv[1], CV_LOAD_IMAGE_COLOR);
    Mat img_2 = imread(argv[2], CV_LOAD_IMAGE_COLOR);
    assert(img_1.data != nullptr && img_2.data != nullptr);

    //—— 初始化
    std::vector<KeyPoint> keypoints_1, keypoints_2;
    Mat descriptors_1, descriptors_2;
    Ptr<FeatureDetector> detector = ORB::create();
    Ptr<DescriptorExtractor> descriptor = ORB::create();
    Ptr<DescriptorMatcher> matcher = DescriptorMatcher::create("BruteForce-Hamming");

    //—— 第一步:检测 Oriented FAST 角点位置
    chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
    detector->detect(img_1, keypoints_1);
    detector->detect(img_2, keypoints_2);

    //—— 第二步:根据角点位置计算 BRIEF 描述子
    descriptor->compute(img_1, keypoints_1, descriptors_1);
    descriptor->compute(img_2, keypoints_2, descriptors_2);
    chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
    chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2 - t1);
    cout << "extract ORB cost = " << time_used.count() << " seconds. " << endl;

    Mat outimg1;
    drawKeypoints(img_1, keypoints_1, outimg1, Scalar::all(-1), DrawMatchesFlags::DEFAULT);
    imshow("ORB features", outimg1);

    //—— 第三步:对两幅图像中的BRIEF描述子进行匹配，使用 Hamming 距离
    vector<DMatch> matches;
    t1 = chrono::steady_clock::now();
    matcher->match(descriptors_1, descriptors_2, matches);
    t2 = chrono::steady_clock::now();
    time_used = chrono::duration_cast<chrono::duration<double>>(t2 - t1);
    cout << "match ORB cost = " << time_used.count() << " seconds. " << endl;

    //—— 第四步:匹配点对筛选
    // 计算最小距离和最大距离
    auto min_max = minmax_element(matches.begin(), matches.end(),
        [](const DMatch &m1, const DMatch &m2) { return m1.distance < m2.distance; });
    double min_dist = min_max.first->distance;
    double max_dist = min_max.second->distance;

    printf("—— Max dist : %f \n", max_dist);
    printf("—— Min dist : %f \n", min_dist);

    //当描述子之间的距离大于两倍的最小距离时,即认为匹配有误.但有时候最小距离会非常小,设置一个经验值30
        作为下限.
    std::vector<DMatch> good_matches;
    for (int i = 0; i < descriptors_1.rows; i++) {
        if (matches[i].distance <= max(2 * min_dist, 30.0)) {
            good_matches.push_back(matches[i]);
```

```
        }
    }

    //— 第五步:绘制匹配结果
    Mat img_match;
    Mat img_goodmatch;
    drawMatches(img_1, keypoints_1, img_2, keypoints_2, matches, img_match);
    drawMatches(img_1, keypoints_1, img_2, keypoints_2, good_matches, img_goodmatch);
    imshow("all matches", img_match);
    imshow("good matches", img_goodmatch);
    waitKey(0);

    return 0;
}
```

运行此程序（需要输入两个图像位置），将输出运行结果：

终端输入：

```
% build/orb_cv 1.png 2.png
extract ORB cost = 0.0229183 seconds.
match ORB cost = 0.000751868 seconds.
— Max dist : 95.000000
— Min dist : 4.000000
```

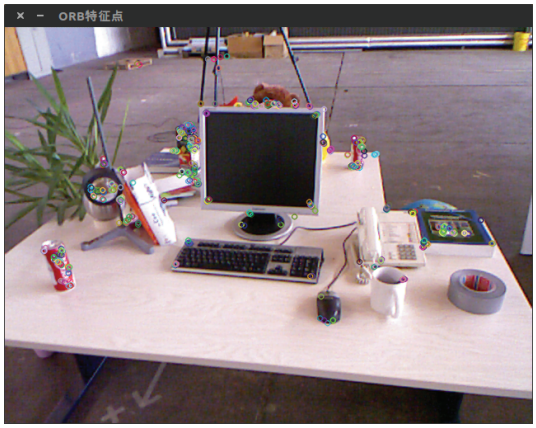
图 7-7 显示了例程的运行结果。我们看到未筛选的匹配中带有大量的误匹配。经过一次筛选之后，匹配数量减少了许多，但大多数匹配都是正确的。这里，筛选的依据是**汉明距离小于最小距离的两倍**，这是一种工程上的经验方法，不一定有理论依据。不过，尽管在示例图像中能够筛选出正确的匹配，但我们仍然不能保证在所有其他图像中得到的匹配都是正确的。因此，在后面的运动估计中，还需要使用去除误匹配的算法。在我的机器上，ORB 提取花费了 22.9 毫秒（两张图像），匹配花费了 0.75 毫秒，可见大部分计算量花在了特征提取上。

7.2.2 手写 ORB 特征

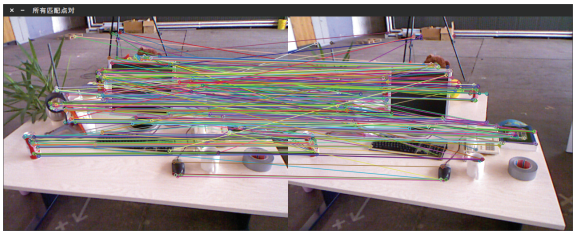
下面我们演示手写 ORB 特征的方法。这部分代码比较多，书上只展示核心部分的代码，其余的周边代码请读者从代码库中获取。

slambook/ch7/orb_self.cpp

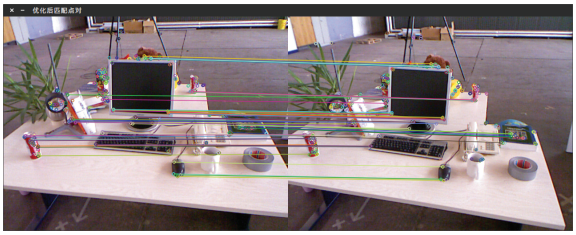
```
typedef vector<uint32_t> DescType;
// ... 省略图片读取部分代码和测试代码
// compute the descriptor
void ComputeORB(const cv::Mat &img, vector<cv::KeyPoint> &keypoints, vector<DescType> &descriptors) {
    const int half_patch_size = 8;
    const int half_boundary = 16;
    int bad_points = 0;
    for (auto &kp: keypoints) {
        if (kp.pt.x < half_boundary || kp.pt.y < half_boundary ||
        kp.pt.x >= img.cols – half_boundary || kp.pt.y >= img.rows – half_boundary) {
```



ORB关键点



未筛选的匹配



筛选后的匹配

图 7-7　特征提取与匹配结果。

```
            // outside
            bad_points++;
            descriptors.push_back({});
            continue;
        }

        float m01 = 0, m10 = 0;
        for (int dx = -half_patch_size; dx < half_patch_size; ++dx) {
            for (int dy = -half_patch_size; dy < half_patch_size; ++dy) {
                uchar pixel = img.at<uchar>(kp.pt.y + dy, kp.pt.x + dx);
                m01 += dx * pixel;
                m10 += dy * pixel;
            }
        }

        // angle should be arc tan(m01/m10);
        float m_sqrt = sqrt(m01 * m01 + m10 * m10);
        float sin_theta = m01 / m_sqrt;
        float cos_theta = m10 / m_sqrt;

        // compute the angle of this point
        DescType desc(8, 0);
        for (int i = 0; i < 8; i++) {
            uint32_t d = 0;
            for (int k = 0; k < 32; k++) {
                int idx_pq = i * 8 + k;
                cv::Point2f p(ORB_pattern[idx_pq * 4], ORB_pattern[idx_pq * 4 + 1]);
                cv::Point2f q(ORB_pattern[idx_pq * 4 + 2], ORB_pattern[idx_pq * 4 + 3]);
```

```
39
40                  // rotate with theta
41                  cv::Point2f pp = cv::Point2f(cos_theta * p.x - sin_theta * p.y, sin_theta * p.x +
                         cos_theta * p.y) + kp.pt;
42                  cv::Point2f qq = cv::Point2f(cos_theta * q.x - sin_theta * q.y, sin_theta * q.x +
                         cos_theta * q.y) + kp.pt;
43                  if (img.at<uchar>(pp.y, pp.x) < img.at<uchar>(qq.y, qq.x)) {
44                      d |= 1 << k;
45                  }
46              }
47              desc[i] = d;
48          }
49          descriptors.push_back(desc);
50      }
51
52      cout << "bad/total: " << bad_points << "/" << keypoints.size() << endl;
53  }
54
55  // brute-force matching
56  void BfMatch(
57      const vector<DescType> &desc1, const vector<DescType> &desc2, vector<cv::DMatch> &matches) {
58      const int d_max = 40;
59
60      for (size_t i1 = 0; i1 < desc1.size(); ++i1) {
61          if (desc1[i1].empty()) continue;
62          cv::DMatch m{i1, 0, 256};
63          for (size_t i2 = 0; i2 < desc2.size(); ++i2) {
64              if (desc2[i2].empty()) continue;
65              int distance = 0;
66              for (int k = 0; k < 8; k++) {
67                  distance += _mm_popcnt_u32(desc1[i1][k] ^desc2[i2][k]);
68              }
69              if (distance < d_max && distance < m.distance) {
70                  m.distance = distance;
71                  m.trainIdx = i2;
72              }
73          }
74          if (m.distance < d_max) {
75              matches.push_back(m);
76          }
77      }
78  }
```

这个演示中我们只展示 ORB 的计算代码和匹配代码。在计算中，我们用 256 位的二进制描述，即对应到 8 个 32 位的 unsigned int 数据，用 typedef 将它表示成 DescType。然后，我们根据前面介绍的原理计算 FAST 特征点的角度，再使用该角度计算描述子。此代码中通过三角函数的原理回避了复杂的 arctan 以及 sin、cos 计算，从而达到加速的效果。在 BfMatch 函数中，我们还使用了 SSE 指令集中的_mm_popcnt_u32 函数来计算一个 unsigned int 变量中 1 的个数，从而达到计算汉明距离的效果。该段程序的运行结果如下，匹配结果如图 7-8所示：

终端输出：

```
bad/total: 43/638
bad/total: 8/595
extract ORB cost = 0.00390721 seconds.
match ORB cost = 0.000862984 seconds.
matches: 51
```

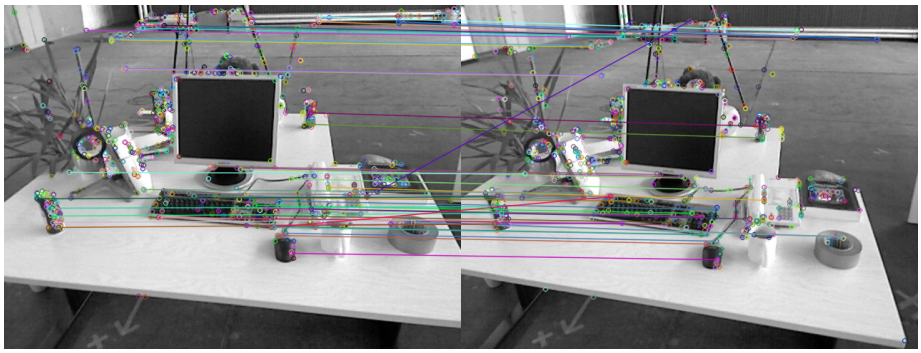


图 7-8　匹配结果

可见，这个程序中，ORB 的提取只需要 3.9 毫秒，匹配只需 0.86 毫秒。我们通过一些简单的算法修改，对 ORB 的提取加速了 5.8 倍。请读者注意，编译这个程序需要你的 CPU 支持 SSE 指令集，这应该在绝大多数现代的家用 CPU 上都已经支持。如果我们能够对提取特征部分进一步并行化处理，算法还可以有加速的空间。

7.2.3　计算相机运动

我们已经有了匹配好的点对，接下来，我们要根据点对来估计相机的运动。这里由于相机的原理不同，情况发生了变化：

1. 当相机为单目时，我们只知道 2D 的像素坐标，因而问题是根据**两组 2D 点**估计运动。该问题用**对极几何**来解决。
2. 当相机为双目、RGB-D 时，或者通过某种方法得到了距离信息，那么问题就是根据**两组 3D 点**估计运动。该问题通常用 ICP 来解决。
3. 如果一组为 3D，一组为 2D，即，我们得到了一些 3D 点和它们在相机的投影位置，也能估计相机的运动。该问题通过 **PnP** 求解。

因此，下面几节就来介绍这三种情形下的相机运动估计。我们将从信息最少的 2D–2D 情形出发，看看它如何求解，求解过程又有哪些麻烦的问题。

7.3 2D−2D: 对极几何

7.3.1 对极约束

现在，假设我们从两张图像中得到了一对配对好的特征点，如图 7-9 所示。如果有若干对这样的匹配点，就可以通过这些二维图像点的对应关系，恢复出在两帧之间摄像机的运动。这里“若干对”具体是多少对呢？我们会在下文介绍。下面先来看看两个图像当中的匹配点有什么几何关系。

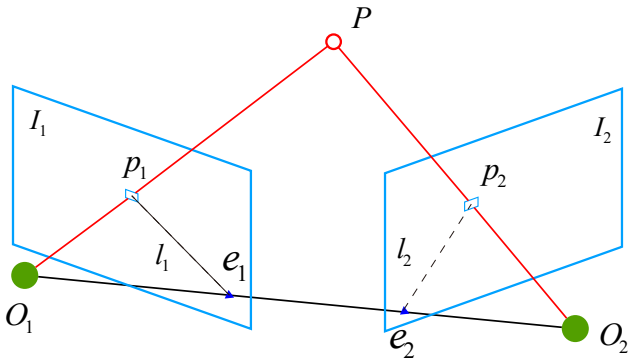


图 7-9 对极几何约束。

以图 7-9 为例，我们希望求取两帧图像 I_1, I_2 之间的运动，设第一帧到第二帧的运动为 $\boldsymbol{R}, \boldsymbol{t}$。两个相机中心分别为 O_1, O_2。现在，考虑 I_1 中有一个特征点 p_1，它在 I_2 中对应着特征点 p_2。我们知道两者是通过特征匹配得到的。如果匹配正确，说明它们确实是**同一个空间点在两个成像平面上的投影**。这里需要一些术语来描述它们之间的几何关系。首先，连线 $\overrightarrow{O_1p_1}$ 和连线 $\overrightarrow{O_2p_2}$ 在三维空间中会相交于点 P。这时候点 O_1, O_2, P 三个点可以确定一个平面，称为**极平面（Epipolar plane）**。O_1O_2 连线与像平面 I_1, I_2 的交点分别为 e_1, e_2。e_1, e_2 称为**极点（Epipoles）**，O_1O_2 被称为**基线（Baseline）**。我们称极平面与两个像平面 I_1, I_2 之间的相交线 l_1, l_2 为**极线（Epipolar line）**。

直观讲，从第一帧的角度看，射线 $\overrightarrow{O_1p_1}$ 是**某个像素可能出现的空间位置**——因为该射线上的所有点都会投影到同一个像素点。同时，如果不知道 P 的位置，那么当我们在第二幅图像上看时，连线 $\overrightarrow{e_2p_2}$（也就是第二幅图像中的极线）就是 P 可能出现的投影的位置，也就是射线 $\overrightarrow{O_1p_1}$ 在第二个相机中的投影。现在，由于我们通过特征点匹配确定了 p_2 的像素位置，所以能够推断 P 的空间位置，以及相机的运动。要提醒读者的是，**这多亏了正确的特征匹配**。如果没有特征匹配，我们就没法确定 p_2 到底在极线的哪个位置了。那时，就必须在极线上搜索以获得正确的匹配，这将在第 12 讲中提到。

现在，我们从代数角度来看一下这里的几何关系。在第一帧的坐标系下，设 P 的空间位置为

$$\boldsymbol{P} = [X, Y, Z]^{\mathrm{T}}.$$

根据第 5 讲介绍的针孔相机模型，我们知道两个像素点 $\boldsymbol{p}_1, \boldsymbol{p}_2$ 的像素位置为

$$s_1\boldsymbol{p}_1 = \boldsymbol{K}\boldsymbol{P}, \quad s_2\boldsymbol{p}_2 = \boldsymbol{K}\left(\boldsymbol{R}\boldsymbol{P} + \boldsymbol{t}\right). \tag{7.1}$$

这里 $\boldsymbol{K}$ 为相机内参矩阵，$\boldsymbol{R}, \boldsymbol{t}$ 为两个坐标系的相机运动。具体来说，这里计算的是 $\boldsymbol{R}_{21}$ 和 $\boldsymbol{t}_{21}$，因为它们把第一个坐标系下的坐标转换到第二个坐标系下。如果我们愿意，也可以把它们写成李代数形式。

有时候，我们会使用齐次坐标表示像素点。在使用齐次坐标时，一个向量将等于它自身乘上任意的非零常数。这通常用于表达一个投影关系。例如 $s_1\boldsymbol{p}_1$ 和 $\boldsymbol{p}_1$ 成投影关系，它们在齐次坐标的意义下是相等的。我们称这种相等关系为**尺度意义下相等**（equal up to a scale），记作：

$$s\boldsymbol{p} \simeq \boldsymbol{p}. \tag{7.2}$$

那么，上述两个投影关系可写为：

$$\boldsymbol{p}_1 \simeq \boldsymbol{K}\boldsymbol{P}, \quad \boldsymbol{p}_2 \simeq \boldsymbol{K}\left(\boldsymbol{R}\boldsymbol{P} + \boldsymbol{t}\right). \tag{7.3}$$

现在，取：

$$\boldsymbol{x}_1 = \boldsymbol{K}^{-1}\boldsymbol{p}_1, \quad \boldsymbol{x}_2 = \boldsymbol{K}^{-1}\boldsymbol{p}_2. \tag{7.4}$$

这里的 $\boldsymbol{x}_1, \boldsymbol{x}_2$ 是两个像素点的归一化平面上的坐标。代入上式，得：

$$\boldsymbol{x}_2 \simeq \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}. \tag{7.5}$$

两边同时左乘 $\boldsymbol{t}^\wedge$。回忆 $\wedge$ 的定义，这相当于两侧同时与 $\boldsymbol{t}$ 做外积：

$$\boldsymbol{t}^\wedge\boldsymbol{x}_2 \simeq \boldsymbol{t}^\wedge\boldsymbol{R}\boldsymbol{x}_1. \tag{7.6}$$

然后，两侧同时左乘 $\boldsymbol{x}_2^{\mathrm{T}}$：

$$\boldsymbol{x}_2^{\mathrm{T}}\boldsymbol{t}^\wedge\boldsymbol{x}_2 \simeq \boldsymbol{x}_2^{\mathrm{T}}\boldsymbol{t}^\wedge\boldsymbol{R}\boldsymbol{x}_1. \tag{7.7}$$

观察等式左侧，$\boldsymbol{t}^\wedge\boldsymbol{x}_2$ 是一个与 $\boldsymbol{t}$ 和 $\boldsymbol{x}_2$ 都垂直的向量。把它再和 $\boldsymbol{x}_2$ 做内积时，将得到 0。由于等式左侧严格为零，那么乘以任意非零常数之后也为零，于是我们可以把 $\simeq$ 写成通常的等号。因此，我们就得到了一个简洁的式子：

$$\boldsymbol{x}_2^{\mathrm{T}}\boldsymbol{t}^\wedge\boldsymbol{R}\boldsymbol{x}_1 = 0. \tag{7.8}$$

重新代入 $\boldsymbol{p}_1, \boldsymbol{p}_2$，有：

$$\boldsymbol{p}_2^{\mathrm{T}}\boldsymbol{K}^{-\mathrm{T}}\boldsymbol{t}^\wedge\boldsymbol{R}\boldsymbol{K}^{-1}\boldsymbol{p}_1 = 0. \tag{7.9}$$

这两个式子都称为**对极约束**，它以形式简洁著名。它的几何意义是 O_1, P, O_2 三者共面。对极约束中同时包含了平移和旋转。我们把中间部分记作两个矩阵：基础矩阵（Fundamental Matrix）$\boldsymbol{F}$ 和本质矩阵（Essential Matrix）$\boldsymbol{E}$，于是可以进一步简化对极约束：

$$\boldsymbol{E} = \boldsymbol{t}^\wedge\boldsymbol{R}, \quad \boldsymbol{F} = \boldsymbol{K}^{-\mathrm{T}}\boldsymbol{E}\boldsymbol{K}^{-1}, \quad \boldsymbol{x}_2^{\mathrm{T}}\boldsymbol{E}\boldsymbol{x}_1 = \boldsymbol{p}_2^{\mathrm{T}}\boldsymbol{F}\boldsymbol{p}_1 = 0. \tag{7.10}$$

对极约束简洁地给出了两个匹配点的空间位置关系。于是，相机位姿估计问题变为以下两步：

1. 根据配对点的像素位置求出 $\boldsymbol{E}$ 或者 $\boldsymbol{F}$。
2. 根据 $\boldsymbol{E}$ 或者 $\boldsymbol{F}$ 求出 $\boldsymbol{R}, \boldsymbol{t}$。

由于 $\boldsymbol{E}$ 和 $\boldsymbol{F}$ 只相差了相机内参，而内参在 SLAM 中通常是已知的[①]，所以实践当中往往使用形式更简单的 $\boldsymbol{E}$。我们以 $\boldsymbol{E}$ 为例，介绍上面两个问题如何求解。

7.3.2 本质矩阵

根据定义，本质矩阵 $\boldsymbol{E} = \boldsymbol{t}^\wedge \boldsymbol{R}$。它是一个 3×3 的矩阵，内有 9 个未知数。那么，是不是任意一个 3×3 的矩阵都可以被当成本质矩阵呢？从 $\boldsymbol{E}$ 的构造方式上看，有以下值得注意的地方：

- 本质矩阵是由对极约束定义的。由于对极约束是**等式为零**的约束，所以对 $\boldsymbol{E}$ 乘以任意非零常数后，**对极约束依然满足**。我们把这件事情称为 $\boldsymbol{E}$ 在不同尺度下是等价的。
- 根据 $\boldsymbol{E} = \boldsymbol{t}^\wedge \boldsymbol{R}$，可以证明[3]，本质矩阵 $\boldsymbol{E}$ 的奇异值必定是 $[\sigma, \sigma, 0]^\mathrm{T}$ 的形式。这称为**本质矩阵的内在性质**。
- 另一方面，由于平移和旋转各有 3 个自由度，故 $\boldsymbol{t}^\wedge \boldsymbol{R}$ 共有 6 个自由度。但由于尺度等价性，故 $\boldsymbol{E}$ 实际上有 5 个自由度。

$\boldsymbol{E}$ 具有 5 个自由度的事实，表明我们最少可以用 5 对点来求解 $\boldsymbol{E}$。但是，$\boldsymbol{E}$ 的内在性质是一种非线性性质，在估计时会带来麻烦，因此，也可以只考虑它的**尺度等价性**，使用 8 对点来估计 $\boldsymbol{E}$——这就是经典的**八点法（Eight-point-algorithm）**[48, 49]。八点法只利用了 $\boldsymbol{E}$ 的线性性质，因此可以在线性代数框架下求解。下面我们来看八点法是如何工作的。

考虑一对匹配点，它们的归一化坐标为 $\boldsymbol{x}_1 = [u_1, v_1, 1]^\mathrm{T}, \boldsymbol{x}_2 = [u_2, v_2, 1]^\mathrm{T}$。根据对极约束，有：

$$\begin{pmatrix} u_2, v_2, 1 \end{pmatrix} \begin{pmatrix} e_1 & e_2 & e_3 \\ e_4 & e_5 & e_6 \\ e_7 & e_8 & e_9 \end{pmatrix} \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = 0. \tag{7.11}$$

我们把矩阵 $\boldsymbol{E}$ 展开，写成向量的形式：

$$\boldsymbol{e} = [e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8, e_9]^\mathrm{T},$$

那么对极约束可以写成与 $\boldsymbol{e}$ 有关的线性形式：

$$[u_2 u_1, u_2 v_1, u_2, v_2 u_1, v_2 v_1, v_2, u_1, v_1, 1] \cdot \boldsymbol{e} = 0. \tag{7.12}$$

同理，对于其他点对也有相同的表示。我们把所有点都放到一个方程中，变成线性方程组（u^i, v^i

[①] 在 SfM 研究中则有可能是未知而有待估计的。

表示第 i 个特征点，依此类推 ）：

$$\begin{pmatrix} u_2^1 u_1^1 & u_2^1 v_1^1 & u_2^1 & v_2^1 u_1^1 & v_2^1 v_1^1 & v_2^1 & u_1^1 & v_1^1 & 1 \\ u_2^2 u_1^2 & u_2^2 v_1^2 & u_2^2 & v_2^2 u_1^2 & v_2^2 v_1^2 & v_2^2 & u_1^2 & v_1^2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \\ u_2^8 u_1^8 & u_2^8 v_1^8 & u_2^8 & v_2^8 u_1^8 & v_2^8 v_1^8 & v_2^8 & u_1^8 & v_1^8 & 1 \end{pmatrix} \begin{pmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \\ e_6 \\ e_7 \\ e_8 \\ e_9 \end{pmatrix} = 0. \tag{7.13}$$

这 8 个方程构成了一个线性方程组。它的系数矩阵由特征点位置构成，大小为 8×9。$\boldsymbol{e}$ 位于该矩阵的零空间中。如果系数矩阵是满秩的（即秩为 8），那么它的零空间维数为 1，也就是 $\boldsymbol{e}$ 构成一条线。这与 $\boldsymbol{e}$ 的尺度等价性是一致的。如果 8 对匹配点组成的矩阵满足秩为 8 的条件，那么 $\boldsymbol{E}$ 的各元素就可由上述方程解得。

接下来的问题是如何根据已经估得的本质矩阵 $\boldsymbol{E}$，恢复出相机的运动 $\boldsymbol{R}, \boldsymbol{t}$。这个过程是由奇异值分解（SVD）得到的。设 $\boldsymbol{E}$ 的 SVD 分解为

$$\boldsymbol{E} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}, \tag{7.14}$$

其中 $\boldsymbol{U}, \boldsymbol{V}$ 为正交阵，$\boldsymbol{\Sigma}$ 为奇异值矩阵。根据 $\boldsymbol{E}$ 的内在性质，我们知道 $\boldsymbol{\Sigma} = \mathrm{diag}(\sigma, \sigma, 0)$。在 SVD 分解中，对于任意一个 $\boldsymbol{E}$，存在两个可能的 $\boldsymbol{t}, \boldsymbol{R}$ 与它对应：

$$\begin{aligned} \boldsymbol{t}_1^\wedge &= \boldsymbol{U}\boldsymbol{R}_Z(\tfrac{\pi}{2})\boldsymbol{\Sigma}\boldsymbol{U}^{\mathrm{T}}, \quad \boldsymbol{R}_1 = \boldsymbol{U}\boldsymbol{R}_Z^{\mathrm{T}}(\tfrac{\pi}{2})\boldsymbol{V}^{\mathrm{T}} \\ \boldsymbol{t}_2^\wedge &= \boldsymbol{U}\boldsymbol{R}_Z(-\tfrac{\pi}{2})\boldsymbol{\Sigma}\boldsymbol{U}^{\mathrm{T}}, \quad \boldsymbol{R}_2 = \boldsymbol{U}\boldsymbol{R}_Z^{\mathrm{T}}(-\tfrac{\pi}{2})\boldsymbol{V}^{\mathrm{T}}. \end{aligned} \tag{7.15}$$

其中 $\boldsymbol{R}_Z(\frac{\pi}{2})$ 表示沿 Z 轴旋转 90° 得到的旋转矩阵。同时，由于 $-\boldsymbol{E}$ 和 $\boldsymbol{E}$ 等价，所以对任意一个 $\boldsymbol{t}$ 取负号，也会得到同样的结果。因此，从 $\boldsymbol{E}$ 分解到 $\boldsymbol{t}, \boldsymbol{R}$ 时，一共存在 **4 个**可能的解。

图 7-10 形象地展示了分解本质矩阵得到的 4 个解。我们已知空间点在相机（蓝色线）上的投影（红色点），想要求解相机的运动。在保持红色点不变的情况下，可以画出 4 种可能的情况。不过幸运的是，只有第一种解中 P 在两个相机中都具有正的深度。因此，只要把任意一点代入 4 种解中，检测该点在两个相机下的深度，就可以确定哪个解是正确的了。

如果利用 $\boldsymbol{E}$ 的内在性质，那么它只有 5 个自由度。所以最少可以通过 5 对点来求解相机运动[50, 51]。然而这种做法形式复杂，从工程实现角度考虑，由于平时通常会有几十对乃至上百对的匹配点，从 8 对减至 5 对意义并不明显。为保持简单，我们这里就只介绍基本的八点法。

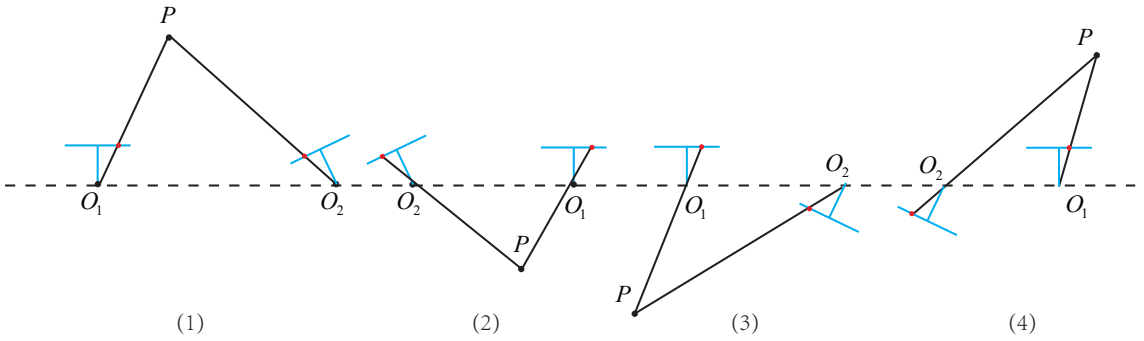


图 7-10 分解本质矩阵得到的 4 个解。在保持投影点（红色点）不变的情况下，两个相机及空间点一共有 4 种可能的情况。

剩下的问题还有一个：根据线性方程解出的 $\boldsymbol{E}$，可能不满足 $\boldsymbol{E}$ 的内在性质——它的奇异值不一定为 $\sigma, \sigma, 0$ 的形式。这时，我们会刻意地把 $\boldsymbol{\Sigma}$ 矩阵调整成上面的样子。通常的做法是，对八点法求得的 $\boldsymbol{E}$ 进行 SVD 分解后，会得到奇异值矩阵 $\boldsymbol{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$，不妨设 $\sigma_1 \geqslant \sigma_2 \geqslant \sigma_3$。取：

$$\boldsymbol{E} = \boldsymbol{U}\text{diag}\left(\frac{\sigma_1 + \sigma_2}{2}, \frac{\sigma_1 + \sigma_2}{2}, 0\right)\boldsymbol{V}^{\mathrm{T}}. \tag{7.16}$$

这相当于是把求出来的矩阵投影到了 $\boldsymbol{E}$ 所在的流形上。当然，更简单的做法是将奇异值矩阵取成 $\text{diag}(1, 1, 0)$，因为 $\boldsymbol{E}$ 具有尺度等价性，所以这样做也是合理的。

7.3.3 单应矩阵

除了基本矩阵和本质矩阵，二视图几何中还存在另一种常见的矩阵：单应矩阵（Homography）$\boldsymbol{H}$，它描述了两个平面之间的映射关系。若场景中的特征点都落在同一平面上（比如墙、地面等），则可以通过单应性来进行运动估计。这种情况在无人机携带的俯视相机或扫地机携带的顶视相机中比较常见。由于之前没有提到过单应，因此这里稍微介绍一下。

单应矩阵通常描述处于共同平面上的一些点在两张图像之间的变换关系。考虑在图像 I_1 和 I_2 有一对匹配好的特征点 p_1 和 p_2。这些特征点落在平面 P 上，设这个平面满足方程：

$$\boldsymbol{n}^{\mathrm{T}}\boldsymbol{P} + d = 0. \tag{7.17}$$

稍加整理，得：

$$-\frac{\boldsymbol{n}^{\mathrm{T}}\boldsymbol{P}}{d} = 1. \tag{7.18}$$

然后，回顾本节开头的式 (7.1)，得：

$$\begin{aligned}\boldsymbol{p}_2 &\simeq \boldsymbol{K}(\boldsymbol{R}\boldsymbol{P}+\boldsymbol{t}) \\ &\simeq \boldsymbol{K}\left(\boldsymbol{R}\boldsymbol{P}+\boldsymbol{t}\cdot(-\frac{\boldsymbol{n}^{\mathrm{T}}\boldsymbol{P}}{d})\right) \\ &\simeq \boldsymbol{K}\left(\boldsymbol{R}-\frac{\boldsymbol{t}\boldsymbol{n}^{\mathrm{T}}}{d}\right)\boldsymbol{P} \\ &\simeq \boldsymbol{K}\left(\boldsymbol{R}-\frac{\boldsymbol{t}\boldsymbol{n}^{\mathrm{T}}}{d}\right)\boldsymbol{K}^{-1}\boldsymbol{p}_1.\end{aligned}$$

于是，我们得到了一个直接描述图像坐标 $\boldsymbol{p}_1$ 和 $\boldsymbol{p}_2$ 之间的变换，把中间这部分记为 $\boldsymbol{H}$，于是：

$$\boldsymbol{p}_2 \simeq \boldsymbol{H}\boldsymbol{p}_1. \tag{7.19}$$

它的定义与旋转、平移及平面的参数有关。与基础矩阵 $\boldsymbol{F}$ 类似，单应矩阵 $\boldsymbol{H}$ 也是一个 3×3 的矩阵，求解时的思路也和 $\boldsymbol{F}$ 类似，同样可以先根据匹配点计算 $\boldsymbol{H}$，然后将它分解以计算旋转和平移。把上式展开，得：

$$\begin{pmatrix}u_2\\ v_2\\ 1\end{pmatrix} = \begin{pmatrix}h_1 & h_2 & h_3\\ h_4 & h_5 & h_6\\ h_7 & h_8 & h_9\end{pmatrix}\begin{pmatrix}u_1\\ v_1\\ 1\end{pmatrix}. \tag{7.20}$$

请注意，这里的等号是在非零因子下成立的。我们在实际处理中通常乘以一个非零因子使得 $h_9=1$（在它取非零值时）。然后根据第 3 行，去掉这个非零因子，于是有：

$$\begin{aligned}u_2 &= \frac{h_1u_1+h_2v_1+h_3}{h_7u_1+h_8v_1+h_9} \\ v_2 &= \frac{h_4u_1+h_5v_1+h_6}{h_7u_1+h_8v_1+h_9}.\end{aligned}$$

整理得：

$$\begin{aligned}h_1u_1+h_2v_1+h_3-h_7u_1u_2-h_8v_1u_2 &= u_2 \\ h_4u_1+h_5v_1+h_6-h_7u_1v_2-h_8v_1v_2 &= v_2.\end{aligned}$$

这样一组匹配点对就可以构造出两项约束（事实上有三个约束，但是因为线性相关，只取前两个），于是自由度为 8 的单应矩阵可以通过 4 对匹配特征点算出（注意，这些特征点不能有三点共

线的情况），即求解以下的线性方程组（当 $h_9 = 0$ 时，右侧为零）：

$$\begin{pmatrix} u_1^1 & v_1^1 & 1 & 0 & 0 & 0 & -u_1^1u_2^1 & -v_1^1u_2^1 \\ 0 & 0 & 0 & u_1^1 & v_1^1 & 1 & -u_1^1v_2^1 & -v_1^1v_2^1 \\ u_1^2 & v_1^2 & 1 & 0 & 0 & 0 & -u_1^2u_2^2 & -v_1^2u_2^2 \\ 0 & 0 & 0 & u_1^2 & v_1^2 & 1 & -u_1^2v_2^2 & -v_1^2v_2^2 \\ u_1^3 & v_1^3 & 1 & 0 & 0 & 0 & -u_1^3u_2^3 & -v_1^3u_2^3 \\ 0 & 0 & 0 & u_1^3 & v_1^3 & 1 & -u_1^3v_2^3 & -v_1^3v_2^3 \\ u_1^4 & v_1^4 & 1 & 0 & 0 & 0 & -u_1^4u_2^4 & -v_1^4u_2^4 \\ 0 & 0 & 0 & u_1^4 & v_1^4 & 1 & -u_1^4v_2^4 & -v_1^4v_2^4 \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{pmatrix} = \begin{pmatrix} u_2^1 \\ v_2^1 \\ u_2^2 \\ v_2^2 \\ u_2^3 \\ v_2^3 \\ u_2^4 \\ v_2^4 \end{pmatrix}. \tag{7.21}$$

这种做法把 $\boldsymbol{H}$ 矩阵看成了向量，通过解该向量的线性方程来恢复 $\boldsymbol{H}$，又称直接线性变换法（Direct Linear Transform）。与本质矩阵相似，求出单应矩阵以后需要对其进行分解，才可以得到相应的旋转矩阵 $\boldsymbol{R}$ 和平移向量 $\boldsymbol{t}$。分解的方法包括数值法[52, 53]与解析法[54]。与本质矩阵的分解类似，单应矩阵的分解同样会返回 4 组旋转矩阵与平移向量，并且同时可以计算出它们分别对应的场景点所在平面的法向量。如果已知成像的地图点的深度全为正值（即在相机前方），则又可以排除两组解。最后仅剩两组解，这时需要通过更多的先验信息进行判断。通常我们可以通过假设已知场景平面的法向量来解决，如场景平面与相机平面平行，那么法向量 $\boldsymbol{n}$ 的理论值为 $\mathbf{1}^{\mathrm{T}}$。

单应性在 SLAM 中具有重要意义。当特征点共面或者相机发生纯旋转时，基础矩阵的自由度下降，这就出现了所谓的退化（degenerate）。现实中的数据总包含一些噪声，这时候如果继续使用八点法求解基础矩阵，基础矩阵多余出来的自由度将会主要由噪声决定。为了能够避免退化现象造成的影响，通常我们会同时估计基础矩阵 $\boldsymbol{F}$ 和单应矩阵 $\boldsymbol{H}$，选择重投影误差比较小的那个作为最终的运动估计矩阵。

7.4 实践：对极约束求解相机运动

下面，我们来练习一下如何通过本质矩阵求解相机运动。上一节实践部分的程序提供了特征匹配，而这次我们就使用匹配好的特征点来计算 $\boldsymbol{E}, \boldsymbol{F}$ 和 $\boldsymbol{H}$，进而分解 $\boldsymbol{E}$ 得到 $\boldsymbol{R}, \boldsymbol{t}$。整个程序使用 OpenCV 提供的算法进行求解。我们把上一节的特征提取封装成函数，以供后面使用。本节只展示位姿估计部分的代码。

slambook/ch7/pose_estimation_2d2d.cpp（片段）

```
void pose_estimation_2d2d (
  std::vector<KeyPoint> keypoints_1,
  std::vector<KeyPoint> keypoints_2,
  std::vector< DMatch > matches,
  Mat& R, Mat& t )
```

```
{
  // 相机内参, TUM Freiburg2
  Mat K = ( Mat_<double> ( 3,3 ) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1 );

  //-- 把匹配点转换为 vector<Point2f> 的形式
  vector<Point2f> points1;
  vector<Point2f> points2;

  for ( int i = 0; i < ( int ) matches.size(); i++ )
  {
    points1.push_back( keypoints_1[matches[i].queryIdx].pt );
    points2.push_back( keypoints_2[matches[i].trainIdx].pt );
  }

  //-- 计算基础矩阵
  Mat fundamental_matrix;
  fundamental_matrix = findFundamentalMat ( points1, points2, CV_FM_8POINT );
  cout<<"fundamental_matrix is "<<endl<< fundamental_matrix<<endl;

  //-- 计算本质矩阵
  Point2d principal_point ( 325.1, 249.7 ); // 光心, TUM dataset 标定值
  int focal_length = 521;           // 焦距, TUM dataset 标定值
  Mat essential_matrix;
  essential_matrix = findEssentialMat ( points1, points2, focal_length, principal_point, RANSAC );
  cout<<"essential_matrix is "<<endl<< essential_matrix<<endl;

  //-- 计算单应矩阵
  Mat homography_matrix;
  homography_matrix = findHomography ( points1, points2, RANSAC, 3, noArray(), 2000, 0.99 );
  cout<<"homography_matrix is "<<endl<<homography_matrix<<endl;

  //-- 从本质矩阵中恢复旋转和平移信息
  recoverPose ( essential_matrix, points1, points2, R, t, focal_length, principal_point );
  cout<<"R is "<<endl<<R<<endl;
  cout<<"t is "<<endl<<t<<endl;
}
```

该函数提供了从特征点求解相机运动的部分，然后，我们在主函数中调用它，就能得到相机的运动：

slambook/ch7/pose_estimation_2d2d.cpp（片段）

```
int main( int argc, char** argv )
{
  if ( argc != 3 )
  {
    cout<<"usage: feature_extraction img1 img2"<<endl;
    return 1;
  }
  //-- 读取图像
  Mat img_1 = imread ( argv[1], CV_LOAD_IMAGE_COLOR );
  Mat img_2 = imread ( argv[2], CV_LOAD_IMAGE_COLOR );

```

```
  vector<KeyPoint> keypoints_1, keypoints_2;
  vector<DMatch> matches;
  find_feature_matches( img_1, img_2, keypoints_1, keypoints_2, matches );
  cout<<"一共找到了"<<matches.size()<<"组匹配点"<<endl;

  //—— 估计两张图像间的运动
  Mat R,t;
  pose_estimation_2d2d( keypoints_1, keypoints_2, matches, R, t );

  //—— 验证 E=t^R*scale
  Mat t_x = (Mat_<double>(3,3) <<
    0,                     -t.at<double>(2,0),     t.at<double>(1,0),
    t.at<double>(2,0),      0,                     -t.at<double>(0,0),
    -t.at<double>(1.0),     t.at<double>(0,0),      0);

  cout<<"t^R="<<endl<<t_x*R<<endl;
  //—— 验证对极约束
  Mat K = ( Mat_<double> ( 3,3 ) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1 );
  for ( DMatch m: matches )
  {
    Point2d pt1 = pixel2cam( keypoints_1[ m.queryIdx ].pt, K );
    Mat y1 = (Mat_<double>(3,1) << pt1.x, pt1.y, 1);
    Point2d pt2 = pixel2cam( keypoints_2[ m.trainIdx ].pt, K );
    Mat y2 = (Mat_<double>(3,1) << pt2.x, pt2.y, 1);
    Mat d = y2.t() * t_x * R * y1;
    cout << "epipolar constraint = " << d << endl;
  }
  return 0;
}
```

我们在函数中输出了 $\boldsymbol{E}, \boldsymbol{F}$ 和 $\boldsymbol{H}$ 的数值，然后验证了对极约束是否成立，以及 $\boldsymbol{t}^\wedge \boldsymbol{R}$ 和 $\boldsymbol{E}$ 在非零数乘下等价的事实。现在，调用此程序即可看到输出结果：

```
% build/pose_estimation_2d2d 1.png 2.png
-- Max dist : 95.000000
-- Min dist : 4.000000
一共找到了 79 组匹配点
fundamental_matrix is
[4.844484382466111e-06, 0.0001222601840188731, -0.01786737827487386;
-0.0001174326832719333, 2.122888800459598e-05, -0.01775877156212593;
0.01799658210895528, 0.008143605989020664, 1]
essential_matrix is
[-0.0203618550523477, -0.4007110038118445, -0.03324074249824097;
0.3939270778216369, -0.03506401846698079, 0.5857110303721015;
-0.006788487241438284, -0.5815434272915686, -0.01438258684486258]
homography_matrix is
[0.9497129583105288, -0.143556453147626, 31.20121878625771;
0.04154536627445031, 0.9715568969832015, 5.306887618807696;
-2.81813676978796e-05, 4.353702039810921e-05, 1]
R is
[0.9985961798781875, -0.05169917220143662, 0.01152671359827873;
```

```
0.051396075089760 55, 0.9983603445075083, 0.02520051547522442;
−0.01281065954813571, −0.02457271064688495, 0.9996159607036126]
t is
[−0.8220841067933337;
−0.03269742706405412;
0.5684264241053522]

t^R=
[0.02879601157010516, 0.5666909361828478, 0.04700950886436416;
−0.5570970160413605, 0.0495880104673049, −0.8283204827837456;
0.009600370724838804, 0.8224266019846683, 0.02034004937801349]
epipolar constraint = [0.002528128704106625]
epipolar constraint = [−0.001663727901710724]
epipolar constraint = [−0.0008009088410884102]
......
```

在程序的输出结果可以看出，对极约束的满足精度约在 10^{-3} 量级。根据前面的讨论，分解得到的 $\boldsymbol{R}, \boldsymbol{t}$ 一共有 4 种可能性。不过，OpenCV 会替我们使用三角化检测角点的深度是否为正，从而选出正确的解。

需要注意的地方是，我们要弄清程序求解出来的 $\boldsymbol{R}, \boldsymbol{t}$ 是什么意义。按照例程的定义，我们的对极约束是从

$$\boldsymbol{x}_2 = \boldsymbol{R}\boldsymbol{x}_1 + \boldsymbol{t}$$

得到的。这里的 $\boldsymbol{R}, \boldsymbol{t}$ 组成的变换矩阵，是第一个图到第二个图的坐标变换矩阵：

$$\boldsymbol{x}_2 = \boldsymbol{T}_{21}\boldsymbol{x}_1. \tag{7.22}$$

请读者在实践中务必清楚这里使用的变换顺序（因为有时我们会用 $\boldsymbol{T}_{12}$），它们**非常容易搞反**。

讨论

从演示程序中可以看到，输出的 $\boldsymbol{E}$ 和 $\boldsymbol{F}$ 之间相差了相机内参矩阵。虽然它们在数值上并不直观，但可以验证它们的数学关系。从 $\boldsymbol{E}, \boldsymbol{F}$ 和 $\boldsymbol{H}$ 都可以分解出运动，不过 $\boldsymbol{H}$ 需要假设特征点位于平面上。对于本实验的数据，这个假设是不好的，所以我们这里主要用 $\boldsymbol{E}$ 来分解运动。

值得一提的是，由于 $\boldsymbol{E}$ 本身具有尺度等价性，它分解得到的 $\boldsymbol{t}, \boldsymbol{R}$ 也有一个尺度等价性。而 $\boldsymbol{R} \in \mathrm{SO}(3)$ 自身具有约束，所以我们认为 $\boldsymbol{t}$ 具有一个**尺度**。换言之，在分解过程中，对 $\boldsymbol{t}$ 乘以任意非零常数，分解都是成立的。因此，我们通常把 $\boldsymbol{t}$ 进行**归一化**，让它的长度等于 1。

尺度不确定性

对 $\boldsymbol{t}$ 长度的归一化，直接导致了**单目视觉的尺度不确定性（Scale Ambiguity）**。例如，程序中输出的 $\boldsymbol{t}$ 第一维约为 0.822。这个 0.822 究竟是指 0.822 米还是 0.822 厘米，我们是没法确定的。因为对 $\boldsymbol{t}$ 乘以任意比例常数后，对极约束依然是成立的。换言之，在单目 SLAM 中，对轨迹和地图同时缩放任意倍数，我们得到的图像依然是一样的。这在第 2 讲中就已经向读者介绍过了。

在单目视觉中，我们对两张图像的 $\boldsymbol{t}$ 归一化相当于**固定了尺度**。虽然我们不知道它的实际长度是多少，但我们以这时的 $\boldsymbol{t}$ 为单位 1，计算相机运动和特征点的 3D 位置。这被称为单目 SLAM 的**初始化**。在初始化之后，就可以用 3D−2D 来计算相机运动了。初始化之后的轨迹和地图的单位，就是初始化时固定的尺度。因此，单目 SLAM 有一步不可避免的**初始化**。初始化的两张图像必须有一定程度的平移，而后的轨迹和地图都将以此步的平移为单位。

除了对 $\boldsymbol{t}$ 进行归一化之外，另一种方法是令初始化时所有的特征点平均深度为 1，也可以固定一个尺度。相比于令 $\boldsymbol{t}$ 长度为 1 的做法，把特征点深度归一化可以控制场景的规模大小，使计算在数值上更稳定些。不过这并没有理论上的差别。

初始化的纯旋转问题

从 $\boldsymbol{E}$ 分解到 $\boldsymbol{R},\boldsymbol{t}$ 的过程中，如果相机发生的是纯旋转，导致 $\boldsymbol{t}$ 为零，那么，得到的 $\boldsymbol{E}$ 也将为零，这将导致我们无从求解 $\boldsymbol{R}$。不过，此时我们可以依靠 $\boldsymbol{H}$ 求取旋转，但仅有旋转时，我们无法用三角测量估计特征点的空间位置（这将在下文提到），于是，另一个结论是，**单目初始化不能只有纯旋转，必须要有一定程度的平移**。如果没有平移，单目将无法初始化。在实践当中，如果初始化时平移太小，会使得位姿求解与三角化结果不稳定，从而导致失败。相对地，如果把相机左右移动而不是原地旋转，就容易让单目 SLAM 初始化。因而，有经验的 SLAM 研究人员，在单目 SLAM 情况下经常选择让相机进行左右平移以顺利地进行初始化。

多于 8 对点的情况

当给定的点数多于 8 对时（比如，例程找到了 79 对匹配），我们可以计算一个最小二乘解。回忆式(7.13)中线性化后的对极约束，我们把左侧的系数矩阵记为 $\boldsymbol{A}$：

$$\boldsymbol{A}\boldsymbol{e} = \boldsymbol{0}. \tag{7.23}$$

对于八点法，$\boldsymbol{A}$ 的大小为 8×9。如果给定的匹配点多于 8，该方程构成一个超定方程，即不一定存在 $\boldsymbol{e}$ 使得上式成立。因此，可以通过最小化一个二次型来求：

$$\min_{\boldsymbol{e}} \|\boldsymbol{A}\boldsymbol{e}\|_2^2 = \min_{\boldsymbol{e}} \boldsymbol{e}^{\mathrm{T}}\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{e}. \tag{7.24}$$

于是就求出了在最小二乘意义下的 $\boldsymbol{E}$ 矩阵。不过，当可能存在误匹配的情况时，我们会更倾向于使用**随机采样一致性（Random Sample Concensus，RANSAC）**来求，而不是最小二乘。RANSAC 是一种通用的做法，适用于很多带错误数据的情况，可以处理带有错误匹配的数据。

7.5　三角测量

之前两节我们使用对极几何约束估计了相机运动，也讨论了这种方法的局限性。在得到运动之后，下一步我们需要用相机的运动估计特征点的空间位置。在单目 SLAM 中，仅通过单张图像无法获得像素的深度信息，我们需要通过**三角测量（Triangulation）（或三角化）**的方法来估计地图点的深度，如图 7-11所示。

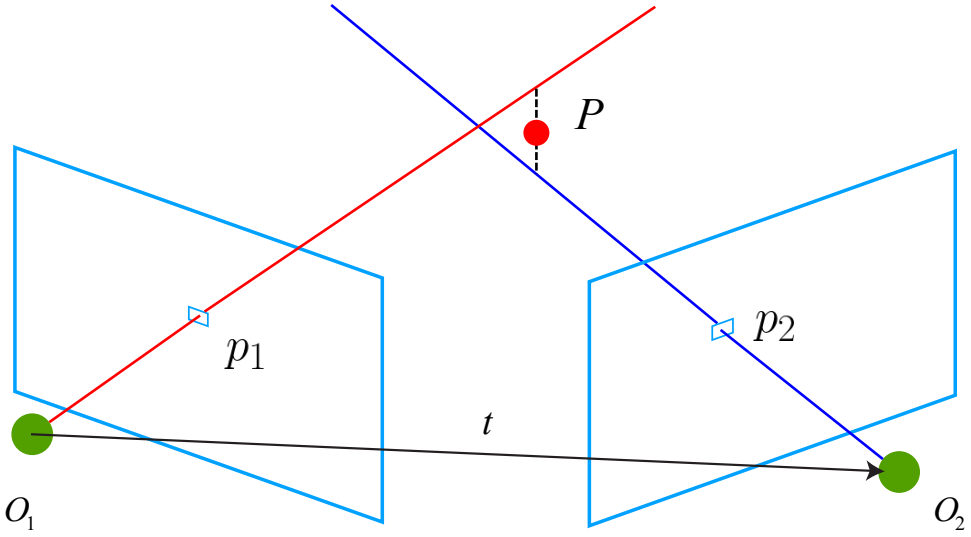


图 7-11　三角化获得地图点深度。

三角测量是指，通过在两处观察同一个点的夹角，从而确定该点的距离。三角测量最早由高斯提出并应用于测量学中，它在天文学、地理学的测量中都有应用。例如，我们可以通过不同季节观察到的星星的角度，估计它离我们的距离。在 SLAM 中，我们主要用三角化来估计像素点的距离。

和上一节类似，考虑图像 I_1 和 I_2，以左图为参考，右图的变换矩阵为 $\boldsymbol{T}$。相机光心为 O_1 和 O_2。在 I_1 中有特征点 p_1，对应 I_2 中有特征点 p_2。理论上直线 O_1p_1 与 O_2p_2 在场景中会相交于一点 P，该点即两个特征点所对应的地图点在三维场景中的位置。然而由于噪声的影响，这两条直线往往无法相交。因此，可以通过最二小乘法求解。

按照对极几何中的定义，设 $\boldsymbol{x}_1, \boldsymbol{x}_2$ 为两个特征点的归一化坐标，那么它们满足：

$$s_1\boldsymbol{x}_1 = s_2\boldsymbol{R}\boldsymbol{x}_2 + \boldsymbol{t}. \tag{7.25}$$

现在我们已经知道了 $\boldsymbol{R}, \boldsymbol{t}$，想要求解的是两个特征点的深度 s_1, s_2。当然这两个深度是可以分开求的，比如，先来看 s_2。如果要算 s_2，那么先对上式两侧左乘一个 $\boldsymbol{x}_1^\wedge$，得：

$$s_1\boldsymbol{x}_1^\wedge\boldsymbol{x}_1 = 0 = s_2\boldsymbol{x}_1^\wedge\boldsymbol{R}\boldsymbol{x}_2 + \boldsymbol{x}_1^\wedge\boldsymbol{t}. \tag{7.26}$$

该式左侧为零，右侧可看成 s_2 的一个方程，可以根据它直接求得 s_2。有了 s_2，s_1 也非常容易求出。于是，我们就得到了两帧下的点的深度，确定了它们的空间坐标。当然，由于噪声的存在，我们估得的 $\boldsymbol{R}, \boldsymbol{t}$ 不一定精确使式(7.26)为零，所以更常见的做法是求最小二乘解而不是零解。

7.6 实践：三角测量

7.6.1 三角测量代码

下面，我们演示如何根据之前利用对极几何求解的相机位姿，通过三角化求出上一节特征点的空间位置。我们调用 OpenCV 提供的 triangulation 函数进行三角化。

slambook/ch7/triangulation.cpp（片断）

```
void triangulation (
  const vector<KeyPoint>& keypoint_1,
  const vector<KeyPoint>& keypoint_2,
  const std::vector< DMatch >& matches,
  const Mat& R, const Mat& t,
  vector<Point3d>& points
);

void triangulation (
  const vector< KeyPoint >& keypoint_1,
  const vector< KeyPoint >& keypoint_2,
  const std::vector< DMatch >& matches,
  const Mat& R, const Mat& t,
  vector< Point3d >& points )
{
  Mat T1 = (Mat_<double> (3,4) <<
    1,0,0,0,
    0,1,0,0,
    0,0,1,0);
  Mat T2 = (Mat_<double> (3,4) <<
    R.at<double>(0,0), R.at<double>(0,1), R.at<double>(0,2), t.at<double>(0,0),
    R.at<double>(1,0), R.at<double>(1,1), R.at<double>(1,2), t.at<double>(1,0),
    R.at<double>(2,0), R.at<double>(2,1), R.at<double>(2,2), t.at<double>(2,0)
  );

  Mat K = ( Mat_<double> ( 3,3 ) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1 );
  vector<Point2d> pts_1, pts_2;
  for ( DMatch m:matches )
  {
    // 将像素坐标转换至相机坐标
    pts_1.push_back ( pixel2cam( keypoint_1[m.queryIdx].pt, K) );
    pts_2.push_back ( pixel2cam( keypoint_2[m.trainIdx].pt, K) );
  }

  Mat pts_4d;
  cv::triangulatePoints( T1, T2, pts_1, pts_2, pts_4d );
```

```
37
38      // 转换成非齐次坐标
39      for ( int i=0; i<pts_4d.cols; i++ )
40      {
41        Mat x = pts_4d.col(i);
42        x /= x.at<float>(3,0); // 归一化
43        Point3d p (
44          x.at<float>(0,0),
45          x.at<float>(1,0),
46          x.at<float>(2,0)
47        );
48        points.push_back( p );
49      }
50  }
```

同时，在 main 函数中增加三角测量部分，并验证重投影关系：

```
int main (int argc, char** argv)
{
  // .....
    //— 三角化
    vector<Point3d> points;
    triangulation( keypoints_1, keypoints_2, matches, R, t, points );

    //— 验证三角化点与特征点的重投影关系
    Mat K = ( Mat_<double> ( 3,3 ) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1 );
    for ( int i=0; i<matches.size(); i++ )
    {
      Point2d pt1_cam = pixel2cam( keypoints_1[ matches[i].queryIdx ].pt, K );
      Point2d pt1_cam_3d (
        points[i].x/points[i].z,
        points[i].y/points[i].z
      );

      cout<<"point in the first camera frame: "<<pt1_cam<<endl;
      cout<<"point projected from 3D "<<pt1_cam_3d<<", d="<<points[i].z<<endl;

      // 第2幅图
      Point2f pt2_cam = pixel2cam( keypoints_2[ matches[i].trainIdx ].pt, K );
      Mat pt2_trans = R*( Mat_<double>(3,1) << points[i].x, points[i].y, points[i].z ) + t;
      pt2_trans /= pt2_trans.at<double>(2,0);
      cout<<"point in the second camera frame: "<<pt2_cam<<endl;
      cout<<"point reprojected from second frame: "<<pt2_trans.t()<<endl;
      cout<<endl;
    }
    // ...
}
```

我们打印了每个空间点在两个相机坐标系下的投影坐标与像素坐标——相当于 P 的投影位置与看到的特征点位置。由于误差的存在，它们会有一些微小的差异。以下是某一特征点的信息：

```
point in the first camera frame: [0.0844072, −0.0734976]
point projected from 3D [0.0843702, −0.0743606], d=14.9895
point in the second camera frame: [0.0431343, −0.0459876]
point reprojected from second frame: [0.04312769812378599, −0.04515455276163744, 1]
```

可以看到，误差的量级大约在小数点后第 3 位。可以看到，三角化特征点的距离大约为 15。但由于尺度不确定性，我们并不知道这里的 15 究竟是多少米。

7.6.2 讨论

关于三角测量，还有一个必须注意的地方。

三角测量是由**平移**得到的，有平移才会有对极几何中的三角形，才谈得上三角测量。因此，纯旋转是无法使用三角测量的，因为对极约束将永远满足。在平移存在的情况下，我们还要关心三角测量的不确定性，这会引出一个**三角测量的矛盾**。

如图 7-12 所示，当平移很小时，像素上的不确定性将导致较大的深度不确定性。也就是说，如果特征点运动一个像素 δx，使得视线角变化了一个角度 $\delta\theta$，那么将测量到深度值有 δd 的变化。从几何关系可以看到，当 $\boldsymbol{t}$ 较大时，δd 将明显变小，这说明平移较大时，在同样的相机分辨率下，三角化测量将更精确。对该过程的定量分析可以使用正弦定理得到，不过这里先考虑定性分析。

因此，要提高三角化的精度，其一是提高特征点的提取精度，也就是提高图像分辨率——但这会导致图像变大，增加计算成本。另一方式是使平移量增大。但是，这会导致图像的**外观**发生明显的变化，比如箱子原先被挡住的侧面显示出来，又比如反射光发生变化，等等。外观变化会使得特征提取与匹配变得困难。总而言之，再增大平移，会导致匹配失效；而平移太小，则三角化精度不够——这就是三角化的矛盾。

虽然本节只介绍了三角化的深度估计，但只要我们愿意，也能够定量地计算每个特征点的**位置及不确定性**。所以，如果假设特征点服从高斯分布，并且不断地对它进行观测，在信息正确的情况下，我们就能够期望**它的方差会不断减小乃至收敛**。这就得到了一个**滤波器**，称为**深度滤波器（Depth Filter）**。不过，由于它的原理较复杂，我们将留到第 13 讲再详细讨论它。下面，我们来讨论从 3D−2D 的匹配点来估计相机运动，以及 3D−3D 的估计方法。

7.7 3D−2D：PnP

PnP（Perspective-n-Point）是求解 3D 到 2D 点对运动的方法。它描述了当知道 n 个 3D 空间点及其投影位置时，如何估计相机的位姿。前面说到，2D−2D 的对极几何方法需要 8 个或 8 个以上的点对（以八点法为例），且存在着初始化、纯旋转和尺度的问题。然而，如果两张图像中其中一张特征点的 3D 位置已知，那么最少只需 3 个点对（需要至少一个额外点验证结果）就可以估计相机运动。特征点的 3D 位置可以由三角化或者 RGB-D 相机的深度图确定。因此，在双目或 RGB-D 的

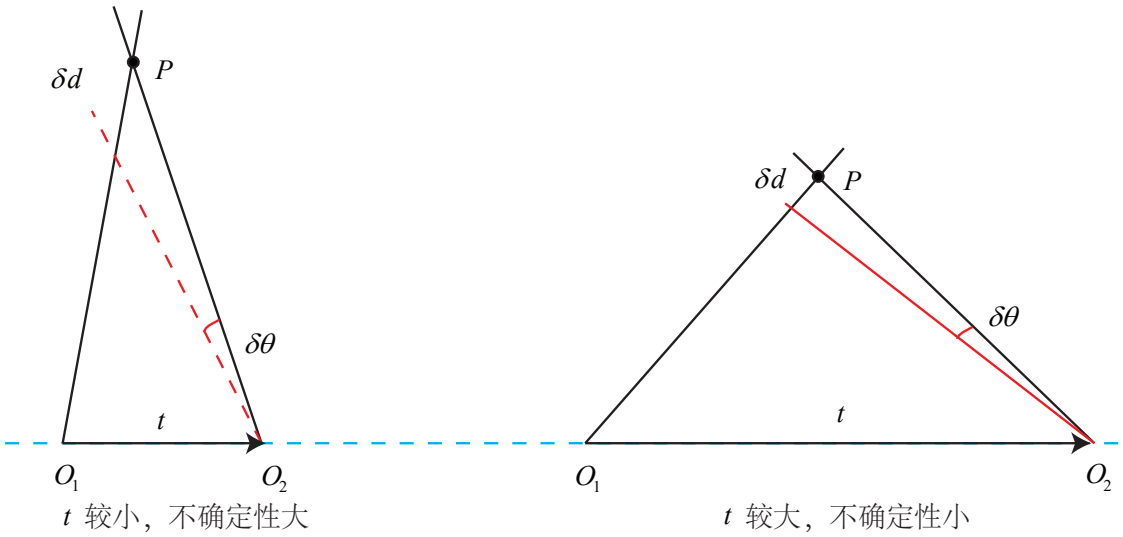


图 7-12　三角测量的矛盾。

视觉里程计中，我们可以直接使用 PnP 估计相机运动。而在单目视觉里程计中，必须先进行初始化，然后才能使用 PnP。3D−2D 方法不需要使用对极约束，又可以在很少的匹配点中获得较好的运动估计，是最重要的一种姿态估计方法。

PnP 问题有很多种求解方法，例如，用 3 对点估计位姿的 P3P[55]、直接线性变换（DLT）、EPnP（Efficient PnP）[56]、UPnP[57]，等等。此外，还能用**非线性优化**的方式，构建最小二乘问题并迭代求解，也就是万金油式的 Bundle Adjustment。我们先来看 DLT，然后再讲解 Bundle Adjustment。

7.7.1 直接线性变换

考虑某个空间点 P，它的齐次坐标为 $\boldsymbol{P} = (X, Y, Z, 1)^{\mathrm{T}}$。在图像 I_1 中，投影到特征点 $\boldsymbol{x}_1 = (u_1, v_1, 1)^{\mathrm{T}}$（以归一化平面齐次坐标表示）。此时相机的位姿 $\boldsymbol{R}, \boldsymbol{t}$ 是未知的。与单应矩阵的求解类似，我们定义增广矩阵 $[\boldsymbol{R}|\boldsymbol{t}]$ 为一个 3×4 的矩阵，包含了旋转与平移信息[①]。我们将其展开形式列写如下：

$$s \begin{pmatrix} u_1 \\ v_1 \\ 1 \end{pmatrix} = \begin{pmatrix} t_1 & t_2 & t_3 & t_4 \\ t_5 & t_6 & t_7 & t_8 \\ t_9 & t_{10} & t_{11} & t_{12} \end{pmatrix} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}. \tag{7.27}$$

用最后一行把 s 消去，得到两个约束：

$$u_1 = \frac{t_1 X + t_2 Y + t_3 Z + t_4}{t_9 X + t_{10} Y + t_{11} Z + t_{12}}, \quad v_1 = \frac{t_5 X + t_6 Y + t_7 Z + t_8}{t_9 X + t_{10} Y + t_{11} Z + t_{12}}.$$

为了简化表示，定义 $\boldsymbol{T}$ 的行向量：

$$\boldsymbol{t}_1 = (t_1, t_2, t_3, t_4)^{\mathrm{T}}, \boldsymbol{t}_2 = (t_5, t_6, t_7, t_8)^{\mathrm{T}}, \boldsymbol{t}_3 = (t_9, t_{10}, t_{11}, t_{12})^{\mathrm{T}},$$

于是有：

$$\boldsymbol{t}_1^{\mathrm{T}} \boldsymbol{P} - \boldsymbol{t}_3^{\mathrm{T}} \boldsymbol{P} u_1 = 0,$$

和

$$\boldsymbol{t}_2^{\mathrm{T}} \boldsymbol{P} - \boldsymbol{t}_3^{\mathrm{T}} \boldsymbol{P} v_1 = 0.$$

请注意，$\boldsymbol{t}$ 是待求的变量，可以看到，每个特征点提供了两个关于 $\boldsymbol{t}$ 的线性约束。假设一共有 N 个特征点，则可以列出如下线性方程组：

[①] 请注意，这和 SE(3) 中的变换矩阵 $\boldsymbol{T}$ 是不同的。

$$\begin{pmatrix} \boldsymbol{P}_1^{\mathrm{T}} & 0 & -u_1\boldsymbol{P}_1^{\mathrm{T}} \\ 0 & \boldsymbol{P}_1^{\mathrm{T}} & -v_1\boldsymbol{P}_1^{\mathrm{T}} \\ \vdots & \vdots & \vdots \\ \boldsymbol{P}_N^{\mathrm{T}} & 0 & -u_N\boldsymbol{P}_N^{\mathrm{T}} \\ 0 & \boldsymbol{P}_N^{\mathrm{T}} & -v_N\boldsymbol{P}_N^{\mathrm{T}} \end{pmatrix} \begin{pmatrix} \boldsymbol{t}_1 \\ \boldsymbol{t}_2 \\ \boldsymbol{t}_3 \end{pmatrix} = 0. \tag{7.28}$$

由于 $\boldsymbol{t}$ 一共有 12 维，因此，最少通过 6 对匹配点即可实现矩阵 $\boldsymbol{T}$ 的线性求解，这种方法称为直接线性变换（Direct Linear Transform，DLT）。当匹配点大于 6 对时，也可以使用 SVD 等方法对超定方程求最小二乘解。

在 DLT 求解中，我们直接将 $\boldsymbol{T}$ 矩阵看成了 12 个未知数，忽略了它们之间的联系。因为旋转矩阵 $\boldsymbol{R} \in \mathrm{SO}(3)$，用 DLT 求出的解不一定满足该约束，它是一个一般矩阵。平移向量比较好办，它属于向量空间。对于旋转矩阵 $\boldsymbol{R}$，我们必须针对 DLT 估计的 $\boldsymbol{T}$ 左边 3×3 的矩阵块，寻找一个最好的旋转矩阵对它进行近似。这可以由 QR 分解完成[3, 58]，相当于把结果从矩阵空间重新投影到 SE(3) 流形上，转换成旋转和平移两部分。

需要解释的是，我们这里的 $\boldsymbol{x}_1$ 使用了归一化平面坐标，去掉了内参矩阵 $\boldsymbol{K}$ 的影响——这是因为内参 $\boldsymbol{K}$ 在 SLAM 中通常假设为已知。即使内参未知，也能用 PnP 去估计 $\boldsymbol{K}, \boldsymbol{R}, \boldsymbol{t}$ 三个量。然而由于未知量增多，效果会差一些。

7.7.2　P3P

下面讲的 P3P 是另一种解 PnP 的方法。它仅使用 3 对匹配点，对数据要求较少，因此这里也简单介绍一下（这部分推导借鉴了文献 [59]）。

P3P 需要利用给定的 3 个点的几何关系。它的输入数据为 3 对 3D–2D 匹配点。记 3D 点为 A, B, C，2D 点为 a, b, c，其中小写字母代表的点为对应大写字母代表的点在相机成像平面上的投影，如图 7-13 所示。此外，P3P 还需要使用一对验证点，以从可能的解中选出正确的那一个（类似于对极几何情形）。记验证点对为 $D - d$，相机光心为 O。请注意，我们知道的是 A, B, C 在**世界坐标系中的坐标**，而不是**在相机坐标系中的坐标**。一旦 3D 点在相机坐标系下的坐标能够算出，我们就得到了 3D–3D 的对应点，把 PnP 问题转换为了 ICP 问题。

首先，显然三角形之间存在对应关系：

$$\Delta Oab - \Delta OAB, \quad \Delta Obc - \Delta OBC, \quad \Delta Oac - \Delta OAC. \tag{7.29}$$

来考虑 Oab 和 OAB 的关系。利用余弦定理，有：

$$OA^2 + OB^2 - 2OA \cdot OB \cdot \cos\langle a, b\rangle = AB^2. \tag{7.30}$$

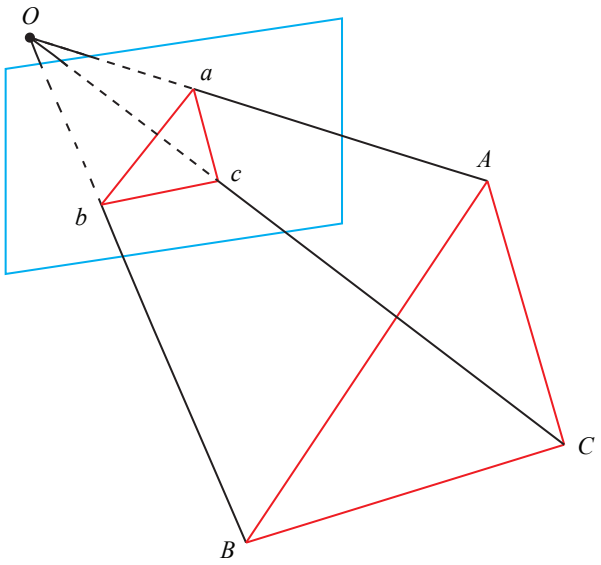


图 7-13　P3P 问题示意图。

对于其他两个三角形亦有类似性质，于是有：

$$\begin{aligned} OA^2 + OB^2 - 2OA \cdot OB \cdot \cos\langle a, b\rangle &= AB^2 \\ OB^2 + OC^2 - 2OB \cdot OC \cdot \cos\langle b, c\rangle &= BC^2 \\ OA^2 + OC^2 - 2OA \cdot OC \cdot \cos\langle a, c\rangle &= AC^2. \end{aligned} \tag{7.31}$$

对以上三式全体除以 OC^2，并且记 $x = OA/OC, y = OB/OC$，得：

$$\begin{aligned} x^2 + y^2 - 2xy\cos\langle a, b\rangle &= AB^2/OC^2 \\ y^2 + 1^2 - 2y\cos\langle b, c\rangle &= BC^2/OC^2 \\ x^2 + 1^2 - 2x\cos\langle a, c\rangle &= AC^2/OC^2. \end{aligned} \tag{7.32}$$

记 $v = AB^2/OC^2, uv = BC^2/OC^2, wv = AC^2/OC^2$，有：

$$\begin{aligned} x^2 + y^2 - 2xy\cos\langle a, b\rangle - v &= 0 \\ y^2 + 1^2 - 2y\cos\langle b, c\rangle - uv &= 0 \\ x^2 + 1^2 - 2x\cos\langle a, c\rangle - wv &= 0. \end{aligned} \tag{7.33}$$

我们可以把第一个式子中的 v 放到等式一边，并代入其后两式，得：

$$\begin{aligned} (1-u)\,y^2 - ux^2 - \cos\langle b, c\rangle\, y + 2uxy\cos\langle a, b\rangle + 1 &= 0 \\ (1-w)\,x^2 - wy^2 - \cos\langle a, c\rangle\, x + 2wxy\cos\langle a, b\rangle + 1 &= 0. \end{aligned} \tag{7.34}$$

注意这些方程中的已知量和未知量。由于我们知道 2D 点的图像位置，3 个余弦角 $\cos\langle a,b\rangle, \cos\langle b,c\rangle, \cos\langle a,c\rangle$ 是已知的。同时，$u = BC^2/AB^2, w = AC^2/AB^2$ 可以通过 A,B,C 在世界坐标系下的坐标算出，变换到相机坐标系下之后，这个比值并不改变。该式中的 x,y 是未知的，随着相机移动会发生变化。因此，该方程组是关于 x,y 的一个二元二次方程（多项式方程）。解析地求解该方程组是一个复杂的过程，需要用吴消元法。这里不展开对该方程解法的介绍，感兴趣的读者请参阅文献 [55]。类似于分解 $\boldsymbol{E}$ 的情况，该方程最多可能得到 4 个解，但我们可以用验证点来计算最可能的解，得到 A,B,C 在相机坐标系下的 3D 坐标。然后，根据 3D−3D 的点对，计算相机的运动 $\boldsymbol{R},\boldsymbol{t}$。这部分将在 7.9 节介绍。

从 P3P 的原理可以看出，为了求解 PnP，我们利用了三角形相似性质，求解投影点 a,b,c 在相机坐标系下的 3D 坐标，最后把问题转换成一个 3D 到 3D 的位姿估计问题。在后文将看到，带有匹配信息的 3D−3D 位姿求解非常容易，所以这种思路是非常有效的。其他的一些方法，例如 EPnP，亦采用了这种思路。然而，P3P 也存在着一些问题：

1. P3P 只利用 3 个点的信息。当给定的配对点多于 3 组时，难以利用更多的信息。
2. 如果 3D 点或 2D 点受噪声影响，或者存在误匹配，则算法失效。

所以后续人们还提出了许多别的方法，如 EPnP、UPnP 等。它们利用更多的信息，而且用迭代的方式对相机位姿进行优化，以尽可能地消除噪声的影响。不过，相对于 P3P 来说，原理会更加复杂一些，所以我们建议读者阅读原始的论文，或通过实践来理解 PnP 过程。在 SLAM 当中，通常的做法是先使用 P3P/EPnP 等方法估计相机位姿，然后构建最小二乘优化问题对估计值进行调整（Bundle Adjustment）。接下来我们从非线性优化角度来看一下 PnP 问题。

7.7.3 Bundle Adjustment

除了使用线性方法之外，我们还可以把 PnP 问题构建成一个定义于李代数上的非线性最小二乘问题。这将用到本书第 4 讲和第 5 讲的知识。前面说的线性方法，往往是**先求相机位姿，再求空间点位置**，而非线性优化则是把它们都看成优化变量，放在一起优化。这是一种非常通用的求解方式，我们可以用它对 PnP 或 ICP 给出的结果进行优化。在 PnP 中，这个 Bundle Adjustment 问题，是一个最小化**重投影误差（Reprojection error）**的问题。我们将在本节给出此问题在两个视图下的基本形式，然后在第 10 讲讨论较大规模的 BA 问题。

考虑 n 个三维空间点 P 及其投影 p，我们希望计算相机的位姿 $\boldsymbol{R},\boldsymbol{t}$，它的李代数表示为 $\boldsymbol{\xi}$。假设某空间点坐标为 $\boldsymbol{P}_i = [X_i, Y_i, Z_i]^{\mathrm{T}}$，其投影的像素坐标为 $\boldsymbol{u}_i = [u_i, v_i]^{\mathrm{T}}$。根据第 5 讲的内容，像素位置与空间点位置的关系如下：

$$s_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \boldsymbol{K} \exp(\boldsymbol{\xi}^\wedge) \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}. \tag{7.35}$$

除了用 $\boldsymbol{\xi}$ 表示相机姿态之外，别的都和前面的定义保持一致。写成矩阵形式就是：

$$s_i \boldsymbol{u}_i = \boldsymbol{K} \exp\left(\boldsymbol{\xi}^\wedge\right) \boldsymbol{P}_i.$$

请读者脑补中间隐含着的齐次坐标到非齐次的转换，否则按矩阵的乘法来说，维度是不对的[①]。现在，由于相机位姿未知及观测点的噪声，该等式存在一个误差。因此，我们把误差求和，构建最小二乘问题，然后寻找最好的相机位姿，使它最小化：

$$\boldsymbol{\xi}^* = \arg\min_{\boldsymbol{\xi}} \frac{1}{2} \sum_{i=1}^{n} \left\| \boldsymbol{u}_i - \frac{1}{s_i} \boldsymbol{K} \exp\left(\boldsymbol{\xi}^\wedge\right) \boldsymbol{P}_i \right\|_2^2. \tag{7.36}$$

该问题的误差项，是将像素坐标（观测到的投影位置）与 3D 点按照当前估计的位姿进行投影得到的位置相比较得到的误差，所以称为**重投影误差**。使用齐次坐标时，这个误差有 3 维。不过，由于 $\boldsymbol{u}$ 最后一维为 1，该维度的误差一直为零，因而我们更多时候使用非齐次坐标，于是误差就只有 2 维了。如图 7-14 所示，我们通过特征匹配知道了 p_1 和 p_2 是同一个空间点 P 的投影，但是不知道相机的位姿。在初始值中，P 的投影 $\hat{p}_2$ 与实际的 p_2 之间有一定的距离。于是我们调整相机的位姿，使得这个距离变小。不过，由于这个调整需要考虑很多个点，所以最后每个点的误差通常都不会精确为零。

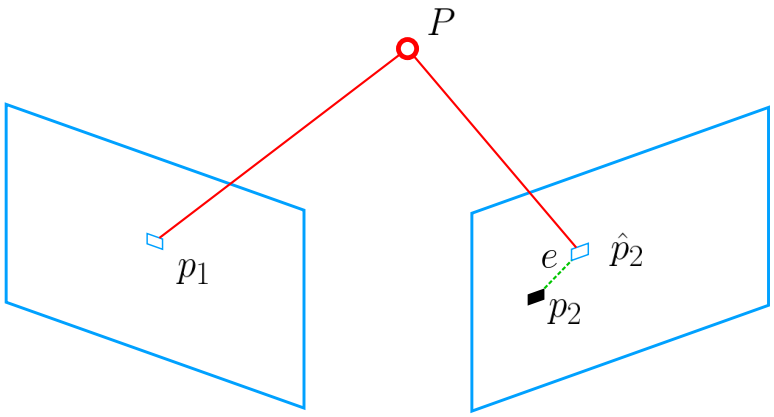


图 7-14　重投影误差示意图。

最小二乘优化问题已经在第 6 讲介绍过了。使用李代数，可以构建无约束的优化问题，很方便地通过高斯牛顿法、列文伯格—马夸尔特方法等优化算法进行求解。不过，在使用高斯牛顿法和列

[①] $\exp\left(\boldsymbol{\xi}^\wedge\right) \boldsymbol{P}_i$ 结果是 4×1 的，而其左侧的 $\boldsymbol{K}$ 是 3×3 的，所以必须把 $\exp\left(\boldsymbol{\xi}^\wedge\right) \boldsymbol{P}_i$ 的前三维取出来，变成三维的非齐次坐标。这在前面讲过。

文伯格—马夸尔特方法之前，我们需要知道每个误差项关于优化变量的导数，也就是**线性化**：

$$\boldsymbol{e}(\boldsymbol{x}+\Delta\boldsymbol{x}) \approx \boldsymbol{e}(\boldsymbol{x}) + \boldsymbol{J}\Delta\boldsymbol{x}. \tag{7.37}$$

这里的 $\boldsymbol{J}$ 的形式是值得讨论的，甚至可以说是关键所在。我们固然可以使用数值导数，但如果能够推导出解析形式，则我们会优先考虑解析导数。现在，当 $\boldsymbol{e}$ 为像素坐标误差（2 维），$\boldsymbol{x}$ 为相机位姿（6 维）时，$\boldsymbol{J}$ 将是一个 2×6 的矩阵。我们来推导 $\boldsymbol{J}$ 的形式。

回忆李代数的内容，我们介绍了如何使用扰动模型来求李代数的导数。首先，记变换到相机坐标系下的空间点坐标为 $\boldsymbol{P}'$，并且将其前 3 维取出来：

$$\boldsymbol{P}' = (\exp(\boldsymbol{\xi}^\wedge)\boldsymbol{P})_{1:3} = [X', Y', Z']^{\mathrm{T}}. \tag{7.38}$$

那么，相机投影模型相对于 $\boldsymbol{P}'$ 为

$$s\boldsymbol{u} = \boldsymbol{K}\boldsymbol{P}'. \tag{7.39}$$

展开：

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}. \tag{7.40}$$

利用第 3 行消去 s（实际上就是 $\boldsymbol{P}'$ 的距离），得：

$$u = f_x\frac{X'}{Z'} + c_x, \quad v = f_y\frac{Y'}{Z'} + c_y. \tag{7.41}$$

这与之前讲的相机模型是一致的。当我们求误差时，可以把这里的 u, v 与实际的测量值比较，求差。在定义了中间变量后，我们对 $\boldsymbol{\xi}^\wedge$ 左乘扰动量 $\delta\boldsymbol{\xi}$，然后考虑 $\boldsymbol{e}$ 的变化关于扰动量的导数。利用链式法则，可以列写如下：

$$\frac{\partial \boldsymbol{e}}{\partial \delta\boldsymbol{\xi}} = \lim_{\delta\boldsymbol{\xi}\to 0}\frac{e(\delta\boldsymbol{\xi}\oplus\boldsymbol{\xi})}{\delta\boldsymbol{\xi}} = \frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}'}\frac{\partial \boldsymbol{P}'}{\partial \delta\boldsymbol{\xi}}. \tag{7.42}$$

这里的 $\oplus$ 指李代数上的左乘扰动。第一项是误差关于投影点的导数，在式(7.41)已经列出了变量之间的关系，易得：

$$\frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}'} = -\begin{bmatrix} \frac{\partial u}{\partial X'} & \frac{\partial u}{\partial Y'} & \frac{\partial u}{\partial Z'} \\ \frac{\partial v}{\partial X'} & \frac{\partial v}{\partial Y'} & \frac{\partial v}{\partial Z'} \end{bmatrix} = -\begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} \end{bmatrix}. \tag{7.43}$$

而第二项为变换后的点关于李代数的导数，根据4.3.5节中的推导，得：

$$\frac{\partial(\boldsymbol{T}\boldsymbol{P})}{\partial \delta\boldsymbol{\xi}} = (\boldsymbol{T}\boldsymbol{P})^{\odot} = \begin{bmatrix} \boldsymbol{I} & -\boldsymbol{P}'^{\wedge} \\ \boldsymbol{0}^{\mathrm{T}} & \boldsymbol{0}^{\mathrm{T}} \end{bmatrix}. \tag{7.44}$$

而在 $\boldsymbol{P}'$ 的定义中，我们取出了前 3 维，于是得：

$$\frac{\partial \boldsymbol{P}'}{\partial \delta \boldsymbol{\xi}} = [\boldsymbol{I}, -\boldsymbol{P}'^{\wedge}]. \tag{7.45}$$

将这两项相乘，就得到了 2×6 的雅可比矩阵：

$$\frac{\partial \boldsymbol{e}}{\partial \delta \boldsymbol{\xi}} = -\begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} & -\frac{f_x X' Y'}{Z'^2} & f_x + \frac{f_x X'^2}{Z'^2} & -\frac{f_x Y'}{Z'} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} & -f_y - \frac{f_y Y'^2}{Z'^2} & \frac{f_y X' Y'}{Z'^2} & \frac{f_y X'}{Z'} \end{bmatrix}. \tag{7.46}$$

这个雅可比矩阵描述了重投影误差关于相机位姿李代数的一阶变化关系。我们保留了前面的负号，这是因为误差是由**观测值减预测值**定义的。它当然也可反过来，定义成“预测值减观测值”的形式。在那种情况下，只要去掉前面的负号即可。此外，如果 $\mathfrak{se}(3)$ 的定义方式是旋转在前，平移在后，只要把这个矩阵的前 3 列与后 3 列对调即可。

另一方面，除了优化位姿，我们还希望优化特征点的空间位置。因此，需要讨论 $\boldsymbol{e}$ 关于空间点 $\boldsymbol{P}$ 的导数。所幸这个导数矩阵相对来说容易一些。仍利用链式法则，有：

$$\frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}} = \frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}'} \frac{\partial \boldsymbol{P}'}{\partial \boldsymbol{P}}. \tag{7.47}$$

第一项在前面已推导，关于第二项，按照定义

$$\boldsymbol{P}' = \exp(\boldsymbol{\xi}^{\wedge})\boldsymbol{P} = \boldsymbol{R}\boldsymbol{P} + \boldsymbol{t},$$

我们发现 $\boldsymbol{P}'$ 对 $\boldsymbol{P}$ 求导后将只剩下 $\boldsymbol{R}$。于是：

$$\frac{\partial \boldsymbol{e}}{\partial \boldsymbol{P}} = -\begin{bmatrix} \frac{f_x}{Z'} & 0 & -\frac{f_x X'}{Z'^2} \\ 0 & \frac{f_y}{Z'} & -\frac{f_y Y'}{Z'^2} \end{bmatrix} \boldsymbol{R}. \tag{7.48}$$

于是，我们推导出了观测相机方程关于相机位姿与特征点的两个导数矩阵。它们**十分重要**，能够在优化过程中提供重要的梯度方向，指导优化的迭代。

7.8 实践：求解 PnP

7.8.1 使用 EPnP 求解位姿

下面，我们通过实验理解一下 PnP 的过程。首先，我们用 OpenCV 提供的 EPnP 求解 PnP 问题，然后通过 g2o 对结果进行优化。由于 PnP 需要使用 3D 点，为了避免初始化带来的麻烦，我们使用了 RGB-D 相机中的深度图（1_depth.png）作为特征点的 3D 位置。首先来看 OpenCV 提供的 PnP 函数：

slambook/ch7/pose_estimation_3d2d.cpp（片段）

```
int main( int argc, char** argv )
{
  ......
    // 建立 3D 点
    Mat d1 = imread( argv[3], CV_LOAD_IMAGE_UNCHANGED );        // 深度图为 16 位无符号数，单通道图像
    Mat K = ( Mat_<double> ( 3,3 ) << 520.9, 0, 325.1, 0, 521.0, 249.7, 0, 0, 1 );
    vector<Point3f> pts_3d;
    vector<Point2f> pts_2d;
    for ( DMatch m:matches )
    {
      ushort d = d1.ptr<unsigned short> ( int(keypoints_1[m.queryIdx].pt.y) )[ int(keypoints_1[m.
          queryIdx].pt.x) ];
      if ( d == 0 )   // bad depth
        continue;
      float dd = d/1000.0;
      Point2d p1 = pixel2cam( keypoints_1[m.queryIdx].pt, K );
      pts_3d.push_back( Point3f(p1.x*dd, p1.y*dd, dd) );
      pts_2d.push_back ( keypoints_2[m.trainIdx].pt );
    }

    cout<<"3d-2d pairs: "<<pts_3d.size()<<endl;

    Mat r, t;
    // 调用 OpenCV 的 PnP 求解，可选择 EPNP、DLS 等方法
    solvePnP( pts_3d, pts_2d, K, Mat(), r, t, false, cv::SOLVEPNP_EPNP );
    Mat R;
    cv::Rodrigues(r, R); // r 为旋转向量形式，用 Rodrigues 公式转换为矩阵

    cout<<"R="<<endl<<R<<endl;
    cout<<"t="<<endl<<t<<endl;
}
```

在例程中，得到配对特征点后，我们在第一个图的深度图中寻找它们的深度，并求出空间位置。以此空间位置为 3D 点，再以第二个图像的像素位置为 2D 点，调用 EPnP 求解 PnP 问题。程序输出如下：

```
% build/pose_estimation_3d2d 1.png 2.png d1.png d2.png
-- Max dist : 95.000000
-- Min dist : 4.000000
一共找到了79组匹配点
3d-2d pairs: 78
R=
[0.9977970937403702, -0.05195299069131867, 0.04125344205637558;
0.05073872610592159, 0.9982626103770279, 0.02995567385972873;
-0.04273805559942161, -0.02779653722084675, 0.9986995599889442]
t=
[-0.6455324432075111;
-0.05776758294184359;
0.2844565219506077]
```

读者可以对比先前 2D−2D 情况下求解的 $\boldsymbol{R}, \boldsymbol{t}$ 看看有什么不同。可以看到，在有 3D 信息时，估计的 $\boldsymbol{R}$ 几乎是相同的，而 $\boldsymbol{t}$ 相差得较多。这是由于引入了新的深度信息所致。不过，由于 Kinect 采集的深度图本身会有一些误差，所以这里的 3D 点也不是准确的。我们会希望把位姿 $\boldsymbol{\xi}$ 和所有三维特征点 P 同时优化。

7.8.2 使用 BA 优化

下面演示如何进行 Bundle Adjustment。我们将使用前一步的估计值作为初始值。优化可以使用前面讲的 Ceres 或 g2o 库实现，这里采用 g2o 作为例子。

g2o 的基本知识在第 6 讲中已经介绍过了。在使用 g2o 之前，我们要把问题建模成一个最小二乘的图优化问题，如图 7-15 所示。

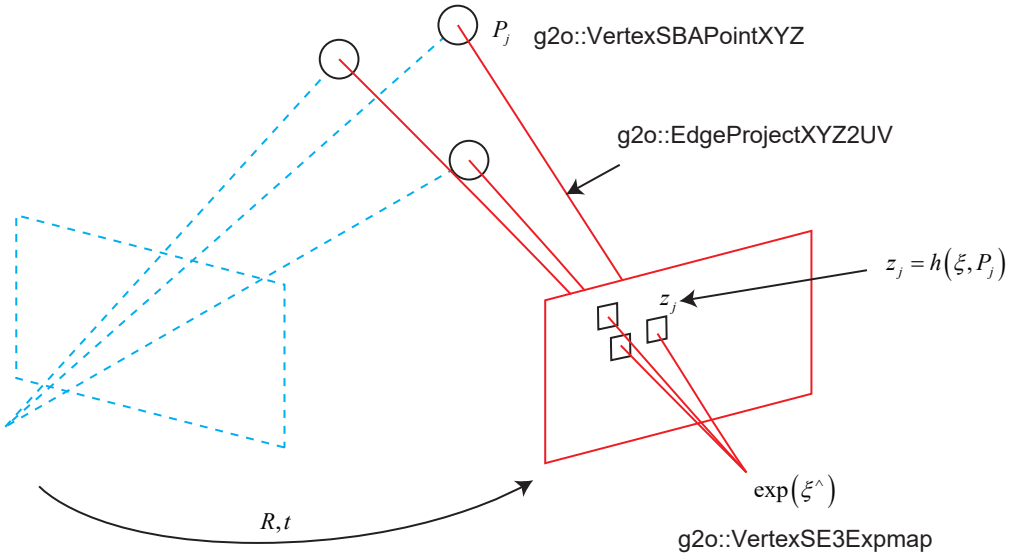


图 7-15　PnP 的 Bundle Adjustment 的图优化表示。

在这个图优化中，节点和边的选择如下：

1. **节点**：第二个相机的位姿节点 $\boldsymbol{\xi} \in \mathfrak{se}(3)$，所有特征点的空间位置 $\boldsymbol{P} \in \mathbb{R}^3$。
2. **边**：每个 3D 点在第二个相机中的投影，以观测方程来描述：

$$\boldsymbol{z}_j = h(\boldsymbol{\xi}, \boldsymbol{P}_j).$$

由于第一个相机位姿固定为零，我们没有把它写到优化变量里，但在习题中，希望你能够把第一个相机的位姿与观测也考虑进来。现在我们根据一组 3D 点和第二个图像中的 2D 投影，估计第二个相机的位姿。所以我们把第一个相机画成虚线，表明不希望考虑它。

g2o 提供了许多关于 BA 的节点和边，我们不必自己从头实现所有的计算。在 g2o/types/sba/types_six_dof_expmap.h 中则提供了李代数表达的节点和边。请读者打开这个文件，找到 VertexSE3Expmap（李代数位姿）、VertexSBAPointXYZ（空间点位置）和 EdgeProjectXYZ2UV（投影方程边）这三个类。我们来简单看一下它们的类定义，例如 VertexSE3Expmap：

```
class G2O_TYPES_SBA_API VertexSE3Expmap : public BaseVertex<6, SE3Quat>{
public:
  EIGEN_MAKE_ALIGNED_OPERATOR_NEW

  VertexSE3Expmap();

  bool read(std::istream& is);

  bool write(std::ostream& os) const;
```

```
10
11   virtual void setToOriginImpl() {
12   _estimate = SE3Quat();
13   }
14
15   virtual void oplusImpl(const double* update_)  {
16     Eigen::Map<const Vector6d> update(update_);
17     setEstimate( SE3Quat::exp(update)*estimate());
18   }
19 };
```

请注意它的模板参数。第一个参数 6 表示它内部存储的优化变量维度，可以看到这是一个 6 维的李代数。第二个参数是优化变量的类型，这里使用了 g2o 定义的相机位姿：SE3Quat。这个类内部使用了**四元数**加位移向量来存储位姿，但同时也支持李代数上的运算，例如对数映射（log 函数）和李代数上增量（update 函数）等操作。我们可以对照它的实现代码，看看 g2o 对李代数是如何操作的：

```
1  class G2O_TYPES_SBA_API VertexSBAPointXYZ : public BaseVertex<3, Vector3D>
2  {
3   ......
4  };
5
6  class G2O_TYPES_SBA_API EdgeProjectXYZ2UV : public  BaseBinaryEdge<2, Vector2D, VertexSBAPointXYZ,
       VertexSE3Expmap>
7  {
8    ......
9      void computeError()  {
10       const VertexSE3Expmap* v1 = static_cast<const VertexSE3Expmap*>(_vertices[1]);
11       const VertexSBAPointXYZ* v2 = static_cast<const VertexSBAPointXYZ*>(_vertices[0]);
12       const CameraParameters * cam
13         = static_cast<const CameraParameters *>(parameter(0));
14       Vector2D obs(_measurement);
15       _error = obs-cam->cam_map(v1->estimate().map(v2->estimate()));
16     }
17 };
```

这里就不把整个类定义都搬过来了。从模板参数可以看到，空间点位置类的维度为 3，类型是 Eigen 的 Vector3D。另一方面，边 EdgeProjectXYZ2UV 连接了前面说的两个顶点，它的观测值为 2 维，由 Vector2D 表示，实际上就是空间点的像素坐标。它的误差计算函数表达了投影方程的误差计算方法，也就是我们前面提到的 $\boldsymbol{z}-h(\boldsymbol{\xi},\boldsymbol{P})$ 的方式。

现在，进一步观察 EdgeProjectXYZ2UV 的 linearizeOplus 函数的实现。这里用到了我们前面推导的雅可比矩阵：

```
1  void EdgeProjectXYZ2UV::linearizeOplus() {
```

```
  VertexSE3Expmap * vj = static_cast<VertexSE3Expmap *>(_vertices[1]);
  SE3Quat T(vj->estimate());
  VertexSBAPointXYZ* vi = static_cast<VertexSBAPointXYZ*>(_vertices[0]);
  Vector3D xyz = vi->estimate();
  Vector3D xyz_trans = T.map(xyz);

  double x = xyz_trans[0];
  double y = xyz_trans[1];
  double z = xyz_trans[2];
  double z_2 = z*z;

  const CameraParameters * cam = static_cast<const CameraParameters *>(parameter(0));

  Matrix<double,2,3,Eigen::ColMajor> tmp;
  tmp(0,0) = cam->focal_length;
  tmp(0,1) = 0;
  tmp(0,2) = -x/z*cam->focal_length;

  tmp(1,0) = 0;
  tmp(1,1) = cam->focal_length;
  tmp(1,2) = -y/z*cam->focal_length;

  _jacobianOplusXi =  -1./z * tmp * T.rotation().toRotationMatrix();

  _jacobianOplusXj(0,0) =  x*y/z_2 *cam->focal_length;
  _jacobianOplusXj(0,1) = -(1+(x*x/z_2)) *cam->focal_length;
  _jacobianOplusXj(0,2) = y/z *cam->focal_length;
  _jacobianOplusXj(0,3) = -1./z *cam->focal_length;
  _jacobianOplusXj(0,4) = 0;
  _jacobianOplusXj(0,5) = x/z_2 *cam->focal_length;

  _jacobianOplusXj(1,0) = (1+y*y/z_2) *cam->focal_length;
  _jacobianOplusXj(1,1) = -x*y/z_2 *cam->focal_length;
  _jacobianOplusXj(1,2) = -x/z *cam->focal_length;
  _jacobianOplusXj(1,3) = 0;
  _jacobianOplusXj(1,4) = -1./z *cam->focal_length;
  _jacobianOplusXj(1,5) = y/z_2 *cam->focal_length;
}
```

仔细研究此段代码，我们会发现它与式(7.46)和(7.48)是一致的。成员变量“_jacobianOplusXi”是误差到空间点的导数，“_jacobianOplusXj”是误差到相机位姿的导数，以李代数的左乘扰动表达。稍有差别的是，g2o 的相机里用 f 统一描述 f_x, f_y，并且李代数定义顺序不同（g2o 是旋转在前，平移在后；我们是平移在前，旋转在后），所以矩阵前 3 列和后 3 列与我们的定义是相反的，此外都一致。

值得一提的是，我们亦可自己实现相机位姿节点，并使用 Sophus::SE3 来表达位姿，提供类似的求导过程。然而，既然 g2o 已经提供了这样的类，在没有额外要求的情况下，自己重新实现就没有必要了。现在，我们在上一个 PnP 例程的基础上，加上 g2o 提供的 Bundle Adjustment。

slambook/ch7/pose_estimation_3d2d.cpp（片段）

```
void bundleAdjustment (
  const vector< Point3f > points_3d,
  const vector< Point2f > points_2d,
  const Mat& K,
  Mat& R, Mat& t )
{
  // 初始化g2o
  typedef g2o::BlockSolver< g2o::BlockSolverTraits<6,3> > Block;  // pose 维度为 6, landmark 维度为 3
  Block::LinearSolverType* linearSolver = new g2o::LinearSolverCSparse<Block::PoseMatrixType>();
  Block* solver_ptr = new Block( linearSolver );
  g2o::OptimizationAlgorithmLevenberg* solver = new g2o::OptimizationAlgorithmLevenberg( solver_ptr );
  g2o::SparseOptimizer optimizer;
  optimizer.setAlgorithm( solver );

  // vertex
  g2o::VertexSE3Expmap* pose = new g2o::VertexSE3Expmap(); // camera pose
  Eigen::Matrix3d R_mat;
  R_mat <<
    R.at<double>(0,0), R.at<double>(0,1), R.at<double>(0,2),
    R.at<double>(1,0), R.at<double>(1,1), R.at<double>(1,2),
    R.at<double>(2,0), R.at<double>(2,1), R.at<double>(2,2);
  pose->setId(0);
  pose->setEstimate( g2o::SE3Quat(
    R_mat,
    Eigen::Vector3d( t.at<double>(0,0), t.at<double>(1,0), t.at<double>(2,0))
  ) );
  optimizer.addVertex( pose );

  int index = 1;
  for ( const Point3f p:points_3d )   // landmarks
  {
    g2o::VertexSBAPointXYZ* point = new g2o::VertexSBAPointXYZ();
    point->setId( index++ );
    point->setEstimate( Eigen::Vector3d(p.x, p.y, p.z) );
    point->setMarginalized( true );
    optimizer.addVertex( point );
  }

  // parameter: camera intrinsics
  g2o::CameraParameters* camera = new g2o::CameraParameters(
```

```
  K.at<double>(0,0), Eigen::Vector2d(K.at<double>(0,2), K.at<double>(1,2)), 0
  );
  camera->setId(0);
  optimizer.addParameter( camera );

  // edges
  index = 1;
  for ( const Point2f p:points_2d )
  {
    g2o::EdgeProjectXYZ2UV* edge = new g2o::EdgeProjectXYZ2UV();
    edge->setId( index );
    edge->setVertex( 0, dynamic_cast<g2o::VertexSBAPointXYZ*> (optimizer.vertex(index)) );
    edge->setVertex( 1, pose );
    edge->setMeasurement( Eigen::Vector2d( p.x, p.y ) );
    edge->setParameterId(0,0);
    edge->setInformation( Eigen::Matrix2d::Identity() );
    optimizer.addEdge(edge);
    index++;
  }

  chrono::steady_clock::time_point t1 = chrono::steady_clock::now();
  optimizer.setVerbose( true );
  optimizer.initializeOptimization();
  optimizer.optimize(100);
  chrono::steady_clock::time_point t2 = chrono::steady_clock::now();
  chrono::duration<double> time_used = chrono::duration_cast<chrono::duration<double>>(t2-t1);
  cout<<"optimization costs time: "<<time_used.count()<<" seconds."<<endl;

  cout<<endl<<"after optimization:"<<endl;
  cout<<"T="<<endl<<Eigen::Isometry3d( pose->estimate() ).matrix()<<endl;
}
```

程序大体上和第 6 讲的 g2o 类似。我们首先声明了 g2o 图优化，并配置优化求解器和梯度下降方法。然后根据估计到的特征点，将位姿和空间点放到图中。最后调用优化函数进行求解。读者可以看到优化的结果如下：

```
calling bundle adjustment
iteration= 0  chi2= 1.083180 time= 0.000107183 cumTime= 0.000107183 edges= 76 schur= 1 lambda=
     78.907222 levenbergIter= 1
iteration= 1  chi2= 0.000798 time= 5.8615e-05  cumTime= 0.000165798 edges= 76 schur= 1 lambda=
     26.302407 levenbergIter= 1
iteration= 2  chi2= 0.000000 time= 3.0203e-05  cumTime= 0.000196001 edges= 76 schur= 1 lambda=
     17.534938 levenbergIter= 1
...... 中间过程略
iteration= 11 chi2= 0.000000 time= 2.8394e-05  cumTime= 0.000525203 edges= 76 schur= 1 lambda=
     11209.703029 levenbergIter= 1
optimization costs time: 0.00132938 seconds.

after optimization:
T=
```

```
0.997776 −0.0519476  0.0417755  −0.649778
0.050735   0.998274  0.0295806 −0.0545231
−0.0432401 −0.0273953   0.998689   0.295564
0          0          0          1
```

迭代 11 轮后，列文伯格—马夸尔特方法发现优化目标函数接近不变，于是停止优化。我们输出了最后得到的位姿变换矩阵 $\boldsymbol{T}$，对比之前直接做 PnP 的结果，大约在小数点后第 3 位发生了一些变化。这主要是由于我们同时优化了特征点和相机位姿导致的。

Bundle Adjustment 是一种通用的做法。它可以不限于两幅图像。我们完全可以放入多幅图像匹配到的位姿和空间点进行迭代优化，甚至可以把整个 SLAM 过程放进来。那种做法规模较大，主要在后端使用，我们会在第 10 讲再次遇到这个问题。在前端，我们通常考虑局部相机位姿和特征点的小型 Bundle Adjustment 问题，希望对它进行实时求解和优化。

7.9 3D−3D：ICP

最后，我们来介绍 3D−3D 的位姿估计问题。假设我们有一组配对好的 3D 点（比如我们对两幅 RGB-D 图像进行了匹配）：

$$\boldsymbol{P} = \{\boldsymbol{p}_1, \cdots, \boldsymbol{p}_n\}, \quad \boldsymbol{P}' = \{\boldsymbol{p}'_1, \cdots, \boldsymbol{p}'_n\},$$

现在，想要找一个欧氏变换 $\boldsymbol{R}, \boldsymbol{t}$，使得：

$$\forall i, \boldsymbol{p}_i = \boldsymbol{R}\boldsymbol{p}_i' + \boldsymbol{t}.$$

这个问题可以用迭代最近点（Iterative Closest Point，ICP）求解。读者应该注意到了，3D−3D 位姿估计问题中并没有出现相机模型，也就是说，仅考虑两组 3D 点之间的变换时，和相机并没有关系。因此，在激光 SLAM 中也会碰到 ICP，不过由于激光数据特征不够丰富，我们无从知道两个点集之间的**匹配关系**，只能认为距离最近的两个点为同一个，所以这个方法称为迭代最近点。而在视觉中，特征点为我们提供了较好的匹配关系，所以整个问题就变得更简单了。在 RGB-D SLAM 中，可以用这种方式估计相机位姿。下文我们用 ICP 指代**匹配好的**两组点间的运动估计问题。

和 PnP 类似，ICP 的求解也分为两种方式：利用线性代数的求解（主要是 SVD），以及利用非线性优化方式的求解（类似于 Bundle Adjustment）。下面分别进行介绍。

7.9.1　SVD 方法

首先来看以 SVD 为代表的代数方法。根据前面描述的 ICP 问题，我们先定义第 i 对点的误差项：

$$\boldsymbol{e}_i = \boldsymbol{p}_i - (\boldsymbol{R}\boldsymbol{p}_i' + \boldsymbol{t}). \tag{7.49}$$

然后，构建最小二乘问题，求使误差平方和达到极小的 $\boldsymbol{R}, \boldsymbol{t}$：

$$\min_{\boldsymbol{R},\boldsymbol{t}} J = \frac{1}{2}\sum_{i=1}^{n} \|(\boldsymbol{p}_i - (\boldsymbol{R}\boldsymbol{p}_i{}' + \boldsymbol{t}))\|_2^2. \tag{7.50}$$

下面来推导它的求解方法。首先，定义两组点的质心：

$$\boldsymbol{p} = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{p}_i), \quad \boldsymbol{p}' = \frac{1}{n}\sum_{i=1}^{n}(\boldsymbol{p}_i'). \tag{7.51}$$

请注意，质心是没有下标的。随后，在误差函数中做如下的处理：

$$\begin{aligned}
\tfrac{1}{2}\sum_{i=1}^{n} \|\boldsymbol{p}_i - (\boldsymbol{R}\boldsymbol{p}_i{}' + \boldsymbol{t})\|^2 &= \tfrac{1}{2}\sum_{i=1}^{n} \|\boldsymbol{p}_i - \boldsymbol{R}\boldsymbol{p}_i{}' - \boldsymbol{t} - \boldsymbol{p} + \boldsymbol{R}\boldsymbol{p}' + \boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}'\|^2 \\
&= \tfrac{1}{2}\sum_{i=1}^{n} \|(\boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i{}' - \boldsymbol{p}')) + (\boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t})\|^2 \\
&= \tfrac{1}{2}\sum_{i=1}^{n} (\|\boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i{}' - \boldsymbol{p}')\|^2 + \|\boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t}\|^2 + \\
&\quad 2(\boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i{}' - \boldsymbol{p}'))^T (\boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t})).
\end{aligned}$$

注意到交叉项部分中 $(\boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i{}' - \boldsymbol{p}'))$ 在求和之后为零，因此优化目标函数可以简化为

$$\min_{\boldsymbol{R},\boldsymbol{t}} J = \frac{1}{2}\sum_{i=1}^{n} \|\boldsymbol{p}_i - \boldsymbol{p} - \boldsymbol{R}(\boldsymbol{p}_i{}' - \boldsymbol{p}')\|^2 + \|\boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}' - \boldsymbol{t}\|^2. \tag{7.52}$$

仔细观察左右两项，我们发现左边只和旋转矩阵 $\boldsymbol{R}$ 相关，而右边既有 $\boldsymbol{R}$ 也有 $\boldsymbol{t}$，但只和质心相关。只要我们获得了 $\boldsymbol{R}$，令第二项为零就能得到 $\boldsymbol{t}$。于是，ICP 可以分为以下三个步骤求解：

1. 计算两组点的质心位置 $\boldsymbol{p}, \boldsymbol{p}'$，然后计算每个点的**去质心坐标**：

$$\boldsymbol{q}_i = \boldsymbol{p}_i - \boldsymbol{p}, \quad \boldsymbol{q}_i' = \boldsymbol{p}_i' - \boldsymbol{p}'.$$

2. 根据以下优化问题计算旋转矩阵：

$$\boldsymbol{R}^* = \arg\min_{\boldsymbol{R}} \frac{1}{2} \sum_{i=1}^{n} \left\| \boldsymbol{q}_i - \boldsymbol{R}\boldsymbol{q}_i' \right\|^2. \tag{7.53}$$

3. 根据第 2 步的 $\boldsymbol{R}$ 计算 $\boldsymbol{t}$：

$$\boldsymbol{t}^* = \boldsymbol{p} - \boldsymbol{R}\boldsymbol{p}'. \tag{7.54}$$

我们看到，只要求出了两组点之间的旋转，平移量是非常容易得到的。所以我们重点关注 $\boldsymbol{R}$ 的计算。展开关于 $\boldsymbol{R}$ 的误差项，得：

$$\frac{1}{2} \sum_{i=1}^{n} \left\| \boldsymbol{q}_i - \boldsymbol{R}\boldsymbol{q}_i' \right\|^2 = \frac{1}{2} \sum_{i=1}^{n} \boldsymbol{q}_i^{\mathrm{T}} \boldsymbol{q}_i + \boldsymbol{q}_i'^{\mathrm{T}} \boldsymbol{R}^{\mathrm{T}} \boldsymbol{R} \boldsymbol{q}_i' - 2\boldsymbol{q}_i^{\mathrm{T}} \boldsymbol{R} \boldsymbol{q}_i'. \tag{7.55}$$

注意到第一项和 $\boldsymbol{R}$ 无关，第二项由于 $\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R} = \boldsymbol{I}$，亦与 $\boldsymbol{R}$ 无关。因此，实际上优化目标函数变为

$$\sum_{i=1}^{n} -\boldsymbol{q}_i^{\mathrm{T}} \boldsymbol{R} \boldsymbol{q}_i' = \sum_{i=1}^{n} -\mathrm{tr}\left(\boldsymbol{R}\boldsymbol{q}_i'\boldsymbol{q}_i^{\mathrm{T}}\right) = -\mathrm{tr}\left(\boldsymbol{R} \sum_{i=1}^{n} \boldsymbol{q}_i'\boldsymbol{q}_i^{\mathrm{T}}\right). \tag{7.56}$$

接下来，我们介绍怎样通过 SVD 解出上述问题中最优的 $\boldsymbol{R}$。关于最优性的证明较为复杂，感兴趣的读者请参考文献 [60, 61]。为了解 $\boldsymbol{R}$，先定义矩阵：

$$\boldsymbol{W} = \sum_{i=1}^{n} \boldsymbol{q}_i \boldsymbol{q}_i'^{\mathrm{T}}. \tag{7.57}$$

$\boldsymbol{W}$ 是一个 3×3 的矩阵，对 $\boldsymbol{W}$ 进行 SVD 分解，得：

$$\boldsymbol{W} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathrm{T}}. \tag{7.58}$$

其中，$\boldsymbol{\Sigma}$ 为奇异值组成的对角矩阵，对角线元素从大到小排列，而 $\boldsymbol{U}$ 和 $\boldsymbol{V}$ 为对角矩阵。当 $\boldsymbol{W}$ 满秩时，$\boldsymbol{R}$ 为

$$\boldsymbol{R} = \boldsymbol{U}\boldsymbol{V}^{\mathrm{T}}. \tag{7.59}$$

解得 $\boldsymbol{R}$ 后，按式(7.54)求解 $\boldsymbol{t}$ 即可。

7.9.2　非线性优化方法

求解 ICP 的另一种方式是使用非线性优化，以迭代的方式去找最优值。该方法和我们前面讲述的 PnP 非常相似。以李代数表达位姿时，目标函数可以写成

$$\min_{\boldsymbol{\xi}} = \frac{1}{2}\sum_{i=1}^{n} \left\| (\boldsymbol{p}_i - \exp\left(\boldsymbol{\xi}^\wedge\right) \boldsymbol{p}_i' \right\|_2^2. \tag{7.60}$$

单个误差项关于位姿的导数在前面已推导，使用李代数扰动模型即可：

$$\frac{\partial \boldsymbol{e}}{\partial \delta \boldsymbol{\xi}} = -\left(\exp\left(\boldsymbol{\xi}^\wedge\right) \boldsymbol{p}_i'\right)^\odot. \tag{7.61}$$

于是，在非线性优化中只需不断迭代，就能找到极小值。而且，可以证明[6]，ICP 问题存在唯一解或无穷多解的情况。在唯一解的情况下，只要能找到极小值解，那么**这个极小值就是全局最优值**——因此不会遇到局部极小而非全局最小的情况。这也意味着 ICP 求解可以任意选定初始值。这是已匹配点时求解 ICP 的一大好处。

需要说明的是，我们这里讲的 ICP 是指已由图像特征给定了匹配的情况下进行位姿估计的问题。在匹配已知的情况下，这个最小二乘问题实际上具有解析解[62–64]，所以并没有必要进行迭代优化。ICP 的研究者们往往更加关心匹配未知的情况。不过，在 RGB-D SLAM 中，由于一个像素的深度数据可能测量不到，所以我们可以混合着使用 PnP 和 ICP 优化：对于深度已知的特征点，建模它们的 3D−3D 误差；对于深度未知的特征点，则建模 3D−2D 的重投影误差。于是，可以将所有的误差放在同一个问题中考虑，使得求解更加方便。

7.10　实践：求解 ICP

7.10.1　SVD 方法

下面演示一下如何使用 SVD 及非线性优化来求解 ICP。本节我们使用两幅 RGB-D 图像，通过特征匹配获取两组 3D 点，最后用 ICP 计算它们的位姿变换。由于 OpenCV 目前还没有计算两组带匹配点的 ICP 的方法，而且它的原理也并不复杂，所以我们自己来实现一个 ICP。

slambook/ch7/pose_estimation_3d3d.cpp（片段）

```
void pose_estimation_3d3d(
  const vector<Point3f>& pts1,
  const vector<Point3f>& pts2,
  Mat& R, Mat& t
)
{
  Point3f p1, p2;     // center of mass
  int N = pts1.size();
  for ( int i=0; i<N; i++ )
```

```
  {
    p1 += pts1[i];
    p2 += pts2[i];
  }
  p1 /= N;    p2 /= N;
  vector<Point3f>     q1(N), q2(N); // remove the center
  for ( int i=0; i<N; i++ )
  {
    q1[i] = pts1[i] - p1;
    q2[i] = pts2[i] - p2;
  }

  // compute q1*q2^T
  Eigen::Matrix3d W = Eigen::Matrix3d::Zero();
  for ( int i=0; i<N; i++ )
  {
    W += Eigen::Vector3d( q1[i].x, q1[i].y, q1[i].z ) * Eigen::Vector3d( q2[i].x, q2[i].y, q2[i].z ).
        transpose();
  }
  cout<<"W="<<W<<endl;

  // SVD on W
  Eigen::JacobiSVD<Eigen::Matrix3d> svd(W, Eigen::ComputeFullU|Eigen::ComputeFullV);
  Eigen::Matrix3d U = svd.matrixU();
  Eigen::Matrix3d V = svd.matrixV();
  cout<<"U="<<U<<endl;
  cout<<"V="<<V<<endl;

  Eigen::Matrix3d R_ = U*(V.transpose());
  Eigen::Vector3d t_ = Eigen::Vector3d( p1.x, p1.y, p1.z ) - R_ * Eigen::Vector3d( p2.x, p2.y, p2.z );

  // convert to cv::Mat
  R = ( Mat_<double>(3,3) <<
    R_(0,0), R_(0,1), R_(0,2),
    R_(1,0), R_(1,1), R_(1,2),
    R_(2,0), R_(2,1), R_(2,2)
  );
  t = ( Mat_<double>(3,1) << t_(0,0), t_(1,0), t_(2,0) );
}
```

ICP 的实现方式和前文讲述的是一致的。我们调用 Eigen 进行 SVD，然后计算 $\boldsymbol{R}, \boldsymbol{t}$ 矩阵。我们输出了匹配后的结果，不过请注意，由于前面的推导是按照 $\boldsymbol{p}_i = \boldsymbol{R}\boldsymbol{p}'_i + \boldsymbol{t}$ 进行的，这里的 $\boldsymbol{R}, \boldsymbol{t}$ 是第二帧到第一帧的变换，与前面 PnP 部分是相反的。所以在输出结果中，我们同时打印了逆变换：

```
% build/pose_estimation_3d3d 1.png 2.png 1_depth.png 2_depth.png
-- Max dist : 95.000000
-- Min dist : 4.000000
一共找到了 79 组匹配点
3d-3d pairs: 74
W=  298.51 -14.1815  41.0456
```

```
−44.8208  107.825 −164.404
78.1978 −163.954  271.439
U=  0.474143  −0.880373 −0.0114952
−0.460275  −0.258979   0.849163
0.750556   0.397334   0.528006
V=  0.535211  −0.844064 −0.0332488
−0.434767  −0.309001    0.84587
0.724242   0.438263   0.532352
ICP via SVD results:
R = [0.9972395976914055, 0.05617039049497474, −0.04855998381307948;
−0.05598344580804095, 0.9984181433274515, 0.005202390798842771;
0.04877538920134394, −0.002469474885032297, 0.998806719591959]
t = [0.7086246277241892;
−0.2775515782948791;
−0.1559573762377209]
R_inv = [0.9972395976914055, −0.05598344580804095, 0.04877538920134394;
0.05617039049497474, 0.9984181433274515, −0.002469474885032297;
−0.04855998381307948, 0.005202390798842771, 0.998806719591959]
t_inv = [−0.7145999506834847;
0.2369236766013986;
0.1916260075851286]
```

读者可以比较一下 ICP 与 PnP、对极几何的运动估计结果之间的差异。可以认为，在这个过程中我们使用了越来越多的信息（没有深度—有一个图的深度—有两个图的深度），因此，在深度准确的情况下，得到的估计也将越来越准确。但是，由于 Kinect 的深度图存在噪声，而且有可能存在数据丢失的情况，使得我们不得不丢弃一些没有深度数据的特征点。这可能导致 ICP 的估计不够准确，并且，如果特征点丢弃得太多，可能引起由于特征点太少，无法进行运动估计的情况。

7.10.2　非线性优化方法

下面考虑用非线性优化来计算 ICP。我们依然使用李代数来表达相机位姿。与 SVD 思路不同的地方在于，在优化中我们不仅考虑相机的位姿，同时会优化 3D 点的空间位置。对我们来说，RGB-D 相机每次可以观测到路标点的三维位置，从而产生一个 3D 观测数据。不过，由于 g2o/sba 中没有提供 3D 到 3D 的边，而我们又想使用 g2o/sba 中李代数实现的位姿节点，所以最好的方式是自定义一种这样的边，并向 g2o 提供解析求导方式。

slambook/ch7/pose_estimation_3d3d.cpp

```
class EdgeProjectXYZRGBDPoseOnly : public g2o::BaseUnaryEdge<3, Eigen::Vector3d, g2o::VertexSE3Expmap>
{
public:
  EIGEN_MAKE_ALIGNED_OPERATOR_NEW;
  EdgeProjectXYZRGBDPoseOnly( const Eigen::Vector3d& point ) :
    _point(point) {}

  virtual void computeError()
  {
    const g2o::VertexSE3Expmap* pose = static_cast<const g2o::VertexSE3Expmap*> ( _vertices[0] );
```

```
    // measurement is p, point is p'
    _error = _measurement - pose->estimate().map( _point );
  }

  virtual void linearizeOplus()
  {
    g2o::VertexSE3Expmap* pose = static_cast<g2o::VertexSE3Expmap *>(_vertices[0]);
    g2o::SE3Quat T(pose->estimate());
    Eigen::Vector3d xyz_trans = T.map(_point);
    double x = xyz_trans[0];
    double y = xyz_trans[1];
    double z = xyz_trans[2];

    _jacobianOplusXi(0,0) = 0;
    _jacobianOplusXi(0,1) = -z;
    _jacobianOplusXi(0,2) = y;
    _jacobianOplusXi(0,3) = -1;
    _jacobianOplusXi(0,4) = 0;
    _jacobianOplusXi(0,5) = 0;

    _jacobianOplusXi(1,0) = z;
    _jacobianOplusXi(1,1) = 0;
    _jacobianOplusXi(1,2) = -x;
    _jacobianOplusXi(1,3) = 0;
    _jacobianOplusXi(1,4) = -1;
    _jacobianOplusXi(1,5) = 0;

    _jacobianOplusXi(2,0) = -y;
    _jacobianOplusXi(2,1) = x;
    _jacobianOplusXi(2,2) = 0;
    _jacobianOplusXi(2,3) = 0;
    _jacobianOplusXi(2,4) = 0;
    _jacobianOplusXi(2,5) = -1;
  }

  bool read ( istream& in ) {}
  bool write ( ostream& out ) const {}
protected:
  Eigen::Vector3d _point;
};
```

这是一个一元边，写法类似于前面提到的 g2o::EdgeSE3ProjectXYZ，不过观测量从 2 维变成了 3 维，内部没有相机模型，并且只关联到一个节点。请读者注意这里雅可比矩阵的书写，它必须与我们前面的推导一致。雅可比矩阵给出了关于相机位姿的导数，是一个 3×6 的矩阵。

调用 g2o 进行优化的代码是相似的，我们设定好图优化的节点和边即可。这部分代码请读者查看源文件，这里不再列出。现在，来看看优化的结果：

```
calling bundle adjustment
iteration= 0 chi2= 452884.696837 time= 3.8443e-05 cumTime= 3.8443e-05  edges= 74 schur= 0
```

```
iteration= 1 chi2= 452762.638918 time= 1.436e-05  cumTime= 5.2803e-05  edges= 74 schur= 0
iteration= 2 chi2= 452762.618632 time= 1.1943e-05
...... 中间略
iteration= 9 chi2= 452762.618615 time= 1.0772e-05 cumTime= 0.000140108 edges= 74 schur= 0
optimization costs time: 0.000528066 seconds.

after optimization:
T=
0.99724   0.0561704    -0.04856    0.708625
-0.0559834    0.998418  0.00520239   -0.277551
0.0487754 -0.00246948    0.998807   -0.155957
0           0           0           1
```

我们发现，只迭代一次后总体误差就已经稳定不变，说明仅在一次迭代之后算法即已收敛。从位姿求解的结果可以看出，它和前面 SVD 给出的位姿结果几乎一模一样，这说明 SVD 已经给出了优化问题的解析解。所以，本实验中可以认为 SVD 给出的结果是相机位姿的最优值。

需要说明的是，在本例的 ICP 中，我们使用了在两个图都有深度读数的特征点。然而事实上，只要其中一个图深度确定，我们就能用类似于 PnP 的误差方式，把它们也加到优化中来。同时，除了相机位姿之外，将空间点也作为优化变量考虑，亦是一种解决问题的方式。我们应当清楚，实际的求解是非常灵活的，不必拘泥于某种固定的形式。如果同时考虑点和相机，整个问题就变得**更自由**了，你可能会得到其他的解。比如，可以让相机少转一些角度，而把点多移动一些。这从另一侧面反映出，在 Bundle Adjustment 里面，我们会希望有尽可能多的约束，因为多次观测会带来更多的信息，使我们能够更准确地估计每个变量。

7.11　小结

本讲介绍了基于特征点的视觉里程计中的几个重要的问题。包括：

1. 特征点是如何提取并匹配的。
2. 如何通过 2D−2D 的特征点估计相机运动。
3. 如何从 2D−2D 的匹配估计一个点的空间位置。
4. 3D−2D 的 PnP 问题，其线性解法和 Bundle Adjustment 解法。
5. 3D−3D 的 ICP 问题，其线性解法和 Bundle Adjustment 解法。

本讲内容较为丰富，且结合应用了前几讲的基本知识。读者若觉得理解有困难，可以对前面知识稍加回顾。最好亲自做一遍实验，以理解整个运动估计的内容。

需要解释的是，为保证行文流畅，我们省略了大量关于某些特殊情况的讨论。例如，如果在对极几何求解过程中给定的特征点共面，会发生什么情况（这在单应矩阵 $\boldsymbol{H}$ 中提到了）？共线又会发生什么情况？在 PnP 和 ICP 中若给定这样的解，又会导致什么情况？求解算法能否识别这些特殊的情况，并报告所得的解可能不可靠？尽管它们都是值得研究和探索的，然而对它们的讨论势必让本书变得特别烦琐。而且在工程实现中，这些情况甚少出现，所以本书介绍的方法，是指在实际工程

中能够有效运行的方法，我们假定了那些少见的情况并不发生。如果你关心这些少见的情况，可以阅读 [3] 等论文，在文献中我们经常会研究一些特殊情况下的解决方案。

习题

1. 除了本书介绍的 ORB 特征点外，你还能找到哪些特征点？请说说 SIFT 或 SURF 的原理，并对比它们与 ORB 之间的优劣。
2. 设计程序调用 OpenCV 中的其他种类特征点。统计在提取 1000 个特征点时在你的机器上所用的时间。
3.* 我们发现，OpenCV 提供的 ORB 特征点在图像当中分布不够均匀。你是否能够找到或提出让特征点分布更加均匀的方法？
4. 研究 FLANN 为何能够快速处理匹配问题。除了 FLANN 之外，还有哪些可以加速匹配的手段？
5. 把演示程序使用的 EPnP 改成其他 PnP 方法，并研究它们的工作原理。
6. 在 PnP 优化中，将第一个相机的观测也考虑进来，程序应如何书写？最后结果会有何变化？
7. 在 ICP 程序中，将空间点也作为优化变量考虑进来，程序应如何书写？最后结果会有何变化？
8.* 在特征点匹配过程中，不可避免地会遇到误匹配的情况。如果我们把错误匹配输入到 PnP 或 ICP 中，会发生怎样的情况？你能想到哪些避免误匹配的方法？
9.* 使用 Sophus 的 SE3 类，自己设计 g2o 的节点与边，实现 PnP 和 ICP 的优化。
10.* 在 Ceres 中实现 PnP 和 ICP 的优化。

第八章　SLAM：现在与未来

主要目标

1. 了解经典的 SLAM 实现方案。
2. 通过实验，比较各种 SLAM 方案的异同。
3. 探讨 SLAM 的未来发展方向。

前面介绍了一个 SLAM 系统中的各个模块的工作原理，这是研究者们多年工作的结晶。目前，除了这些理论框架之外，我们也积累了许多优秀的开源 SLAM 方案。不过，由于它们大部分的实现都比较复杂，不适合作为初学者的上手材料，所以我们放到了本书的最后加以介绍。相信读者通过阅读之前的内容，应该能明白其基本原理。

8.1 当前的开源方案

本讲是全书的总结，我们将带着读者去看看现有的 SLAM 方案能做到怎样的程度。特别地，我们重点关注那些提供开源实现的方案。在 SLAM 研究领域，能见到开源方案是很不容易的。往往论文中介绍理论只占 20% 的内容，其他 80% 都写在代码中，是论文里没有提到的。正是这些研究者们的无私奉献，推动了整个 SLAM 行业的快速前进，使后续研究者有了更高的起点。在我们开始做 SLAM 之前，应该对相似的方案有深入的了解，然后再进行自己的研究，这样才会更有意义。

本讲的前半部分将带领读者参观一下当前的视觉 SLAM 方案，评述其历史地位和优缺点。表 8-1 列举了一些常见的开源 SLAM 方案，读者可以选择感兴趣的方案进行研究和实验。限于篇幅，我们只选了一部分有代表性的方案，这肯定是不全面的。在后半部分，我们将探讨未来可能的一些发展方向，并给出当前的一些研究成果。

表 8-1 常用开源 SLAM 方案

方案名称	传感器形式	地址
MonoSLAM	单目	https://github.com/hanmekim/SceneLib2
PTAM	单目	http://www.robots.ox.ac.uk/~gk/PTAM/
ORB-SLAM	单目为主	http://webdiis.unizar.es/~raulmur/orbslam/
LSD-SLAM	单目为主	http://vision.in.tum.de/research/vslam/lsdslam
SVO	单目	https://github.com/uzh-rpg/rpg_svo
DTAM	RGB-D	https://github.com/anuranbaka/OpenDTAM
DVO	RGB-D	https://github.com/tum-vision/dvo_slam
DSO	单目	https://github.com/JakobEngel/dso
RTAB-MAP	双目/RGB-D	https://github.com/introlab/rtabmap
RGBD-SLAM-V2	RGB-D	https://github.com/felixendres/rgbdslam_v2
Elastic Fusion	RGB-D	https://github.com/mp3guy/ElasticFusion
Hector SLAM	激光	http://wiki.ros.org/hector_slam
GMapping	激光	http://wiki.ros.org/gmapping
OKVIS	多目 +IMU	https://github.com/ethz-asl/okvis
ROVIO	单目 +IMU	https://github.com/ethz-asl/rovio

8.1.1　MonoSLAM

说到视觉 SLAM，很多研究者第一个想到的是 A. J. Davison 的单目 SLAM 工作[2, 65]。Davison 教授是视觉 SLAM 研究领域的先驱，他在 2007 年提出的 MonoSLAM 是第一个实时的单目视觉 SLAM 系统[2]，被认为是许多工作的发源地[①]。MonoSLAM 以扩展卡尔曼滤波为后端，追踪前端非常稀疏的特征点。由于 EKF 在早期 SLAM 中占据着明显主导地位，所以 MonoSLAM 亦是建立在 EKF 的基础之上，以相机的当前状态和所有路标点为状态量，更新其均值和协方差。

图 8-1 所示是 MonoSLAM 在运行时的情形。可以看到，单目相机在一幅图像当中追踪了非常稀疏的特征点（且用到了主动追踪技术）。在 EKF 中，每个特征点的位置服从高斯分布，所以我们能够以一个椭球的形式表达它的均值和不确定性。在该图的右半部分，我们可以找到一些在空间中分布着的小球。它们在某个方向上显得越长，说明在该方向的位置就越不确定。我们可以想象，如果一个特征点收敛，我们应该能看到它从一个很长的椭球（相机 Z 方向上非常不确定）最后变成一个小点的样子。

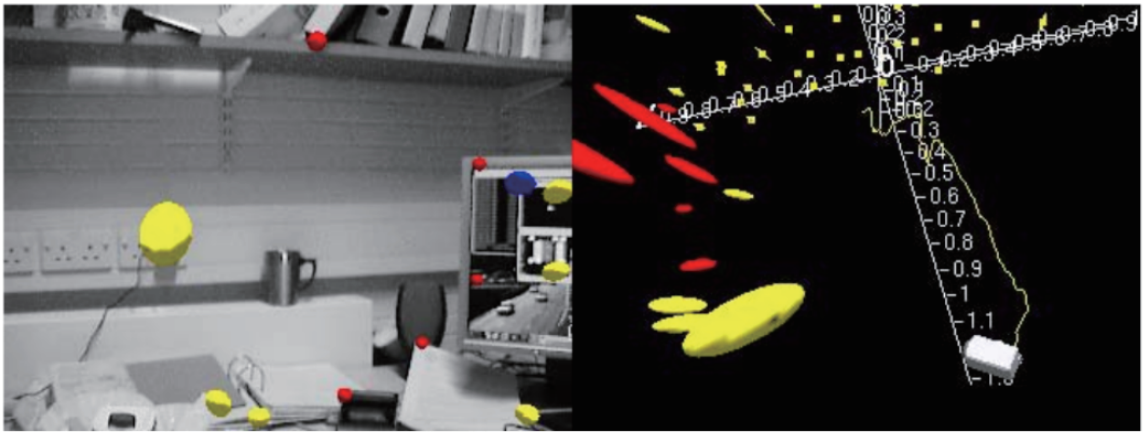


图 8-1　MonoSLAM 的运行时截图。左侧：追踪特征点在图像中的表示；右侧：特征点在三维空间中的表示。

这种做法在今天看来固然存在许多弊端，但在当时已经是里程碑式的工作了，因为在此之前的视觉 SLAM 系统基本不能在线运行，只能靠机器人携带相机采集数据，再离线地进行定位与建图。计算机性能的进步，以及用稀疏的方式处理图像，加在一起才使得一个 SLAM 系统能够在线地运行。从现代的角度来看，MonoSLAM 存在诸如应用场景很窄，路标数量有限，稀疏特征点非常容易丢失的情况，对它的开发也已经停止，取而代之的是更先进的理论和编程工具。不过这并不妨碍我们对前人工作的理解和尊敬。

[①] 这是他博士期间工作的延续。他现在也在致力于将 SLAM 小型化、低功率化。

8.1.2 PTAM

2007 年，Klein 等人提出了 PTAM（Parallel Tracking and Mapping）[66]，这也是视觉 SLAM 发展过程中的重要事件。PTAM 的重要意义在于以下两点：

1. PTAM 提出并实现了跟踪与建图过程的并行化。我们现在已然清楚，跟踪部分需要实时响应图像数据，而对地图的优化则没必要实时地计算。后端优化可以在后台慢慢进行，然后在必要的时候进行线程同步即可。这是视觉 SLAM 中首次区分出前后端的概念，引领了后来许多视觉 SLAM 系统的设计（我们现在看到的 SLAM 多半都分前后端）。
2. PTAM 是第一个使用非线性优化，而不是使用传统的滤波器作为后端的方案。它引入了关键帧机制：我们不必精细地处理每一幅图像，而是把几个关键图像串起来，然后优化其轨迹和地图。早期的 SLAM 大多数使用 EKF 滤波器或其变种，以及粒子滤波器等；在 PTAM 之后，视觉 SLAM 研究逐渐转向了以非线性优化为主导的后端。由于之前人们未认识到后端优化的稀疏性，所以觉得优化后端无法实时处理那样大规模的数据，而 PTAM 则是一个显著的反例。

   PTAM 同时是一个增强现实软件，演示了酷炫的 AR 效果（如图 8-2所示）。根据 PTAM 估计的相机位姿，我们可以在一个虚拟的平面上放置虚拟物体，看起来就像在真实的场景中一样。

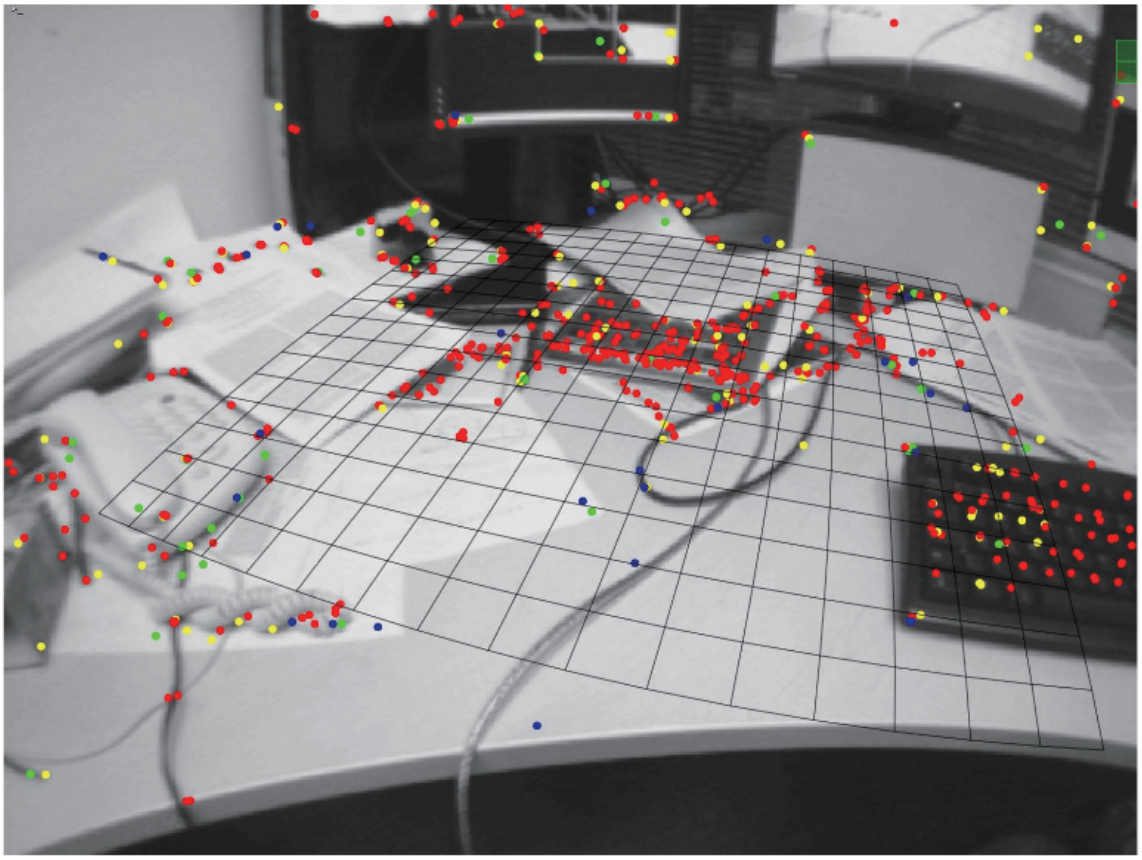


图 8-2　PTAM 的演示截图。它既可以提供实时的定位和建图，也可以在虚拟平面上叠加虚拟物体。

不过，从现代的眼光看来，PTAM 也算是早期的结合 AR 的 SLAM 工作之一。与许多早期工作相似，存在着明显的缺陷：场景小，跟踪容易丢失，等等。这些又在后续的方案中得以修正。

8.1.3　ORB-SLAM

介绍了历史上的几种方案之后，我们来看现代的一些 SLAM 系统。ORB-SLAM 是 PTAM 的继承者中非常有名的一位[67]（见图 8-3）。它提出于 2015 年，是现代 SLAM 系统中做得非常完善、非常易用的系统之一（如果不是最完善易用的话）。ORB-SLAM 代表着主流的特征点 SLAM 的一个高峰。相比于之前的工作，ORB-SLAM 具有以下几条明显的优势：

1. 支持单目、双目、RGB-D 三种模式。这使得无论我们拿到了哪种常见的传感器，都可以先放到 ORB-SLAM 上测试一下，它具有良好的泛用性。
2. 整个系统围绕 ORB 特征进行计算，包括视觉里程计与回环检测的 ORB 字典。它体现出 ORB 特征是现阶段计算平台的一种优秀的效率与精度之间的折中方式。ORB 不像 SIFT 或 SURF 那样费时，在 CPU 上面即可实时计算；相比 Harris 角点等简单角点特征，又具有良好的旋转和缩放不变性。并且，ORB 提供描述子，使我们在大范围运动时能够进行回环检测和重定位。
3. ORB 的回环检测是它的亮点。优秀的回环检测算法保证了 ORB-SLAM 有效地防止累积误差，并且在丢失之后还能迅速找回，这一点许多现有的 SLAM 系统都不够完善。为此，ORB-SLAM 在运行之前必须加载一个很大的 ORB 字典文件[①]。
4. ORB-SLAM 创新式地使用了三个线程完成 SLAM：实时跟踪特征点的 Tracking 线程，局部 Bundle Adjustment 的优化线程（Co-visibility Graph，俗称**小图**），以及全局 Pose Graph 的回环检测与优化线程（Essential Graph 俗称**大图**）。其中，Tracking 线程负责对每幅新来的图像提取 ORB 特征点，并与最近的关键帧进行比较，计算特征点的位置并粗略估计相机位姿。小图线程求解一个 Bundle Adjustment 问题，它包括局部空间内的特征点与相机位姿。这个线程负责求解更精细的相机位姿与特征点空间位置。不过，仅有前两个线程，只完成了一个比较好的视觉里程计。第三个线程，也就是大图线程，对全局的地图与关键帧进行回环检测，消除累积误差。由于全局地图中的地图点太多，所以这个线程的优化不包括地图点，而只有相机位姿组成的位姿图。

   继 PTAM 的双线程结构之后，ORB-SLAM 的三线程结构取得了非常好的跟踪和建图效果，能够保证轨迹与地图的全局一致性。这种三线程结构也将被后续的研究者认同和采用。
5. ORB-SLAM 围绕特征点进行了不少的优化。例如，在 OpenCV 的特征提取基础上保证了特征点的均匀分布，在优化位姿时使用了一种循环优化 4 遍以得到更多正确匹配的方法，比 PTAM 更为宽松的关键帧选取策略，等等。这些细小的改进使得 ORB-SLAM 具有远超其他方案的稳健性：即使对于较差的场景，较差的标定内参，ORB-SLAM 都能够顺利地工作。

[①] 目前开源版 ORB-SLAM 使用了文本格式的字典，改成二进制格式字典之后可以加速不少。

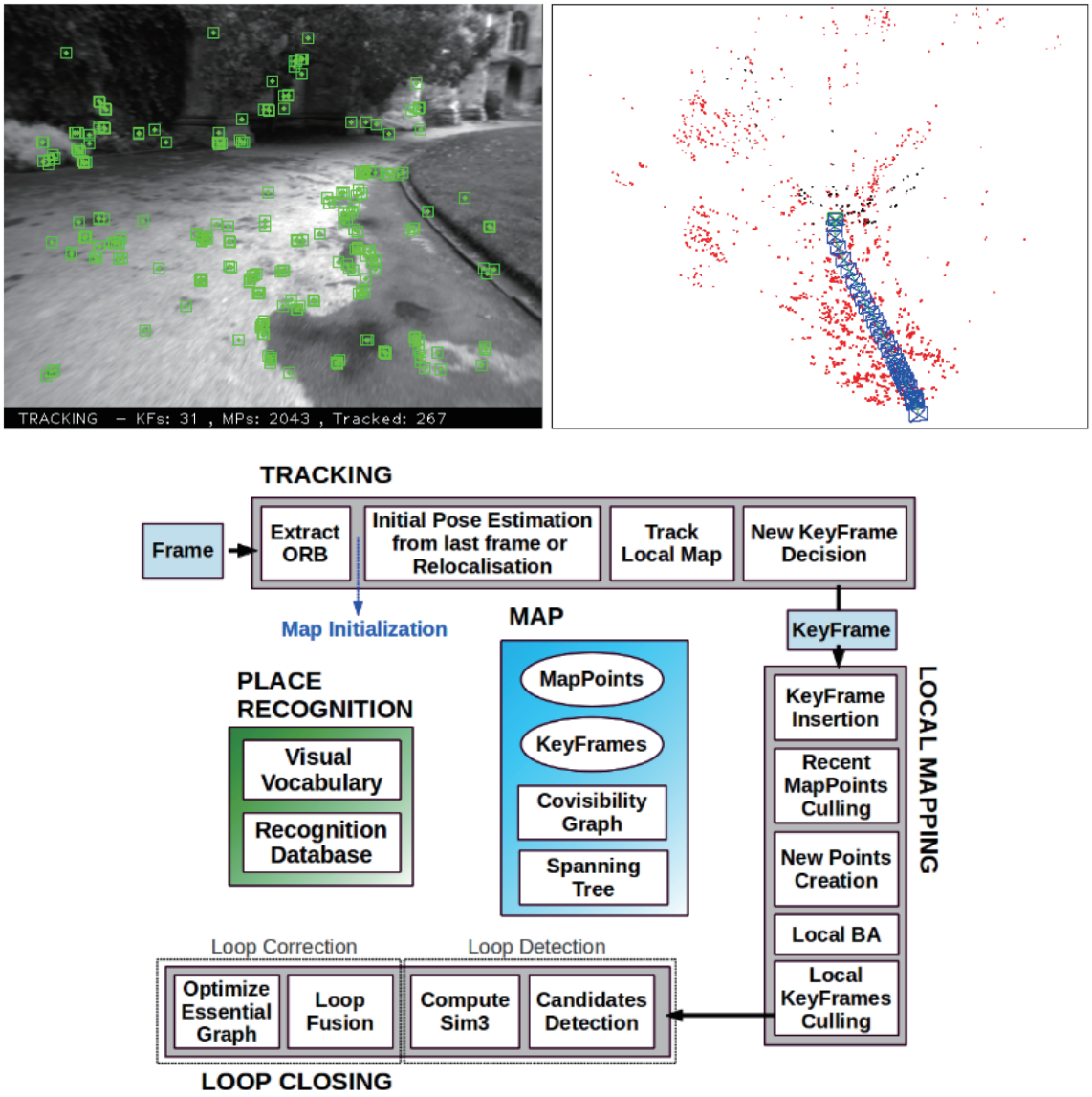


图 8-3 ORB-SLAM 运行截图。左侧为图像与追踪到的特征点，右侧为相机轨迹与建模的特征点地图。下方为其标志性的三线程结构。

上述这些优势使得 ORB-SLAM 在特征点 SLAM 中达到顶峰，许多研究工作都以 ORB-SLAM 作为标准，或者在它的基础上进行后续的开发。它的代码以清晰易读著称，有着完善的注释，可供后来的研究者进一步理解。

当然，ORB-SLAM 也存在一些不足之处。首先，由于整个 SLAM 系统都采用特征点进行计算，

我们必须对每幅图像都计算一遍 ORB 特征，这是非常耗时的。ORB-SLAM 的三线程结构也给 CPU 带来了较重的负担，使得它只有在当前 PC 架构的 CPU 上才能实时运算，移植到嵌入式设备上则有一定困难。其次，ORB-SLAM 的建图为稀疏特征点，目前还没有开放存储和读取地图后重新定位的功能（虽然从实现上来讲并不困难）。根据我们在建图部分的分析，稀疏特征点地图只能满足我们对定位的需求，而无法提供导航、避障、交互等诸多功能。然而，如果我们仅用 ORB-SLAM 处理定位问题，似乎又显得有些过于重量级了。相比之下，另外一些方案提供了更为轻量级的定位，使我们能够在低端的处理器上运行 SLAM，或者让 CPU 有余力处理其他的事务。

8.1.4　LSD-SLAM

LSD-SLAM（Large Scale Direct monocular SLAM）是 J. Engel 等人于 2014 年提出的 SLAM 工作[68, 69]。类比于 ORB-SLAM 之于特征点，LSD-SLAM 则标志着单目直接法在 SLAM 中的成功应用。LSD-SLAM 的核心贡献是将直接法应用到了半稠密的单目 SLAM 中。它不仅不需要计算特征点，还能构建半稠密的地图——这里半稠密的意思主要是指估计梯度明显的像素位置。它的主要优点如下：

1. LSD-SLAM 的直接法是针对像素进行的。作者有创见地提出了像素梯度与直接法的关系，以及像素梯度与极线方向在稠密重建中的角度关系。这些在本书的第 8 讲和第 13 讲均有讨论。不过，LSD-SLAM 是在单目图像进行半稠密的跟踪，实现原理要比本书的例程更加复杂。
2. LSD-SLAM 在 CPU 上实现了半稠密场景的重建，这在之前的方案中是很少见到的。基于特征点的方法只能是稀疏的，而进行稠密重建的方案大多要使用 RGB-D 传感器，或者使用 GPU 构建稠密地图[70]。TUM 计算机视觉组在多年对直接法研究的基础上，实现了这种 CPU 上的实时半稠密 SLAM。
3. 之前也说过，LSD-SLAM 的半稠密追踪使用了一些精妙的手段来保证追踪的实时性与稳定性。例如，LSD-SLAM 既不是利用单个像素，也不是利用图像块，而是在极线上等距离取 5 个点，度量其 SSD；在深度估计时，LSD-SLAM 首先用随机数初始化深度，在估计完后又把深度均值归一化，以调整尺度；在度量深度不确定性时，不仅考虑三角化的几何关系，而且考虑了极线与深度的夹角，归纳成一个光度不确定性项；关键帧之间的约束使用了相似变换群及与之对应的李代数 $\zeta \in \mathfrak{sim}(3)$ 显式地表达出尺度，在后端优化中可以将不同尺度的场景考虑进来，减小了尺度飘移现象。

图 8-4 显示了 LSD 的运行情况。我们可以观察一下这种微妙的半稠密地图是怎样一种介于稀疏地图与稠密地图之间的形式。半稠密地图建模了灰度图中有明显梯度的部分，显示在地图中，很大一部分都是物体的边缘或表面上带纹理的部分。LSD-SLAM 对它们进行跟踪并建立关键帧，最后优化得到这样的地图。看起来比稀疏的地图具有更多的信息，但又不像稠密地图那样拥有完整的表面（稠密地图一般认为无法仅用 CPU 实现实时性）。

由于 LSD-SLAM 使用了直接法进行跟踪，所以它既有直接法的优点（对特征缺失区域不敏感），也继承了直接法的缺点。例如，LSD-SLAM 对相机内参和曝光非常敏感，并且在相机快速运动时容

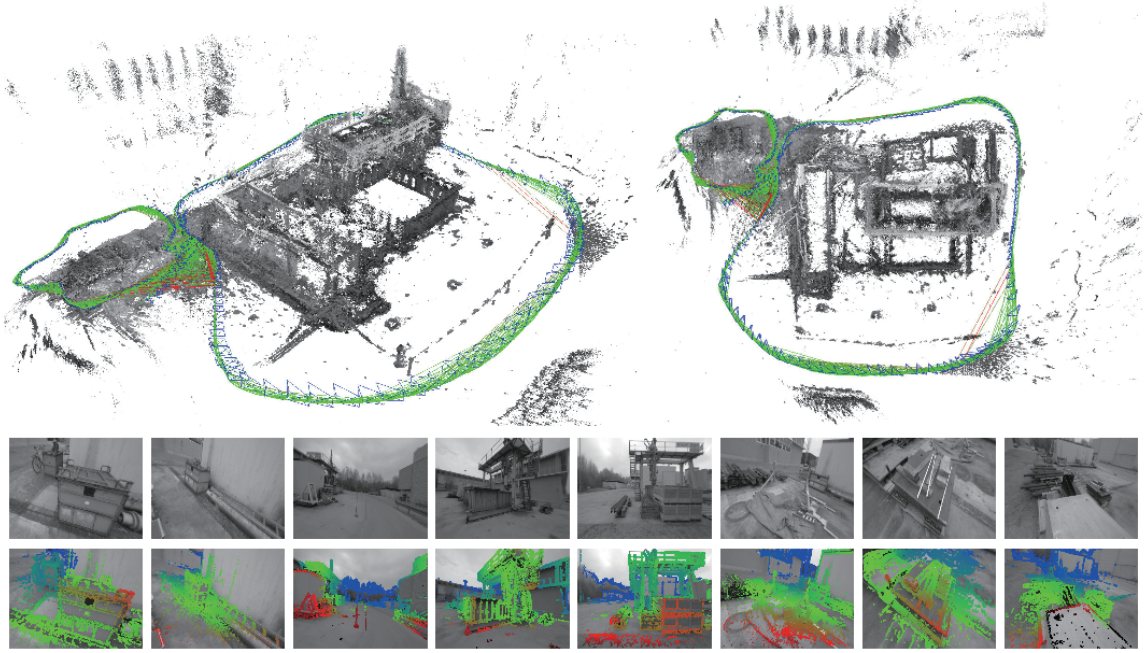


图 8-4 LSD-SLAM 运行图片。上半部分为估计的轨迹与地图，下半部分为图像中被建模的部分，即具有较好的像素梯度的部分。

易丢失。另外，在回环检测部分，由于目前并没有基于直接法实现的回环检测方式，因此 LSD-SLAM 必须依赖于特征点方法进行回环检测，尚未完全摆脱特征点的计算。

8.1.5 SVO

SVO 是 Semi-direct Visual Odoemtry 的缩写[71]。它是由 Forster 等人于 2014 年提出的一种基于稀疏直接法的视觉里程计。按作者的称呼应该叫“半直接”法，然而按照本书的理念框架，称为“稀疏直接法”可能更好一些。**半直接**在原文中的意思是指特征点与直接法的混合使用：SVO 跟踪了一些关键点（角点，没有描述子），然后像直接法那样，根据这些关键点周围的信息估计相机运动及其位置（如图 8-4所示）。在实现中，SVO 使用了关键点周围的 4×4 的小块进行块匹配，估计相机自身的运动。

相比于其他方案，SVO 的最大优势是速度极快。由于使用稀疏的直接法，它既不必费力去计算描述子，也不必处理像稠密和半稠密那么多的信息，因此，即使在低端计算平台上也能达到实时性，而在 PC 平台上则可以达到每秒 100 多帧的速度。在后续的 SVO 2.0 中，速度更达到了惊人的每秒 400 帧。这使得 SVO 非常适用于计算平台受限的场合，例如无人机、手持 AR/VR 设备的定位。无人机也是作者开发 SVO 的目标应用平台。

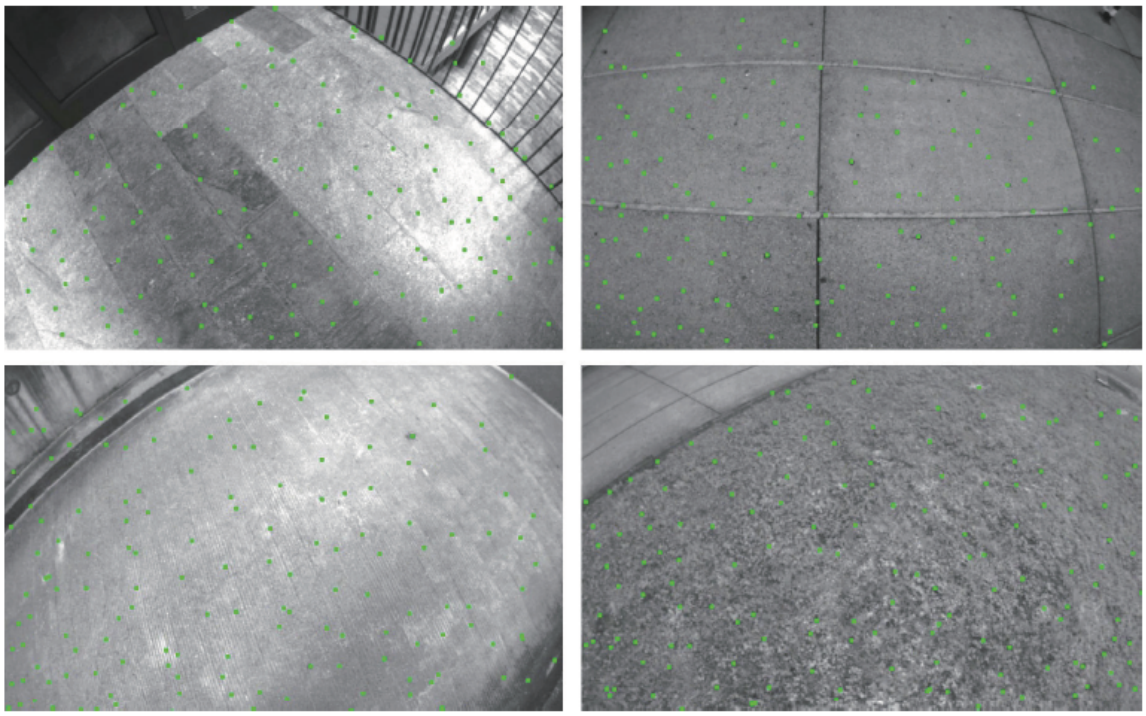


图 8-5　SVO 跟踪关键点的图片。

SVO 的另一创新之处是提出了深度滤波器的概念，并推导了基于均匀 – 高斯混合分布的深度滤波器。这在本书的第 13 讲有提及，但由于原理较为复杂，我们没有详细解释。SVO 将这种滤波器用于关键点的位置估计，并使用了逆深度作为参数化形式，使之能够更好地计算特征点位置。

开源版的 SVO 代码清晰易读，十分适合读者作为第一个 SLAM 实例进行分析。不过，开源版 SVO 也存在一些问题：

1. 由于目标应用平台为无人机的俯视相机，其视野内的物体主要是地面，而且相机的运动主要为水平和上下的移动，SVO 的许多细节是围绕这个应用设计的，这使得它在平视相机中表现不佳。例如，SVO 在单目初始化时，使用了分解 $\boldsymbol{H}$ 矩阵而不是传统的 $\boldsymbol{F}$ 或 $\boldsymbol{E}$ 矩阵的方式，这需要假设特征点位于平面上。该假设对俯视相机是成立的，但对平视相机通常是不成立的，可能导致初始化失败。再如，SVO 在关键帧选择时，使用了平移量作为确定新的关键帧的策略，而没有考虑旋转量。这同样在无人机俯视配置下是有效的，但在平视相机中则会容易丢失。所以，如果读者想要在平视相机中使用 SVO，必须自己加以修改。
2. SVO 为了速度和轻量化，舍弃了后端优化和回环检测部分，也基本没有建图功能。这意味着 SVO 的位姿估计必然存在累积误差，而且丢失后不太容易进行重定位（因为没有描述子用来回环检测）。所以，我们称它为一个 VO，而不是称它为完整的 SLAM。

8.1.6 RTAB-MAP

介绍了几款单目 SLAM 方案后，我们再来看一些 RGB-D 传感器上的 SLAM 方案。相比于单目和双目，RGB-D SLAM 的原理要简单很多（尽管实现上不一定），而且能够在 CPU 上实时建立稠密的地图。

RTAB-MAP（Real Time Appearance-Based Mapping）[72] 是 RGB-D SLAM 中比较经典的一个方案。它实现了 RGB-D SLAM 中所有应该有的东西：基于特征的视觉里程计、基于词袋的回环检测、后端的位姿图优化，以及点云和三角网格地图。因此，RTAB-MAP 给出了一套完整的（但有些庞大的）RGB-D SLAM 方案。目前我们已经可以直接从 ROS 中获得其二进制程序，此外，在 Google Project Tango 上也可以获取其 App 使用（如图 8-6 所示）。

图 8-6 RTAB-MAP 在 Google Project Tango 上的运行样例。

RTAB-MAP 支持一些常见的 RGB-D 和双目传感器，像 Kinect、Xtion 等，且提供实时的定位和建图功能。不过由于集成度较高，使得其他开发者在它的基础上进行二次开发变得困难，所以 RTAB-MAP 更适合作为 SLAM 应用而非研究使用。

8.1.7　其他

除了这些开源方案之外，读者还能在openslam.org之类的网站上找到许多其他的研究，例如，DVO-SLAM[73]、RGBD-SLAM-V2[74]、DSO[75]，以及一些 Kinect Fusion 相关的工作，等等。随着时代发展，更新颖、更优秀的开源 SLAM 作品亦将出现在人们的视野中，限于篇幅这里就不逐一介绍了。

8.2　未来的 SLAM 话题

看过了现有的方案，我们再来讨论一些未来的发展方向[①]。大体上讲，SLAM 将来的发展趋势有两大类：一是朝轻量级、小型化方向发展，让 SLAM 能够在嵌入式或手机等小型设备上良好运行，然后考虑以它为底层功能的应用。毕竟，大部分场合中，我们的真正目的都是实现机器人、AR/VR 设备的功能，比如说运动、导航、教学、娱乐，而 SLAM 是为上层应用提供自身的一个位姿估计。在这些应用中，我们不希望 SLAM 占用所有计算资源，所以对 SLAM 的小型化和轻量化有非常强烈的需求。另一方面则是利用高性能计算设备，实现精密的三维重建、场景理解等功能。在这些应用中，我们的目的是完美地重建场景，而对于计算资源和设备的便携性则没有多大限制。由于可以利用 GPU，这个方向和深度学习亦有结合点。

8.2.1　视觉 + 惯性导航 SLAM

首先，我们要谈一个有很强应用背景的方向：视觉－惯性导航融合 SLAM 方案。实际的机器人也好，硬件设备也好，通常都不会只携带一种传感器，往往是多种传感器的融合。学术界的研究人员喜爱“大而且干净的问题”（Big Clean Problem），比如说仅用单个摄像头实现视觉 SLAM。但产业界的朋友们则更注重让算法更加实用，不得不面对一些复杂而琐碎的场景。在这种应用背景下，用视觉与惯性导航融合进行 SLAM 成为了一个关注热点。

惯性传感器（IMU）能够测量传感器本体的角速度和加速度，被认为与相机传感器具有明显的互补性，而且十分有潜力在融合之后得到更完善的 SLAM 系统[76]。为什么这么说呢？

1. IMU 虽然可以测得角速度和加速度，但这些量都存在明显的漂移（Drift），使得积分两次得到的位姿数据非常不可靠。好比说，我们将 IMU 放在桌上不动，用它的读数积分得到的位姿也会漂出十万八千里。但是，对于短时间内的快速运动，IMU 能够提供一些较好的估计。这正是相机的弱点。

[①] 这里有一部分是笔者个人的理解，不一定完全正确。

当运动过快时，（卷帘快门的）相机会出现运动模糊，或者两帧之间重叠区域太少以至于无法进行特征匹配，所以纯视觉 SLAM 非常害怕快速的运动。而有了 IMU，即使在相机数据无效的那段时间内，我们也能保持一个较好的位姿估计，这是纯视觉 SLAM 无法做到的。

2. 相比于 IMU，相机数据基本不会有漂移。如果相机放在原地固定不动，那么（在静态场景下）视觉 SLAM 的位姿估计也是固定不动的。所以，相机数据可以有效地估计并修正 IMU 读数中的漂移，使得在慢速运动后的位姿估计依然有效。
3. 当图像发生变化时，本质上我们没法知道是相机自身发生了运动，还是外界条件发生了变化，所以纯视觉 SLAM 难以处理动态的障碍物。而 IMU 能够感受到自己的运动信息，从某种程度上减轻动态物体的影响。

总而言之，我们看到 IMU 为快速运动提供了较好的解决方式，而相机又能在慢速运动下解决 IMU 的漂移问题——在这个意义下，它们二者是互补的。

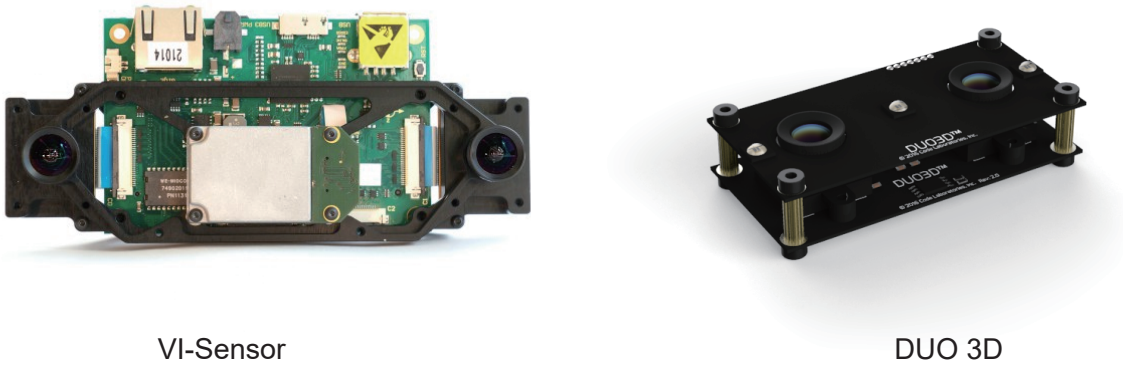


图 8-7　越来越多的相机开始集成 IMU 设备。

当然，虽然说得很好听，不管是理论还是实践，VIO（Visual Inertial Odometry）都是相当复杂的。其复杂性主要来源于 IMU 测量加速度和角速度这两个量的事实，所以不得不引入运动学计算。目前 VIO 的框架已经定型为两大类：松耦合（Loosely Coupled）和紧耦合（Tightly Coupled）[77]。松耦合是指 IMU 和相机分别进行自身的运动估计，然后对其位姿估计结果进行融合。紧耦合是指把 IMU 的状态与相机的状态合并在一起，共同构建运动方程和观测方程，然后进行状态估计——这和我们之前介绍的理论非常相似。我们可以预见，紧耦合理论也必将分为基于滤波和基于优化两个方向。在滤波方面，传统的 EKF[78] 以及改进的 MSCKF（Multi-State Constraint KF）[79] 都取得了一定的成果，研究者对 EKF 也进行了深入的讨论（例如能观性[80]）；优化方面亦有相应的方案[81, 82]。值得一提的是，尽管在纯视觉 SLAM 中优化方法已经占了主流，但在 VIO 中，由于 IMU 的数据频率非常高，对状态进行优化需要的计算量就更大，因此目前仍处于滤波与优化并存的阶段[83, 84]。由于过于复杂，限于篇幅，这里就只能大概地介绍一下这个方向了。

VIO 为将来 SLAM 的小型化与低成本化提供了一个非常有效的方向。而且结合稀疏直接法，我们有望在低端硬件上取得良好的 SLAM 或 VO 效果，是非常有前景的。

8.2.2 语义 SLAM

SLAM 的另一个大方向就是和深度学习技术结合。到目前为止，SLAM 的方案都处于特征点或者像素的层级。关于这些特征点或像素到底来自于什么东西，我们一无所知。这使得计算机视觉中的 SLAM 与我们人类的做法不怎么相似，至少我们自己从来看不到特征点，也不会去根据特征点判断自身的运动方向。我们看到的是一个个物体，通过左右眼判断它们的远近，然后基于它们在图像当中的运动推测相机的移动。

很久之前，研究者就试图将物体信息结合到 SLAM 中。例如文献 [85–88] 中就曾把物体识别与视觉 SLAM 结合起来，构建带物体标签的地图。另一方面，把标签信息引入到 BA 或优化端的目标函数和约束中，我们可以结合特征点的位置与标签信息进行优化[89]。这些工作都可以称为语义 SLAM。综合来说，SLAM 和语义的结合点主要有两个方面[9]：

1. 语义帮助 SLAM。传统的物体识别、分割算法往往只考虑一幅图，而在 SLAM 中我们拥有一台移动的相机。如果我们把运动过程中的图片都带上物体标签，就能得到一个带有标签的地图。另外，物体信息亦可为回环检测、BA 优化带来更多的条件。
2. SLAM 帮助语义。物体识别和分割都需要大量的训练数据。要让分类器识别各个角度的物体，需要从不同视角采集该物体的数据，然后进行人工标定，非常辛苦。而 SLAM 中，由于我们可以估计相机的运动，可以自动地计算物体在图像中的位置，节省人工标定的成本。如果有自动生成的带高质量标注的样本数据，能够很大程度上加速分类器的训练过程。

图 8-8　语义 SLAM 的一些结果，左图和右图分别来自文献 [88, 90]。

在深度学习广泛应用之前，我们只能利用支持向量机、条件随机场等传统工具对物体或场景进行分割和识别，或者直接将观测数据与数据库中的样本进行比较[90, 91]，尝试构建语义地图[88, 92–94]。由于这些工具本身在分类正确率上存在限制，所以效果也往往不尽如人意。随着深度学习的发展，我

们开始使用网络，越来越准确地对图像进行识别、检测和分割[95–100]。这为构建准确的语义地图打下了更好的基础[101]。我们正看到，逐渐开始有学者将神经网络方法引入到 SLAM 中的物体识别和分割，甚至 SLAM 本身的位姿估计与回环检测中[102–104]。虽然这些方法目前还没有成为主流，但将 SLAM 与深度学习结合来处理图像，亦是一个很有前景的研究方向。

8.2.3 SLAM 的未来

除此之外，基于线/面特征的 SLAM[105–107]、动态场景下的 SLAM[108–110]、多机器人的 SLAM[111–113]，等等，都是研究者感兴趣并发力的地方。按照文献 [9] 的观点，视觉 SLAM 经过了三个大时代：提出问题、寻找算法、完善算法。而我们目前正处于第三个时代，面对着如何在已有的框架中进一步改善，使视觉 SLAM 系统能够在各种干扰的条件下稳定运行。这一步需要许多研究者的不懈努力。

当然，没有人能够预测未来，我们也说不准会不会突然有一天，整个框架都被新的技术推倒重写。不过即使是那样，今天我们的付出仍将是有意义的。没有今天的研究，也就不会有将来的发展。最后，希望读者能在读完本书之后，对现有的整个 SLAM 系统有了充分的认识。我们也期待你能够为 SLAM 研究做出贡献！

习题

1. 选择本讲提到的任意一个开源 SLAM 系统，在你的机器上编译运行它，直观体验其过程。
2. 你应该已经能够看懂绝大多数 SLAM 相关论文了。拿起纸和笔，开始你的研究吧！

附录 A　高斯分布的性质

这里总结一下常见的高斯分布的性质，它在本书的很多地方都会用到。

A.1　高斯分布

如果一个随机变量 x 服从高斯分布 $N(\mu,\sigma)$，那么它的概率密度函数为

$$p\left(x\right)=\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{1}{2}\frac{\left(x-\mu\right)^{2}}{\sigma^{2}}\right). \tag{A.1}$$

其高维形式为

$$p\left(x\right)=\frac{1}{\sqrt{\left(2\pi\right)^{N}\det\left(\boldsymbol{\Sigma}\right)}}\exp\left(-\frac{1}{2}\left(\boldsymbol{x}-\boldsymbol{\mu}\right)^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}\left(\boldsymbol{x}-\boldsymbol{\mu}\right)\right). \tag{A.2}$$

A.2　高斯分布的运算

A.2.1　线性运算

设两个独立的高斯分布：

$$\boldsymbol{x}\sim N(\boldsymbol{\mu}_{x},\boldsymbol{\Sigma}_{xx}),\quad \boldsymbol{y}\sim N(\boldsymbol{\mu}_{y},\boldsymbol{\Sigma}_{yy}),$$

那么，它们的和仍是高斯分布：

$$\boldsymbol{x}+\boldsymbol{y}\sim N(\boldsymbol{\mu}_{x}+\boldsymbol{\mu}_{y},\boldsymbol{\Sigma}_{xx}+\boldsymbol{\Sigma}_{yy}). \tag{A.3}$$

如果以常数 a 乘以 $\boldsymbol{x}$，那么 $a\boldsymbol{x}$ 满足：

$$a\boldsymbol{x}\sim N(a\boldsymbol{\mu}_{x},a^{2}\boldsymbol{\Sigma}_{xx}). \tag{A.4}$$

如果取 $\boldsymbol{y}=\boldsymbol{A}\boldsymbol{x}$，那么 $\boldsymbol{y}$ 满足：

$$\boldsymbol{y}\sim N(\boldsymbol{A}\boldsymbol{\mu}_{x},\boldsymbol{A}\boldsymbol{\Sigma}_{xx}\boldsymbol{A}^{\mathrm{T}}). \tag{A.5}$$

A.2.2 乘积

设两个高斯分布的乘积满足 $p(\boldsymbol{xy}) = N(\boldsymbol{\mu}, \boldsymbol{\Sigma})$，那么：

$$\begin{aligned} \boldsymbol{\Sigma}^{-1} &= \boldsymbol{\Sigma}_{xx}^{-1} + \boldsymbol{\Sigma}_{yy}^{-1} \\ \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} &= \boldsymbol{\Sigma}_{xx}^{-1}\boldsymbol{\mu}_x + \boldsymbol{\Sigma}_{yy}^{-1}\boldsymbol{\mu}_y. \end{aligned} \tag{A.6}$$

该公式可以推广到任意多个高斯分布之乘积。

A.2.3 复合运算

同样考虑 $\boldsymbol{x}$ 和 $\boldsymbol{y}$，若其不独立，则其复合分布为

$$p(\boldsymbol{x}, \boldsymbol{y}) = N\left(\begin{bmatrix} \boldsymbol{\mu}_x \\ \boldsymbol{\mu}_y \end{bmatrix}, \begin{bmatrix} \boldsymbol{\Sigma}_{xx} & \boldsymbol{\Sigma}_{xy} \\ \boldsymbol{\Sigma}_{yx} & \boldsymbol{\Sigma}_{yy} \end{bmatrix}\right). \tag{A.7}$$

由条件分布展开式 $p(\boldsymbol{x}, \boldsymbol{y}) = p(\boldsymbol{x}|\boldsymbol{y})\, p(\boldsymbol{y})$ 可以推出，条件概率 $p(\boldsymbol{x}|\boldsymbol{y})$ 满足：

$$p(\boldsymbol{x}|\boldsymbol{y}) = N\left(\boldsymbol{\mu}_x + \boldsymbol{\Sigma}_{xy}\boldsymbol{\Sigma}_{yy}^{-1}(\boldsymbol{y} - \boldsymbol{\mu}_y), \boldsymbol{\Sigma}_{xx} - \boldsymbol{\Sigma}_{xy}\boldsymbol{\Sigma}_{yy}^{-1}\boldsymbol{\Sigma}_{yx}\right). \tag{A.8}$$

A.3 复合的例子

下面举一个和卡尔曼滤波器相关的例子。考虑随机变量 $\boldsymbol{x} \sim N(\boldsymbol{\mu}_x, \boldsymbol{\Sigma}_{xx})$，另一变量 $\boldsymbol{y}$ 满足：

$$\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b} + \boldsymbol{w} \tag{A.9}$$

其中 $\boldsymbol{A}, \boldsymbol{b}$ 为线性变量的系数矩阵和偏移量，$\boldsymbol{w}$ 为噪声项，为零均值的高斯分布：$\boldsymbol{w} \sim N(\boldsymbol{0}, \boldsymbol{R})$。我们来看 $\boldsymbol{y}$ 的分布。根据前面的介绍，可以推出：

$$p(\boldsymbol{y}) = N\left(\boldsymbol{A}\boldsymbol{\mu}_x + \boldsymbol{b}, \boldsymbol{R} + \boldsymbol{A}\boldsymbol{\Sigma}_{xx}\boldsymbol{A}^{\mathrm{T}}\right). \tag{A.10}$$

这为卡尔曼滤波器的预测部分提供了理论基础。

附录 B　矩阵求导

B.0.1　标量函数对向量求导

B.0.2　标量函数对矩阵求导

B.0.3　向量函数对向量求导

附录 C　ROS 入门

ROS 是机器人研究领域一个广为探讨的主题。为了避免使本书阅读门槛太高，我们没有在正文和例程中提到它。然而近年来，ROS 正逐步在各大高校的学生中间得到推广，渐渐为人们所熟知和接受，所以这里也介绍一下 ROS，希望对读者能有所帮助。

C.1　ROS 是什么

ROS（Robot Operating System）是 Willow Garage 公司于 2007 年发布的一个开源机器人操作系统，它为软件开发人员开发机器人应用程序提供了许多优秀的工具和库。同时，还有优秀的开发者不断地为它贡献代码。本质上，ROS 并不是一个真正意义上的操作系统，而更像是基于操作系统之上的一个软件包。它提供了众多在实际机器人中可能遇到的算法：导航、通信、路径规划，等等。

ROS 的版本代号是按照字母顺序来编排的，并随着 Ubuntu 系统发布更新。通常一个 ROS 版本会支持两到三个 Ubuntu 系统版本。ROS 从 Box Turtle 开始，截止到本书写作时（2016 年），已经更新到了 Kinetic Kame（见图 C-1）。同时，ROS 也已经彻底重构，推出了实时性更强的 2.0 版本。

图 C-1　ROS 各版本命名方式。

ROS 支持很多操作系统，支持最完善的是 Ubuntu 及其衍生版本（Kubuntu、Linux Mint、Ubuntu GNOME 等），对其他 Linux 发布版本、Windows 等的支持也有，不过没有那么完善。因此，推荐读者使用 Ubuntu 操作系统来进行开发和研究。

ROS 支持目前被广泛使用的面向对象的编程语言 C++，以及脚本语言 Python。你可以选择自己喜欢的语言进行开发。

C.2 ROS 的特点

ROS 的设计初衷，就是使机器人开发能够像计算机开发一样，屏蔽底层硬件及其接口的不一致性，最终使得软件可以复用。

而软件复用也正是软件工程优美性最集中的体现之一，ROS 能够以统一消息格式来使得大家只需要关注算法层面的设计，而底层硬件的根本目的是接收各种各样的消息，如图像、数据等。各个硬件厂商将接收到的数据都统一到 ROS 所规定的统一消息格式下，即可让用户方便地使用各种开源的机器人相关算法。

在第 14 讲中提到的常见的开源 SLAM 方案中，ORB-SLAM、ORB-SLAM2、LSD-SLAM、SVO、DVO、RTAB-MAP、RGBD-SLAM-V2、Hector SLAM、Gmapping、ROVIO 等均有 ROS 版本的开源代码，你可以很方便地在 ROS 中运行、调试和修改它们。

在调试 SLAM 程序时，数据的来源通常有 3 种：传感器、数据集，以及 bag 文件。若手头没有相应的传感器，通常就需要利用虚拟的数据来跑 SLAM 程序。其中，最方便的方式当属利用 ROS 下的 bag 文件发布 topic，然后 SLAM 程序就可以监视 topic 发出的数据，就像使用真实的传感器采集数据一样。后面我们会简单介绍一下如何利用 bag 文件来模拟真实的传感器数据。

C.3 如何快速上手 ROS

ROS 有完善的维基系统。因而，按照官网的介绍在机器上安装对应版本的 ROS：`http://wiki.ros.org/ROS/Installation`；然后，阅读 ROS 自带的教学程序即可。你会学习到 ROS 的基本概念、主题的发布和订阅，以及用 Python 和 C++ 控制它们。如果你觉得麻烦，也可以使用针对 ROS 定制的 Ubuntu：`http://www.aicrobo.com/ubuntu_for_ros.html`。

除了基本知识之外，你还可以学到一些 ROS 的常用工具，例如：

1. rqt。rqt 是 ROS 下的一个软件框架，它以插件的方式提供了各种各样方便好用的 GUI（用户图形界面）。rqt 的功能非常强大，可以实时地查看 ROS 中流动的消息。
2. rosbag。rosbag 是 ROS 提供的一个非常好用的录制及播放 topic 数据的工具。当你想实际跑一下 SLAM 程序，但囿于手头没有实际的传感器时，可以考虑使用公开提供的 bag 文件来进行图像或者数据的模拟，这种方式与使用一个真实的传感器在感觉上并无不同。rosbag 的使用方式请参考 ROS 的维基页面。此外，许多公开数据集也会提供 bag 格式的数据文件。

3. rviz。rviz 是 ROS 提供的可视化模块，你可以通过它实时地查看 ROS 中的图像、点云、地图、规划的路径，等等，从而更方便地调试程序。

我们相信，机器人的硬件层面和软件层面一定都会向着统一架构的方向前行，而 ROS 正是软件架构层面标准化一个重要的里程碑。其中，ROS 1.x 在之前被大量用于实验室的研究，或者公司产品 demo 的研发阶段，而 ROS2 则解决了 ROS 实时性的问题，未来很有可能被直接用于实际产品的研发，为推进工业级机器人和服务机器人的应用做出重要的贡献。

本附录概述性地介绍了有关 ROS 的历史、优点，以及如何利用 ROS 中的一些可视化工具来辅助 SLAM 程序开发等。我们希望读者系统地学习 ROS，并使用 ROS 开发自己的 SLAM 程序。

参考文献

[1] L. Haomin, Z. Guofeng, and B. Hujun, "A survey of monocular simultaneous localization and mapping," *Journal of Computer-Aided Design and Compute Graphics*, vol. 28, no. 6, pp. 855–868, 2016. in Chinese.

[2] A. Davison, I. Reid, N. Molton, and O. Stasse, "Monoslam: Real-time single camera SLAM," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 29, no. 6, pp. 1052–1067, 2007.

[3] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*. Cambridge university press, 2003.

[4] R. C. Smith and P. Cheeseman, "On the representation and estimation of spatial uncertainty," *International Journal of Robotics Research*, vol. 5, no. 4, pp. 56–68, 1986.

[5] S. Thrun, W. Burgard, and D. Fox, *Probabilistic robotics*. MIT Press, 2005.

[6] T. Barfoot, "State estimation for robotics: A matrix lie group approach," 2016.

[7] A. Pretto, E. Menegatti, and E. Pagello, "Omnidirectional dense large-scale mapping and navigation based on meaningful triangulation," *2011 IEEE International Conference on Robotics and Automation (ICRA 2011)*, pp. 3289–96, 2011.

[8] B. Rueckauer and T. Delbruck, "Evaluation of event-based algorithms for optical flow with ground-truth from inertial measurement sensor," *Frontiers in neuroscience*, vol. 10, 2016.

[9] C. Cesar, L. Carlone, H. C., Y. Latif, D. Scaramuzza, J. Neira, I. D. Reid, and L. John J., "Past, present, and future of simultaneous localization and mapping: Towards the robust-perception age," *arXiv preprint arXiv:1606.05830*, 2016.

[10] P. Newman and K. Ho, "Slam-loop closing with visually salient features," in *proceedings of the 2005 IEEE International Conference on Robotics and Automation*, pp. 635–642, IEEE, 2005.

[11] R. Smith, M. Self, and P. Cheeseman, "Estimating uncertain spatial relationships in robotics," in *Autonomous robot vehicles*, pp. 167–193, Springer, 1990.

[12] P. Beeson, J. Modayil, and B. Kuipers, "Factoring the mapping problem: Mobile robot map-building in the hybrid spatial semantic hierarchy," *International Journal of Robotics Research*, vol. 29, no. 4, pp. 428–459, 2010.

[13] H. Strasdat, J. M. Montiel, and A. J. Davison, "Visual slam: Why filter?," *Image and Vision Computing*, vol. 30, no. 2, pp. 65–77, 2012.

[14] M. Liang, H. Min, and R. Luo, "Graph-based slam: A survey," *ROBOT*, vol. 35, no. 4, pp. 500–512, 2013. in Chinese.

[15] J. Fuentes-Pacheco, J. Ruiz-Ascencio, and J. M. Rendón-Mancha, "Visual simultaneous localization and mapping: a survey," *Artificial Intelligence Review*, vol. 43, no. 1, pp. 55–81, 2015.

[16] J. Boal, Á. Sánchez-Miralles, and Á. Arranz, "Topological simultaneous localization and mapping: a survey," *Robotica*, vol. 32, pp. 803–821, 2014.

[17] S. Y. Chen, "Kalman filter for robot vision: A survey," *IEEE Transactions on Industrial Electronics*, vol. 59, no. 11, pp. 4409–4420, 2012.

[18] Z. Chen, J. Samarabandu, and R. Rodrigo, "Recent advances in simultaneous localization and map-building using computer vision," *Advanced Robotics*, vol. 21, no. 3-4, pp. 233–265, 2007.

[19] J. Stuelpnagel, "On the parametrization of the three-dimensional rotation group," *SIAM Review*, vol. 6, no. 4, pp. 422–430, 1964.

[20] T. Barfoot, J. R. Forbes, and P. T. Furgale, "Pose estimation using linearized rotations and quaternion algebra," *Acta Astronautica*, vol. 68, no. 1-2, pp. 101–112, 2011.

[21] V. S. Varadarajan, *Lie groups, Lie algebras, and their representations*, vol. 102. Springer Science & Business Media, 2013.

[22] Y. Ma, S. Soatto, J. Kosecka, and S. S. Sastry, *An invitation to 3-d vision: from images to geometric models*, vol. 26. Springer Science & Business Media, 2012.

[23] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of rgb-d SLAM systems," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 573–580, IEEE, 2012.

[24] H. Strasdat, *Local accuracy and global consistency for efficient visual slam*. PhD thesis, Citeseer, 2012.

[25] Z. Zhang, "Flexible camera calibration by viewing a plane from unknown orientations," in *Computer Vision, 1999. The Proceedings of the Seventh IEEE International Conference on*, vol. 1, pp. 666–673, Ieee, 1999.

[26] H. Hirschmuller, "Stereo processing by semiglobal matching and mutual information," *IEEE Transactions on pattern analysis and machine intelligence*, vol. 30, no. 2, pp. 328–341, 2008.

[27] D. Scharstein and R. Szeliski, "A taxonomy and evaluation of dense two-frame stereo correspondence algorithms," *International journal of computer vision*, vol. 47, no. 1-3, pp. 7–42, 2002.

[28] S. M. Seitz, B. Curless, J. Diebel, D. Scharstein, and R. Szeliski, “A comparison and evaluation of multi-view stereo reconstruction algorithms,” in *null*, pp. 519–528, IEEE, 2006.

[29] S. Agarwal, N. Snavely, I. Simon, S. M. Seitz, and R. Szeliski, “Building rome in a day,” in *2009 IEEE 12th international conference on computer vision*, pp. 72–79, IEEE, 2009.

[30] P. Wolfe, “Convergence conditions for ascent methods,” *SIAM review*, vol. 11, no. 2, pp. 226–235, 1969.

[31] J. Nocedal and S. Wright, *Numerical Optimization*. Springer Science & Business Media, 2006.

[32] M. I. Lourakis and A. A. Argyros, “Sba: A software package for generic sparse bundle adjustment,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 36, no. 1, p. 2, 2009.

[33] G. Sibley, “Relative bundle adjustment,” *Department of Engineering Science, Oxford University, Tech. Rep*, vol. 2307, no. 09, 2009.

[34] B. Triggs, P. F. McLauchlan, R. I. Hartley, and A. W. Fitzgibbon, “Bundle adjustment: a modern synthesis,” in *Vision algorithms: theory and practice*, pp. 298–372, Springer, 2000.

[35] S. Agarwal, K. Mierle, and Others, “Ceres solver.” `http://ceres-solver.org`.

[36] R. Kummerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, “G2o: a general framework for graph optimization,” in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3607–3613, IEEE, 2011.

[37] Wikipedia, “Feature (computer vision).” `"https://en.wikipedia.org/wiki/Feature_(computer_vision)"`, 2016. [Online; accessed 09-July-2016].

[38] C. Harris and M. Stephens, “A combined corner and edge detector.,” in *Alvey vision conference*, vol. 15, pp. 10–5244, Citeseer, 1988.

[39] E. Rosten and T. Drummond, “Machine learning for high-speed corner detection,” in *Computer Vision–ECCV 2006*, pp. 430–443, Springer, 2006.

[40] J. Shi and C. Tomasi, “Good features to track,” in *Computer Vision and Pattern Recognition, 1994. Proceedings CVPR’94., 1994 IEEE Computer Society Conference on*, pp. 593–600, IEEE, 1994.

[41] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[42] H. Bay, T. Tuytelaars, and L. Van Gool, “Surf: Speeded up robust features,” in *Computer Vision–ECCV 2006*, pp. 404–417, Springer, 2006.

[43] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “Orb: an efficient alternative to sift or surf,” in *2011 IEEE International Conference on Computer Vision (ICCV)*, pp. 2564–2571, IEEE, 2011.

[44] M. Calonder, V. Lepetit, C. Strecha, and P. Fua, "Brief: Binary robust independent elementary features," in *European conference on computer vision*, pp. 778–792, Springer, 2010.

[45] M. Nixon and A. S. Aguado, *Feature extraction and image processing for computer vision*. Academic Press, 2012.

[46] P. L. Rosin, "Measuring corner properties," *Computer Vision and Image Understanding*, vol. 73, no. 2, pp. 291–307, 1999.

[47] M. Muja and D. G. Lowe, "Fast approximate nearest neighbors with automatic algorithm configuration.," in *VISAPP (1)*, pp. 331–340, 2009.

[48] R. I. Hartley, "In defense of the eight-point algorithm," *IEEE Transactions on pattern analysis and machine intelligence*, vol. 19, no. 6, pp. 580–593, 1997.

[49] H. C. Longuet-Higgins, "A computer algorithm for reconstructing a scene from two projections," *Readings in Computer Vision: Issues, Problems, Principles, and Paradigms, MA Fischler and O. Firschein, eds*, pp. 61–62, 1987.

[50] H. Li and R. Hartley, "Five-point motion estimation made easy," in *18th International Conference on Pattern Recognition (ICPR'06)*, vol. 1, pp. 630–633, IEEE, 2006.

[51] D. Nistér, "An efficient solution to the five-point relative pose problem," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 26, no. 6, pp. 756–770, 2004.

[52] O. D. Faugeras and F. Lustman, "Motion and structure from motion in a piecewise planar environment," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 2, no. 03, pp. 485–508, 1988.

[53] Z. Zhang and A. R. Hanson, "3d reconstruction based on homography mapping," *ARPA Image Understanding Workshop*, pp. 1007–1012, 1996.

[54] E. Malis and M. Vargas, *Deeper understanding of the homography decomposition for vision-based control*. PhD thesis, INRIA, 2007.

[55] X.-S. Gao, X.-R. Hou, J. Tang, and H.-F. Cheng, "Complete solution classification for the perspective-three-point problem," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 25, pp. 930–943, Aug 2003.

[56] V. Lepetit, F. Moreno-Noguer, and P. Fua, "Epnp: An accurate o(n) solution to the pnp problem," *International Journal of Computer Vision*, vol. 81, no. 2, pp. 155–166, 2008.

[57] A. Penate-Sanchez, J. Andrade-Cetto, and F. Moreno-Noguer, "Exhaustive linearization for robust camera pose and focal length estimation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 10, pp. 2387–2400, 2013.

[58] L. Chen, C. W. Armstrong, and D. D. Raftopoulos, “An investigation on the accuracy of three-dimensional space reconstruction using the direct linear transformation technique,” *Journal of Biomechanics*, vol. 27, no. 4, pp. 493–500, 1994.

[59] iplimage, “P3p(blog).” "http://iplimage.com/blog/p3p-perspective-point-overview/", 2016.

[60] K. S. Arun, T. S. Huang, and S. D. Blostein, “Least-squares fitting of two 3-d point sets,” *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, no. 5, pp. 698–700, 1987.

[61] F. Pomerleau, F. Colas, and R. Siegwart, “A review of point cloud registration algorithms for mobile robotics,” *Foundations and Trends in Robotics (FnTROB)*, vol. 4, no. 1, pp. 1–104, 2015.

[62] O. D. Faugeras and M. Hebert, “The representation, recognition, and locating of 3-d objects,” *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 27–52, 1986.

[63] B. K. Horn, “Closed-form solution of absolute orientation using unit quaternions,” *JOSA A*, vol. 4, no. 4, pp. 629–642, 1987.

[64] G. C. Sharp, S. W. Lee, and D. K. Wehe, “Icp registration using invariant features,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 1, pp. 90–102, 2002.

[65] A. J. Davison, “Real-time simultaneous localisation and mapping with a single camera,” in *Computer Vision, 2003. Proceedings. Ninth IEEE International Conference on*, pp. 1403–1410, IEEE, 2003.

[66] G. Klein and D. Murray, “Parallel tracking and mapping for small ar workspaces,” in *Mixed and Augmented Reality, 2007. ISMAR 2007. 6th IEEE and ACM International Symposium on*, pp. 225–234, IEEE, 2007.

[67] R. Mur-Artal, J. Montiel, and J. D. Tardos, “Orb-slam: a versatile and accurate monocular slam system,” *arXiv preprint arXiv:1502.00956*, 2015.

[68] J. Engel, J. Sturm, and D. Cremers, “Semi-dense visual odometry for a monocular camera,” in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1449–1456, 2013.

[69] J. Engel, T. Schöps, and D. Cremers, “Lsd-slam: Large-scale direct monocular slam,” in *Computer Vision–ECCV 2014*, pp. 834–849, Springer, 2014.

[70] C. Kerl, J. Sturm, and D. Cremers, “Robust odometry estimation for rgb-d cameras,” in *Robotics and Automation (ICRA), 2013 IEEE International Conference on*, pp. 3748–3754, IEEE, 2013.

[71] C. Forster, M. Pizzoli, and D. Scaramuzza, “Svo: Fast semi-direct monocular visual odometry,” in *Robotics and Automation (ICRA), 2014 IEEE International Conference on* (rs, ed.), pp. 15–22, IEEE, 2014.

[72] M. Labbé and F. Michaud, “Online global loop closure detection for large-scale multi-session graph-based slam,” in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 2661–2666, IEEE, 2014.

[73] C. Kerl, J. Sturm, and D. Cremers, “Dense visual slam for rgb-d cameras,” in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 2100–2106, IEEE, 2013.

[74] F. Endres, J. Hess, J. Sturm, D. Cremers, and W. Burgard, “3-d mapping with an rgb-d camera,” *IEEE Transactions on Robotics*, vol. 30, no. 1, pp. 177–187, 2014.

[75] J. Engel, V. Koltun, and D. Cremers, “Direct sparse odometry,” *arXiv preprint arXiv:1607.02565*, 2016.

[76] J. Gui, D. Gu, S. Wang, and H. Hu, “A review of visual inertial odometry from filtering and optimisation perspectives,” *Advanced Robotics*, vol. 29, pp. 1289–1301, Oct 18 2015.

[77] A. Martinelli, “Closed-form solution of visual-inertial structure from motion,” *International Journal of Computer Vision*, vol. 106, no. 2, pp. 138–152, 2014.

[78] M. Bloesch, S. Omari, M. Hutter, and R. Siegwart, “Robust visual inertial odometry using a direct ekf-based approach,” in *Intelligent Robots and Systems (IROS), 2015 IEEE/RSJ International Conference on*, pp. 298–304, IEEE, 2015.

[79] M. Li and A. I. Mourikis, “High-precision, consistent ekf-based visual-inertial odometry,” *International Journal of Robotics Research*, vol. 32, pp. 690–711, MAY 2013.

[80] G. Huang, M. Kaess, and J. J. Leonard, “Towards consistent visual-inertial navigation,” in *2014 IEEE International Conference on Robotics and Automation (icra)*, IEEE International Conference on Robotics and Automation ICRA, pp. 4926–4933, 2014. IEEE International Conference on Robotics and Automation (ICRA), Hong Kong, PEOPLES R CHINA, MAY 31-JUN 07, 2014.

[81] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, “Keyframe-based visual–inertial odometry using nonlinear optimization,” *The International Journal of Robotics Research*, vol. 34, no. 3, pp. 314–334, 2015.

[82] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, “Imu preintegration on manifold for efficient visual-inertial maximum-a-posteriori estimation,” in *Robotics: Science and Systems XI*, no. EPFL-CONF-214687, 2015.

[83] M. Tkocz and K. Janschek, “Towards consistent state and covariance initialization for monocular slam filters,” *Journal of Intelligent & Robotic Systems*, vol. 80, pp. 475–489, DEC 2015.

[84] V. Usenko, J. Engel, J. Stueckler, and D. Cremers, “Direct visual-inertial odometry with stereo cameras,” in *IEEE International Conference on Robotics and Automation (ICRA)*, May 2016.

[85] A. Nüchter and J. Hertzberg, “Towards semantic maps for mobile robots,” *Robotics and Autonomous Systems*, vol. 56, no. 11, pp. 915–926, 2008.

[86] J. Civera, D. Gálvez-López, L. Riazuelo, J. D. Tardós, and J. Montiel, “Towards semantic slam using a monocular camera,” in *Intelligent Robots and Systems (IROS), 2011 IEEE/RSJ International Conference on*, pp. 1277–1284, IEEE, 2011.

[87] H. S. Koppula, A. Anand, T. Joachims, and A. Saxena, “Semantic labeling of 3d point clouds for indoor scenes,” in *Advances in Neural Information Processing Systems*, pp. 244–252, 2011.

[88] A. Anand, H. S. Koppula, T. Joachims, and A. Saxena, “Contextually guided semantic labeling and search for three-dimensional point clouds,” *The International Journal of Robotics Research*, p. 0278364912461538, 2012.

[89] N. Fioraio and L. Di Stefano, “Joint detection, tracking and mapping by semantic bundle adjustment,” *2013 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1538–45, 2013.

[90] R. F. Salas-Moreno, B. Glocken, P. H. Kelly, and A. J. Davison, “Dense planar slam,” in *Mixed and Augmented Reality (ISMAR), 2014 IEEE International Symposium on*, pp. 157–164, IEEE, 2014.

[91] R. F. Salas-Moreno, R. A. Newcombe, H. Strasdat, P. H. J. Kelly, and A. J. Davison, “Slam++: Simultaneous localisation and mapping at the level of objects,” *2013 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 1352–9, 2013.

[92] J. Stückler, N. Biresev, and S. Behnke, “Semantic mapping using object-class segmentation of rgb-d images,” in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 3005–3010, IEEE, 2012.

[93] I. Kostavelis and A. Gasteratos, “Learning spatially semantic representations for cognitive robot navigation,” *Robotics and Autonomous Systems*, vol. 61, no. 12, pp. 1460–1475, 2013.

[94] C. Couprie, C. Farabet, L. Najman, and Y. LeCun, “Indoor semantic segmentation using depth information,” *arXiv preprint arXiv:1301.3572*, 2013.

[95] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “Imagenet: A large-scale hierarchical image database,” in *CVPR09*, 2009.

[96] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in neural information processing systems*, pp. 1097–1105, 2012.

[97] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *arXiv preprint arXiv:1512.03385*, 2015.

[98] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” in *Advances in neural information processing systems*, pp. 91–99, 2015.

[99] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” *arXiv preprint arXiv:1411.4038*, 2014.

[100] S. Zheng, S. Jayasumana, B. Romera-Paredes, V. Vineet, Z. Su, D. Du, C. Huang, and P. Torr, “Conditional random fields as recurrent neural networks,” in *International Conference on Computer Vision (ICCV)*, 2015.

[101] S. Gupta, P. Arbeláez, R. Girshick, and J. Malik, “Indoor scene understanding with rgb-d images: Bottom-up segmentation, object detection and semantic segmentation,” *International Journal of Computer Vision*, pp. 1–17, 2014.

[102] K. Konda and R. Memisevic, “Learning visual odometry with a convolutional network,” in *International Conference on Computer Vision Theory and Applications*, 2015.

[103] A. Kendall, M. Grimes, and R. Cipolla, “Posenet: A convolutional network for real-time 6-dof camera relocalization,” in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2938–2946, 2015.

[104] Y. Hou, H. Zhang, and S. Zhou, “Convolutional neural network-based image representation for visual loop closure detection,” *arXiv preprint arXiv:1504.05241*, 2015.

[105] S. Y. An, J. G. Kang, L. K. Lee, and S. Y. Oh, “Line segment-based indoor mapping with salient line feature extraction,” *Advanced Robotics*, vol. 26, no. 5-6, pp. 437–460, 2012.

[106] H. Zhou, D. Zou, L. Pei, R. Ying, P. Liu, and W. Yu, “Structslam: Visual slam with building structure lines,” *Vehicular Technology, IEEE Transactions on*, vol. 64, pp. 1364–1375, April 2015.

[107] D. Benedettelli, A. Garulli, and A. Giannitrapani, “Cooperative slam using m-space representation of linear features,” *Robotics and Autonomous Systems*, vol. 60, no. 10, pp. 1267–1278, 2012.

[108] J. P. Saarinen, H. Andreasson, T. Stoyanov, and A. J. Lilienthal, “3d normal distributions transform occupancy maps: An efficient representation for mapping in dynamic environments,” *The International Journal of Robotics Research*, vol. 32, no. 14, pp. 1627–1644, 2013.

[109] W. Maddern, M. Milford, and G. Wyeth, “Cat-slam: probabilistic localisation and mapping using a continuous appearance-based trajectory,” *International Journal of Robotics Research*, vol. 31, no. 4SI, pp. 429–451, 2012.

[110] H. Wang, Z.-G. Hou, L. Cheng, and M. Tan, “Online mapping with a mobile robot in dynamic and unknown environments,” *International Journal of Modelling, Identification and Control*, vol. 4, no. 4, pp. 415–423, 2008.

[111] D. Zou and P. Tan, “Coslam: Collaborative visual SLAM in dynamic environments,” *IEEE Transactions On Pattern Analysis And Machine Intelligence*, vol. 35, no. 2, pp. 354–366, 2013.

[112] A. Gil, O. Reinoso, M. Ballesta, and M. Julia, “Multi-robot visual slam using a rao-blackwellized particle filter,” *Robotics and Autonomous Systems*, vol. 58, no. 1, pp. 68–80, 2010.

[113] T. A. Vidal-Calleja, C. Berger, J. Sola, and S. Lacroix, “Large scale multiple robot visual mapping with heterogeneous landmarks in semi-structured terrain,” *Robotics and Autonomous Systems*, vol. 59, no. 9, pp. 654–674, 2011.